Lectures on Quantum Computing and Completely Positive Maps

A formalisation in Isabelle/HOL of Vern Paulsen's Houston 2011 lecture notes

Dustin Bryant

formal statements and proofs: the theories listed in the table of contents

October 19, 2026

Abstract

This document is generated by Isabelle from the project theories. Every definition and theorem shown here has been checked by Isabelle/HOL (proofs are folded). The commentary restates Paulsen's lecture notes lecture by lecture (Day 1 = 22 August 2011, Day 34 = 18 November 2011): definitions and results in the notes' own language, each followed by the formal statement. The informal proofs of the notes are omitted; the formal proofs replace them. Where the formalisation deviates from the notes, the commentary says so. The companion file `NOTES_AUDIT.md` lists, for every lecture, what is formalised and what is not. Part VI goes beyond the notes: the functional calculus, phase estimation and the HHL algorithm, built on the same library.

Contents

1 Reading guide **2**

2 Conventions **3**

I Lectures 1–10: basic mathematics and the postulates of quantum mechanics **4**

3 Preliminaries: linear-space isomorphisms **4**

4 Day 1 (22 August 2011): the inner product **5**
4.1 The Hilbert–Schmidt inner product on matrices . . . 6
4.2 Properties of the inner product . . . 6

5 Day 2 (24 August 2011): bases, matrix units, trace, projections **7**
5.1 The trace . . . 8
5.2 Matrix-unit algebra and cyclicity of the trace . . . 10
5.3 Orthogonal projections and Parseval's identity . . . 10

6 Unitary algebra **13**

7 Tensor product infrastructure **15**
7.1 Tensor of Identities . . . 15
7.2 Tensor Adjoint . . . 15
7.3 Mixed-Product Property . . . 15
7.4 Corollaries of the Mixed-Product Property . . . 15
7.5 Tensor Unitarity . . . 15

8 Day 3 (31 August 2011): unitary matrices, Householder unitaries, Schur's theorem **16**
8.1 Unitary matrices . . . 16
8.2 Householder unitaries . . . 17

9 Day 4 (2 September 2011): Hermitian matrices; positive semidefinite and positive definite matrices **23**
9.1 Hermitian matrices . . . 23
9.2 The spectral theorem for Hermitian matrices . . . 26
9.3 Positive semidefinite and positive definite matrices . . . 27
9.4 The positive square root . . . 30

10 Day 5 (7 September 2011): tests for positivity; direct sums and partitioned matrices **30**
10.1 Practical tests for positivity . . . 30
10.2 Direct sums of vector spaces and partitioned matrices . . . 31

11 Day 6 (9 September 2011): tensor products **32**

12 Day 7 (12 September 2011): tensor products of Hilbert spaces; the postulates **33**
12.1 Postulates of quantum mechanics . . . 34

13 Day 8 (14 September 2011): the quantum game; projective measurements and expected values **35**
13.1 Distinguishing states . . . 35
13.2 Projective measurements, expected values and self-adjoint operators . . . 37

14 Day 9 (16 September 2011): positive operator-valued measures; composite systems; entanglement **38**
14.1 Positive operator-valued measures . . . 38
14.2 Composite systems . . . 39
14.3 Entanglement . . . 39

15 Day 10 (19 September 2011): superdense coding; classical and quantum gates **40**
15.1 One-qubit gates . . . 40
15.2 Two- and three-qubit gates . . . 41
15.3 Superdense coding . . . 42
15.4 Day 2: the projection does not depend on the orthonormal basis . . . 42
15.5 Day 6: coordinates in a tensor product . . . 43
15.6 Day 8: non-orthogonal states cannot be distinguished . . . 45
15.7 Day 9: measuring one half of a Bell pair . . . 45
15.8 Day 11: SWAP from three CNOTs . . . 46

II Lectures 11–22 and 29–34: quantum computing, density matrices, completely positive maps, error correction **47**

16 Day 11 (21 September 2011): gates revisited, circuit diagrams, no-cloning **47**
16.1 Cloning and no-cloning . . . 48

17 Day 12 (23 September 2011): quantum parallelism **48**

18 Day 13 (26 September 2011): ensembles and density matrices **49**
18.1 Ensembles . . . 49
18.2 Von Neumann's density matrix . . . 49
18.3 Probabilities via the trace . . . 51

19 Day 14 (28 September 2011): density matrices, pure states, composite ensembles **52**
19.1 Pure states . . . 52
19.2 Composite ensembles . . . 53
19.3 Partial traces: motivation . . . 54

20 Day 15 (30 September 2011): traces, tensor products of operators, partial traces **54**
20.1 The isomorphism $L(\mathcal{H}_A) \otimes L(\mathcal{H}_B) \cong L(\mathcal{H}_A \otimes \mathcal{H}_B)$ . . . 54
20.2 Partial traces . . . 57
20.3 Partial traces and measurements . . . 58

21 Day 16 (3 October 2011): partial traces in block form; the postulates for density matrices **59**

22 Day 17 (5 October 2011): measurement maps, quantum noise, completely positive maps **59**
22.1 Measurement maps . . . 59
22.2 Kraus maps, positive maps, complete positivity . . . 61

23 Day 18 (7 October 2011): the theory of completely positive maps **65**

24 Day 19 (10 October 2011): Choi's theorem **65**
24.1 From a positive Choi matrix to a Kraus representation . . . 66

25 Day 20 (12 October 2011): positive functionals, adjoints, Choi rank **70**
25.1 The adjoint of a Kraus map . . . 70
25.2 Complete positivity and the UCP/CPTP duality . . . 71
25.3 Positive functionals are completely positive . . . 72
25.4 Choi rank . . . 72

26 Day 21 (14 October 2011): the Kraus rank bound; unitary freedom of representations **73**

27 Day 22 (17 October 2011): minimal Kraus representations, convex sets, UCP and CPTP **74**
27.1 Convex sets and extreme points . . . 74
27.2 The convex sets UCP and CPTP . . . 75

28 Day 29 (7 November 2011): classical error detection and correction **76**
28.1 Binary strings, Hamming weight and distance . . . 76
28.2 Code distance . . . 77
28.3 The parity-check and repetition codes . . . 77

29 Day 30 (9 November 2011): error probabilities; the three-qubit bit-flip code **78**
29.1 The Pauli matrices . . . 78
29.2 Pauli magic (Day 32) . . . 79
29.3 The bit-flip channel (Day 31) . . . 80

30 Day 31 (11 November 2011): the bit-flip code as a channel; fidelity **80**
30.1 Fidelity . . . 81

31 Day 32 (14 November 2011): the phase-flip code, the Shor code, Pauli magic **81**
31.1 The phase-flip code . . . 81
31.2 Projections . . . 83
31.3 More on the repetition code . . . 84
31.4 Algebra of projections; towards Days 33 and 34 . . . 84

32 Extensions beyond the notes **85**
32.1 Rank-one projections . . . 86
32.2 The bit-flip channel at $p = 0$ and $p = 1$ . . . 86
32.3 The phase-flip channel . . . 86
32.4 Unitary-conjugation channels . . . 87
32.5 The bit-phase-flip channel and the identity channel . . . 88
32.6 Composition of channels . . . 89
32.7 Conjugation channels are completely positive . . . 89
32.8 Unitary-conjugation channels are UCP and CPTP . . . 90
32.9 The Pauli channels are UCP and CPTP . . . 91
32.10 Pauli matrices as Kraus operators . . . 92
32.11 Kraus maps of Kraus-operator lists are CPTP . . . 92
32.12 Concatenation of Kraus lists . . . 93
32.13 Fixed points of Pauli channels . . . 93
32.14 The maximally mixed state . . . 94
32.15 Fixed points of unitary-conjugation channels . . . 94
32.16 Channels preserve density matrices . . . 95
32.17 Pauli product identities . . . 96

33 Small matrix and vector lemmas shared by the gap-closure theories **96**

34 Day 12: $H^{\otimes N}$ on $|0\cdots0\rangle$ and the counting probability (D12-1, D12-4) **99**

35 Days 17–18: densities to densities iff positive and trace-preserving (D17-4, D18-2) **101**

36 Day 18: the transpose is positive but not 2-positive (D18-4) **102**

37 Day 20: Choi's rank theorem $\mathrm{cr}(\Phi) = \mathrm{rank}\, P_\Phi$ (D20-4) **103**
37.1 $\mathrm{rank}(\sum_{l<L} w_l w_l^*) \le L$ . . . 103
37.2 Independent vectors inside the column space bound the rank from below . . . 104
37.3 The theorem . . . 104

38 Day 21: unitary freedom of rank-one decompositions and Kraus representations (D21-3, D21-5, D21-6) **105**
38.1 The Gram matrix of an independent list is invertible; dual vectors . . . 105
38.2 D21-3: independent vectors give independent rank-one matrices . . . 105
38.3 Isometric mixing of columns preserves VV^* . . . 105
38.4 D21-5: two rank-one decompositions are related by an isometry . . . 106
38.5 D21-6 (Choi 2): unitary freedom of Kraus representations . . . 106

39 Day 22: linear independence bounds the number of Kraus operators (D22-6) **107**

40 Day 22: a minimal Kraus representation is linearly independent (D22-1) 108
40.1 Mixing a Kraus list by an isometry does not change the map . . . 108
40.2 A unitary with a prescribed first row . . . 109
40.3 D22-1 . . . 109
40.4 The notes' "Hamming [7,1,3]" code does not correct one error . . . 110
40.5 The Hamming [7,4,3] code corrects one error . . . 111
40.6 Probabilities of undetected and uncorrected errors (Day 30) . . . 111

41 Day 30: the three-qubit bit-flip code and its recovery map (D30-3, D30-4) 112
41.1 Errors followed by recovery . . . 116
41.2 Minimum fidelities (D31-4) . . . 117

42 Day 32: the Pauli expansion identity for unitaries (D32-7) 117

43 Days 32–33: composition of Kraus maps and the general recovery proposition (D32-4, D33-2) 118
43.1 Algebra of *foldr*-sums of matrices . . . 118
43.2 Composition of Kraus maps . . . 119
43.3 The general recovery proposition (D32-4) and its linear-combination form (D33-2) 120
43.4 Matrix sums acting on vectors . . . 120
43.5 Codes with orthogonal error images . . . 123
43.6 The Y-flip code . . . 124
43.7 Tensor products of matrices . . . 125
43.8 Single-qubit operators on nine qubits, and on a block of three . . . 125
43.9 The block vectors $|m\rangle \pm |7-m\rangle$ . . . 126
43.10 Code words and the images of the errors . . . 127
43.11 The 22 correctable errors . . . 129
43.12 The code space and its projection . . . 130
43.13 Proposition (Day 33): orthogonal images, and $Z_1 v = Z_2 v = Z_3 v$ . . . 130
43.14 Shor's theorem . . . 131

44 Day 34: the Knill–Laflamme conditions (D34-1, D34-2, D34-3) 132
44.1 Vectors in the range of a projection; rank-one decompositions inside V . . . 132
44.2 Linearity facts for the composite map $\mathcal{R} \circ \mathbb{E}$ . . . 133
44.3 Knill–Laflamme, Theorem 1, necessity: $PE_i^* E_j P = \alpha_{ij} P$ . . . 133
44.4 Kraus lists consisting of multiples of one operator . . . 134
44.5 Theorem 2 . . . 134

III Lectures 23–26: extreme points, operator systems and Arveson's theorem 136

45 Day 23 (19 October 2011): Choi's Theorem 3, extreme points of UCP and CPTP 136
45.1 Extreme UCP maps . . . 137
45.2 UCP-Kraus representation . . . 138
45.3 Unital condition from the Kraus list . . . 139
45.4 UCP-Kraus existence . . . 139
45.5 Sanity facts about *kraus-products* . . . 139
45.6 Quadratic Kraus map (perturbation primitive) . . . 139
45.7 Lemma A: PSD coefficient matrix gives a CP perturbation . . . 143
45.8 Small-ε PSD perturbation of the identity . . . 144

45.9 Symmetric / anti-hermitian split of the coefficient matrix . . . 144
45.10Bridge: ¬ *mat-lin-indep-real* gives a real *A-orig* witness . . . 146
45.11Auxiliary lemmas for the assembly . . . 148
45.12Minimality and the distinctness lemma (Lemma B) . . . 148
45.13CPTP-Kraus representation and extreme CPTP maps . . . 149
45.14Vectorisation of Kraus operators . . . 150
45.15Choi matrix of a Kraus map . . . 150
45.16Choi matrix is linear in the map . . . 151
45.17The Choi matrix of a *kraus-quad-map* . . . 151
45.18CP Radon–Nikodym — the dominated-PSD kernel . . . 152
45.18.1 Diagonal pseudo-inverse list and its identity . . . 152
45.19Linearity of *kraus-quad-map* in its matrix argument . . . 154
45.20Choi matrix determines a linear CP map on carrier . . . 154
45.21CP Radon–Nikodym . . . 155
45.22Choi's Theorem 3 — reverse direction . . . 156
45.23Trace pairing duality and TP-converse . . . 159
45.24Choi's Theorem 3 — forward direction . . . 159
45.25Choi's Theorem 3 (statement) . . . 159
45.26Sanity / easy facts about extreme UCP . . . 161
45.27Linear dependence tests . . . 162
45.28Example 1: unitary conjugation is extreme . . . 162
45.29Example 2: an average of two unitary conjugations . . . 163
45.30Example 4: the diagonal map . . . 163
45.31Example 3: $X \mapsto \mathrm{Tr}(X)\, I_d$ . . . 164
45.32$M_p(OS)$ is an operator system . . . 166
45.33Every element is a combination of four positives . . . 166
45.34Finite sums of matrices . . . 167
45.35$E_{ij} \otimes M$ and the block decomposition . . . 168
45.36The correspondence . . . 168
45.37Theorem (Arveson): completely positive iff d-positive iff S_Φ positive . . . 170
45.38Vectors . . . 171
45.39The operator norm . . . 172
45.40Unital functionals . . . 173
45.41Auxiliary facts . . . 174
45.42Matrices as points of the Euclidean space *lmat* . . . 175
45.43Extension of positive functionals . . . 176
45.44Arveson's extension theorem . . . 176

IV Functional-analytic infrastructure (Lecture 25: Hahn–Banach) 176

46 Typed-dimension matrix wrapper 177

47 Typeclass instances 177

48 Real scalar multiplication and Hahn-Banach overloading 178

49 Real-vectorspace locale on the full type 179

50 The Hilbert-Schmidt norm on (*complex*, ′*m*, ′*n*) *lmat* 179

51 Triangle inequality for the HS norm 179

52 Subspace, seminorm, normed-vectorspace instances 180

53 Complex scalar multiplication on lmat 180

54 Day 25 (26 October 2011): the Hahn–Banach theorem 182

55 Bounded complex-linear functionals on lmat 182

56 Real-part functional and linearform/continuous instances 182

57 Application of HOL/Hahn_Banach (norm-preserving extension) 183

58 Bohnenblust-Sobczyk: complex extension from real extension 183

59 The matrix-space Hahn-Banach theorem 184

60 Overview 185

61 HS inner-product algebra 185

62 Trace lemmas 186

63 HS norm definition and basic facts 186

64 HS-norm convergence vs entry-wise convergence 187

65 Carathéodory's theorem for conic combinations 189

66 Overview 192

67 Real scalar multiplication and *real-vector* instance 193

68 Auxiliary: pointwise unwrapping (used by later instances) 193

69 Distance, uniformity, open: *metric-space* instance 194

70 Norm and *real-normed-vector* instance 194

71 Real inner product 195

72 Basis: matrix units carrying a basis vector 195

73 Basis set 196

74 Euclidean space instance 197

75 Heine-Borel 197

76 Dimension 197

77 Component-wise bounds via the L2 norm 197

78 Convergence: lmat sequence convergence implies entry convergence 197

V Lectures 26–28: separability and entanglement witnesses 198

79 Overview (Days 27–28 of the notes) 198

80 Typed PSD predicate 199

81 Typed tensor product 199

82 The separable cone 199

83 Step 1: PSD set is closed 200

84 Trace on lmat 200

85 Compactness of the unit-trace PSD slice 201

86 Tensor continuity 201

87 The unit-trace product slice 201

88 Closedness of the separable cone 202

89 The lmat inner product as a real-trace inner product 204

90 Separating hyperplane for non-separable states 205

91 Entanglement witness in real-trace form 205

92 Hermitian entanglement witness 205

93 Hermitian closure of the separable cone 206

94 Main separability characterisation 207

95 Density-matrix specialisation 207

96 Days 26–28 (28 October – 2 November 2011): separability and entanglement witnesses 210

96.1 The separable cone (Day 27) . . . 210
96.2 Separable density matrices (Day 26) . . . 210
96.3 The separable cone is closed (Day 28) . . . 211
96.4 Separation (Days 27–28) . . . 211
96.5 Entanglement witnesses (Day 28) . . . 212
96.6 Tensor products with a matrix in the first factor . . . 213
96.7 The quadratic form of Φ_f . . . 214
96.8 Proposition (Day 27): $f(\mathrm{Sep}) \geq 0$ iff Φ_f is positive . . . 214
96.9 Proposition (Day 27): positive maps keep separable matrices positive . . . 214
96.10 Theorem (Day 28): the positive-map criterion . . . 215

97 Day 28: the Horodecki–Peres criterion for two qubits (D28-5) 215

VI Quantum Algorithms (beyond the notes) 216

98 Functional calculus for Hermitian matrices **216**
98.1 Spectral decompositions . . . 217
98.2 The definition . . . 217
98.3 Algebra . . . 218
98.4 Time evolution . . . 219
98.5 The inverse . . . 219

99 Phase estimation **220**
99.1 Sums of roots of unity . . . 220
99.2 The Fourier matrix . . . 220
99.3 Controlled powers and phase kickback . . . 221
99.4 The phase estimation circuit . . . 222
99.5 Measuring the register . . . 222
99.6 The success probability . . . 222

100 The AFP Fourier transform circuit is the Fourier matrix **223**

101 The HHL algorithm **224**
101.1 Block-diagonal controlled operations . . . 224
101.2 The circuit . . . 225
101.3 The circuit on an eigenvector . . . 227
101.4 All right-hand sides . . . 227
101.5 The success probability . . . 228
101.6 Non-Hermitian systems . . . 228

102 HHL with inexact phase estimation **229**
102.1 Register sums . . . 229
102.2 Phase estimation on a branch . . . 230
102.3 Bounds on the branch distances and amplitudes . . . 232
102.4 The error bound . . . 233

theory *QC-Notes-Overview*
imports
Vern-Paulsen-QC.Day23-ChoiExtremeUCP
Vern-Paulsen-QC.Day24-Separability
Vern-Paulsen-QC.Day12-Counting-Probability
Vern-Paulsen-QC.Day18-Positive-Maps
Vern-Paulsen-QC.Day22-Kraus-Independence
Vern-Paulsen-QC.Day28-Horodecki-Peres
Vern-Paulsen-QC.Day30-BitFlip-Code
Vern-Paulsen-QC.Day32-Pauli-Expansion
Vern-Paulsen-QC.Day21-Unitary-Freedom
Vern-Paulsen-QC.Day32-Recovery-Proposition
Vern-Paulsen-QC.Day22-Minimal-Kraus
Vern-Paulsen-QC.Day34-Knill-Laflamme
Vern-Paulsen-QC.Day20-Choi-Rank
Vern-Paulsen-QC.Day34-KL-Sufficiency
Vern-Paulsen-QC.Day33-Shor-Code
Vern-Paulsen-QC.Day17-Environment-Model
Vern-Paulsen-QC.Day27-Positive-Map-Criterion
Vern-Paulsen-QC.Day02-11-Supplement
Vern-Paulsen-QC.Day29-Classical-Codes
Vern-Paulsen-QC.Day31-Code-Fidelity
Vern-Paulsen-QC.Day32-Y-Code
Vern-Paulsen-QC.Day19-Choi-TP-Unital
Vern-Paulsen-QC.Day23-Extremality-Examples
Vern-Paulsen-QC.Day25-Unital-Functional-Norm
Vern-Paulsen-QC-Algorithms.HHL-Approximate
Vern-Paulsen-QC-Algorithms.QFT-Fourier
begin

1 Reading guide

Paulsen's course had five parts: (I) basic mathematics, (II) an introduction to quantum computing, (III) the theory of completely positive maps, (IV) entanglement and entanglement witnesses, (V) quantum error correction. The formal development keeps the mathematics of the notes but is organised by theory, since later lectures reuse earlier infrastructure. The table gives the correspondence.

Lectures	Topic
1–2	inner products, matrix units, trace, projections, Parseval
3–10	unitaries, Schur, Hermitian and positive matrices, tensor products, postulates, POVMs, gates
11–22	no-cloning, oracles, density matrices, partial traces, CP maps, Choi's theorem, adjoints, convexit
23	extreme points of CPTP/UCP maps (Choi's Theorem 3), CP Radon–Nikodym
25	Hahn–Banach on matrix spaces
–	Hilbert–Schmidt analysis, Carathéodory, typed matrices as a Euclidean space
26–28	separable cone, separating hyperplanes, entanglement witnesses
29–32	classical codes, bit/phase-flip channels, fidelity, Pauli algebra
12	$H^{\otimes N}\lvert 0\cdots 0\rangle$, the counting probability
17	the environment model of noise, $\mathrm{tr}_E(U(\rho\otimes\lvert\varphi\rangle\langle\varphi\rvert)U^*)=\sum W_i\rho W_i^*$
17–18	densities to densities iff positive and TP; the transpose
20	$\mathrm{cr}(\Phi)=\operatorname{rank}P_\Phi$
21	rank-one decompositions and Kraus representations up to an isometry (Choi 2)
22	$\mathrm{cr}(\Phi)\le n$ for extreme maps; minimal Kraus lists are independent
24	operator systems, $M_p(S)$, completely positive maps on S
24–25	Arveson: CP $\iff$ d-positive $\iff$ $S_\Phi\ge 0$; the correspondence $\Phi\leftrightarrow S_\Phi$
26	Arveson's extension theorem; Kraus form on operator systems
27–28	positive maps and separability: $f(\mathrm{Sep})\ge 0 \iff \Phi_f$ positive, the positive-map criterion
28	the two-qubit Horodecki–Peres criterion
30	the three-qubit bit-flip code and its recovery theorem
32–33	Pauli expansion of unitaries; the general recovery proposition
32–33	the nine-qubit Shor code and Shor's theorem
34	Knill–Laflamme: states vs. PXP, Theorem 1 (both directions), closure
2–11	basis-independent projections, tensor coordinates, non-orthogonal states, a Bell measurement, S
29–30	the notes' Hamming code, the $[7,4,3]$ code, undetected-error probabilities
31	the bit-flip code against independent flips: $R\circ E$, minimum fidelities
32	the code for Y flips; codes with orthogonal error images
19	trace preservation and unitality read off the Choi matrix
23	extreme and non-extreme CPTP maps: U^*XU, averages, $\mathrm{Tr}(X)I/d$, diagonals
25	a unital functional is positive iff it has norm one
–	functional calculus $g(A)$ for Hermitian A; e^{itA}, A^{-1}
–	Fourier matrix, phase kickback, phase estimation (exact; $\ge 4/\pi^2$)
–	the AFP circuit *ordered-QFT* is the Fourier matrix F_{2^n}
–	the HHL algorithm: output state, $Ax=b$, success $\ge 1/\kappa^2$, non-Hermitian A
–	inexact phase estimation: the error identity, and for the clamped circuit $\lVert\text{output}-\text{ideal}\rVert^2\le\lVert u\rVert$

The theories of the second and third blocks (shared lemmas in `QC_Prelims`) close the gaps recorded by the first and second audits; the remaining unformalised items are listed in `NOTES_AUDIT.md`. The last block, Part VI, goes beyond the notes (session `Vern_Paulsen_QC_Algorithms`).

2 Conventions

- Vectors in $\mathbb{C}^n$ are *complex Matrix.vec* with *dim-vec* $v = n$; matrices in $M_{m,n}$ are *complex Matrix.mat* with $A \in$ *carrier-mat* m n (the Jordan-Normal-Form library). Dimensions are hypotheses, not types, except in the typed wrapper *lmat* used for the topological arguments of Part IV.
- The physicists' adjoint $X^\dagger = X^*$ is $A^\dagger$; $\langle y|x\rangle$ is $\langle y|x\rangle$ (conjugate-linear in the first slot, as in the notes); $|v\rangle\langle w|$ is $v \oslash w$; the Hilbert–Schmidt inner product $\langle Y|X\rangle = \mathrm{Tr}(Y^*X)$ is $\langle\langle$

$Y \mid X \rangle\rangle$; the matrix unit $E_{ij} \in M_{m,n}$ is $E[\![i,j \ ; \ m,n]\!]$.

- Indices start at 0, as the physicists in the notes prefer: the canonical basis of $\mathbb{C}^n$ is $|0\rangle, \ldots, |n-1\rangle$, formally *unit-vec n i*.
- $\sum_i |v_i\rangle\langle v_i|$ for a list of vectors is *outer-sum n vs* $= VV^\dagger$ with V the matrix whose columns are the v_i.
- Tensor products are Kronecker products: $v \otimes w$ and $A \bigotimes B$, with the first factor as the major index.
- Kraus sums $\sum_i W_i X W_i^\dagger$ over a list are *kraus-map-dim d Ws X*.

Verification status. All theories build with *quick-and-dirty = false*; there are no *sorry*s and no axioms beyond those of Isabelle/HOL and the imported libraries.

end

Part I
Lectures 1–10: basic mathematics and the postulates of quantum mechanics

theory *Basic-Math*
imports
Quantum-Fourier-Transform.QFT
Isabelle-Marries-Dirac.No-Cloning
begin

Isabelle_Marries_Dirac names its gates *X*, *Y*, *Z*, *H*, *S*, *T*; hidden here so that these letters are variables. The gates remain *Quantum.X* etc.; the project names are *Pauli-X*, *Pauli-Y*, *Pauli-Z*, *Hadamard*, *Phase-S*. Theories importing this one must not import Isabelle_Marries_Dirac directly, as that undoes the hiding.

hide-const (**open**) *Quantum.X Quantum.Y Quantum.Z Quantum.H Quantum.S Quantum.T*

No-Cloning is imported for *cauchy-schwarz-ineq*; its simp rules (e.g. $v = w$ from inner products) slow simp on every vector equation.

declare *cauchy-schwarz-eq* [*simp del*] *cauchy-schwarz-col* [*simp del*] *eq-from-inner-prod* [*simp del*]

3 Preliminaries: linear-space isomorphisms

Matrix spaces are represented by carrier predicates on the single type *complex Matrix.mat*. The record below packages a carrier with its zero, addition and scalar multiplication, so that statements such as $L(\mathcal{H}_A) \otimes L(\mathcal{H}_B) \cong L(\mathcal{H}_A \otimes \mathcal{H}_B)$ (Day 15) can be made as explicit linear isomorphisms with explicit inverses.

record (*'a*, *'v*) *linear-structure* =
ls-carrier :: *'v set*
ls-zero :: *'v*

ls-add :: *'v* ⇒ *'v* ⇒ *'v*
ls-smul :: *'a* ⇒ *'v* ⇒ *'v*

definition *linear-structure-map* ::
(*'a*, *'v*) *linear-structure* ⇒ (*'a*, *'w*) *linear-structure* ⇒ (*'v* ⇒ *'w*) ⇒ *bool* **where**
linear-structure-map V W f ≡
f (*ls-zero V*) = *ls-zero W* ∧
(∀ *x* ∈ *ls-carrier V*. *f x* ∈ *ls-carrier W*) ∧
(∀ *x* ∈ *ls-carrier V*. ∀ *y* ∈ *ls-carrier V*.
f (*ls-add V x y*) = *ls-add W* (*f x*) (*f y*)) ∧
(∀ *c x*. *x* ∈ *ls-carrier V* ⟶
f (*ls-smul V c x*) = *ls-smul W c* (*f x*))

definition *is-isomorphism-between* ::
(*'a*, *'v*) *linear-structure* ⇒ (*'a*, *'w*) *linear-structure* ⇒ (*'v* ⇒ *'w*) ⇒ *bool* **where**
is-isomorphism-between V W f ≡
linear-structure-map V W f ∧
(∃ *g*. *linear-structure-map W V g* ∧
(∀ *x* ∈ *ls-carrier V*. *g* (*f x*) = *x*) ∧
(∀ *y* ∈ *ls-carrier W*. *f* (*g y*) = *y*))

definition *is-isomorphic-to* ::
(*'a*, *'v*) *linear-structure* ⇒ (*'a*, *'w*) *linear-structure* ⇒ *bool* **where**
is-isomorphic-to V W ≡ ∃ *f*. *is-isomorphism-between V W f*

lemma *is-isomorphism-betweenD-map*:
assumes *is-isomorphism-between V W f*
shows *linear-structure-map V W f*
⟨*proof*⟩

lemma *is-isomorphism-betweenD-inverse*:
assumes *is-isomorphism-between V W f*
obtains *g* **where** *linear-structure-map W V g*
and ⋀*x*. *x* ∈ *ls-carrier V* ⟹ *g* (*f x*) = *x*
and ⋀*y*. *y* ∈ *ls-carrier W* ⟹ *f* (*g y*) = *y*
⟨*proof*⟩

definition *mat-linear-structure* :: *nat* ⇒ *nat* ⇒ (*complex*, *complex Matrix.mat*) *linear-structure* **where**
mat-linear-structure m n ≡
(‖ *ls-carrier* = *carrier-mat m n*,
ls-zero = 0_m *m n*,
ls-add = (λ*x y*. *x* + *y*),
ls-smul = (λ*c x*. *c* ·$_m$ *x*) ‖)

4 Day 1 (22 August 2011): the inner product

Paulsen opens with the dictionary between classical and quantum computing: a bit is an element of $\mathbb{Z}_2$, a qubit a unit vector in $\mathbb{C}^2$; N bits form $\mathbb{Z}_2^N$, N qubits a unit vector in $\mathbb{C}^{2^N} \cong \mathbb{C}^2 \otimes \cdots \otimes \mathbb{C}^2$; operations are Boolean gates versus completely positive maps on M_{2^N}; information is a probability vector versus a positive semidefinite matrix of trace one. He then fixes the physicists' notation.

Definition (notes). $\mathbb{C}^n$ is the space of complex n-tuples with the inner product

$$\langle y|x\rangle = \sum_{i=1}^{n} \overline{y_i}\, x_i,$$

which is linear in the right-hand slot, conjugate linear in the left-hand slot ("sesquilinear"), and positive definite: $\langle x|x\rangle \geq 0$ with equality only for $x = 0$. The Euclidean length is $\|x\| = \sqrt{\langle x|x\rangle}$. A *Hilbert space* is a complex vector space with a sesquilinear positive definite form, complete for this norm.

The library's inner product $\langle y|x\rangle$ is exactly this form, with indices running from 0 to $n - 1$.

lemma *inner-prod-expand*:
assumes *dim-vec u* = *n* **and** *dim-vec v* = *n*
shows $\langle u|v\rangle = (\sum\ i \in \{0..< n\}.\ cnj(u\ \$\ i) * (v\ \$\ i))$
⟨*proof*⟩

4.1 The Hilbert–Schmidt inner product on matrices

Definition (notes) (Example)**.** The space $M_{n,k}$ of complex $n \times k$ matrices is a Hilbert space for

$$\langle (y_{ij}) \,|\, (x_{ij})\rangle = \sum_i \sum_j \overline{y_{ij}}\, x_{ij},$$

and $M_{n,k} \cong \mathbb{C}^{nk}$.

We write $\langle\langle y|x\rangle\rangle$ for this Hilbert–Schmidt (Frobenius) inner product; the double brackets distinguish it from the inner product of vectors.

definition *hs-inner-prod* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* ⇒ *complex* **where**
hs-inner-prod = (λ *y x*.
$\sum\ i \in \{0\ ..<\ dim\text{-}row\ x\}.$
$\sum\ j \in \{0\ ..<\ dim\text{-}col\ x\}.$
cnj (*y* $$ (*i*,*j*)) ∗ (*x* $$ (*i*,*j*)))

notation *hs-inner-prod* (⟨⟨ - | - ⟩⟩)

lemma *hs-inner-prod-expand*:
assumes *dim-row x* = *m* **and** *dim-col x* = *n*
shows ⟨⟨ *y* | *x* ⟩⟩ =
$(\sum\ i \in \{0\ ..<\ m\}.\ \sum\ j \in \{0\ ..<\ n\}.\ cnj\ (y\ \$\$\ (i,j)) * (x\ \$\$\ (i,j)))$
⟨*proof*⟩

lemma *hs-inner-prod-square*:
assumes *dim-row x* = *n* **and** *dim-col x* = *n*
shows ⟨⟨ *y* | *x* ⟩⟩ =
$(\sum\ i \in \{0\ ..<\ n\}.\ \sum\ j \in \{0\ ..<\ n\}.\ cnj\ (y\ \$\$\ (i,j)) * (x\ \$\$\ (i,j)))$
⟨*proof*⟩

4.2 Properties of the inner product

The properties listed in the notes, verified for the matrix inner product: conjugate symmetry, linearity in the right-hand slot, conjugate linearity in the left-hand slot, and positive definiteness.

lemma *hs-inner-prod-cnj*:
assumes *dim-row x* = *dim-row y* **and** *dim-col x* = *dim-col y*
shows ⟨⟨ *x* | *y* ⟩⟩ = *cnj* (⟨⟨ *y* | *x* ⟩⟩)
⟨*proof*⟩

lemma *hs-inner-prod-right-add*:
assumes *dim-row X1 = dim-row y* **and** *dim-col X1 = dim-col y*
and *dim-row X2 = dim-row y* **and** *dim-col X2 = dim-col y*
shows ⟨⟨ *y* | *X1 + X2* ⟩⟩ = ⟨⟨ *y* | *X1* ⟩⟩ + ⟨⟨ *y* | *X2* ⟩⟩
⟨*proof*⟩

lemma *hs-inner-prod-right-smult*:
assumes *dim-row x = dim-row y* **and** *dim-col x = dim-col y*
shows ⟨⟨ *y* | *c* ·$_m$ *x* ⟩⟩ = *c* ∗ ⟨⟨ *y* | *x* ⟩⟩
⟨*proof*⟩

lemma *hs-inner-prod-left-smult*:
assumes *dim-row x = dim-row y* **and** *dim-col x = dim-col y*
shows ⟨⟨ *c* ·$_m$ *y* | *x* ⟩⟩ = *cnj c* ∗ ⟨⟨ *y* | *x* ⟩⟩
⟨*proof*⟩

lemma *hs-inner-prod-self-real*:
shows ⟨⟨ *x* | *x* ⟩⟩ ∈ ℝ
⟨*proof*⟩

lemma *hs-inner-prod-self-nonneg*:
shows *Re* (⟨⟨ *x* | *x* ⟩⟩) ≥ *0*
⟨*proof*⟩

lemma *hs-inner-prod-eq-zero-imp*:
assumes ⟨⟨ *x* | *x* ⟩⟩ = *0*
and *i* < *dim-row x* **and** *j* < *dim-col x*
shows *x* $$ (*i*,*j*) = *0*
⟨*proof*⟩

5 Day 2 (24 August 2011): bases, matrix units, trace, projections

Definition (notes) (Examples)**.** $\mathbb{C}^n$ has the orthonormal basis $e_1, \ldots, e_n$ and every x satisfies $x = \sum_i x_i e_i = \sum_i \langle e_i | x \rangle e_i$. The *matrix units* E_{ij} (entry 1 in position (i, j), 0 elsewhere) form an orthonormal basis of $M_{n,m}$, and

$$X = \sum_{i,j} X_{ij} E_{ij} = \sum_{i,j} \langle E_{ij} | X \rangle E_{ij}.$$

Formally *E*⟦*i*,*j* ; *m*,*n*⟧ is the $m \times n$ matrix unit (indices first, dimensions second). The notes also recall the conjugate $\overline{X}$, the transpose X^t, the adjoint $X^* = \overline{X}^t$ (written $X^\dagger$ by physicists, formally $X^\dagger$) and the row-times-column rule for matrix products; these are library notions.

definition *mat-unit* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
mat-unit i j m n = *Matrix.mat m n* (λ(*r*,*c*). *if r* = *i* ∧ *c* = *j then 1 else 0*)

notation *mat-unit* (*E*⟦-,- ; -,-⟧ [*1000*, *1000*, *1000*, *1000*] *999*)

lemma *mat-unit-dim-row* [*simp*]:
fixes *i j m n* :: *nat*
shows *dim-row* (*E*⟦*i*,*j* ; *m*,*n*⟧) = *m*
⟨*proof*⟩

lemma *mat-unit-dim-col* [*simp*]:
fixes *i j m n* :: *nat*
shows *dim-col* (*E*⟦*i*,*j* ; *m*,*n*⟧) = *n*
⟨*proof*⟩

lemma *mat-unit-index* [*simp*]:
assumes *r* < *m* **and** *c* < *n*
shows (*E*⟦*i*,*j* ; *m*,*n*⟧) $$ (*r*,*c*) = (*if r* = *i* ∧ *c* = *j then 1 else 0*)
⟨*proof*⟩

Pairing with E_{ij} extracts the (i, j) entry: the matrix analogue of $\langle e_i|v\rangle = v_i$.

lemma *hs-inner-prod-mat-unit-left*:
assumes *i* < *dim-row x* **and** *j* < *dim-col x*
shows ⟨⟨ *E*⟦*i*,*j* ; (*dim-row x*), (*dim-col x*)⟧ | *x* ⟩⟩ = *x* $$ (*i*,*j*)
⟨*proof*⟩

lemma *mat-unit-orthonormal*:
assumes *i-lt*: *i* < *m*
and *j-lt*: *j* < *n*
and *k-lt*: *k* < *m*
and *l-lt*: *l* < *n*
shows ⟨⟨ *E*⟦*i*,*j* ; *m*,*n*⟧ | *E*⟦*k*,*l* ; *m*,*n*⟧ ⟩⟩ =
(*if i* = *k* ∧ *j* = *l then 1 else 0*)
⟨*proof*⟩

Expansion in the basis $\{E_{ij}\}$, stated entrywise because matrices of different dimensions cannot be added in the library.

lemma *mat-unit-expansion*:
assumes *r* < *dim-row x* **and** *c* < *dim-col x*
shows *x* $$ (*r*,*c*) =
(∑ *i* ∈ {*0*..<*dim-row x*}. ∑ *j* ∈ {*0*..<*dim-col x*}.
x $$ (*i*,*j*) ∗ (*mat-unit i j* (*dim-row x*) (*dim-col x*)) $$ (*r*,*c*))
⟨*proof*⟩

lemma *mat-unit-hs-expansion*:
assumes *r* < *dim-row x* **and** *c* < *dim-col x*
shows *x* $$ (*r*,*c*) =
(∑ *i* ∈ {*0*..<*dim-row x*}. ∑ *j* ∈ {*0*..<*dim-col x*}.
⟨⟨ *mat-unit i j* (*dim-row x*) (*dim-col x*) | *x* ⟩⟩ ∗
(*mat-unit i j* (*dim-row x*) (*dim-col x*)) $$ (*r*,*c*))
⟨*proof*⟩

5.1 The trace

Definition (notes). $\mathrm{Tr} : M_{n,n} \to \mathbb{C}$, $\mathrm{Tr}(X) = \sum_{i=1}^{n} x_{ii}$; we sometimes write Tr_n.

The formal *Tr* sums the diagonal of any matrix. On matrix units, $\mathrm{Tr}(E_{ii}) = 1$ and $\mathrm{Tr}(E_{ij}) = 0$ for $i \neq j$, so the trace is the sum of the diagonal Hilbert–Schmidt coefficients.

definition *Tr* :: *complex Matrix.mat* ⇒ *complex* **where**
Tr x = (∑ *r* < *dim-row x*. *if r* < *dim-col x then x* $$ (*r*,*r*) *else 0*)

lemma *Tr-mat-unit*:
assumes *i-lt*: *i* < *m*
and *j-lt*: *j* < *n*
shows *Tr* (*E*⟦*i*,*j* ; *m*,*n*⟧) = (*if i* = *j then 1 else 0*)
⟨*proof*⟩

corollary *Tr-mat-unit-diag*:
assumes *i* < *n*
shows *Tr* (*E*⟦*i*,*i* ; *n*,*n*⟧) = *1*
⟨*proof*⟩

corollary *Tr-mat-unit-offdiag*:
assumes *i* < *m* **and** *j* < *n* **and** *i* ≠ *j*
shows *Tr* (*E*⟦*i*,*j* ; *m*,*n*⟧) = *0*
⟨*proof*⟩

lemma *Tr-hs-diagonal-sum*:
shows *Tr x* =
(∑ *r* < *dim-row x*.
if r < *dim-col x*
then ⟨⟨ *E*⟦*r*,*r* ; (*dim-row x*), (*dim-col x*)⟧ | *x* ⟩⟩
else 0)
⟨*proof*⟩

corollary *Tr-hs-diagonal-sum-square*:
assumes *rows*: *dim-row x* = *n*
and *cols*: *dim-col x* = *n*
shows *Tr x* = (∑ *r*<*n*. ⟨⟨ *E*⟦*r*,*r* ; *n*,*n*⟧ | *x* ⟩⟩)
⟨*proof*⟩

The trace is linear.

lemma *Tr-add*:
assumes *rows*: *dim-row x* = *dim-row y*
and *cols*: *dim-col x* = *dim-col y*
shows *Tr* (*x* + *y*) = *Tr x* + *Tr y*
⟨*proof*⟩

lemma *Tr-smult*: *Tr* (*a* ·$_m$ *x*) = *a* ∗ *Tr x*
⟨*proof*⟩

corollary *Tr-uminus*: *Tr* (− *x*) = − *Tr x*
⟨*proof*⟩

corollary *Tr-diff*:
assumes *rows*: *dim-row x* = *dim-row y*
and *cols*: *dim-col x* = *dim-col y*
shows *Tr* (*x* − *y*) = *Tr x* − *Tr y*
⟨*proof*⟩

lemma *Tr-zero-mat* [*simp*]:
shows *Tr* (*Matrix.mat m n* (λ(*r*,*c*). *0*)) = *0*
⟨*proof*⟩

5.2 Matrix-unit algebra and cyclicity of the trace

$E_{ij}^* = E_{ji}$, $E_{ij}E_{kl} = \delta_{jk}E_{il}$, $\mathrm{Tr}(AB) = \mathrm{Tr}(BA)$, $\mathrm{Tr}(U^*AU) = \mathrm{Tr}(A)$ for unitary U, and $\langle A|B\rangle = \mathrm{Tr}(A^*B)$: the facts about the trace that the notes use from Day 3 onwards.

lemma *mat-unit-dagger*:
shows (*E*⟦*i*,*j* ; *m*,*n*⟧)† = *E*⟦*j*,*i* ; *n*,*m*⟧
⟨*proof*⟩

lemma *mat-unit-mult-entry*:
assumes *r-lt*: *r* < *m*
and *c-lt*: *c* < *p*
and *j-lt*: *j* < *n*
shows (*E*⟦*i*,*j* ; *m*,*n*⟧ ∗ *E*⟦*k*,*l* ; *n*,*p*⟧) $$ (*r*,*c*) =
(*if j* = *k* ∧ *r* = *i* ∧ *c* = *l then 1 else 0*)
⟨*proof*⟩

lemma *Tr-cyclic*:
assumes *AB*: *dim-col A* = *dim-row B*
and *BA*: *dim-row A* = *dim-col B*
shows *Tr* (*A* ∗ *B*) = *Tr* (*B* ∗ *A*)
⟨*proof*⟩

lemma *Tr-unitary-conjugation*:
assumes *Uu*: *unitary U*
and *Arow*: *dim-row A* = *dim-row U*
and *Acol*: *dim-col A* = *dim-row U*
shows *Tr* ((*U*†) ∗ *A* ∗ *U*) = *Tr*(*A*)
⟨*proof*⟩

lemma *hs-inner-prod-as-trace*:
assumes *rows*: *dim-row A* = *dim-row B*
and *cols*: *dim-col A* = *dim-col B*
shows ⟨⟨ *A* | *B* ⟩⟩ = *Tr* ((*A*†) ∗ *B*)
⟨*proof*⟩

corollary *hs-inner-prod-self-as-trace*:
shows ⟨⟨ *A* | *A* ⟩⟩ = *Tr* ((*A*†) ∗ *A*)
⟨*proof*⟩

5.3 Orthogonal projections and Parseval's identity

Definition (notes) (Orthogonal projections)**.** Let $V \subseteq \mathbb{C}^n$ be a subspace with orthonormal basis $\{v_1, \ldots, v_k\}$ and put $P = \sum_{i=1}^{k} |v_i\rangle\langle v_i|$. Then $Px = \sum_i \langle v_i|x\rangle v_i \in V$ and $x - Px \perp V$, so every $x \in \mathbb{C}^n$ is uniquely $x = v + w$ with $v \in V$ and $w \in V^{\perp}$. Consequently P does not depend on the orthonormal basis chosen for V, $Pv = v$ for $v \in V$, $P^2 = P$ and $P = P^*$: P is *the* orthogonal projection onto V.

Theorem (notes) (Parseval)**.** If $\{v_1, \ldots, v_k\}$ is an orthonormal basis of V and $v \in V$, then $\|v\|^2 = \sum_{i=1}^{k} |\langle v_i|v\rangle|^2$. For $V = \mathbb{C}^n$ this holds for every v.

Theorem (notes). If $P \in M_n$ satisfies $P^2 = P$ and $P = P^*$, and $V = \text{range}(P)$, then P is the orthogonal projection onto V.

Formally, an orthonormal list *vs* of vectors in $\mathbb{C}^n$ is *orthonormal-list n vs*, the matrix with columns $v_1, \ldots, v_k$ is *mat-of-cols n vs*, and $P = \sum_i |v_i\rangle\langle v_i| = VV^\dagger$ is *outer-sum n vs*. The results below are the projection formula, $V^\dagger V = I_k$, the completeness relation $VV^\dagger = I_n$ for a full basis, $P = P^\dagger$, $P^2 = P$, Parseval, the orthogonality of the residual, and the uniqueness of the decomposition. Independence of P from the basis is *outer-sum-basis-independent* in theory Day02_11_Supplement; the characterisation of Hermitian idempotents as orthogonal projections is *projection-annihilates-orthogonal* in theory Quantum_Computing11.

definition *orthonormal-list* :: *nat* ⇒ *complex Matrix.vec list* ⇒ *bool* **where**
orthonormal-list n vs ≡
(∀ *v* ∈ *set vs*. *dim-vec v* = *n*) ∧
(∀ *i* < *length vs*. ∀ *j* < *length vs*.
⟨*vs* ! *i* | *vs* ! *j*⟩ = (*if i* = *j then 1 else 0*))

lemma *orthonormal-listD-dim*:
assumes *orthonormal-list n vs* **and** *i* < *length vs*
shows *dim-vec* (*vs* ! *i*) = *n*
⟨*proof*⟩

lemma *orthonormal-listD-inner*:
assumes *orthonormal-list n vs* **and** *i* < *length vs* **and** *j* < *length vs*
shows ⟨*vs* ! *i* | *vs* ! *j*⟩ = (*if i* = *j then 1 else 0*)
⟨*proof*⟩

lemma *orthonormal-listD-carrier*:
assumes *orthonormal-list n vs*
shows *set vs* ⊆ *carrier-vec n*
⟨*proof*⟩

definition *outer-sum* :: *nat* ⇒ *complex Matrix.vec list* ⇒ *complex Matrix.mat* **where**
outer-sum n vs = *mat-of-cols n vs* ∗ ((*mat-of-cols n vs*)†)

lemma *outer-sum-dim-row* [*simp*]: *dim-row* (*outer-sum n vs*) = *n*
⟨*proof*⟩

lemma *outer-sum-dim-col* [*simp*]: *dim-col* (*outer-sum n vs*) = *n*
⟨*proof*⟩

lemma *outer-sum-hermitian*:
shows (*outer-sum n vs*)† = *outer-sum n vs*
⟨*proof*⟩

lemma *cols-mat-dagger-left*:
assumes *orth*: *orthonormal-list n vs*
shows (*mat-of-cols n vs*)† ∗ (*mat-of-cols n vs*) = 1_m (*length vs*)
⟨*proof*⟩

lemma *cols-mat-dagger-right*:
assumes *orth*: *orthonormal-list n vs*

and *full*: *length vs* = *n*
shows (*mat-of-cols n vs*) ∗ ((*mat-of-cols n vs*)†) = 1_m *n*
⟨*proof*⟩

lemma *completeness-relation*:
assumes *orth*: *orthonormal-list n vs*
and *full*: *length vs* = *n*
and *i-lt*: *i* < *n* **and** *j-lt*: *j* < *n*
shows (∑ *l*<*n*. (*vs* ! *l*) $ *i* ∗ *cnj* ((*vs* ! *l*) $ *j*)) = (*if i* = *j then 1 else 0*)
⟨*proof*⟩

lemma *outer-sum-index*:
assumes *orth*: *orthonormal-list n vs*
and *i-lt*: *i* < *n* **and** *j-lt*: *j* < *n*
shows (*outer-sum n vs*) $$ (*i*,*j*) =
(∑ *l* < *length vs*. (*vs* ! *l*) $ *i* ∗ *cnj* ((*vs* ! *l*) $ *j*))
⟨*proof*⟩

lemma *outer-sum-apply*:
assumes *orth*: *orthonormal-list n vs*
and *xdim*: *dim-vec x* = *n*
and *i-lt*: *i* < *n*
shows (*outer-sum n vs* ∗$_v$ *x*) $ *i* = (∑ *l* < *length vs*. ⟨*vs* ! *l* | *x*⟩ ∗ (*vs* ! *l*) $ *i*)
⟨*proof*⟩

lemma *outer-sum-idempotent*:
assumes *orth*: *orthonormal-list n vs*
shows *outer-sum n vs* ∗ *outer-sum n vs* = *outer-sum n vs*
⟨*proof*⟩

lemma *parseval*:
assumes *orth*: *orthonormal-list n vs*
and *full*: *length vs* = *n*
and *vdim*: *dim-vec v* = *n*
and *i-lt*: *i* < *n*
shows *v* $ *i* = (∑ *l*<*n*. ⟨*vs* ! *l* | *v*⟩ ∗ (*vs* ! *l*) $ *i*)
⟨*proof*⟩

corollary *parseval-norm*:
assumes *orth*: *orthonormal-list n vs*
and *full*: *length vs* = *n*
and *vdim*: *dim-vec v* = *n*
shows $\|v\|^2$ = (∑ *l*<*n*. (*cmod* (⟨*vs* ! *l* | *v*⟩))^2)
⟨*proof*⟩

The residual $x - Px$ is orthogonal to every v_l, because $\langle v_l | Px\rangle = \langle v_l | x\rangle$; hence the orthogonal decomposition $x = Px + (x - Px)$.

lemma *inner-prod-outer-sum*:
  assumes *orth*: *orthonormal-list n vs*
    and *xdim*: *dim-vec x = n*
    and *l-lt*: *l < length vs*
  shows ⟨*vs ! l* | *outer-sum n vs ∗v x*⟩ = ⟨*vs ! l* | *x*⟩
⟨*proof*⟩

lemma *residual-orthogonal*:
  assumes *orth*: *orthonormal-list n vs*
    and *xdim*: *dim-vec x = n*
    and *l-lt*: *l < length vs*
  shows ⟨*vs ! l* | *x − outer-sum n vs ∗v x*⟩ = *0*
⟨*proof*⟩

corollary *orthogonal-decomposition*:
  assumes *xdim*: *dim-vec x = n*
  shows *x = outer-sum n vs ∗v x* + (*x − outer-sum n vs ∗v x*)
⟨*proof*⟩

Uniqueness: if $x = v + w$ with $Pv = v$ and $w \perp v_l$ for all l, then $v = Px$ and $w = x - Px$.

lemma *outer-sum-orthogonal-zero*:
  assumes *orth*: *orthonormal-list n vs*
    and *wdim*: *dim-vec w = n*
    and *ortho*: ∀ *l < length vs*. ⟨*vs ! l* | *w*⟩ = *0*
  shows *outer-sum n vs ∗v w = 0v n*
⟨*proof*⟩

lemma *outer-sum-add*:
  assumes *vdim*: *dim-vec v = n* **and** *wdim*: *dim-vec w = n*
  shows *outer-sum n vs ∗v* (*v* + *w*) =
      outer-sum n vs ∗v v + *outer-sum n vs ∗v w*
⟨*proof*⟩

lemma *orthogonal-decomposition-unique*:
  assumes *orth*: *orthonormal-list n vs*
    and *xdim*: *dim-vec x = n*
    and *vdim*: *dim-vec v = n*
    and *wdim*: *dim-vec w = n*
    and *decomp*: *x = v + w*
    and *v-fixed*: *outer-sum n vs ∗v v = v*
    and *w-ortho*: ∀ *l < length vs*. ⟨*vs ! l* | *w*⟩ = *0*
  shows *v = outer-sum n vs ∗v x* **and** *w = x − outer-sum n vs ∗v x*
⟨*proof*⟩

6 Unitary algebra

Library-level consequences of unitarity ($U^\dagger U = I$ and $UU^\dagger = I$ in the library's definition) used throughout; Paulsen's Day 3 characterisations of unitaries are in theory Quantum_Computing_3_10.

lemma *unitary-square-dim*:
assumes *uA*: *unitary A*
shows *dim-row A* = *dim-col A*
⟨*proof*⟩

lemma *unitary-inv-l-dimcol*:
assumes *uA*: *unitary A*
shows $(A^\dagger) * A = 1_m$ (*dim-col A*)
⟨*proof*⟩

lemma *unitary-inv-r-dimrow*:
assumes *uA*: *unitary A*
shows $A * (A^\dagger) = 1_m$ (*dim-row A*)
⟨*proof*⟩

lemma *unitary-inv-l-dimrow*:
assumes *uA*: *unitary A*
shows $(A^\dagger) * A = 1_m$ (*dim-row A*)
⟨*proof*⟩

lemma *unitary-inv-r-dimcol*:
assumes *uA*: *unitary A*
shows $A * (A^\dagger) = 1_m$ (*dim-col A*)
⟨*proof*⟩

lemma *unitary-carrier-square-col*:
assumes *uA*: *unitary A*
shows *A* ∈ *carrier-mat* (*dim-col A*) (*dim-col A*)
⟨*proof*⟩

lemma *unitary-carrier-square-row*:
assumes *uA*: *unitary A*
shows *A* ∈ *carrier-mat* (*dim-row A*) (*dim-row A*)
⟨*proof*⟩

lemma *unitary-dagger*:
assumes *uA*: *unitary A*
shows *unitary* $(A^\dagger)$
⟨*proof*⟩

lemma *unitary-mult*:
assumes *UA*: *unitary A*
and *UB*: *unitary B*
and *dimA*: *A* ∈ *carrier-mat n n*
and *dimB*: *B* ∈ *carrier-mat n n*
shows *unitary* (*A* ∗ *B*)
⟨*proof*⟩

lemma *unitary-mult2*:
assumes *UA*: *unitary A*
and *UB*: *unitary B*
and *dims*: *dim-row A* = *dim-row B*
shows *unitary* (*A* ∗ *B*)
⟨*proof*⟩

7 Tensor product infrastructure

Kronecker products of matrices, $A \bigotimes B$: the tensor of identities, the adjoint $(R \otimes T)^* = R^* \otimes T^*$, the mixed-product property $(R_1 \otimes T_1)(R_2 \otimes T_2) = R_1 R_2 \otimes T_1 T_2$ (both from Day 9), and unitarity of tensors of unitaries.

7.1 Tensor of Identities

lemma *tensor-mat-id*:
Tensor.tensor-mat $(1_m\ n)\ (1_m\ m) = 1_m\ (n * m)$
⟨*proof*⟩

7.2 Tensor Adjoint

lemma *tensor-mat-adjoint*:
$(Tensor.tensor\text{-}mat\ A\ B)^\dagger = Tensor.tensor\text{-}mat\ (A^\dagger)\ (B^\dagger)$
⟨*proof*⟩

7.3 Mixed-Product Property

lemma *tensor-mat-mult*:
assumes *dimAC*: *dim-col A* = *dim-row C*
and *dimBD*: *dim-col B* = *dim-row D*
shows *Tensor.tensor-mat A B* ∗ *Tensor.tensor-mat C D* =
Tensor.tensor-mat $(A * C)\ (B * D)$
⟨*proof*⟩

7.4 Corollaries of the Mixed-Product Property

lemma *tensor-adjoint-mult*:
$(Tensor.tensor\text{-}mat\ (A^\dagger)\ (B^\dagger)) * (Tensor.tensor\text{-}mat\ A\ B) =$
$Tensor.tensor\text{-}mat\ ((A^\dagger) * A)\ ((B^\dagger) * B)$
⟨*proof*⟩

lemma *tensor-mult-adjoint*:
$(Tensor.tensor\text{-}mat\ A\ B) * (Tensor.tensor\text{-}mat\ (A^\dagger)\ (B^\dagger)) =$
$Tensor.tensor\text{-}mat\ (A * (A^\dagger))\ (B * (B^\dagger))$
⟨*proof*⟩

7.5 Tensor Unitarity

lemma *tensor-mat-unitary-carrier*:
assumes *dimA*: $A \in carrier\text{-}mat\ n\ n$
and *dimB*: $B \in carrier\text{-}mat\ m\ m$
and *uA*: *unitary A*
and *uB*: *unitary B*
shows *unitary* (*Tensor.tensor-mat A B*)
⟨*proof*⟩

lemma *tensor-mat-unitary*:
assumes *uA*: *unitary A*
and *uB*: *unitary B*
shows *unitary* (*Tensor.tensor-mat A B*)
⟨*proof*⟩

lemma *tensor-mat-unitary-one-left* [*simp*]:
assumes *uA*: *unitary A*
shows *unitary* (*Tensor.tensor-mat* $(1_m\ n)$ *A*)

⟨*proof*⟩

lemma *tensor-mat-unitary-one-right* [*simp*]:
assumes *uA*: *unitary A*
shows *unitary* (*Tensor.tensor-mat A* (*1*$_m$ *n*))
⟨*proof*⟩

end

theory *Quantum-Computing-3-10*
imports *Basic-Math Jordan-Normal-Form.Schur-Decomposition*
begin

8 Day 3 (31 August 2011): unitary matrices, Householder unitaries, Schur's theorem

8.1 Unitary matrices

Definition (notes). $U \in M_n$ is *unitary* if $U^*U = I$.

Theorem (notes). For $U \in M_n$ the following are equivalent: (a) U is unitary; (b) U is invertible and $U^{-1} = U^*$; (c) $UU^* = I$; (d) U^* is unitary; (e) the columns of U are orthonormal; (f) the rows of U are orthonormal; (g) $\|Ux\| = \|x\|$ for all x (isometry); (h) $\langle Ux|Uy\rangle = \langle x|y\rangle$ for all x, y.

The library predicate *unitary U* bundles (a) and (c). The lemmas below give (a)⇒(b), (d), (e), (f), (g), (h); (e)⇒(a) is *orthonormal-mat-unitary*, (g)⇒(a) and (h)⇒(a) are *unitary-of-isometry* and *unitary-of-inner-prod-preserving* (proved on Day 4, after the polarisation lemma); (d) is self-dual and (f) follows from (e) for U^*.

lemma *unitary-left-right*:
assumes *unitary U*
shows $U^\dagger * U = 1_m$ (*dim-col U*) **and** $U * (U^\dagger) = 1_m$ (*dim-row U*)
⟨*proof*⟩

(b) U is invertible with inverse U^*:

lemma *unitary-invertible*:
assumes *unitary U*
shows *invertible-mat U*
⟨*proof*⟩

(d) U^* is unitary:

lemma *unitary-dagger-is-unitary*:
assumes *unitary U*
shows *unitary* ($U^\dagger$)
⟨*proof*⟩

(e), (f) the columns and the rows of U are orthonormal:

lemma *unitary-cols-orthonormal*:
assumes *U*: *unitary U*
and *i*: *i* < *dim-col U*
and *j*: *j* < *dim-col U*
shows ⟨*Matrix.col U i* | *Matrix.col U j*⟩ = (*if i* = *j then 1 else 0* :: *complex*)
⟨*proof*⟩

lemma *unitary-rows-orthonormal*:
assumes *U*: *unitary U*
and *i*: *i* < *dim-row U*
and *j*: *j* < *dim-row U*
shows ⟨*Matrix.row U i* | *Matrix.row U j*⟩ = (*if i* = *j then 1 else 0* :: *complex*)
⟨*proof*⟩

(g), (h) U is an isometry and preserves inner products:

lemma *unitary-isometry*:
assumes *unitary U* **and** *dim-vec v* = *dim-col U*
shows ‖*U* ∗*v v*‖ = ‖*v*‖
⟨*proof*⟩

lemma *unitary-preserves-inner-prod*:
assumes *U*: *unitary U*
and *u*: *dim-vec u* = *dim-col U*
and *v*: *dim-vec v* = *dim-col U*
shows ⟨*U* ∗*v u* | *U* ∗*v v*⟩ = ⟨*u* | *v*⟩
⟨*proof*⟩

8.2 Householder unitaries

Definition (notes). For $w \in \mathbb{C}^n$ with $\|w\| = 1$ let $P_w = |w\rangle\langle w|$, the projection onto $\mathrm{span}\{w\}$, and $U_w = I - 2P_w$. Then $U_w^* U_w = (I - 2P_w)^2 = I - 4P_w + 4P_w^2 = I$, so U_w is unitary: the *Householder unitary* of w.

Remark. Geometrically U_w is the reflection through the hyperplane $\{w\}^\perp$.

Formally *householder w* is defined entrywise as $(\delta_{ij} - 2w_i\overline{w_j})$ and shown to equal $I - 2|w\rangle\langle w|$.

abbreviation *outer-prod* :: *complex Matrix.vec* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.mat*
(**infixr** ⊘ *75*)
where *v* ⊘ *w* ≡ |*v*⟩ ∗ *bra-vec w*

lemma *bra-vec-index-0*:
assumes *j*: *j* < *dim-vec w*
shows *bra-vec w* $$ (*0*,*j*) = *cnj* (*w* $ *j*)
⟨*proof*⟩

lemma *outer-prod-index*:
fixes *v w* :: *complex Matrix.vec*
assumes *i*: *i* < *dim-vec v*
and *j*: *j* < *dim-vec w*
shows (*v* ⊘ *w*) $$ (*i*,*j*) = (*v* $ *i*) ∗ *cnj* (*w* $ *j*)
⟨*proof*⟩

lemma *outer-prod-mult-vec-index*:
fixes *v w x* :: *complex Matrix.vec*
assumes *xdim*: *dim-vec x* = *dim-vec w*
and *i*: *i* < *dim-vec v*
shows ((*v* ⊘ *w*) ∗*v x*) $ *i* = (*v* $ *i*) ∗ ⟨*w* | *x*⟩
⟨*proof*⟩

lemma *outer-prod-mult-vec*:
fixes *v w x* :: *complex Matrix.vec*
assumes *xdim*: *dim-vec x* = *dim-vec w*

shows $(v \oslash w) *_v x = \langle w \mid x \rangle \cdot_v v$
⟨*proof*⟩

lemma *smult-mat-mult-vec*:
fixes A :: *complex Matrix.mat* **and** x :: *complex Matrix.vec*
assumes A: $A \in$ *carrier-mat nr nc*
and x: $x \in$ *carrier-vec nc*
shows $(a \cdot_m A) *_v x = a \cdot_v (A *_v x)$
⟨*proof*⟩

lemma *cpx-vec-length-zero-iff*:
fixes v :: *complex Matrix.vec*
shows $\|v\| = 0 \longleftrightarrow v = 0_v$ (*dim-vec v*)
⟨*proof*⟩

$\langle v|v\rangle = 0$ iff $v = 0$.

lemma *inner-prod-self-eq-zero-iff*:
fixes v :: *complex Matrix.vec*
shows $\langle v \mid v \rangle = 0 \longleftrightarrow v = 0_v$ (*dim-vec v*)
⟨*proof*⟩

lemma *cpx-vec-length-smult*:
fixes a :: *complex* **and** v :: *complex Matrix.vec*
shows $\|a \cdot_v v\| =$ *cmod a* $* \|v\|$
⟨*proof*⟩

lemma *inner-prod-right-smult'*:
fixes u v :: *complex Matrix.vec*
shows $\langle u \mid a \cdot_v v \rangle = a * \langle u \mid v \rangle$
⟨*proof*⟩

lemma *inner-prod-left-smult'*:
fixes u v :: *complex Matrix.vec*
assumes *uv*: *dim-vec u* = *dim-vec v*
shows $\langle a \cdot_v u \mid v \rangle =$ *cnj a* $* \langle u \mid v \rangle$
⟨*proof*⟩

lemma *inner-prod-right-add'*:
fixes u v w :: *complex Matrix.vec*
assumes *vw*: *dim-vec v* = *dim-vec w*
shows $\langle u \mid v + w \rangle = \langle u \mid v \rangle + \langle u \mid w \rangle$
⟨*proof*⟩

lemma *inner-prod-right-minus'*:
fixes u v w :: *complex Matrix.vec*
assumes *vw*: *dim-vec v* = *dim-vec w*
shows $\langle u \mid v - w \rangle = \langle u \mid v \rangle - \langle u \mid w \rangle$
⟨*proof*⟩

lemma *inner-prod-left-minus'*:
fixes u v w :: *complex Matrix.vec*
assumes *uv*: *dim-vec u* = *dim-vec v*
and *uw*: *dim-vec u* = *dim-vec w*
shows $\langle u - v \mid w \rangle = \langle u \mid w \rangle - \langle v \mid w \rangle$
⟨*proof*⟩

lemma *dagger-of-smult*:
fixes γ :: *complex*
and U :: *complex Matrix.mat*
shows $(\gamma \cdot_m U)^\dagger = (cnj\ \gamma) \cdot_m (U^\dagger)$
⟨*proof*⟩

The adjoint of an $m \times n$ matrix is $n \times m$.

lemma *dagger-carrier*:
assumes $M \in$ *carrier-mat m n*
shows $(M^\dagger) \in$ *carrier-mat n m*
⟨*proof*⟩

definition *householder* :: *complex Matrix.vec* ⇒ *complex Matrix.mat* **where**
householder w ≡
Matrix.mat (*dim-vec w*) (*dim-vec w*)
(λ(*i*,*j*). (*if i* = *j then 1 else 0*) − *2* ∗ (*w* $ *i*) ∗ *cnj* (*w* $ *j*))

notation *householder* (U_H)

lemma *householder-index*:
assumes *i* < *dim-vec w* **and** *j* < *dim-vec w*
shows (*householder w*) $$ (*i*,*j*) =
(*if i* = *j then 1 else 0*) − *2* ∗ (*w* $ *i*) ∗ *cnj* (*w* $ *j*)
⟨*proof*⟩

lemma *householder-outer-prod*:
householder w = 1_m (*dim-vec w*) − *2* $\cdot_m$ (*w* ⊘ *w*)
⟨*proof*⟩

lemma *householder-hermitian*:
shows $(householder\ w)^\dagger = householder\ w$
⟨*proof*⟩

lemma *householder-unitary*:
assumes *norm-w*: ‖*w*‖ = (*1* :: *real*)
shows *unitary* (*householder w*)
⟨*proof*⟩

lemma *householder-mult-vec-index*:
fixes *w x* :: *complex Matrix.vec*
assumes *xdim*: *dim-vec x* = *dim-vec w*
and *i*: *i* < *dim-vec w*
shows (*householder w* ∗$_v$ *x*) $ *i* =
x $ *i* − *2* ∗ (*w* $ *i*) ∗ ⟨*w* | *x*⟩
⟨*proof*⟩

lemma *householder-mult-vec*:
fixes *w x* :: *complex Matrix.vec*
assumes *xdim*: *dim-vec x* = *dim-vec w*
shows *householder w* ∗$_v$ *x* =
x − (*2* ∗ ⟨*w* | *x*⟩) $\cdot_v$ *w*
⟨*proof*⟩

Lemma (notes) (Schur)**.** Given $x, y \in \mathbb{C}^n$ with $\|x\| = \|y\|$ there are $w \in \mathbb{C}^n$ with $\|w\| = 1$ and

θ such that $U_w x = e^{i\theta} y$.

The formal statement writes $e^{i\theta}$ as *Complex* (*cos* ϑ) (*sin* ϑ); the case $x = 0$ is treated separately.

lemma *schur-lemma-zero-case*:
  fixes *x y* :: *complex Matrix.vec*
  assumes *n-pos*: $n > 0$
    and *xdim*: *dim-vec x* $= n$
    and *ydim*: *dim-vec y* $= n$
    and *norm-eq*: $\|x\| = \|y\|$
    and *x-zero-norm*: $\|x\| = 0$
  shows $\exists\, w$ :: *complex Matrix.vec*. $\exists\, \vartheta$ :: *real*.
      dim-vec w $= n \wedge \|w\| = 1 \wedge$
      householder w $*_v$ *x* $=$ *Complex* (*cos* ϑ) (*sin* ϑ) $\cdot_v$ *y*
⟨*proof*⟩

thm *of-real-divide*[**where** *x* $=$ *cmod* ($\langle y \mid x\rangle$) ^ 2 **and** *y* $=$ *cmod* ($\langle y \mid x\rangle$), *symmetric*]

lemma *schur-lemma*:
  fixes *x y* :: *complex Matrix.vec*
  assumes *n-pos*: $n > 0$
    and *xdim*: *dim-vec x* $= n$
    and *ydim*: *dim-vec y* $= n$
    and *norm-eq*: $\|x\| = \|y\|$
  shows $\exists\, w$ :: *complex Matrix.vec*. $\exists\, \vartheta$ :: *real*.
      dim-vec w $= n \wedge \|w\| = 1 \wedge$
      householder w $*_v$ *x* $=$ *Complex* (*cos* ϑ) (*sin* ϑ) $\cdot_v$ *y*
⟨*proof*⟩

Theorem (notes) (Schur)**.** For every $A \in M_n$ there is a unitary U such that U^*AU is upper triangular.

The notes prove this by induction, peeling off one eigenvector with the Householder unitary of Schur's lemma and recursing on the $(n-1) \times (n-1)$ block. The formal proof follows the same route: *schur-triangular-similar* is the similarity version available from the Jordan-Normal-Form library, and the block-matrix lemmas that follow assemble the unitary step by step.

lemma *schur-triangular-similar*:
  fixes *A* :: *complex Matrix.mat*
  assumes *sq*: *A* ∈ *carrier-mat n n*
  shows $\exists$ *T* :: *complex Matrix.mat*.
      T ∈ *carrier-mat n n* $\wedge$
      upper-triangular T $\wedge$
      similar-mat A T
⟨*proof*⟩

lemma *cpx-mat-cnj-four-block*:
  assumes *A*: *A* ∈ *carrier-mat nr1 nc1*
    and *B*: *B* ∈ *carrier-mat nr1 nc2*
    and *C*: *C* ∈ *carrier-mat nr2 nc1*
    and *D*: *D* ∈ *carrier-mat nr2 nc2*
  shows (*four-block-mat A B C D*)* $=$
      four-block-mat (*A**) (*B**) (*C**) (*D**)
⟨*proof*⟩

lemma *dagger-four-block-mat-complex*:
  assumes *A*: *A* ∈ *carrier-mat nr1 nc1*

and B: $B \in$ *carrier-mat nr1 nc2*
and C: $C \in$ *carrier-mat nr2 nc1*
and D: $D \in$ *carrier-mat nr2 nc2*
shows $(\textit{four-block-mat}\ A\ B\ C\ D)^\dagger =$
$\textit{four-block-mat}\ (A^\dagger)\ (C^\dagger)\ (B^\dagger)\ (D^\dagger)$
⟨*proof*⟩

lemma *dagger-zero-mat*[*simp*]:
$(0_m\ m\ n :: \textit{complex Matrix.mat})^\dagger = 0_m\ n\ m$
⟨*proof*⟩

lemma *block-diag-mult*:
fixes $A\ B\ D\ E ::$ *complex Matrix.mat*
assumes A: $A \in$ *carrier-mat n n*
and B: $B \in$ *carrier-mat n n*
and D: $D \in$ *carrier-mat m m*
and E: $E \in$ *carrier-mat m m*
shows $\textit{four-block-mat}\ A\ (0_m\ n\ m)\ (0_m\ m\ n)\ D *$
$\textit{four-block-mat}\ B\ (0_m\ n\ m)\ (0_m\ m\ n)\ E =$
$\textit{four-block-mat}\ (A * B)\ (0_m\ n\ m)\ (0_m\ m\ n)\ (D * E)$
⟨*proof*⟩

lemma *block-diag-unitary*:
fixes $U\ V ::$ *complex Matrix.mat*
assumes U: $U \in$ *carrier-mat n n*
and V: $V \in$ *carrier-mat m m*
and uU: *unitary U*
and uV: *unitary V*
shows $\textit{unitary}\ (\textit{four-block-mat}\ U\ (0_m\ n\ m)\ (0_m\ m\ n)\ V)$
⟨*proof*⟩

lemma *complex-matrix-has-eigenvector*:
fixes $A ::$ *complex Matrix.mat*
assumes *sq*: $A \in$ *carrier-mat n n*
and *n-pos*: $n > 0$
shows $\exists\, e\ v.$ *eigenvector A v e*
⟨*proof*⟩

lemma *orthonormal-mat-unitary*:
assumes *orth*: *orthonormal-list n vs*
and *full*: *length vs = n*
shows $\textit{unitary}\ (\textit{mat-of-cols}\ n\ vs)$
⟨*proof*⟩

lemma *eigenvec-normalize*:
fixes $A ::$ *complex Matrix.mat* **and** $v ::$ *complex Matrix.vec*
assumes *sq*: $A \in$ *carrier-mat n n*
and *vdim*: $v \in$ *carrier-vec n*
and *vne*: $v \neq 0_v\ n$
and *ev*: $A *_v\ v = e \cdot_v v$
defines $v_1 \equiv (1\ /\ \|v\|) \cdot_v v$
shows $A *_v\ v_1 = e \cdot_v v_1$ **and** $\|v_1\| = 1$
⟨*proof*⟩

definition *vec-normalize* :: *complex Matrix.vec* ⇒ *complex Matrix.vec* **where**
vec-normalize v = (*if v* = 0_v (*dim-vec v*) *then v else* (1 / ‖*v*‖) ·$_v$ *v*)

lemma *normalized-vec-norm*:

assumes *vdim*: $v \in$ *carrier-vec n*
and *vne*: $v \neq 0_v\ n$
shows $\|$*vec-normalize v*$\| = 1$
⟨*proof*⟩

lemma *four-block-mat-mult-left-block-diag*:
fixes *U* :: *complex Matrix.mat*
and *A11* :: *complex Matrix.mat*
and *A12* :: *complex Matrix.mat*
and *A21* :: *complex Matrix.mat*
and *A22* :: *complex Matrix.mat*
assumes *U*: *U* ∈ *carrier-mat n n*
and *A11*: *A11* ∈ *carrier-mat 1 1*
and *A12*: *A12* ∈ *carrier-mat 1 n*
and *A21*: *A21* ∈ *carrier-mat n 1*
and *A22*: *A22* ∈ *carrier-mat n n*
shows
(*four-block-mat* (1_m *1* :: *complex Matrix.mat*)
(0_m *1 n* :: *complex Matrix.mat*)
(0_m *n 1* :: *complex Matrix.mat*)
U)
∗
(*four-block-mat A11 A12 A21 A22*)
=
four-block-mat A11 A12 (*U* ∗ *A21*) (*U* ∗ *A22*)
⟨*proof*⟩

lemma *cscalar-prod-is-inner-prod-swap*:
fixes *u v* :: *complex Matrix.vec*
assumes *u* ∈ *carrier-vec n* **and** *v* ∈ *carrier-vec n*
shows *u* ·*c v* = ⟨*v*|*u*⟩
⟨*proof*⟩

lemma *vec-normalize-dim*[*simp*]:
dim-vec (*vec-normalize v*) = *dim-vec v*
⟨*proof*⟩

lemma *vec-normalize-carrier*[*simp*]:
assumes *v* ∈ *carrier-vec n*
shows *vec-normalize v* ∈ *carrier-vec n*
⟨*proof*⟩

lemma *vec-normalize-unit*:
assumes *vdim*: *v* ∈ *carrier-vec n* **and** *vnorm*: $\|v\| = 1$
shows *vec-normalize v* = *v*
⟨*proof*⟩

Any unit vector extends to an orthonormal basis (needed for the induction).

lemma *unit-vec-extends-to-onb*:
fixes *v* :: *complex Matrix.vec*
assumes *vdim*: *v* ∈ *carrier-vec n*
and *vnorm*: $\|v\| = 1$
shows ∃ *vs*. *orthonormal-list n vs* ∧ *length vs* = *n* ∧ *vs* ! *0* = *v*
⟨*proof*⟩

lemma *schur-induct*:
∀ *A* :: *complex Matrix.mat*.
A ∈ *carrier-mat n n* ⟶

```
    (∃ U :: complex Matrix.mat. ∃ T :: complex Matrix.mat.
      U ∈ carrier-mat n n ∧
      T ∈ carrier-mat n n ∧
      unitary U ∧
      upper-triangular T ∧
      (((U†) * A) * U) = T)
⟨proof⟩

theorem schur-theorem:
  fixes A :: complex Matrix.mat
  assumes sq: A ∈ carrier-mat n n
  shows ∃ U :: complex Matrix.mat. ∃ T :: complex Matrix.mat.
        U ∈ carrier-mat n n ∧
        T ∈ carrier-mat n n ∧
        unitary U ∧
        upper-triangular T ∧
        (((U†) * A) * U) = T
  ⟨proof⟩
```

Remark. The characteristic polynomial of A is $p_A(t) = \det(tI - A)$, a polynomial of degree n.

Corollary (notes). $\mathrm{Tr}(A) = \lambda_1 + \cdots + \lambda_n$, where the λ_j are the eigenvalues of A.

Formally the eigenvalues are listed as the roots of the characteristic polynomial, $p_A(t) = \prod_j (t - \lambda_j)$, read off the diagonal of the Schur form.

```
corollary trace-eq-sum-eigenvalues:
  fixes A :: complex Matrix.mat
  assumes sq: A ∈ carrier-mat n n
  obtains eigs :: complex list
  where length eigs = n
    and Tr A = sum-list eigs
    and char-poly A = (∏ e ← eigs. [:−e, 1:])
⟨proof⟩
```

9 Day 4 (2 September 2011): Hermitian matrices; positive semidefinite and positive definite matrices

9.1 Hermitian matrices

Definition (notes). $H \in M_n$ is *Hermitian* (self-adjoint) if $H = H^*$.

Theorem (notes). The following are equivalent: (a) H is Hermitian; (b) there is a unitary U with $U^*HU = D$ diagonal with real entries; (c) $\mathbb{C}^n$ has an orthonormal basis of eigenvectors of H with real eigenvalues; (d) $\langle x|Hx\rangle \in \mathbb{R}$ for all $x \in \mathbb{C}^n$.

Formally *hermitian H* is $H^\dagger = H$; (a)⇒(b) is the spectral theorem *hermitian-spectral-decomp* below, (a)⇔(d) is *hermitian-iff-real-quad-form*, and (c) is read off from (b): the columns of U are an orthonormal eigenbasis. The proof of (d)⇒(a) needs the following lemma, which the notes note fails over $\mathbb{R}$ (take $T = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$).

```
definition hermitian :: complex Matrix.mat ⇒ bool where
  hermitian A ≡ A† = A


lemma hermitian-square:
```

assumes *hermitian A*
shows *dim-row A = dim-col A*
⟨*proof*⟩

lemma *hermitian-index*:
assumes *hermitian A*
and *i < dim-row A* **and** *j < dim-col A*
shows *A $$ (i,j) = cnj (A $$ (j,i))*
⟨*proof*⟩

lemma *inner-T-double-sum*:
fixes *T :: complex Matrix.mat* **and** *u v :: complex Matrix.vec*
assumes *T-carr*: *T ∈ carrier-mat n n*
and *udim*: *dim-vec u = n* **and** *vdim*: *dim-vec v = n*
shows ⟨*u | T *$_v$ v*⟩ =
(∑ *k = 0..<n.* ∑ *l = 0..<n. cnj (u $ k) * T $$ (k, l) * v $ l*)
⟨*proof*⟩

lemma *inner-unit-T-unit*:
fixes *T :: complex Matrix.mat*
assumes *T-carr*: *T ∈ carrier-mat n n*
and *p*: *p < n* **and** *q*: *q < n*
shows ⟨*unit-vec n p | T *$_v$ unit-vec n q*⟩ = *T $$ (p, q)*
⟨*proof*⟩

lemma *sum-indicator-T-indicator*:
fixes *T :: complex Matrix.mat*
assumes *p*: *p < n* **and** *q*: *q < n*
shows (∑ *k = 0..<n.* ∑ *l = 0..<n.*
(*if k = p then (1 :: complex) else 0*) * *T $$ (k, l)* *
(*if l = q then (1 :: complex) else 0*))
= *T $$ (p, q)*
⟨*proof*⟩

lemma *polarization-helper*:
fixes *T :: complex Matrix.mat* **and** *a b c d :: complex*
assumes *sq*: *T ∈ carrier-mat n n*
and *i*: *i < n* **and** *j*: *j < n*
shows ⟨*a *$\cdot_v$* unit-vec n i + b *$\cdot_v$* unit-vec n j*
| *T *$*_v$* (c *$\cdot_v$* unit-vec n i + d *$\cdot_v$* unit-vec n j)*⟩
= *cnj a * c * T $$ (i, i) + cnj a * d * T $$ (i, j)*
+ *cnj b * c * T $$ (j, i) + cnj b * d * T $$ (j, j)*
⟨*proof*⟩

Lemma (notes). For $T \in M_n$, if $\langle x|Tx\rangle = 0$ for all $x \in \mathbb{C}^n$, then $T = 0$.

lemma *quadratic-form-zero*:
fixes *T :: complex Matrix.mat*
assumes *sq*: *T ∈ carrier-mat n n*
and *zero*: *∀ x :: complex Matrix.vec. dim-vec x = n ⟶ ⟨x | T *$_v$ x*⟩ = 0*
shows *T = 0$_m$ n n*
⟨*proof*⟩

lemma *mult-mat-vec-index*:
fixes *A :: 'a::{comm-ring-1} Matrix.mat*

and $x :: 'a\ Matrix.vec$
assumes A: $A \in carrier\text{-}mat\ m\ n$
and x: $x \in carrier\text{-}vec\ n$
and j: $j < m$
shows $(A *_v x)\ \$\ j = (\sum i = 0..<n.\ A\ \$\$\ (j, i) * x\ \$\ i)$
⟨*proof*⟩

The adjoint moves across the inner product, $\langle x|My\rangle = \langle M^\dagger x|y\rangle$: the defining property of $M^\dagger$.

lemma *inner-prod-adjoint-swap*:
fixes $M :: complex\ Matrix.mat$
and $x\ y :: complex\ Matrix.vec$
assumes M: $M \in carrier\text{-}mat\ n\ m$
and x: $dim\text{-}vec\ x = n$
and y: $dim\text{-}vec\ y = m$
shows $\langle x \mid M *_v y\rangle = \langle M^\dagger *_v x \mid y\rangle$
⟨*proof*⟩

Sums and real multiples of Hermitian matrices are Hermitian; the identity is Hermitian.

lemma *hermitian-plus*:
fixes $A\ B :: complex\ Matrix.mat$
assumes $Aherm$: $hermitian\ A$ **and** $Bherm$: $hermitian\ B$
and Ac: $A \in carrier\text{-}mat\ k\ k$ **and** Bc: $B \in carrier\text{-}mat\ k\ k$
shows $hermitian\ (A + B)$
⟨*proof*⟩

lemma *hermitian-identity*:
$hermitian\ (1_m\ k :: complex\ Matrix.mat)$
⟨*proof*⟩

lemma *hermitian-real-scale*:
fixes $A :: complex\ Matrix.mat$
assumes $hermitian\ A\ A \in carrier\text{-}mat\ k\ k$
shows $hermitian\ (complex\text{-}of\text{-}real\ d \cdot_m A)$
⟨*proof*⟩

The converses (g)⇒(a) and (h)⇒(a) of the Day 3 theorem on unitaries, postponed until the lemma $\langle x|Tx\rangle = 0\ \forall x \Rightarrow T = 0$ was available: a square matrix that preserves lengths, or inner products, is unitary (the library proves the length-preserving case in its own matrix notation as *length-preserving-is-unitary*).

lemma *unitary-of-isometry*:
fixes $U :: complex\ Matrix.mat$
assumes U: $U \in carrier\text{-}mat\ n\ n$
and iso: $\bigwedge v :: complex\ Matrix.vec.\ dim\text{-}vec\ v = n \Longrightarrow \|U *_v v\| = \|v\|$
shows $unitary\ U$
⟨*proof*⟩

lemma *unitary-of-inner-prod-preserving*:
fixes $U :: complex\ Matrix.mat$
assumes U: $U \in carrier\text{-}mat\ n\ n$
and $pres$: $\bigwedge u\ v :: complex\ Matrix.vec.\ dim\text{-}vec\ u = n \Longrightarrow dim\text{-}vec\ v = n \Longrightarrow$
$\langle U *_v u \mid U *_v v\rangle = \langle u \mid v\rangle$
shows $unitary\ U$
⟨*proof*⟩

lemma *hermitian-iff-real-quad-form*:
assumes sq: $A \in carrier\text{-}mat\ n\ n$
shows $hermitian\ A \longleftrightarrow (\forall x :: complex\ Matrix.vec.\ dim\text{-}vec\ x = n \longrightarrow \langle x \mid A *_v x\rangle \in \mathbb{R})$
⟨*proof*⟩

9.2 The spectral theorem for Hermitian matrices

(a)⇒(b) of the theorem above: by Schur, $U^*HU = T$ is upper triangular, and $T^* = (U^*HU)^* = T$ forces T to be diagonal with real entries. *diag-mat-of-real D* is the complex diagonal matrix with the real list D on its diagonal.

definition *diag-mat-of-real* :: *real list* ⇒ *complex Matrix.mat* **where**
diag-mat-of-real D =
Matrix.mat (*length D*) (*length D*)
(λ(*i*,*j*). *if i* = *j then complex-of-real* (*D* ! *i*) *else 0*)

lemma *dim-row-diag-mat-of-real*[*simp*]: *dim-row* (*diag-mat-of-real D*) = *length D*
⟨*proof*⟩

lemma *dim-col-diag-mat-of-real*[*simp*]: *dim-col* (*diag-mat-of-real D*) = *length D*
⟨*proof*⟩

lemma *diag-mat-of-real-carrier*[*simp*]:
diag-mat-of-real D ∈ *carrier-mat* (*length D*) (*length D*)
⟨*proof*⟩

lemma *diag-mat-of-real-index*[*simp*]:
assumes *i* < *length D* **and** *j* < *length D*
shows *diag-mat-of-real D* $$ (*i*, *j*) =
(*if i* = *j then complex-of-real* (*D* ! *i*) *else 0*)
⟨*proof*⟩

lemma *diag-mat-of-real-mult-vec-index*:
fixes *y* :: *complex Matrix.vec*
assumes *ydim*: *dim-vec y* = *length D*
and *i*: *i* < *length D*
shows (*diag-mat-of-real D* ∗$_v$ *y*) $ *i* = *complex-of-real* (*D* ! *i*) ∗ *y* $ *i*
⟨*proof*⟩

lemma *diag-mat-of-real-quad-Re-nonneg*:
fixes *y* :: *complex Matrix.vec*
assumes *ydim*: *dim-vec y* = *length D*
and *D-nonneg*: ⋀*i*. *i* < *length D* ⟹ 0 ≤ *D* ! *i*
shows 0 ≤ *Re* ⟨*y* | *diag-mat-of-real D* ∗$_v$ *y*⟩
⟨*proof*⟩

theorem *hermitian-spectral-decomp*:
fixes *H* :: *complex Matrix.mat*
assumes *sq*: *H* ∈ *carrier-mat n n*
and *herm*: *hermitian H*
shows ∃(*U* :: *complex Matrix.mat*) (*D* :: *real list*).
U ∈ *carrier-mat n n* ∧
unitary U ∧
length D = *n* ∧
($U^\dagger$) ∗ *H* ∗ *U* = *diag-mat-of-real D*
⟨*proof*⟩

corollary *hermitian-real-eigenvalues*:
fixes *H* :: *complex Matrix.mat*
assumes *sq*: *H* ∈ *carrier-mat n n*
and *herm*: *hermitian H*
and *eig*: *H* ∗$_v$ *v* = *lam* ·$_v$ *v*

and *v-nonzero*: $v \neq 0_v\ n$
and *vdim*: *dim-vec* $v = n$
shows *lam* $\in \mathbb{R}$
⟨*proof*⟩

9.3 Positive semidefinite and positive definite matrices

Definition (notes). $P \in M_n$ is *positive semidefinite*, $P \geq 0$, if $\langle x|Px\rangle \geq 0$ for all $x \in \mathbb{C}^n$, and *positive definite*, $P > 0$, if $\langle x|Px\rangle > 0$ for all $x \neq 0$.

By part (d) of the previous theorem, $P \geq 0$ implies $P = P^*$. The formal predicates *psd* and *pd* carry Hermiticity as an explicit conjunct together with the real-part inequality; by *hermitian-iff-real-quad-form* this is equivalent to the notes' definition.

Theorem (notes). $P \geq 0$ if and only if $P = P^*$ and all eigenvalues of P are non-negative.

Corollary (notes). $P > 0$ if and only if $P = P^*$ and all eigenvalues of P are strictly positive.

Theorem (notes). $P \geq 0$ if and only if $P = \sum_{i=1}^m |v_i\rangle\langle v_i|$ for some vectors $v_1, \ldots, v_m$.

Key: if $P = \sum_i |v_i\rangle\langle v_i|$ then $\langle x|Px\rangle = \sum_i |\langle v_i|x\rangle|^2$.

Theorem (notes). Let $P = \sum_{i=1}^m |v_i\rangle\langle v_i| \in M_n$. Then $P > 0$ if and only if $\mathrm{span}\{v_1, \ldots, v_m\} = \mathbb{C}^n$.

The notes' form of the definition is *psd-iff-quad-form-nonneg*; the corollary on positive definite matrices is *pd-iff-pos-eigenvalues* (proved after the square root); the last theorem is formalised with "the v_i span $\mathbb{C}^n$" phrased as "no nonzero x is orthogonal to every v_i".

definition *psd* :: *complex Matrix.mat* ⇒ *bool* **where**
psd P ≡ *hermitian P* ∧
(∀ *x* :: *complex Matrix.vec*. *dim-vec x* = *dim-row P* ⟶
Re (⟨*x* | *P* ∗*v x*⟩) ≥ *0*)

definition *pd* :: *complex Matrix.mat* ⇒ *bool* **where**
pd P ≡ *hermitian P* ∧
(∀ *x* :: *complex Matrix.vec*. *dim-vec x* = *dim-row P* ⟶
x ≠ 0_v (*dim-row P*) ⟶
Re (⟨*x* | *P* ∗*v x*⟩) > *0*)

notation *psd* (- ≥*M* *0*)
notation *pd* (- >*M* *0*)

lemma *psd-hermitian*: *psd P* ⟹ *hermitian P*
⟨*proof*⟩

The notes' definition, without the explicit Hermiticity conjunct: $P \geq 0$ iff every quadratic form $\langle x|Px\rangle$ is a non-negative real number.

lemma *psd-iff-quad-form-nonneg*:
fixes *P* :: *complex Matrix.mat*
assumes *sq*: *P* ∈ *carrier-mat n n*
shows *psd P* ⟷
(∀ *x* :: *complex Matrix.vec*. *dim-vec x* = *n* ⟶
⟨*x* | *P* ∗*v x*⟩ ∈ ℝ ∧ *0* ≤ *Re* (⟨*x* | *P* ∗*v x*⟩))
⟨*proof*⟩

lemma *psd-MdagM*:

assumes M: $M \in$ *carrier-mat n m*
shows *psd* $(M^\dagger * M)$
⟨*proof*⟩

$P \geq 0$ iff P is Hermitian with non-negative eigenvalues:

theorem *psd-iff-nonneg-eigenvalues*:
fixes P :: *complex Matrix.mat*
assumes *sq*: $P \in$ *carrier-mat n n*
shows *psd* $P \longleftrightarrow$
$($*hermitian* $P \wedge$
$(\forall$ *mu* :: *complex*. $\forall$ v :: *complex Matrix.vec*.
$v \neq 0_v$ $n \wedge$ *dim-vec* $v = n \wedge P *_v v = mu \cdot_v v \longrightarrow$ *Re mu* $\geq 0))$
⟨*proof*⟩

$P \geq 0$ iff $P = \sum_i |v_i\rangle\langle v_i|$. First the direction $\Leftarrow$:

lemma *outer-sum-psd*:
assumes *len*: *length vs* $= k$
and *vdim*: $\forall$ $v \in$ *set vs*. *dim-vec* $v = n$
shows *psd* (*outer-sum n vs*)
⟨*proof*⟩

lemma *hermitian-spectral-decomp-eq*:
assumes *Uc*: $U \in$ *carrier-mat n n*
and *Pcar*: $P \in$ *carrier-mat n n*
and *unit*: *unitary U*
and *spec*: $U^\dagger * P * U =$ *diag-mat-of-real D*
shows $P = U *$ *diag-mat-of-real D* $* (U^\dagger)$
⟨*proof*⟩

lemma *hermitian-spectral-decomp-right*:
assumes *Uc*: $U \in$ *carrier-mat n n*
and *Pcar*: $P \in$ *carrier-mat n n*
and *unit*: *unitary U*
and *spec*: $U^\dagger * P * U =$ *diag-mat-of-real D*
shows $P * U = U *$ *diag-mat-of-real D*
⟨*proof*⟩

The direction $\Rightarrow$, from the spectral theorem: $P = \sum_j \lambda_j |u_j\rangle\langle u_j| = \sum_j |\lambda_j^{1/2} u_j\rangle\langle\lambda_j^{1/2} u_j|$.

The spectral theorem for positive matrices: $P = U \operatorname{diag}(D)\, U^\dagger$ with $D \geq 0$.

lemma *psd-spectral-decomp-nonneg*:
fixes P :: *complex Matrix.mat*
assumes *sq*: $P \in$ *carrier-mat n n*
and *pos*: *psd P*
shows $\exists$ $(U$ :: *complex Matrix.mat*$)$ $(D$ :: *real list*$)$.
$U \in$ *carrier-mat n n* $\wedge$ *unitary U* $\wedge$ *length D* $= n \wedge$
$(\forall\, i < n.\ 0 \leq D\ !\ i) \wedge$
$P = U *$ *diag-mat-of-real D* $* (U^\dagger)$
⟨*proof*⟩

theorem *psd-kraus-decomp*:
fixes P :: *complex Matrix.mat*
assumes *sq*: $P \in$ *carrier-mat n n*
and *pos*: *psd P*
shows $\exists$ *vs* :: *complex Matrix.vec list*.
length vs $= n \wedge$

$(\forall\, v \in \mathit{set}\ \mathit{vs}.\ \mathit{dim\text{-}vec}\ v = n) \wedge$
$\mathit{outer\text{-}sum}\ n\ \mathit{vs} = P$
⟨*proof*⟩

$P = \sum_i |v_i\rangle\langle v_i|$ is positive definite iff the v_i span $\mathbb{C}^n$:

Entries of $\sum_i |v_i\rangle\langle v_i|$ for an arbitrary list of vectors.

lemma *outer-sum-entry-sum-list*:
assumes *vs-dim*: $\forall\, v \in \mathit{set}\ \mathit{vs}.\ \mathit{dim\text{-}vec}\ v = n$
and *i-lt*: $i < n$ **and** *j-lt*: $j < n$
shows $\mathit{outer\text{-}sum}\ n\ \mathit{vs}\ \$\$\ (i, j) =$
$\mathit{sum\text{-}list}\ (\mathit{map}\ (\lambda v.\ v\ \$\ i * \mathit{cnj}\ (v\ \$\ j))\ \mathit{vs})$
⟨*proof*⟩

lemma *mat-of-cols-dagger-mult-vec-index*:
assumes *xdim*: $\mathit{dim\text{-}vec}\ x = n$
and *vs-dim*: $\forall\, v \in \mathit{set}\ \mathit{vs}.\ \mathit{dim\text{-}vec}\ v = n$
and *i*: $i < \mathit{length}\ \mathit{vs}$
shows $(((\mathit{mat\text{-}of\text{-}cols}\ n\ \mathit{vs})^\dagger)\ *_v\ x)\ \$\ i = \langle \mathit{vs}\ !\ i \mid x\rangle$
⟨*proof*⟩

lemma *inner-prod-outer-sum-general*:
assumes *xdim*: $\mathit{dim\text{-}vec}\ x = n$
and *vs-dim*: $\forall\, v \in \mathit{set}\ \mathit{vs}.\ \mathit{dim\text{-}vec}\ v = n$
shows $\langle x \mid \mathit{outer\text{-}sum}\ n\ \mathit{vs} *_v x\rangle =$
$\langle((\mathit{mat\text{-}of\text{-}cols}\ n\ \mathit{vs})^\dagger)\ *_v\ x \mid$
$((\mathit{mat\text{-}of\text{-}cols}\ n\ \mathit{vs})^\dagger)\ *_v\ x\rangle$
⟨*proof*⟩

lemma *diag-mat-of-real-hermitian*:
shows *hermitian* (*diag-mat-of-real D*)
⟨*proof*⟩

$U\,\mathrm{diag}(D)\,U^\dagger$ is positive whenever the real entries of D are non-negative.

lemma *psd-U-diag-Udag*:
fixes *U* :: *complex Matrix.mat* **and** *D* :: *real list*
assumes *Uc*: $U \in \mathit{carrier\text{-}mat}\ n\ n$
and *Dlen*: $\mathit{length}\ D = n$
and *Dnn*: $\bigwedge i.\ i < n \Longrightarrow 0 \le D\ !\ i$
shows $\mathit{psd}\ (U * \mathit{diag\text{-}mat\text{-}of\text{-}real}\ D * (U^\dagger))$
⟨*proof*⟩

lemma *diag-mat-of-real-sqrt-square*:
assumes *D-nonneg*: $\bigwedge i.\ i < \mathit{length}\ D \Longrightarrow 0 \le D\ !\ i$
shows $\mathit{diag\text{-}mat\text{-}of\text{-}real}\ (\mathit{map}\ \mathit{sqrt}\ D) * \mathit{diag\text{-}mat\text{-}of\text{-}real}\ (\mathit{map}\ \mathit{sqrt}\ D) = \mathit{diag\text{-}mat\text{-}of\text{-}real}\ D$
⟨*proof*⟩

theorem *pd-iff-spanning*:
fixes *P* :: *complex Matrix.mat*
assumes *sq*: $P \in \mathit{carrier\text{-}mat}\ n\ n$
and *decomp*: $P = \mathit{outer\text{-}sum}\ n\ \mathit{vs}$
and *vs-dim*: $\forall\, v \in \mathit{set}\ \mathit{vs}.\ \mathit{dim\text{-}vec}\ v = n$
shows $\mathit{pd}\ P \longleftrightarrow$
$(\forall\, x :: \mathit{complex\ Matrix.vec}.\ \mathit{dim\text{-}vec}\ x = n \longrightarrow x \neq 0_v\ n \longrightarrow$
$(\exists\, v \in \mathit{set}\ \mathit{vs}.\ \langle v \mid x\rangle \neq 0))$
⟨*proof*⟩

9.4 The positive square root

Remark (Day 8)**.** Given $P \geq 0$ with $U^*PU = \mathrm{diag}(\lambda_1, \ldots, \lambda_n)$, $\lambda_i \geq 0$, put $P^{1/2} = U\,\mathrm{diag}(\lambda_1^{1/2}, \ldots, \lambda_n^{1/2})U^*$. Then $P^{1/2} \geq 0$ and $(P^{1/2})^2 = P$.

psd-sqrt P is defined as a choice of such a square root; *psd-sqrt-exists* is the existence statement and *psd-sqrt-props* the defining properties.

definition *psd-sqrt* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
psd-sqrt P ≡ (*SOME Q*. *Q* ∈ *carrier-mat* (*dim-row P*) (*dim-row P*) ∧ *psd Q* ∧ *Q* ∗ *Q* = *P*)

theorem *psd-sqrt-exists*:
fixes *P* :: *complex Matrix.mat*
assumes *sq*: *P* ∈ *carrier-mat n n*
and *pos*: *psd P*
shows ∃ *Q* :: *complex Matrix.mat*.
Q ∈ *carrier-mat n n* ∧ *psd Q* ∧ *Q* ∗ *Q* = *P*
⟨*proof*⟩
lemma *psd-sqrt-props*:
fixes *P* :: *complex Matrix.mat*
assumes *sq*: *P* ∈ *carrier-mat n n*
and *pos*: *psd P*
shows *psd-sqrt P* ∈ *carrier-mat n n*
and *psd* (*psd-sqrt P*)
and *psd-sqrt P* ∗ *psd-sqrt P* = *P*
⟨*proof*⟩

The corollary of Day 4: $P > 0$ iff $P = P^*$ and all eigenvalues are strictly positive.

lemma *pd-iff-pos-eigenvalues*:
fixes *P* :: *complex Matrix.mat*
assumes *sq*: *P* ∈ *carrier-mat n n*
shows *pd P* ⟷
(*hermitian P* ∧
(∀ *mu* :: *complex*. ∀ *v* :: *complex Matrix.vec*.
v ≠ 0$_v$ *n* ∧ *dim-vec v* = *n* ∧ *P* ∗$_v$ *v* = *mu* ·$_v$ *v* ⟶ 0 < *Re mu*))
⟨*proof*⟩

10 Day 5 (7 September 2011): tests for positivity; direct sums and partitioned matrices

10.1 Practical tests for positivity

Given $P = P^*$, how does one decide whether $P \geq 0$ or $P > 0$? The notes give two tests. (1) $P > 0$ iff every leading principal minor $\det(p_{ij})_{i,j \leq k}$, $k = 1, \ldots, n$, is positive; the same test with ≥ 0 does *not* characterise $P \geq 0$. This criterion (Sylvester's) is stated without proof in the notes and is not formalised. (2) Cholesky's algorithm:

Theorem (notes). If $P = (p_{ij})$ is $n \times n$ with $P = P^*$, then $P \geq 0$ if and only if $P - \left(\frac{p_{i1}p_{1j}}{p_{11}}\right)_{ij} \geq 0$.

The subtracted matrix agrees with P in the first row and column, so the difference is really an $(n-1) \times (n-1)$ matrix and the test can be repeated; when $P \geq 0$ the algorithm also writes P as a sum of rank-one matrices. The formal theorem adds the hypothesis $p_{11} > 0$, which the notes leave implicit (for $p_{11} = 0$ the quotient is undefined).

theorem *cholesky-reduction*:
fixes *P* :: *complex Matrix.mat*
assumes *sq*: *P* ∈ *carrier-mat n n*

and *herm*: *hermitian P*
and *n-pos*: *0 < n*
and *p11-pos*: *P $$ (0,0) > 0*
shows *psd P ⟷*
psd (Matrix.mat (n−1) (n−1)
(λ(i,j). P $$ (i+1, j+1) −
*P $$ (i+1, 0) * cnj (P $$ (j+1, 0)) / P $$ (0,0)))*
⟨*proof*⟩

10.2 Direct sums of vector spaces and partitioned matrices

Definition (notes). The *direct sum* of vector spaces V, W is $V \oplus W = \{(v, w) : v \in V, w \in W\}$ with componentwise operations and inner product $\langle (v_1, w_1) | (v_2, w_2) \rangle = \langle v_1 | v_2 \rangle_V + \langle w_1 | w_2 \rangle_W$.

Proposition (notes). If $v_1, \ldots, v_n$ is a basis of V and $w_1, \ldots, w_k$ a basis of W, then $(v_1, 0), \ldots, (v_n, 0), (0, w_1), \ldots$ is a basis of $V \oplus W$; hence $\dim(V \oplus W) = \dim V + \dim W$.

Key examples: $\mathbb{C}^{n+k} = \mathbb{C}^n \oplus \mathbb{C}^k$, and a linear map $T : \mathbb{C}^{n_1+k_1} \to \mathbb{C}^{n_2+k_2}$ has the block form $T = \begin{pmatrix} T_{11} & T_{12} \\ T_{21} & T_{22} \end{pmatrix}$ with $T\binom{v}{w} = \binom{T_{11}v+T_{12}w}{T_{21}v+T_{22}w}$. Formally the direct sum of vectors is the concatenation $v \oplus_v w$, and block matrices are the library's *four-block-mat*.

definition *direct-sum-vec* :: *complex Matrix.vec ⇒ complex Matrix.vec ⇒ complex Matrix.vec* (**infixr** $\oplus_v$ *65*) **where**
v ⊕ᵥ w ≡ Matrix.vec (dim-vec v + dim-vec w) (λi. if i < dim-vec v then v $ i else w $ (i − dim-vec v))

lemma *direct-sum-vec-dim* [*simp*]:
dim-vec (v ⊕ᵥ w) = dim-vec v + dim-vec w
⟨*proof*⟩

lemma *direct-sum-vec-left* [*simp*]:
i < dim-vec v ⟹ (v ⊕ᵥ w) $ i = v $ i
⟨*proof*⟩

lemma *direct-sum-vec-right* [*simp*]:
i < dim-vec w ⟹ (v ⊕ᵥ w) $ (dim-vec v + i) = w $ i
⟨*proof*⟩

lemma *direct-sum-inner-prod*:
assumes *vd1*: *dim-vec v1 = m* **and** *vd2*: *dim-vec v2 = m*
and *wd1*: *dim-vec w1 = p* **and** *wd2*: *dim-vec w2 = p*
shows *⟨v1 ⊕ᵥ w1 | v2 ⊕ᵥ w2⟩ = ⟨v1 | v2⟩ + ⟨w1 | w2⟩*
⟨*proof*⟩

corollary *direct-sum-dim*:
dim-vec (v ⊕ᵥ w) = dim-vec v + dim-vec w
⟨*proof*⟩

lemma *block-mat-apply*:
assumes *dim-vec v = k1* **and** *dim-vec w = k2*
and *T11 ∈ carrier-mat n1 k1* **and** *T12 ∈ carrier-mat n1 k2*
and *T21 ∈ carrier-mat n2 k1* **and** *T22 ∈ carrier-mat n2 k2*
shows *four-block-mat T11 T12 T21 T22 *ᵥ (v ⊕ᵥ w) =*
*(T11 *ᵥ v + T12 *ᵥ w) ⊕ᵥ (T21 *ᵥ v + T22 *ᵥ w)*
⟨*proof*⟩

11 Day 6 (9 September 2011): tensor products

Definition (notes). $B : X \times Y \to Z$ is *bilinear* if $B(x_1 + x_2, y) = B(x_1, y) + B(x_2, y)$, $B(x, y_1 + y_2) = B(x, y_1) + B(x, y_2)$ and $B(\lambda x, y) = B(x, \lambda y) = \lambda B(x, y)$.

The tensor product $X \otimes Y$ is the span of elementary tensors $x \otimes y$ with the universal property: every bilinear B factors as $B(x, y) = L_B(x \otimes y)$ for a unique linear L_B. Key relations: $(x_1 + x_2) \otimes y = x_1 \otimes y + x_2 \otimes y$, $x \otimes (y_1 + y_2) = x \otimes y_1 + x \otimes y_2$, $\lambda(x \otimes y) = (\lambda x) \otimes y = x \otimes (\lambda y)$.

Theorem (notes). If $\{e_1, \ldots, e_k\}$ and $\{f_1, \ldots, f_m\}$ are bases of X and Y, then $\{e_i \otimes f_j\}$ is a basis of $X \otimes Y$; hence $\dim(X \otimes Y) = \dim X \cdot \dim Y$.

Proposition (notes). With bases as above, every $u \in X \otimes Y$ is uniquely $u = \sum_j x_j \otimes f_j$ with $x_j \in X$, and uniquely $u = \sum_i e_i \otimes y_i$ with $y_i \in Y$; consequently $X \otimes Y \cong X \oplus \cdots \oplus X$ (m copies).

Definition (notes). The *Schmidt rank* of $u \in X \otimes Y$ is the least number of elementary tensors in an expression for u.

The formalisation uses the concrete model: $\mathbb{C}^n \otimes \mathbb{C}^m = \mathbb{C}^{nm}$ with $(v \otimes w)_{im+j} = v_i w_j$ (*tensor-vec* v w from the library). The three key relations are the bilinearity lemmas below; the product basis theorem is *tensor-onb* (Day 7); the unique decomposition along a basis of Y is not stated for vectors (its operator analogue is in theory Quantum_Computing11).

lemma *tensor-vec-dim*:
assumes *vd*: *dim-vec v = n dim-vec w = m*
shows *dim-vec (tensor-vec v w) = n * m*
⟨*proof*⟩

lemma *tensor-vec-index*:
assumes *dim-vec v = n dim-vec w = m i < n * m*
shows *tensor-vec v w $ i = v $ (i div m) * w $ (i mod m)*
⟨*proof*⟩

lemma *tensor-vec-bilinear-left*:
assumes *vd*: *dim-vec v1 = n dim-vec v2 = n dim-vec w = m*
shows *tensor-vec (v1 + v2) w = tensor-vec v1 w + tensor-vec v2 w*
⟨*proof*⟩

lemma *tensor-vec-bilinear-right*:
assumes *vd*: *dim-vec v = n dim-vec w1 = m dim-vec w2 = m*
shows *tensor-vec v (w1 + w2) = tensor-vec v w1 + tensor-vec v w2*
⟨*proof*⟩

lemma *tensor-vec-smult-left*:
assumes *vd*: *dim-vec v = n dim-vec w = m*
shows *tensor-vec (α ·$_v$ v) w = α ·$_v$ tensor-vec v w*
⟨*proof*⟩

lemma *tensor-vec-smult-right*:
assumes *vd*: *dim-vec v = n dim-vec w = m*
shows *tensor-vec v (α ·$_v$ w) = α ·$_v$ tensor-vec v w*
⟨*proof*⟩

The Schmidt rank, as the minimum length of a representation by elementary tensors (no properties of it are needed later):

definition *schmidt-rank* :: *complex Matrix.vec* ⇒ *nat* ⇒ *nat* **where**
schmidt-rank u m ≡
Min {*k*. ∃ *vs* :: *complex Matrix.vec list*. ∃ *ws* :: *complex Matrix.vec list*.
length vs = *k* ∧ *length ws* = *k* ∧
(∀ *i* < *k*. *dim-vec* (*vs* ! *i*) = *dim-vec u div m*) ∧
(∀ *i* < *k*. *dim-vec* (*ws* ! *i*) = *m*) ∧
u = *foldr* (λ(*v*,*w*) *acc*. *tensor-vec v w* + *acc*)
(*zip vs ws*) (0$_v$ (*dim-vec u*))}

12 Day 7 (12 September 2011): tensor products of Hilbert spaces; the postulates

Remark. For Hilbert spaces $\mathcal{H}, \mathcal{K}$ set $\langle h_1 \otimes k_1 | h_2 \otimes k_2 \rangle = \langle h_1 | h_2 \rangle_{\mathcal{H}} \langle k_1 | k_2 \rangle_{\mathcal{K}}$.

Theorem (notes). This extends to a sesquilinear form on $\mathcal{H} \otimes \mathcal{K}$ with $\langle u | u \rangle = 0$ only for $u = 0$, hence to an inner product; in finite dimensions $\mathcal{H} \otimes \mathcal{K}$ with this inner product is the Hilbert space tensor product (in infinite dimensions one completes).

Proposition (notes). If $\{h_i\}$ and $\{k_j\}$ are orthonormal bases of $\mathcal{H}$ and $\mathcal{K}$, then $\{h_i \otimes k_j\}$ is an orthonormal basis of $\mathcal{H} \otimes \mathcal{K}$.

In physics notation $e_i \otimes e_j = |ij\rangle$, and for N qubits $e_{i_1} \otimes \cdots \otimes e_{i_N} = |i_1 \cdots i_N\rangle$ with the multi-index $I = (i_1, \ldots, i_N) \in \mathbb{Z}_2^N$. In the concrete model positivity of the form is automatic; the multiplicativity of the inner product and the orthonormal product basis are proved below.

lemma *tensor-inner-prod*:
assumes *dim-vec h1* = *n dim-vec h2* = *n*
dim-vec k1 = *m dim-vec k2* = *m*
shows ⟨*tensor-vec h1 k1* | *tensor-vec h2 k2*⟩ =
⟨*h1* | *h2*⟩ ∗ ⟨*k1* | *k2*⟩
⟨*proof*⟩

lemma *length-concat-map-upt*:
length (*concat* (*map* (λ*i*. *map* (λ*j*. *f i j*) [0..<*m*]) [0..<*n*])) = *n* ∗ *m*
⟨*proof*⟩

lemma *nth-concat-map-upt*:
assumes *p-lt*: *p* < *n* ∗ *m*
shows *concat* (*map* (λ*i*. *map* (λ*j*. *f i j*) [0..<*m*]) [0..<*n*]) ! *p* =
f (*p div m*) (*p mod m*)
⟨*proof*⟩

lemma *tensor-onb*:
assumes *hs-onb*: *orthonormal-list n hs*
and *hs-len*: *length hs* = *n*
and *ks-onb*: *orthonormal-list m ks*
and *ks-len*: *length ks* = *m*
shows *orthonormal-list* (*n*∗*m*)
[*tensor-vec* (*hs* ! *i*) (*ks* ! *j*). *i* ← [0..<*n*], *j* ← [0..<*m*]]
⟨*proof*⟩

12.1 Postulates of quantum mechanics

Postulate (notes) (1)**.** To each isolated physical system corresponds a Hilbert space, the *state space*; the state of the system is completely described by a unit vector in it, the *state vector*.

Postulate (notes) (2)**.** The time evolution of a closed system from t_1 to t_2 is described by a unitary $U : \mathcal{H} \to \mathcal{H}$: a system in state ψ at t_1 is in state $U\psi$ at t_2.

Postulate (notes) (3)**.** Quantum measurements are described by a collection $\{M_m\}$ of *measurement operators* on $\mathcal{H}$. If the system is in state ψ, the probability of observing m is $p_m(\psi) = \langle\psi|M_m^* M_m\psi\rangle = \|M_m\psi\|^2$; since the probabilities sum to one, $\sum_m M_m^* M_m = I$. After observing m the state is $M_m\psi/p_m(\psi)^{1/2}$.

Formally a state is *quantum-state n ψ* (a unit vector in $\mathbb{C}^n$), evolution is *evolve U ψ*, a measurement system is a list *Ms* with *measurement-system n Ms* (all $n \times n$ with $\sum_m M_m^\dagger M_m = I$), and $p_m(\psi)$ is *meas-prob M ψ*.

definition *quantum-state* :: *nat* ⇒ *complex Matrix.vec* ⇒ *bool* **where**
quantum-state n ψ ≡ *dim-vec ψ* = *n* ∧ ‖*ψ*‖ = *1*

lemma *quantum-state-norm1*:
assumes *quantum-state n ψ*
shows ‖*ψ*‖ = *1*
⟨*proof*⟩

lemma *quantum-state-dim*:
assumes *quantum-state n ψ*
shows *dim-vec ψ* = *n*
⟨*proof*⟩

definition *evolve* :: *complex Matrix.mat* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.vec* **where**
evolve U ψ ≡ *U* ∗*v ψ*

lemma *unitary-preserves-state*:
assumes *unitary U* **and** *quantum-state n ψ* **and** *dim-col U* = *n*
shows *quantum-state* (*dim-row U*) (*evolve U ψ*)
⟨*proof*⟩

definition *measurement-system* :: *nat* ⇒ *complex Matrix.mat list* ⇒ *bool* **where**
measurement-system n Ms ≡
(∀ *M* ∈ *set Ms*. *M* ∈ *carrier-mat n n*) ∧
foldr (λ*M acc*. (*M*†) ∗ *M* + *acc*) *Ms* (*0m n n*) = *1m n*

definition *meas-prob* :: *complex Matrix.mat* ⇒ *complex Matrix.vec* ⇒ *real* **where**
meas-prob M ψ ≡ ‖*M* ∗*v ψ*‖²

lemma *measurement-probs-sum-one*:
assumes *ms*: *measurement-system n Ms*
and *st*: *quantum-state n ψ*
shows (∑ *M* ← *Ms*. *meas-prob M ψ*) = *1*
⟨*proof*⟩

$p_m(\psi) = \langle\psi|M_m^* M_m\psi\rangle$, the form in which the notes state Postulate 3.

lemma *meas-prob-quad-form*:
fixes *M* :: *complex Matrix.mat* **and** *ψ* :: *complex Matrix.vec*

assumes M: $M \in$ *carrier-mat n n* **and** ψ: *dim-vec* $\psi = n$
shows *meas-prob M* ψ = *Re* $\langle \psi \mid (M^\dagger * M) *_v \psi \rangle$
⟨*proof*⟩

After outcome m the state becomes $M_m\psi/\|M_m\psi\|$:

definition *post-meas-state* :: *complex Matrix.mat* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.vec* **where**
post-meas-state M $\psi \equiv$ (*1 / complex-of-real* ($\|M *_v \psi\|$)) $\cdot_v$ ($M *_v \psi$)

Postulate (notes) (4, Day 9)**.** The state space of a composite system is the tensor product of the state spaces of its components; if the i-th component is in state ψ_i, the system is in state $\psi_1 \otimes \cdots \otimes \psi_n$.

lemma *tensor-of-states-is-state*:
assumes *quantum-state n ψ1* **and** *quantum-state m ψ2*
shows *quantum-state* (*n*∗*m*) (*tensor-vec ψ1 ψ2*)
⟨*proof*⟩

13 Day 8 (14 September 2011): the quantum game; projective measurements and expected values

13.1 Distinguishing states

Alice has two states ψ_1, ψ_2 known to Bob; she sends one. Can Bob build a measurement system that decides with certainty which one he received? Formally one wants M_1, M_2 with $\|M_1\psi_2\| = 0$, $\|M_1\psi_1\|^2 = 1$, $\|M_2\psi_1\| = 0$, $\|M_2\psi_2\|^2 = 1$.
Case I, $\psi_1 \perp \psi_2$: take $M_1 = |\psi_1\rangle\langle\psi_1|$, $M_2 = |\psi_2\rangle\langle\psi_2|$ and $M_3 = I - M_1 - M_2$; this distinguishes with certainty (formalised below).
Case II, $\psi_1 \not\perp \psi_2$: write $\psi_2 = \alpha\psi_1 + \beta\gamma$ with $\gamma \perp \psi_1$ a unit vector, so $\alpha \neq 0$; any measurement system with $\|M_1\psi_1\| = 1$ satisfies $|\alpha|^2 \leq \|M_1\psi_2\|^2$, so outcome 1 also occurs with probability at least $|\alpha|^2$ when ψ_2 is sent (*nonorthogonal-outcome-prob* in theory Day02_11_Supplement).

Theorem (notes). Given linearly independent states $\psi_1, \ldots, \psi_n$ there is a measurement system $M_0, \ldots, M_n$ such that if outcome $i \geq 1$ occurs then ψ_i was received.

The notes' proof takes E_i the projection onto $\mathrm{span}\{\psi_j : j \neq i\}^\perp$, $M_i = E_i/\sqrt{n}$, and $M_0 = (I - \sum_i M_i^* M_i)^{1/2}$; the remark that follows observes that the inconclusive outcome 0 then has probability at least $(n-1)/n$. Formally this is *lin-indep-states-unambiguous* below.

lemma *outer-proj-self*:
assumes *q*: *quantum-state n ψ*
shows *outer-prod ψ ψ* $*_v$ *ψ* = *ψ*
⟨*proof*⟩

lemma *outer-proj-orth-zero*:
assumes *q1*: *quantum-state n ψ1*
and *q2*: *quantum-state n ψ2*
and *orth*: $\langle \psi 1 \mid \psi 2 \rangle = 0$
shows *outer-prod ψ1 ψ1* $*_v$ *ψ2* = 0_v *n*
⟨*proof*⟩

lemma *outer-proj-idem*:
assumes *q*: *quantum-state n ψ*
shows (*outer-prod ψ ψ*) ∗ (*outer-prod ψ ψ*) = *outer-prod ψ ψ*
⟨*proof*⟩

lemma *outer-proj-hermitian*:
assumes *dim-vec* $\psi = n$
shows $(\textit{outer-prod}\ \psi\ \psi)^\dagger = \textit{outer-prod}\ \psi\ \psi$
⟨*proof*⟩

lemma *outer-proj-cross-zero*:
assumes *q1*: *quantum-state n ψ1*
and *q2*: *quantum-state n ψ2*
and *orth*: $\langle \psi 1 \mid \psi 2 \rangle = 0$
shows $(\textit{outer-prod}\ \psi 1\ \psi 1) * (\textit{outer-prod}\ \psi 2\ \psi 2) = 0_m\ n\ n$
⟨*proof*⟩

Case I: orthogonal states can be distinguished perfectly.

lemma *orthogonal-states-distinguishable*:
fixes $\psi 1\ \psi 2$ :: *complex Matrix.vec*
assumes *q1*: *quantum-state n ψ1*
and *q2*: *quantum-state n ψ2*
and *orth*: $\langle \psi 1 \mid \psi 2 \rangle = 0$
shows ∃ *Ms* :: *complex Matrix.mat list*.
measurement-system n Ms ∧
(∃ *M1 M2*. *M1* ∈ *set Ms* ∧ *M2* ∈ *set Ms* ∧
meas-prob M1 ψ1 = 1 ∧ *meas-prob M2 ψ1* = 0 ∧
meas-prob M1 ψ2 = 0 ∧ *meas-prob M2 ψ2* = 1)
⟨*proof*⟩

The theorem on linearly independent states. Linear independence of a list of vectors is injectivity of the matrix with those columns; the formal statement produces the measurement system $M_0, \ldots, M_k$ in the order $M_1, \ldots, M_k, M_0$ (outcome $i < k$ identifies ψ_i, outcome k is inconclusive). The proof follows the notes with one change: instead of the projections E_i onto $\{\psi_j : j \neq i\}^\perp$ it uses the dual vectors ϕ_i with $\langle \phi_i | \psi_j \rangle = \delta_{ij}$ (columns of $V(V^*V)^{-1}$) and the operators $M_i = \sqrt{c}\,|e_0\rangle\langle\phi_i|$, where $c > 0$ makes $I - c\sum_i |\phi_i\rangle\langle\phi_i|$ positive; M_0 is its square root.

definition *lin-indep-vecs* :: *nat* ⇒ *complex Matrix.vec list* ⇒ *bool* **where**
lin-indep-vecs n vs ≡ (∀ *v* ∈ *set vs*. *dim-vec v* = *n*) ∧
(∀ *x* :: *complex Matrix.vec*. *x* ∈ *carrier-vec* (*length vs*) ⟶
mat-of-cols n vs $*_v$ *x* = 0_v *n* ⟶ *x* = 0_v (*length vs*))

lemma *outer-prod-carrier*:
fixes *v w* :: *complex Matrix.vec*
assumes *dim-vec v* = *n* **and** *dim-vec w* = *m*
shows *v* ⊘ *w* ∈ *carrier-mat n m*
⟨*proof*⟩

lemma *outer-prod-dagger*:
fixes *v w* :: *complex Matrix.vec*
shows $(v \oslash w)^\dagger = w \oslash v$
⟨*proof*⟩

lemma *foldr-add-mat-carrier*:
fixes *f* :: *'a* ⇒ *complex Matrix.mat*
assumes ∀ *x* ∈ *set xs*. *f x* ∈ *carrier-mat n n*
shows *foldr* (λ*x acc*. *f x* + *acc*) *xs* (0_m *n n*) ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *foldr-add-mat-index*:
fixes *f* :: *'a* ⇒ *complex Matrix.mat*
assumes ∀ *x* ∈ *set xs*. *f x* ∈ *carrier-mat n n* **and** *i* < *n* **and** *j* < *n*
shows *foldr* (λ*x acc*. *f x* + *acc*) *xs* (0_m *n n*) $$ (*i*, *j*) = *sum-list* (*map* (λ*x*. *f x* $$ (*i*, *j*)) *xs*)

⟨*proof*⟩

lemma *foldr-MdagM-shift*:
assumes *Ms-c*: ∀ *M* ∈ *set Ms*. *M* ∈ *carrier-mat n n* **and** *z*: *z* ∈ *carrier-mat n n*
shows *foldr* (λ*M acc*. (*M*†) ∗ *M* + *acc*) *Ms z*
= *foldr* (λ*M acc*. (*M*†) ∗ *M* + *acc*) *Ms* (0*m n n*) + *z*
⟨*proof*⟩

theorem *lin-indep-states-unambiguous*:
fixes ψ*s* :: *complex Matrix.vec list*
assumes *qs*: ∀ ψ ∈ *set* ψ*s*. *quantum-state n* ψ
and *li*: *lin-indep-vecs n* ψ*s*
and *k*: *length* ψ*s* = *k* **and** *kpos*: 0 < *k*
shows ∃ *Ms* :: *complex Matrix.mat list*.
measurement-system n Ms ∧ *length Ms* = *k* + 1 ∧
(∀ *i*<*k*. ∀ *j*<*k*. *i* ≠ *j* ⟶ *meas-prob* (*Ms* ! *i*) (ψ*s* ! *j*) = 0) ∧
(∀ *i*<*k*. 0 < *meas-prob* (*Ms* ! *i*) (ψ*s* ! *i*))
⟨*proof*⟩

13.2 Projective measurements, expected values and self-adjoint operators

If a measurement $\{M_m\}$ has real outcomes λ_m, the expected value of the outcome in state ψ is $E = \sum_m p_m(\psi)\lambda_m = \langle\psi|(\sum_m \lambda_m M_m^* M_m)\psi\rangle = \langle\psi|H\psi\rangle$ with $H = \sum_m \lambda_m M_m^* M_m$ self-adjoint: expected values of observables are quadratic forms of Hermitian matrices (*expected-value-eq-quad-form* below).

Formally: a measurement whose operator is the rank-one projection $|\gamma\rangle\langle\gamma|$ has $p(\psi) = |\langle\gamma|\psi\rangle|^2$ (the proposition of Day 12); an *observable* with outcomes λ_m attached to the measurement operators M_m is $H = \sum_m \lambda_m M_m^* M_m$, which is Hermitian, and the expected outcome $\sum_m \lambda_m p_m(\psi)$ equals $\langle\psi|H\psi\rangle$.

lemma *rank-one-meas-prob*:
fixes γ ψ :: *complex Matrix.vec*
assumes *g*: *quantum-state n* γ **and** ψ: *dim-vec* ψ = *n*
shows *meas-prob* (γ ⊘ γ) ψ = (*cmod* ⟨γ | ψ⟩)²
⟨*proof*⟩

definition *observable* :: *nat* ⇒ (*real* × *complex Matrix.mat*) *list* ⇒ *complex Matrix.mat* **where**
observable n lMs ≡
foldr (λ(*l*, *M*) *acc*. *complex-of-real l* ·*m* ((*M*†) ∗ *M*) + *acc*) *lMs* (0*m n n*)

definition *expected-value* :: (*real* × *complex Matrix.mat*) *list* ⇒ *complex Matrix.vec* ⇒ *real* **where**
expected-value lMs ψ ≡ (∑ (*l*, *M*) ← *lMs*. *l* ∗ *meas-prob M* ψ)

lemma *observable-Nil* [*simp*]: *observable n* [] = 0*m n n*
⟨*proof*⟩

lemma *observable-Cons* [*simp*]:
observable n ((*l*, *M*) # *lMs*) = *complex-of-real l* ·*m* ((*M*†) ∗ *M*) + *observable n lMs*
⟨*proof*⟩

lemma *observable-carrier*:
assumes ∀ (*l*, *M*) ∈ *set lMs*. *M* ∈ *carrier-mat n n*
shows *observable n lMs* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *observable-hermitian*:
assumes ∀ (*l*, *M*) ∈ *set lMs*. *M* ∈ *carrier-mat n n*

shows *hermitian* (*observable n lMs*)
⟨*proof*⟩

lemma *expected-value-eq-quad-form*:
fixes ψ :: *complex Matrix.vec*
assumes ψ: *dim-vec* $\psi = n$
and *lMs*: $\forall (l, M) \in$ *set lMs*. $M \in$ *carrier-mat n n*
shows *expected-value lMs* $\psi =$ *Re* $\langle \psi \mid$ *observable n lMs* $*_v \psi\rangle$
⟨*proof*⟩

14 Day 9 (16 September 2011): positive operator-valued measures; composite systems; entanglement

14.1 Positive operator-valued measures

If two measurement systems satisfy $M_m^* M_m = \tilde{M}_m^* \tilde{M}_m$ for all m then $p_m = \tilde{p}_m$: only the operators $M_m^* M_m$ matter, and $\sum_m M_m^* M_m = I$. Conversely if $P_m \geq 0$ with $\sum_m P_m = I$ then $M_m = P_m^{1/2}$ is a measurement system.

Definition (notes). A *positive operator-valued measure* (POVM) is a set $\{P_m\}$ of positive operators with $\sum_m P_m = I$.

definition *povm* :: *nat* ⇒ *complex Matrix.mat list* ⇒ *bool* **where**
povm n Ps ≡
($\forall P \in$ *set Ps*. $P \in$ *carrier-mat n n* ∧ *psd P*) ∧
foldr (λP *acc*. P + *acc*) *Ps* (0_m *n n*) = 1_m *n*

lemma *measurement-induces-povm*:
assumes *ms*: *measurement-system n Ms*
shows *povm n* (*map* (λM. ($M^\dagger$) ∗ M) *Ms*)
⟨*proof*⟩

lemma *psd-sqrt-measurement-factor*:
assumes *Pcar*: $P \in$ *carrier-mat n n*
and *Ppsd*: *psd P*
shows ∃ M. $M \in$ *carrier-mat n n* ∧ $M^\dagger * M = P$
⟨*proof*⟩

lemma *psd-sqrt-measurement-factors*:
assumes *Ps*: $\forall P \in$ *set Ps*. $P \in$ *carrier-mat n n* ∧ *psd P*
shows ∃ *Ms*. ($\forall M \in$ *set Ms*. $M \in$ *carrier-mat n n*) ∧
map (λM. $M^\dagger * M$) *Ms* = *Ps*
⟨*proof*⟩

lemma *povm-from-sqrt*:
assumes *pv*: *povm n Ps*
shows ∃ *Ms*. *measurement-system n Ms* ∧
map (λM. $M^\dagger * M$) *Ms* = *Ps*
⟨*proof*⟩

lemma *povm-prob*:
assumes *pv*: *povm n Ps* **and** *st*: *quantum-state n* ψ
and *Pin*: $P \in$ *set Ps*
shows *Re* ($\langle \psi \mid P *_v \psi\rangle$) ≥ *0*

⟨*proof*⟩

14.2 Composite systems

Postulate 4 was stated with Day 7. The notes' example: photons in states $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and $\frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle)$ form the pair $\frac{1}{2}(|00\rangle + i|01\rangle + |10\rangle + i|11\rangle) \in \mathbb{C}^2 \otimes \mathbb{C}^2$, a vector of Schmidt rank 1. Two photons described by a unit vector $\psi \in \mathbb{C}^2 \otimes \mathbb{C}^2$ can be regarded as two separate subsystems iff $\psi = \psi_1 \otimes \psi_2$ iff the Schmidt rank of ψ is 1; otherwise ψ is *entangled*, which does happen in nature.

Tensor products of operators: for $R : \mathcal{H} \to \mathcal{H}$ and $T : \mathcal{K} \to \mathcal{K}$ there is a unique $R \otimes T$ with $(R \otimes T)(\sum_l h_l \otimes k_l) = \sum_l Rh_l \otimes Tk_l$, and (1) $(R_1 \otimes T_1)(R_2 \otimes T_2) = R_1R_2 \otimes T_1T_2$, (2) $(R \otimes T)^* = R^* \otimes T^*$ (both proved in Basic_Math for the Kronecker product).

Remark. If lab A has the measurement system $\{M_m\}$ on $\mathcal{H}$ and lab B has state space $\mathcal{K}$, the measurement system of the composite is $\{M_m \otimes I_{\mathcal{K}}\}$ and $p_m^{AB}(\psi \otimes \phi) = \|(M_m \otimes I)(\psi \otimes \phi)\|^2 = p_m^A(\psi)$.

lemma *tensor-mat-vec-mult*:
  assumes *A*: $A \in$ *carrier-mat nr nc*
    and *B*: $B \in$ *carrier-mat mr mc*
    and *v*: *dim-vec v* $= nc$
    and *w*: *dim-vec w* $= mc$
  shows $(A \bigotimes B) *_v$ *tensor-vec v w* $=$ *tensor-vec* $(A *_v v)$ $(B *_v w)$
⟨*proof*⟩

lemma *tensor-mat-vec-mult-right-id*:
  assumes *A*: $A \in$ *carrier-mat n n*
    and *v*: *dim-vec v* $= n$
    and *w*: *dim-vec w* $= m$
  shows $(A \bigotimes 1_m\ m) *_v$ *tensor-vec v w* $=$ *tensor-vec* $(A *_v v)$ *w*
⟨*proof*⟩

lemma *composite-measurement-prob*:
  fixes *M* :: *complex Matrix.mat* **and** ψ :: *complex Matrix.vec* **and** φ :: *complex Matrix.vec*
  assumes *psi-st*: *quantum-state n* ψ **and** *phi-st*: *quantum-state m* φ
    and *M-sq*: $M \in$ *carrier-mat n n*
  shows *meas-prob* $(M \bigotimes 1_m\ m)$ *(tensor-vec* ψ φ*)* $=$ *meas-prob M* ψ
⟨*proof*⟩

14.3 Entanglement

A state $\psi \in \mathcal{H}_A \otimes \mathcal{H}_B$ is *entangled* if it is not a product state $u \otimes v$. The Bell state $\gamma = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ is entangled. The notes' first application of entanglement (A and B share γ; A measures $|0\rangle\langle 0|, |1\rangle\langle 1|$ and gets each outcome with probability $\frac{1}{2}$, after which B's measurement is determined) is *bell-measurement* in theory Day02_11_Supplement.

definition *entangled* :: *nat* ⇒ *nat* ⇒ *complex Matrix.vec* ⇒ *bool* **where**
  entangled n m $\psi \equiv$
    dim-vec $\psi = n * m \wedge$
    $\neg$ $(\exists u\ v.$ *dim-vec u* $= n \wedge$ *dim-vec v* $= m \wedge \psi =$ *tensor-vec u v*$)$

definition *bell-state* :: *complex Matrix.vec* **where**
  bell-state $\equiv$ *(1 / csqrt 2)* $\cdot_v$
    (tensor-vec (unit-vec 2 0) (unit-vec 2 0) +
     tensor-vec (unit-vec 2 1) (unit-vec 2 1))

lemma *bell-state-norm*:
‖bell-state‖ = 1
⟨*proof*⟩

lemma *bell-state-is-quantum-state*:
quantum-state 4 bell-state
⟨*proof*⟩

lemma *bell-state-entangled*:
entangled 2 2 bell-state
⟨*proof*⟩

15 Day 10 (19 September 2011): superdense coding; classical and quantum gates

15.1 One-qubit gates

The one-qubit gates of the notes:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \text{ (NOT)}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad H = \tfrac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}.$$

H is the Hadamard gate; the notes write "$H^2 = X$" here and correct it to $H^2 = I$ on Day 11, which is what is proved below. H and S together generate all 2×2 unitaries (not formalised). Formally H is *Hadamard* (the library's *Quantum.H*) and S is *Phase-S*.

definition *Hadamard* :: *complex Matrix.mat* **where**
Hadamard ≡ Quantum.H

lemma *csqrt-two*: *csqrt (2*::*complex) = (sqrt 2* :: *complex)*
⟨*proof*⟩

lemma *hadamard-squared*:
Hadamard ∗ Hadamard = 1$_m$ *2*
⟨*proof*⟩

lemma *hadamard-e0*:
Hadamard ∗$_v$ *unit-vec 2 0 =*
(1 / csqrt 2) ·$_v$ *(unit-vec 2 0 + unit-vec 2 1)*
⟨*proof*⟩

lemma *hadamard-e1*:
Hadamard ∗$_v$ *unit-vec 2 1 =*
(1 / csqrt 2) ·$_v$ *(unit-vec 2 0 − unit-vec 2 1)*
⟨*proof*⟩

The phase gate $S = \mathrm{diag}(1, i)$ is unitary.

definition *Phase-S* :: *complex Matrix.mat* **where**
Phase-S ≡ Matrix.mat 2 2 (λ(i,j). if i = 0 ∧ j = 0 then 1 else if i = 1 ∧ j = 1 then i *else 0)*

lemma *Phase-S-carrier* [*simp*]: *Phase-S ∈ carrier-mat 2 2*
⟨*proof*⟩

lemma *Phase-S-dagger-S*: *Phase-S*† *∗ Phase-S = 1*$_m$ *2*

⟨*proof*⟩

lemma *Phase-S-S-dagger*: *Phase-S* ∗ (*Phase-S*†) = 1_m *2*
⟨*proof*⟩

lemma *Phase-S-unitary*: *unitary Phase-S*
⟨*proof*⟩

15.2 Two- and three-qubit gates

The notes tabulate the classical gates AND, OR, XOR, NAND, NOR (Peirce: NAND generates all Boolean operations; Sheffer: so does NOR). No two-bit classical gate is a quantum gate, since it maps two dimensions to one and is not reversible. The controlled NOT acts by $|a, b\rangle \mapsto |a, b \oplus a\rangle$:

$$\mathrm{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix},$$

which is unitary, and Toffoli's CCNOT (identity on the first six basis vectors, swapping $|110\rangle$ and $|111\rangle$) is a reversible gate generating all Boolean operations; it is defined in theory Quantum_Computing11.

lemma *CNOT-unitary*: *unitary CNOT*
⟨*proof*⟩

lemma *CNOT-00*:
CNOT ∗$_v$ (*tensor-vec* (*unit-vec 2 0*) (*unit-vec 2 0*)) =
tensor-vec (*unit-vec 2 0*) (*unit-vec 2 0*)
⟨*proof*⟩

lemma *CNOT-10*:
CNOT ∗$_v$ (*tensor-vec* (*unit-vec 2 1*) (*unit-vec 2 0*)) =
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 1*)
⟨*proof*⟩

lemma *CNOT-01*:
CNOT ∗$_v$ (*tensor-vec* (*unit-vec 2 0*) (*unit-vec 2 1*)) =
tensor-vec (*unit-vec 2 0*) (*unit-vec 2 1*)
⟨*proof*⟩

lemma *CNOT-11*:
CNOT ∗$_v$ (*tensor-vec* (*unit-vec 2 1*) (*unit-vec 2 1*)) =
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 0*)
⟨*proof*⟩

lemma *tensor-unit-vec-2-00*:
tensor-vec (*unit-vec 2 0*) (*unit-vec 2 0*) = *unit-vec 4 0*
⟨*proof*⟩

lemma *tensor-unit-vec-2-01*:
tensor-vec (*unit-vec 2 0*) (*unit-vec 2 1*) = *unit-vec 4 1*
⟨*proof*⟩

lemma *tensor-unit-vec-2-10*:
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 0*) = *unit-vec 4 2*
⟨*proof*⟩

lemma *tensor-unit-vec-2-11*:
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 1*) = *unit-vec 4 3*
⟨*proof*⟩

15.3 Superdense coding

A and B share $\gamma = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. Applying nothing, $\begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix}$, $\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix}$ or $\begin{pmatrix}0 & 1\\ -1 & 0\end{pmatrix}$ to its qubit, A turns γ into one of the four orthonormal Bell states $\frac{1}{\sqrt{2}}(|00\rangle \pm |11\rangle)$, $\frac{1}{\sqrt{2}}(|10\rangle \pm |01\rangle)$, so by sending one qubit A communicates two bits; with $2m$ shared qubits, 4^m messages. Below: the four Bell states and their orthonormality (the action table itself is not formalised).

definition *bell-basis* :: *complex Matrix.vec list* **where**
bell-basis ≡
[(*1/csqrt 2*) ·$_v$ (*tensor-vec* (*unit-vec 2 0*) (*unit-vec 2 0*) +
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 1*)),
(*1/csqrt 2*) ·$_v$ (*tensor-vec* (*unit-vec 2 0*) (*unit-vec 2 0*) −
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 1*)),
(*1/csqrt 2*) ·$_v$ (*tensor-vec* (*unit-vec 2 1*) (*unit-vec 2 0*) +
tensor-vec (*unit-vec 2 0*) (*unit-vec 2 1*)),
(*1/csqrt 2*) ·$_v$ (*tensor-vec* (*unit-vec 2 0*) (*unit-vec 2 1*) −
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 0*))]

lemma *bell-basis-orthonormal*: *orthonormal-list 4 bell-basis*
⟨*proof*⟩

end
theory *Day02-11-Supplement*
imports *Quantum-Computing11*
begin

Supplementary results for Days 2–11 recorded as gaps by the first audit.

15.4 Day 2: the projection does not depend on the orthonormal basis

definition *col-span* :: *nat* ⇒ *complex Matrix.vec list* ⇒ *complex Matrix.vec set* **where**
col-span n vs = {*mat-of-cols n vs* ∗$_v$ *c* | *c*. *dim-vec c* = *length vs*}

lemma *outer-sum-fixes-col-span*:
assumes *orth*: *orthonormal-list n vs* **and** *w*: *w* ∈ *col-span n vs*
shows *outer-sum n vs* ∗$_v$ *w* = *w*
⟨*proof*⟩

lemma *mem-col-span-self*:
assumes ∀ *v* ∈ *set vs*. *dim-vec v* = *n* **and** *j*: *j* < *length vs*
shows *vs* ! *j* ∈ *col-span n vs*
⟨*proof*⟩

lemma *outer-sum-absorbs*:
assumes *ov*: *orthonormal-list n vs* **and** *ow*: *orthonormal-list n ws*
and *sub*: *set ws* ⊆ *col-span n vs*
shows *outer-sum n vs* ∗ *outer-sum n ws* = *outer-sum n ws*
⟨*proof*⟩

D2-8: two orthonormal bases of the same subspace give the same projection.

theorem *outer-sum-basis-independent*:
assumes *ov*: *orthonormal-list n vs* **and** *ow*: *orthonormal-list n ws*

and *span*: *col-span n vs = col-span n ws*
shows *outer-sum n vs = outer-sum n ws*
⟨*proof*⟩

15.5 Day 6: coordinates in a tensor product

$\sum_j x_j \otimes y_j$ in $\mathbb{C}^N$, written as in *schmidt-rank*.

definition *tensor-combination* :: *nat* ⇒ *complex Matrix.vec list* ⇒ *complex Matrix.vec list* ⇒ *complex Matrix.vec* **where**
tensor-combination N xs ys = foldr (λ(*x*,*y*) *acc*. *tensor-vec x y* + *acc*) (*zip xs ys*) (*0*$_v$ *N*)

lemma *tensor-combination-dim* [*simp*]: *dim-vec* (*tensor-combination N xs ys*) = *N*
⟨*proof*⟩

lemma *tensor-combination-index*:
assumes ∀ *x* ∈ *set xs*. *dim-vec x* = *k* ∀ *y* ∈ *set ys*. *dim-vec y* = *m length xs* = *length ys p* < *k* ∗ *m*
shows *tensor-combination* (*k* ∗ *m*) *xs ys* $ *p* = (∑*j*<*length xs*. *xs* ! *j* $ (*p div m*) ∗ *ys* ! *j* $ (*p mod m*))
⟨*proof*⟩

lemma *idx-bound*: *i* < *k* ⟹ *l* < (*m*::*nat*) ⟹ *i* ∗ *m* + *l* < *k* ∗ *m*
⟨*proof*⟩

lemma *idx-div-mod*: *l* < (*m*::*nat*) ⟹ (*i* ∗ *m* + *l*) *div m* = *i* ∧ (*i* ∗ *m* + *l*) *mod m* = *l*
⟨*proof*⟩

A basis $f_1, \ldots, f_m$ of $\mathbb{C}^m$ is a list whose column matrix F is invertible; the coordinates use $G = F^{-1}$.

definition *right-coord* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.vec* ⇒ *nat* ⇒ *complex Matrix.vec* **where**
right-coord k m G u j = Matrix.vec k (λ*i*. ∑ *l*<*m*. *u* $ (*i* ∗ *m* + *l*) ∗ *G* $$ (*j*,*l*))

definition *left-coord* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.vec* ⇒ *nat* ⇒ *complex Matrix.vec* **where**
left-coord k m Hm u i = Matrix.vec m (λ*l*. ∑ *r*<*k*. *Hm* $$ (*i*,*r*) ∗ *u* $ (*r* ∗ *m* + *l*))

lemma *basis-inverse*:
fixes *fs* :: *complex Matrix.vec list*
assumes *fd*: ∀ *f* ∈ *set fs*. *dim-vec f* = *m length fs* = *m* **and** *inv*: *invertible-mat* (*mat-of-cols m fs*)
obtains *G* **where** *G* ∈ *carrier-mat m m G* ∗ *mat-of-cols m fs* = *1*$_m$ *m mat-of-cols m fs* ∗ *G* = *1*$_m$ *m*
⟨*proof*⟩

lemma *inv-entry-left*:
fixes *fs* :: *complex Matrix.vec list*
assumes *fd*: ∀ *f* ∈ *set fs*. *dim-vec f* = *m length fs* = *m* **and** *G*: *G* ∈ *carrier-mat m m*
and *GF*: *G* ∗ *mat-of-cols m fs* = *1*$_m$ *m* **and** *j*: *j* < *m* **and** *j'*: *j'* < *m*
shows (∑ *l*<*m*. *fs* ! *j'* $ *l* ∗ *G* $$ (*j*,*l*)) = (*if j* = *j' then 1 else 0*)
⟨*proof*⟩

lemma *inv-entry-right*:
fixes *fs* :: *complex Matrix.vec list*
assumes *fd*: ∀ *f* ∈ *set fs*. *dim-vec f* = *m length fs* = *m* **and** *G*: *G* ∈ *carrier-mat m m*
and *FG*: *mat-of-cols m fs* ∗ *G* = *1*$_m$ *m* **and** *r*: *r* < *m* **and** *l*: *l* < *m*
shows (∑ *j*<*m*. *fs* ! *j* $ *r* ∗ *G* $$ (*j*,*l*)) = (*if r* = *l then 1 else 0*)
⟨*proof*⟩

lemma *right-coord-unique*:

assumes *fd*: $\forall f \in$ *set fs*. *dim-vec f* $=$ *m length fs* $=$ *m* **and** *G*: *G* $\in$ *carrier-mat m m*
and *GF*: *G* $*$ *mat-of-cols m fs* $=$ 1_m *m*
and *xs*: *length xs* $=$ *m* $\forall x \in$ *set xs*. *dim-vec x* $=$ *k* **and** *j*: *j* $<$ *m*
shows *right-coord k m G* (*tensor-combination* (*k* $*$ *m*) *xs fs*) *j* $=$ *xs* ! *j*
⟨*proof*⟩

lemma *right-coord-exists*:
assumes *fd*: $\forall f \in$ *set fs*. *dim-vec f* $=$ *m length fs* $=$ *m* **and** *G*: *G* $\in$ *carrier-mat m m*
and *FG*: *mat-of-cols m fs* $*$ *G* $=$ 1_m *m* **and** *u*: *dim-vec u* $=$ *k* $*$ *m*
shows *tensor-combination* (*k* $*$ *m*) (*map* (*right-coord k m G u*) [*0*..<*m*]) *fs* $=$ *u*
⟨*proof*⟩

D6-3 (i): for a basis $f_1, \ldots, f_m$ of $\mathbb{C}^m$, every $u \in \mathbb{C}^k \otimes \mathbb{C}^m$ is $\sum_j x_j \otimes f_j$ for unique $x_1, \ldots, x_m \in \mathbb{C}^k$.

theorem *tensor-right-coordinates*:
assumes *fd*: $\forall f \in$ *set fs*. *dim-vec f* $=$ *m length fs* $=$ *m* **and** *inv*: *invertible-mat* (*mat-of-cols m fs*)
and *u*: *dim-vec u* $=$ *k* $*$ *m*
shows $\exists!$*xs*. *length xs* $=$ *m* $\wedge$ ($\forall x \in$ *set xs*. *dim-vec x* $=$ *k*) $\wedge$ *u* $=$ *tensor-combination* (*k* $*$ *m*) *xs fs*
⟨*proof*⟩

lemma *left-coord-unique*:
assumes *ed*: $\forall e \in$ *set es*. *dim-vec e* $=$ *k length es* $=$ *k* **and** *Hm*: *Hm* $\in$ *carrier-mat k k*
and *HE*: *Hm* $*$ *mat-of-cols k es* $=$ 1_m *k*
and *ys*: *length ys* $=$ *k* $\forall y \in$ *set ys*. *dim-vec y* $=$ *m* **and** *i*: *i* $<$ *k*
shows *left-coord k m Hm* (*tensor-combination* (*k* $*$ *m*) *es ys*) *i* $=$ *ys* ! *i*
⟨*proof*⟩

lemma *left-coord-exists*:
assumes *ed*: $\forall e \in$ *set es*. *dim-vec e* $=$ *k length es* $=$ *k* **and** *Hm*: *Hm* $\in$ *carrier-mat k k*
and *EH*: *mat-of-cols k es* $*$ *Hm* $=$ 1_m *k* **and** *u*: *dim-vec u* $=$ *k* $*$ *m*
shows *tensor-combination* (*k* $*$ *m*) *es* (*map* (*left-coord k m Hm u*) [*0*..<*k*]) $=$ *u*
⟨*proof*⟩

D6-3 (ii): likewise $u = \sum_i e_i \otimes y_i$ for unique $y_1, \ldots, y_k \in \mathbb{C}^m$.

theorem *tensor-left-coordinates*:
assumes *ed*: $\forall e \in$ *set es*. *dim-vec e* $=$ *k length es* $=$ *k* **and** *inv*: *invertible-mat* (*mat-of-cols k es*)
and *u*: *dim-vec u* $=$ *k* $*$ *m*
shows $\exists!$*ys*. *length ys* $=$ *k* $\wedge$ ($\forall y \in$ *set ys*. *dim-vec y* $=$ *m*) $\wedge$ *u* $=$ *tensor-combination* (*k* $*$ *m*) *es ys*
⟨*proof*⟩

Corollary: $\mathbb{C}^k \otimes \mathbb{C}^m \cong \mathbb{C}^k \oplus \cdots \oplus \mathbb{C}^k$ (m times) by the linear bijection $(x_1, \ldots, x_m) \mapsto \sum_j x_j \otimes f_j$, and $\cong \mathbb{C}^m \oplus \cdots \oplus \mathbb{C}^m$ (k times) likewise; the identification depends on the chosen basis.

lemma *tensor-combination-add-left*:
assumes *l*: *length xs* $=$ *length fs length xs′* $=$ *length fs*
and *d*: $\forall x \in$ *set xs*. *dim-vec x* $=$ *k* $\forall x \in$ *set xs′*. *dim-vec x* $=$ *k* **and** *fd*: $\forall y \in$ *set fs*. *dim-vec y* $=$ *m*
shows *tensor-combination* (*k* $*$ *m*) (*map2* (+) *xs xs′*) *fs* $=$ *tensor-combination* (*k* $*$ *m*) *xs fs* $+$ *tensor-combination* (*k* $*$ *m*) *xs′ fs*
⟨*proof*⟩

lemma *tensor-combination-smult-left*:
assumes *l*: *length xs* $=$ *length fs* **and** *d*: $\forall x \in$ *set xs*. *dim-vec x* $=$ *k* **and** *fd*: $\forall y \in$ *set fs*. *dim-vec y* $=$ *m*
shows *tensor-combination* (*k* $*$ *m*) (*map* (λx. *a* $\cdot_v$ *x*) *xs*) *fs* $=$ *a* $\cdot_v$ *tensor-combination* (*k* $*$ *m*) *xs fs*
⟨*proof*⟩

corollary *tensor-iso-direct-sum*:
assumes *fd*: $\forall f \in$ *set fs*. *dim-vec f* $=$ *m length fs* $=$ *m* **and** *inv*: *invertible-mat* (*mat-of-cols m fs*)
shows *bij-betw* (λxs. *tensor-combination* (*k* $*$ *m*) *xs fs*)

$\{xs.\ length\ xs = m \wedge (\forall x \in set\ xs.\ dim\text{-}vec\ x = k)\}\ (carrier\text{-}vec\ (k * m))$
⟨*proof*⟩

15.6 Day 8: non-orthogonal states cannot be distinguished

lemma *meas-sum-norm*:
assumes *ms*: *measurement-system n Ms* **and** *v*: *dim-vec v = n*
shows $(\sum l{<}length\ Ms.\ \|Ms\ !\ l *_v v\|^2) = \|v\|^2$
⟨*proof*⟩

lemma *inner-minus-smult*:
assumes *d1*: *dim-vec u = n* **and** *d2*: *dim-vec w = n*
shows $\langle w - a \cdot_v u \mid w - a \cdot_v u\rangle = \langle w|w\rangle - a * \langle w|u\rangle - cnj\ a * \langle u|w\rangle + a * cnj\ a * \langle u|u\rangle$
⟨*proof*⟩

lemma *norm-sq-minus-component*:
assumes *d1*: *dim-vec u = n* **and** *d2*: *dim-vec w = n* **and** *u*: $\|u\| = 1$
shows $\|w - \langle u|w\rangle \cdot_v u\|^2 = \|w\|^2 - (cmod\ \langle u|w\rangle)^2$
⟨*proof*⟩

D8-2: if outcome i certainly occurs on ψ_1, it occurs on ψ_2 with probability at least $|\langle\psi_1|\psi_2\rangle|^2$.

theorem *nonorthogonal-outcome-prob*:
assumes *ms*: *measurement-system n Ms* **and** *q1*: *quantum-state n ψ1* **and** *q2*: *quantum-state n ψ2*
and *i*: *i < length Ms* **and** *p1*: *meas-prob (Ms ! i) ψ1 = 1*
shows $(cmod\ \langle\psi 1|\psi 2\rangle)^2 \leq meas\text{-}prob\ (Ms\ !\ i)\ \psi 2$
⟨*proof*⟩

corollary *nonorthogonal-not-distinguishable*:
assumes *ms*: *measurement-system n Ms* **and** *q1*: *quantum-state n ψ1* **and** *q2*: *quantum-state n ψ2*
and *no*: $\langle\psi 1|\psi 2\rangle \neq 0$ **and** *i*: *i < length Ms* **and** *p1*: *meas-prob (Ms ! i) ψ1 = 1*
shows *meas-prob (Ms ! i) ψ2 > 0*
⟨*proof*⟩

15.7 Day 9: measuring one half of a Bell pair

definition *e0* :: *complex Matrix.vec* **where** *e0 = unit-vec 2 0*
definition *e1* :: *complex Matrix.vec* **where** *e1 = unit-vec 2 1*
definition *proj0* :: *complex Matrix.mat* **where** *proj0 = e0 ⊘ e0*
definition *proj1* :: *complex Matrix.mat* **where** *proj1 = e1 ⊘ e1*

lemma *e-dims* [*simp*]: *dim-vec e0 = 2 dim-vec e1 = 2*
⟨*proof*⟩

lemma *e-inner* [*simp*]: $\langle e0|e0\rangle = 1\ \langle e0|e1\rangle = 0\ \langle e1|e0\rangle = 0\ \langle e1|e1\rangle = 1$
⟨*proof*⟩

lemma *proj-carrier* [*simp*]: *proj0 ∈ carrier-mat 2 2 proj1 ∈ carrier-mat 2 2*
⟨*proof*⟩

lemma *proj-apply* [*simp*]:
$proj0 *_v e0 = e0\ proj0 *_v e1 = 0_v\ 2\ proj1 *_v e0 = 0_v\ 2\ proj1 *_v e1 = e1$
⟨*proof*⟩

lemma *bell-state-e*: *bell-state = (1 / csqrt 2) ·v (tensor-vec e0 e0 + tensor-vec e1 e1)*
⟨*proof*⟩

lemma *tensor-zero-left*: *dim-vec w = 2 ⟹ tensor-vec* $(0_v\ 2)\ w = 0_v\ 4$
⟨*proof*⟩

lemma *tensor-zero-right*: *dim-vec v = 2 ⟹ tensor-vec v* (0_v *2*) = 0_v *4*
⟨*proof*⟩

lemma *A-meas-bell*:
(*proj0* ⨂ 1_m *2*) $*_v$ *bell-state* = (*1 / csqrt 2*) $\cdot_v$ *tensor-vec e0 e0*
(*proj1* ⨂ 1_m *2*) $*_v$ *bell-state* = (*1 / csqrt 2*) $\cdot_v$ *tensor-vec e1 e1*
⟨*proof*⟩

lemma *tensor-e-norm*: ‖*tensor-vec e0 e0*‖ = *1* ‖*tensor-vec e1 e1*‖ = *1*
⟨*proof*⟩

D9-6: A measures $|0\rangle\langle 0|, |1\rangle\langle 1|$ on the shared state γ: each outcome has probability $\frac{1}{2}$, and γ collapses to $e_0 \otimes e_0$ resp. $e_1 \otimes e_1$, after which B's measurement $I \otimes M_m$ gives A's outcome with probability 1.

theorem *bell-measurement*:
meas-prob (*proj0* ⨂ 1_m *2*) *bell-state* = *1/2*
meas-prob (*proj1* ⨂ 1_m *2*) *bell-state* = *1/2*
post-meas-state (*proj0* ⨂ 1_m *2*) *bell-state* = *tensor-vec e0 e0*
post-meas-state (*proj1* ⨂ 1_m *2*) *bell-state* = *tensor-vec e1 e1*
meas-prob (1_m *2* ⨂ *proj0*) (*tensor-vec e0 e0*) = *1*
meas-prob (1_m *2* ⨂ *proj1*) (*tensor-vec e1 e1*) = *1*
⟨*proof*⟩

lemma *proj-explicit*:
proj0 = *Matrix.mat 2 2* (λ(*i*,*j*). *if i = 0 ∧ j = 0 then 1 else 0*)
proj1 = *Matrix.mat 2 2* (λ(*i*,*j*). *if i = 1 ∧ j = 1 then 1 else 0*)
⟨*proof*⟩

lemma *proj-measurement-system*: *measurement-system 2* [*proj0*, *proj1*]
⟨*proof*⟩

15.8 Day 11: SWAP from three CNOTs

lemma *less4-cases*: (*i*::*nat*) < *4* ⟷ *i = 0 ∨ i = 1 ∨ i = 2 ∨ i = 3*
⟨*proof*⟩

CNOT′ is CNOT controlled by the second qubit.

definition *CNOT′* :: *complex Matrix.mat* **where**
CNOT′ = *Matrix.mat 4 4* (λ(*i*,*j*). *if* (*i*,*j*) ∈ {(*0*,*0*),(*1*,*3*),(*2*,*2*),(*3*,*1*)} *then 1 else 0*)

D11-3: SWAP = CNOT · CNOT′ · CNOT (*SWAP* is the AFP's).

theorem *SWAP-three-CNOTs*: *SWAP* = *CNOT* ∗ *CNOT′* ∗ *CNOT*
⟨*proof*⟩

lemma *SWAP-basis*:
assumes *a < 2 b < 2*
shows *SWAP* $*_v$ *tensor-vec* (*unit-vec 2 a*) (*unit-vec 2 b*) = *tensor-vec* (*unit-vec 2 b*) (*unit-vec 2 a*)
⟨*proof*⟩

lemma *CNOT′-basis*:
assumes *a < 2 b < 2*
shows *CNOT′* $*_v$ *tensor-vec* (*unit-vec 2 a*) (*unit-vec 2 b*) = *tensor-vec* (*unit-vec 2* ((*a* + *b*) *mod 2*)) (*unit-vec 2 b*)
⟨*proof*⟩

end

Part II

Lectures 11–22 and 29–34: quantum computing, density matrices, completely positive maps, error correction

theory *Quantum-Computing11*
imports
Vern-Paulsen-QC-3-10.Basic-Math
Vern-Paulsen-QC-3-10.Quantum-Computing-3-10
begin

16 Day 11 (21 September 2011): gates revisited, circuit diagrams, no-cloning

The lecture opens with two corrections: $H^2 = I$ (not X), with $He_0 = (e_0 + e_1)/\sqrt{2}$, $Xe_0 = e_1$, $Xe_1 = e_0$, $He_1 = (e_0 - e_1)/\sqrt{2}$, so that X and H are reflections; and the truth tables of CNOT ($|a, b\rangle \mapsto |a, b + a\rangle$, arithmetic in $\mathbb{Z}_2$) and of CCNOT ($|a, b, c\rangle \mapsto |a, b, c + ab\rangle$). Circuit diagrams are introduced; the CNOT with control on the second wire has matrix $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$, and three alternating CNOTs compose to the swap of the two wires (*SWAP-three-CNOTs* in theory Day02_11_Supplement).

Definition (notes). The Toffoli gate CCNOT is the 8×8 permutation matrix that fixes the first six basis vectors and exchanges $|110\rangle$ and $|111\rangle$.

definition *CCNOT* :: *complex Matrix.mat* **where**
CCNOT ≡ *Matrix.mat 8 8*
(λ(*i*,*j*). *if* (*i* = *6* ∧ *j* = *7*) ∨ (*i* = *7* ∧ *j* = *6*)
then 1
else if i = *j* ∧ *i* < *6 then 1 else 0*)

lemma *CCNOT-dagger* [*simp*]:
$CCNOT^\dagger = CCNOT$
⟨*proof*⟩

lemma *CCNOT-dim-row* [*simp*]:
dim-row CCNOT = *8*
⟨*proof*⟩

lemma *CCNOT-dim-col* [*simp*]:
dim-col CCNOT = *8*
⟨*proof*⟩

lemma *CCNOT-inv* [*simp*]:
CCNOT ∗ *CCNOT* = 1_m *8*
⟨*proof*⟩

lemma *CCNOT-unitary*: *unitary CCNOT*
⟨*proof*⟩

16.1 Cloning and no-cloning

CNOT clones the computational basis: $U(e_a \otimes e_0) = e_a \otimes e_a$, and more generally $|i_1 \cdots i_n\rangle \otimes |0 \cdots 0\rangle \mapsto |i_1 \cdots i_n\rangle \otimes |i_1 \cdots i_n\rangle$ for some unitary. But for $\psi = \alpha e_0 + \beta e_1$ with $\alpha\beta \neq 0$, $U(\psi \otimes e_0) \neq \psi \otimes \psi$.

Theorem (notes) (No-cloning)**.** There is no state ϕ and unitary U on $\mathcal{H} \otimes \mathcal{H}$ with $U(\psi \otimes \phi) = \psi \otimes \psi$ for all $\psi \in \mathcal{H}$.

The notes argue with $-\psi$; the formal proof (for $\dim \mathcal{H} = n \geq 2$ and unit vectors ψ) uses two states with inner product neither 0 nor 1.

definition *is-cloner* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.vec* ⇒ *bool* **where**
is-cloner n U φ ≡
U ∈ *carrier-mat* (*n*^2) (*n*^2) ∧ *unitary U* ∧ *dim-vec φ* = *n* ∧
(∀ *ψ*. *quantum-state n ψ* ⟶
U ∗*v tensor-vec ψ φ* = *tensor-vec ψ ψ*)

Two states whose inner product is neither 0 nor 1 exist once $n \geq 2$.

lemma *two-states-nontrivial-inner-prod*:
assumes *n*: *2* ≤ *n*
obtains *ψ*1 *ψ*2 :: *complex Matrix.vec*
where *quantum-state n ψ*1 **and** *quantum-state n ψ*2
and ⟨*ψ*1 | *ψ*2⟩ ≠ (*0* :: *real*) **and** ⟨*ψ*1 | *ψ*2⟩ ≠ (*1* :: *real*)
⟨*proof*⟩

theorem *no-cloning*:
assumes *n2*: *2* ≤ *n*
shows ¬ (∃ *U φ*. *is-cloner n U φ*)
⟨*proof*⟩

17 Day 12 (23 September 2011): quantum parallelism

Since $He_0 = (e_0 + e_1)/\sqrt{2}$,

$$H^{\otimes N}(e_0 \otimes \cdots \otimes e_0) = \left(\tfrac{1}{\sqrt{2}}\right)^N \sum_{J \in \mathbb{Z}_2^N} e_J = \left(\tfrac{1}{\sqrt{2}}\right)^N \sum_J |J\rangle.$$

Only the case $N = 1$ is stated here; the general identity is *hadamard-uniform-N* in *Day12-Counting-Probability* (with *tensor-pow-mat* for $H^{\otimes N}$).

Definition (notes) (Oracle)**.** For $f : \mathbb{Z}_2^N \to \mathbb{Z}_2$, the vectors $e_J \otimes e_{i+f(J)}$ form an orthonormal basis of $(\mathbb{C}^2)^{\otimes N} \otimes \mathbb{C}^2$, so there is a unitary U_f with $U_f(e_J \otimes e_i) = e_J \otimes e_{i+f(J)}$, the *oracle* for f.

Proposition (notes). For a unit vector γ, the measurement $P_0 = |\gamma\rangle\langle\gamma|$, $P_1 = I - P_0$ has $p_0(\psi) = |\langle\gamma|\psi\rangle|^2$ and $p_1(\psi) = 1 - |\langle\gamma|\psi\rangle|^2$.

Application: with $x = (He_0)^{\otimes N}$, $U_f(x \otimes e_0) = 2^{-N/2} \sum_J e_J \otimes e_{f(J)}$, and measuring against $\gamma = x \otimes e_0$ gives $\langle\gamma|U_f(x \otimes e_0)\rangle = M/2^N$ with $M = \#\{J : f(J) = 0\}$, hence $p_0 = (M/2^N)^2$: repeating the experiment estimates M (a Bernoulli trial). The existence of the oracle is formalised below; the proposition is *rank-one-meas-prob* (Day 8); the counting identity is *counting-probability* in *Day12-Counting-Probability*.

lemma *hadamard-uniform-1*:
Hadamard $*_v$ *unit-vec 2 0* =
(*1/csqrt 2*) $\cdot_v$ (*unit-vec 2 0* + *unit-vec 2 1*)
⟨*proof*⟩

definition *oracle-unitary* :: (*nat* ⇒ *nat*) ⇒ *nat* ⇒ *bool* **where**
oracle-unitary f N ≡
∃ *U* :: *complex Matrix.mat*.
U ∈ *carrier-mat* (*2^(N+1)*) (*2^(N+1)*) ∧
unitary U ∧
(∀ *J* < *2^N*. ∀ *i* < *2*.
U $*_v$ (*tensor-vec* (*unit-vec* (*2^N*) *J*) (*unit-vec 2 i*)) =
tensor-vec (*unit-vec* (*2^N*) *J*) (*unit-vec 2* ((*i* + *f J*) *mod 2*)))

theorem *oracle-exists*:
fixes *f* :: *nat* ⇒ *nat*
assumes *f-range*: ∀ *J* < *2^(N* :: *nat*). *f J* < *2*
shows *oracle-unitary f N*
⟨*proof*⟩

$M = \#\{J < 2^N : f(J) = 0\}$, the quantity estimated by the counting experiment.

definition *count-zeros* :: (*nat* ⇒ *nat*) ⇒ *nat* ⇒ *nat* **where**
count-zeros f N ≡ *card* {*J*. *J* < *2^N* ∧ *f J* = *0*}

18 Day 13 (26 September 2011): ensembles and density matrices

18.1 Ensembles

Motivation: measuring $\psi = a|0\rangle + b|1\rangle$ with $M_0 = |0\rangle\langle 0|$, $M_1 = |1\rangle\langle 1|$ leaves the state $\frac{a}{|a|}|0\rangle$ with probability $|a|^2$ and $\frac{b}{|b|}|1\rangle$ with probability $|b|^2$; a later measurement $\{\tilde{M}_\alpha\}$ then has outcome probabilities $p_\alpha = |a|^2\|\tilde{M}_\alpha \frac{a}{|a|}|0\rangle\|^2 + |b|^2\|\tilde{M}_\alpha \frac{b}{|b|}|1\rangle\|^2$.

Definition (notes). An *ensemble* $\{p_i, \psi_i\}_{i=1}^L$ is a set of states with probabilities $p_i \geq 0$, $\sum_i p_i = 1$. For a measurement system $\{M_\alpha\}$ the probability of outcome α is $p_\alpha(\{p_i, \psi_i\}) = \sum_i p_i \|M_\alpha \psi_i\|^2$.

A second source of ensembles: a unit vector $\psi = \sum_l \phi_l \otimes f_l \in \mathcal{H}_A \otimes \mathcal{H}_E$ (environment $\mathcal{H}_E$ with orthonormal basis f_l) behaves, for measurements $M_\alpha \otimes I$, exactly like the ensemble $\{\|\phi_l\|^2, \phi_l/\|\phi_l\|\}$.

definition *ensemble* :: *nat* ⇒ (*real* × *complex Matrix.vec*) *list* ⇒ *bool* **where**
ensemble n E ≡
(∀ (*p*, *ψ*) ∈ *set E*. *p* ≥ *0* ∧ *quantum-state n ψ*) ∧
(∑ (*p*, -) ← *E*. *p*) = *1*

18.2 Von Neumann's density matrix

Measurements cannot distinguish ψ from $e^{i\theta}\psi$, and $|e^{i\theta}\psi\rangle\langle e^{i\theta}\psi| = |\psi\rangle\langle\psi| = (\alpha_i\overline{\alpha_j})$: the object attached to a state is the projection, not the vector. For any γ, $\|\gamma\|^2 = \mathrm{Tr}\,|\gamma\rangle\langle\gamma|$, and

$$p_\alpha(\psi) = \|M_\alpha\psi\|^2 = \mathrm{Tr}\big((M_\alpha^* M_\alpha)(\psi\psi^*)\big), \qquad p_\alpha(\{p_l, \psi_l\}) = \mathrm{Tr}\Big((M_\alpha^* M_\alpha)\sum_l p_l \psi_l \psi_l^*\Big).$$

Definition (notes). $P = \sum_l p_l |\psi_l\rangle\langle\psi_l|$ is the *density matrix* of the ensemble.

Formally *pure-density* ψ is $|\psi\rangle\langle\psi|$, *ensemble-density* n E is P, and *density-matrix* n ϱ is the Day 14 characterisation ($\rho \geq 0$, $\mathrm{Tr}\,\rho = 1$).

definition *density-matrix* :: *nat* ⇒ *complex Matrix.mat* ⇒ *bool* **where**
density-matrix n ϱ ≡ ϱ ∈ carrier-mat n n ∧ psd ϱ ∧ Tr ϱ = 1

definition *pure-density* :: *complex Matrix.vec* ⇒ *complex Matrix.mat* **where**
pure-density ψ ≡
let n = dim-vec ψ in
outer-sum n [ψ]

lemma *Tr-outer-sum-single*:
assumes *dim*: *dim-vec ψ = n*
shows *Tr* (*outer-sum n [ψ]*) = ⟨*ψ* | *ψ*⟩
⟨*proof*⟩

lemma *pure-density-is-density-matrix*:
assumes *quantum-state n ψ*
shows *density-matrix n* (*pure-density ψ*)
⟨*proof*⟩

definition *ensemble-density* :: *nat* ⇒ (*real* × *complex Matrix.vec*) *list* ⇒ *complex Matrix.mat* **where**
ensemble-density n E ≡
foldr (λ(*p*,*ψ*) *acc*. *complex-of-real p* ·$_m$ *outer-sum n* [*ψ*] + *acc*)
E (*0*$_m$ *n n*)

lemma *psd-zero-mat* [*simp*]:
psd (*0*$_m$ *n n*)
⟨*proof*⟩

lemma *psd-smult-nonneg*:
assumes *c*: *0* ≤ *c*
and *A*: *psd A* **and** *Ac*: *A* ∈ *carrier-mat n n*
shows *psd* (*complex-of-real c* ·$_m$ *A*)
⟨*proof*⟩

lemma *psd-add-psd*:
assumes *A*: *psd A* **and** *B*: *psd B*
and *Ac*: *A* ∈ *carrier-mat n n* **and** *Bc*: *B* ∈ *carrier-mat n n*
shows *psd* (*A* + *B*)
⟨*proof*⟩

lemma *ensemble-density-Nil* [*simp*]:
ensemble-density n [] = *0*$_m$ *n n*
⟨*proof*⟩

lemma *ensemble-density-Cons*:
ensemble-density n ((*p*, *ψ*) # *E*) =
complex-of-real p ·$_m$ *outer-sum n* [*ψ*] + *ensemble-density n E*
⟨*proof*⟩

lemma *ensemble-density-carrier*:
assumes ∀ (*p*, *ψ*) ∈ *set E*. *quantum-state n ψ*
shows *ensemble-density n E* ∈ *carrier-mat n n*

⟨*proof*⟩

lemma *ensemble-density-psd-aux*:
assumes $\forall (p, \psi) \in$ *set E*. $p \geq 0 \wedge$ *quantum-state n* ψ
shows *psd* (*ensemble-density n E*)
⟨*proof*⟩

lemma *Tr-zero-mat'* [*simp*]:
Tr (0_m *n n*) = *0*
⟨*proof*⟩

lemma *ensemble-density-Tr*:
assumes $\forall (p, \psi) \in$ *set E*. *quantum-state n* ψ
shows *Tr* (*ensemble-density n E*) = *complex-of-real* ($\sum (p, \text{-}) \leftarrow E.\ p$)
⟨*proof*⟩

theorem *ensemble-density-is-dm*:
assumes *E*: *ensemble n E*
shows *density-matrix n* (*ensemble-density n E*)
⟨*proof*⟩

18.3 Probabilities via the trace

$p_\alpha(\psi) = \|M_\alpha \psi\|^2 = \mathrm{Tr}(M_\alpha^* M_\alpha |\psi\rangle\langle\psi|)$. For an ensemble with density matrix ρ, $p_\alpha = \mathrm{Tr}(M_\alpha^* M_\alpha \rho) = \langle M_\alpha^* M_\alpha | \rho \rangle$ (*ensemble-prob-trace-eq* below).

lemma *pure-state-prob-trace*:
fixes *M* :: *complex Matrix.mat* **and** ψ :: *complex Matrix.vec*
assumes *st*: *quantum-state n* ψ
and *Msq*: *M* $\in$ *carrier-mat n n*
shows *meas-prob M* ψ = *Re* (*Tr* (($M^\dagger$) $*$ *M* $*$ *outer-sum n* [ψ]))
⟨*proof*⟩

For an ensemble: $p_\alpha(\{p_i, \psi_i\}) = \sum_i p_i \|M_\alpha \psi_i\|^2 = \mathrm{Tr}(M_\alpha^* M_\alpha \rho)$.

lemma *ensemble-prob-trace-eq*:
fixes *M* :: *complex Matrix.mat*
assumes *M*: *M* $\in$ *carrier-mat n n*
and *E*: $\forall (p, \psi) \in$ *set E*. *quantum-state n* ψ
shows ($\sum (p, \psi) \leftarrow E.\ p *$ *meas-prob M* ψ)
= *Re* (*Tr* ((($M^\dagger$) $*$ *M*) $*$ *ensemble-density n E*))
⟨*proof*⟩

lemma *Tr-psd-Re-nonneg*:
assumes *P*: *psd P* **and** *Pc*: *P* $\in$ *carrier-mat n n*
shows *0* $\leq$ *Re* (*Tr P*)
⟨*proof*⟩

lemma *ensemble-prob-trace*:
fixes *Ms* :: *complex Matrix.mat list*
assumes *ms*: *measurement-system n Ms*
and *dm*: *density-matrix n* ϱ
and *Min*: *M* $\in$ *set Ms*
shows *Re* (*Tr* (($M^\dagger$) $*$ *M* $*$ ϱ)) $\geq$ *0*
⟨*proof*⟩

19 Day 14 (28 September 2011): density matrices, pure states, composite ensembles

Proposition (notes). $\rho \in L(\mathcal{H}_A)$ is the density matrix of some ensemble if and only if $\rho \geq 0$ and $\operatorname{Tr}\rho = 1$.

Definition (notes). ρ is a *density matrix* when $\rho \geq 0$ and $\operatorname{Tr}\rho = 1$. The ensemble $\{\lambda_i, \psi_i\}$ of eigenvalues and an orthonormal eigenbasis is the *spectral ensemble*.

Many ensembles have the same density matrix: writing $\rho = XX^* = \sum_i c_i c_i^*$ with columns c_i gives the ensemble $\{\|c_i\|^2, c_i/\|c_i\|\}$.

theorem *density-matrix-characterisation*:
fixes ϱ :: *complex Matrix.mat*
assumes *sq*: $\varrho \in$ *carrier-mat n n*
shows *density-matrix n* $\varrho \longleftrightarrow$ *psd* $\varrho \wedge$ *Tr* ϱ = *1*
⟨*proof*⟩

lemma *ensemble-density-carrier-dim*:
shows *ensemble-density n E* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *outer-sum-singleton-index*:
assumes *r*: $r < n$ **and** *c*: $c < n$
shows *outer-sum n* [ψ] $$ (*r*, *c*) = ψ $ *r* ∗ *cnj* (ψ $ *c*)
⟨*proof*⟩

Day 13: $|e^{i\theta}\psi\rangle\langle e^{i\theta}\psi| = |\psi\rangle\langle\psi|$; the density matrix forgets the phase.

lemma *pure-density-smult-unit*:
fixes ψ :: *complex Matrix.vec* **and** *c* :: *complex*
assumes *c*: *cmod c* = *1*
shows *pure-density* (*c* ·$_v$ ψ) = *pure-density* ψ
⟨*proof*⟩

lemma *ensemble-density-index*:
assumes *r*: $r < n$ **and** *c*: $c < n$
shows *ensemble-density n E* $$ (*r*, *c*) =
(∑ (*p*, ψ) ← *E*. *complex-of-real p* ∗ ψ $ *r* ∗ *cnj* (ψ $ *c*))
⟨*proof*⟩

The spectral ensemble: every density matrix comes from an ensemble.

theorem *spectral-ensemble-exists*:
fixes ϱ :: *complex Matrix.mat*
assumes *dm*: *density-matrix n* ϱ
shows ∃ *E*. *ensemble n E* ∧ *ensemble-density n E* = ϱ
⟨*proof*⟩

corollary *density-matrix-iff-ensemble*:
fixes ϱ :: *complex Matrix.mat*
shows *density-matrix n* $\varrho \longleftrightarrow$
(∃ *E*. *ensemble n E* ∧ *ensemble-density n E* = ϱ)
⟨*proof*⟩

19.1 Pure states

If $\rho = |\psi\rangle\langle\psi| = \sum_i p_i |\psi_i\rangle\langle\psi_i|$ then for $\gamma \perp \psi$, $0 = \langle\gamma|\rho\gamma\rangle = \sum_i p_i |\langle\psi_i|\gamma\rangle|^2$, so every ψ_i with $p_i > 0$ is parallel to ψ: $\psi_i = \beta_i\psi$ with $|\beta_i| = 1$ and $|\psi_i\rangle\langle\psi_i| = |\psi\rangle\langle\psi|$. These density matrices

are the *pure states*.

definition *pure-state-density* :: *nat* ⇒ *complex Matrix.mat* ⇒ *bool* **where**
  pure-state-density n ϱ ≡
    density-matrix n ϱ ∧
    (∃ ψ. *quantum-state n ψ* ∧ *ϱ* = *pure-density ψ*)

lemma *pure-state-density-is-density*:
  assumes *pure-state-density n ϱ*
  shows *density-matrix n ϱ*
  ⟨*proof*⟩

lemma *pure-density-is-pure-state-density*:
  assumes *quantum-state n ψ*
  shows *pure-state-density n* (*pure-density ψ*)
  ⟨*proof*⟩

A companion fact not in the notes: a density matrix is pure iff it is idempotent.

theorem *pure-density-iff-idempotent*:
  fixes *ϱ* :: *complex Matrix.mat*
  assumes *dm*: *density-matrix n ϱ*
  shows *pure-state-density n ϱ* ⟷ *ϱ* ∗ *ϱ* = *ϱ*
⟨*proof*⟩

lemma *outer-sum-singleton-mult-vec*:
  assumes *ψ-dim*: *dim-vec ψ* = *n* **and** *γ-dim*: *dim-vec γ* = *n*
  shows *outer-sum n* [*ψ*] ∗*v γ* = ⟨*ψ*|*γ*⟩ ·*v ψ*
⟨*proof*⟩

lemma *ensemble-density-quad-form*:
  assumes *E-dim*: ∀ (*p*, *ψ*) ∈ *set E*. *dim-vec ψ* = *n*
    and *γ-dim*: *dim-vec γ* = *n*
  shows ⟨*γ* | *ensemble-density n E* ∗*v γ*⟩ =
      (∑ (*p*, *ψ*) ← *E*. *complex-of-real p* ∗ ⟨*ψ*|*γ*⟩ ∗ *cnj* ⟨*ψ*|*γ*⟩)
  ⟨*proof*⟩

theorem *pure-ensemble-parallel*:
  fixes *ψ* :: *complex Matrix.vec*
  assumes *st*: *quantum-state n ψ*
    and *ens*: *ensemble n E*
    and *dens*: *ensemble-density n E* = *pure-density ψ*
    and *mem*: (*p*, *ψi*) ∈ *set E*
    and *ppos*: *0* < *p*
  shows ∃ *β*. *ψi* = *β* ·*v ψ*
⟨*proof*⟩

19.2 Composite ensembles

For ensembles $\{p_i, \psi_i\}$ in lab A and $\{q_l, \phi_l\}$ in lab B the composite is the ensemble $\{p_i q_l, \psi_i \otimes \phi_l\}$ with density matrix $\rho = \sum_{i,l} p_i q_l |\psi_i\rangle\langle\psi_i| \otimes |\phi_l\rangle\langle\phi_l|$, because $|\psi \otimes \phi\rangle\langle\psi \otimes \phi| = |\psi\rangle\langle\psi| \otimes |\phi\rangle\langle\phi|$ (checked on elementary tensors $h \otimes k$). The identity is formalised below; that $\rho_1 \otimes \rho_2$ is again a density matrix is *density-matrix-tensor* (Day 15).

lemma *tensor-pure-density*:
  fixes *ψ φ* :: *complex Matrix.vec*

assumes *ψ-dim*: *dim-vec ψ = m*
and *φ-dim*: *dim-vec φ = p*
and *m-pos*: *0 < m* **and** *p-pos*: *0 < p*
shows *pure-density* (*tensor-vec ψ φ*) =
(*pure-density ψ*) ⨂ (*pure-density φ*)
⟨*proof*⟩

19.3 Partial traces: motivation

For a density matrix $\rho \in L(\mathcal{H}_A \otimes \mathcal{H}_B)$ and measurements $\{M_\alpha\}$ in lab A, which act as $M_\alpha \otimes I$, $p_\alpha(\rho) = \mathrm{Tr}((M_\alpha^* M_\alpha \otimes I)\rho)$. The notes announce a density matrix $\rho_A \in L(\mathcal{H}_A)$ with $\mathrm{Tr}_{\mathcal{H}_A \otimes \mathcal{H}_B}((M^*M \otimes I)\rho) = \mathrm{Tr}_{\mathcal{H}_A}(M^* M \rho_A)$ for all M; it is constructed on Day 15.

20 Day 15 (30 September 2011): traces, tensor products of operators, partial traces

Proposition (notes). For any orthonormal basis $\{u_1, \ldots, u_n\}$ of $\mathbb{C}^n$, $\mathrm{tr}(A) = \sum_i \langle u_i | A u_i \rangle$.

theorem *trace-via-onb*:
fixes *A* :: *complex Matrix.mat*
assumes *sq*: *A* ∈ *carrier-mat n n*
and *orth*: *orthonormal-list n vs*
and *full*: *length vs = n*
shows *Tr A* = (∑ *i*<*n*. ⟨*vs* ! *i* | *A* ∗$_v$ *vs* ! *i*⟩)
⟨*proof*⟩

Proposition (notes). $\mathrm{tr}(R \otimes T) = \mathrm{tr}(R)\,\mathrm{tr}(T)$.

theorem *trace-tensor*:
fixes *R T* :: *complex Matrix.mat*
assumes *Rsq*: *R* ∈ *carrier-mat m m*
and *Tsq*: *T* ∈ *carrier-mat p p*
and *m-pos*: *0 < m* **and** *p-pos*: *0 < p*
shows *Tr* (*R* ⨂ *T*) = *Tr R* ∗ *Tr T*
⟨*proof*⟩

20.1 The isomorphism $L(\mathcal{H}_A) \otimes L(\mathcal{H}_B) \cong L(\mathcal{H}_A \otimes \mathcal{H}_B)$

Proposition (notes). In finite dimensions the map $\Gamma : L(\mathcal{H}_A) \otimes L(\mathcal{H}_B) \to L(\mathcal{H}_A \otimes \mathcal{H}_B)$, $R \otimes T \mapsto R \otimes T$, is a vector-space isomorphism (both sides have dimension $m^2 p^2$ and Γ is injective).

Formally, $L(\mathcal{H}_A) \otimes L(\mathcal{H}_B)$ is represented in coordinates as M_{m^2,p^2} and Γ as the reindexing $M_{m^2,p^2} \to M_{mp}$; it is an explicit linear isomorphism with explicit inverse.

Postulate $4'$ (Day 16) and the composite ensembles of Day 14: tensor products of positive matrices are positive, and tensor products of density matrices are density matrices.

lemma *psd-tensor*:
fixes *A B* :: *complex Matrix.mat*
assumes *A*: *A* ∈ *carrier-mat m m* **and** *B*: *B* ∈ *carrier-mat p p*
and *pA*: *psd A* **and** *pB*: *psd B*
shows *psd* (*A* ⨂ *B*)
⟨*proof*⟩

lemma *density-matrix-tensor*:
fixes A B :: *complex Matrix.mat*
assumes *dA*: *density-matrix m A* **and** *dB*: *density-matrix p B*
and m: $0 < m$ **and** p: $0 < p$
shows *density-matrix* $(m * p)$ $(A \bigotimes B)$
⟨*proof*⟩

proposition *L-tensor-dim*:
fixes m p :: *nat*
shows $m\hat{}2 * p\hat{}2 = (m * p)\hat{}2$
⟨*proof*⟩

definition *operator-tensor-iso-map* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
operator-tensor-iso-map m p c ≡
Matrix.mat $(m*p)$ $(m*p)$
$(\lambda(r,s).\ c\ \$\$\ ((r\ div\ p) * m + (s\ div\ p), (r\ mod\ p) * p + (s\ mod\ p)))$

lemma *operator-tensor-iso-map-carrier* [*simp*]:
operator-tensor-iso-map m p c ∈ *carrier-mat* $(m*p)$ $(m*p)$
⟨*proof*⟩

lemma *operator-tensor-iso-map-entry*:
assumes $r < m*p$ **and** $s < m*p$
shows *operator-tensor-iso-map m p c* $\$\$\ (r,s) =$
$c\ \$\$\ ((r\ div\ p) * m + (s\ div\ p), (r\ mod\ p) * p + (s\ mod\ p))$
⟨*proof*⟩

lemma *operator-tensor-iso-map-row-bound*:
fixes m p r s :: *nat*
assumes *p-pos*: $0 < p$ **and** $r < m*p$ **and** $s < m*p$
shows $(r\ div\ p) * m + (s\ div\ p) < m*m$
⟨*proof*⟩

lemma *operator-tensor-iso-map-col-bound*:
fixes p r s :: *nat*
assumes *p-pos*: $0 < p$
shows $(r\ mod\ p) * p + (s\ mod\ p) < p*p$
⟨*proof*⟩

definition *operator-tensor-iso-inv* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
operator-tensor-iso-inv m p x ≡
Matrix.mat $(m*m)$ $(p*p)$
$(\lambda(a,b).\ x\ \$\$\ ((a\ div\ m) * p + (b\ div\ p), (a\ mod\ m) * p + (b\ mod\ p)))$

definition *operator-tensor-coordinate-space* ::
nat ⇒ *nat* ⇒ (*complex*, *complex Matrix.mat*) *linear-structure* **where**
operator-tensor-coordinate-space m p ≡ *mat-linear-structure* $(m*m)$ $(p*p)$

definition *bipartite-operator-space* ::
nat ⇒ *nat* ⇒ (*complex*, *complex Matrix.mat*) *linear-structure* **where**
bipartite-operator-space m p ≡ *mat-linear-structure* $(m*p)$ $(m*p)$

lemma *operator-tensor-iso-inv-carrier* [*simp*]:
operator-tensor-iso-inv m p x ∈ *carrier-mat* $(m*m)$ $(p*p)$
⟨*proof*⟩

lemma *operator-tensor-iso-inv-entry*:
assumes $a < m*m$ **and** $b < p*p$

shows *operator-tensor-iso-inv m p x $$ (a,b) =*
*x $$ ((a div m) * p + (b div p), (a mod m) * p + (b mod p))*
⟨*proof*⟩

lemma *operator-tensor-iso-inv-row-bound*:
fixes *m p a b :: nat*
assumes *m-pos*: *0 < m* **and** *p-pos*: *0 < p* **and** *a < m*m* **and** *b < p*p*
shows *(a div m) * p + (b div p) < m*p*
⟨*proof*⟩

lemma *operator-tensor-iso-inv-col-bound*:
fixes *m p a b :: nat*
assumes *m-pos*: *0 < m* **and** *p-pos*: *0 < p*
shows *(a mod m) * p + (b mod p) < m*p*
⟨*proof*⟩

lemma *operator-tensor-iso-map-add*:
assumes *p-pos*: *0 < p*
and *x*: *x ∈ carrier-mat (m*m) (p*p)*
and *y*: *y ∈ carrier-mat (m*m) (p*p)*
shows *operator-tensor-iso-map m p (x + y) =*
operator-tensor-iso-map m p x + operator-tensor-iso-map m p y
⟨*proof*⟩

lemma *operator-tensor-iso-map-zero*:
assumes *p-pos*: *0 < p*
shows *operator-tensor-iso-map m p (0$_m$ (m*m) (p*p)) = 0$_m$ (m*p) (m*p)*
⟨*proof*⟩

lemma *operator-tensor-iso-map-smult*:
assumes *p-pos*: *0 < p*
and *x*: *x ∈ carrier-mat (m*m) (p*p)*
shows *operator-tensor-iso-map m p (c ·$_m$ x) =*
c ·$_m$ operator-tensor-iso-map m p x
⟨*proof*⟩

lemma *operator-tensor-iso-inv-add*:
assumes *m-pos*: *0 < m*
and *p-pos*: *0 < p*
and *x*: *x ∈ carrier-mat (m*p) (m*p)*
and *y*: *y ∈ carrier-mat (m*p) (m*p)*
shows *operator-tensor-iso-inv m p (x + y) =*
operator-tensor-iso-inv m p x + operator-tensor-iso-inv m p y
⟨*proof*⟩

lemma *operator-tensor-iso-inv-zero*:
assumes *m-pos*: *0 < m* **and** *p-pos*: *0 < p*
shows *operator-tensor-iso-inv m p (0$_m$ (m*p) (m*p)) = 0$_m$ (m*m) (p*p)*
⟨*proof*⟩

lemma *operator-tensor-iso-inv-smult*:
assumes *m-pos*: *0 < m*
and *p-pos*: *0 < p*
and *x*: *x ∈ carrier-mat (m*p) (m*p)*
shows *operator-tensor-iso-inv m p (c ·$_m$ x) =*
c ·$_m$ operator-tensor-iso-inv m p x
⟨*proof*⟩

proposition *operator-tensor-space-isomorphism*:
assumes *m-pos*: $0 < m$
and *p-pos*: $0 < p$
shows *is-isomorphism-between*
(*operator-tensor-coordinate-space m p*)
(*bipartite-operator-space m p*)
(*operator-tensor-iso-map m p*)
⟨*proof*⟩

corollary *operator-tensor-space-isomorphic*:
assumes $0 < m$ **and** $0 < p$
shows *is-isomorphic-to*
(*operator-tensor-coordinate-space m p*)
(*bipartite-operator-space m p*)
⟨*proof*⟩

20.2 Partial traces

Definition (notes). Identify $L(\mathcal{H}_A \otimes \mathcal{H}_B) = L(\mathcal{H}_A) \otimes L(\mathcal{H}_B)$ and define $\mathrm{tr}_B(\sum_l R_l \otimes T_l) = \sum_l \mathrm{tr}(T_l) R_l \in L(\mathcal{H}_A)$ and $\mathrm{tr}_A(\sum_l R_l \otimes T_l) = \sum_l \mathrm{tr}(R_l) T_l \in L(\mathcal{H}_B)$; also written $X^A = \mathrm{tr}_B(X)$, $X^B = \mathrm{tr}_A(X)$. Well defined by the universal property, since $(R, T) \mapsto \mathrm{tr}(T)R$ is bilinear.

The formal definitions use the block formulas of Day 16: an $mp \times mp$ matrix is indexed by pairs (i, k), $i < m$, $k < p$, with flat index $ip + k$ ($\mathcal{H}_A$ is the major factor, matching $(A \otimes B)_{(r,c)} = A_{(r \div p,\, c \div p)} B_{(r \bmod p,\, c \bmod p)}$), and $(\mathrm{tr}_B\, X)_{ij} = \sum_{k<p} X_{ip+k,\, jp+k}$, $(\mathrm{tr}_A\, X)_{ij} = \sum_{k<m} X_{kp+i,\, kp+j}$. The defining property on elementary tensors is proved below.

record *'lab qspace* =
qdim :: *nat*
qlabel :: *'lab*

record *'lab bipartite-space* =
left-space :: *'lab qspace*
right-space :: *'lab qspace*

definition *partial-trace-B* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
partial-trace-B m p x ≡
Matrix.mat m m (λ(*i*, *j*). ∑ *k*<*p*. *x* $$ (*i* ∗ *p* + *k*, *j* ∗ *p* + *k*))

definition *partial-trace-A* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
partial-trace-A m p x ≡
Matrix.mat p p (λ(*i*, *j*). ∑ *k*<*m*. *x* $$ (*k* ∗ *p* + *i*, *k* ∗ *p* + *j*))

definition *partial-trace-on* ::
'lab qspace ⇒ *'lab bipartite-space* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
partial-trace-on s ab x ≡
(*if s* = *right-space ab then*
partial-trace-B (*qdim* (*left-space ab*)) (*qdim* (*right-space ab*)) *x*
else if s = *left-space ab then*
partial-trace-A (*qdim* (*left-space ab*)) (*qdim* (*right-space ab*)) *x*
else 0_m *0 0*)

notation *partial-trace-on* (*tr*_‾ - [*1000*, *1000*, *1000*] *1000*)

lemma *partial-trace-on-right*:
fixes *S* :: *'lab qspace* **and** *AB* :: *'lab bipartite-space*
assumes *S* = *right-space AB*

shows $tr_S^{AB}\ x =$
$partial\text{-}trace\text{-}B\ (qdim\ (left\text{-}space\ AB))\ (qdim\ (right\text{-}space\ AB))\ x$
⟨*proof*⟩

lemma *partial-trace-on-left*:
fixes $S :: 'lab\ qspace$ **and** $AB :: 'lab\ bipartite\text{-}space$
assumes $S = left\text{-}space\ AB$
and $left\text{-}space\ AB \neq right\text{-}space\ AB$
shows $tr_S^{AB}\ x =$
$partial\text{-}trace\text{-}A\ (qdim\ (left\text{-}space\ AB))\ (qdim\ (right\text{-}space\ AB))\ x$
⟨*proof*⟩

lemma *dim-row-partial-trace-B* [*simp*]: $dim\text{-}row\ (partial\text{-}trace\text{-}B\ m\ p\ x) = m$
⟨*proof*⟩

lemma *dim-col-partial-trace-B* [*simp*]: $dim\text{-}col\ (partial\text{-}trace\text{-}B\ m\ p\ x) = m$
⟨*proof*⟩

lemma *partial-trace-B-carrier* [*simp*]:
$partial\text{-}trace\text{-}B\ m\ p\ x \in carrier\text{-}mat\ m\ m$
⟨*proof*⟩

lemma *dim-row-partial-trace-A* [*simp*]: $dim\text{-}row\ (partial\text{-}trace\text{-}A\ m\ p\ x) = p$
⟨*proof*⟩

lemma *dim-col-partial-trace-A* [*simp*]: $dim\text{-}col\ (partial\text{-}trace\text{-}A\ m\ p\ x) = p$
⟨*proof*⟩

lemma *partial-trace-A-carrier* [*simp*]:
$partial\text{-}trace\text{-}A\ m\ p\ x \in carrier\text{-}mat\ p\ p$
⟨*proof*⟩

lemma *partial-trace-B-tensor*:
fixes $R\ T :: complex\ Matrix.mat$
assumes *Rsq*: $R \in carrier\text{-}mat\ m\ m$
and *Tsq*: $T \in carrier\text{-}mat\ p\ p$
and *m-pos*: $0 < m$ **and** *p-pos*: $0 < p$
shows $partial\text{-}trace\text{-}B\ m\ p\ (R \otimes T) = Tr\ T \cdot_m R$
⟨*proof*⟩

lemma *partial-trace-A-tensor*:
fixes $R\ T :: complex\ Matrix.mat$
assumes *Rsq*: $R \in carrier\text{-}mat\ m\ m$
and *Tsq*: $T \in carrier\text{-}mat\ p\ p$
and *m-pos*: $0 < m$ **and** *p-pos*: $0 < p$
shows $partial\text{-}trace\text{-}A\ m\ p\ (R \otimes T) = Tr\ R \cdot_m T$
⟨*proof*⟩

20.3 Partial traces and measurements

Writing $\rho = \sum_l R_l \otimes T_l$,

$$p_\alpha(\rho) = \mathrm{tr}((M_\alpha^* M_\alpha \otimes I)\rho) = \sum_l \mathrm{tr}(M_\alpha^* M_\alpha R_l)\,\mathrm{tr}(T_l) = \mathrm{tr}\big(M_\alpha^* M_\alpha\ \mathrm{tr}_B(\rho)\big) = p_\alpha(\rho^A)\ :$$

lab A only sees ρ^A, and symmetrically lab B only sees ρ^B. Check: $\mathrm{tr}(\rho^A) = \mathrm{tr}\,\rho = 1$, and $\rho \geq 0$ implies $\rho^A \geq 0$.

21 Day 16 (3 October 2011): partial traces in block form; the postulates for density matrices

With matrix units E_{ij} of $L(\mathcal{H}_B)$, every $X \in L(\mathcal{H}_A) \otimes L(\mathcal{H}_B)$ is uniquely $X = \sum_{i,j} X_{ij} \otimes E_{ij}$ with $X_{ij} \in L(\mathcal{H}_A)$, and $\mathrm{tr}_B(X) = \sum_i X_{ii}$, $\mathrm{tr}_A(X) = (\mathrm{tr}\, X_{ij})_{ij}$; symmetrically for $X = \sum E_{kl} \otimes Y_{kl}$. Choosing a basis $f_1, \ldots, f_p$ of $\mathcal{H}_B$ identifies $\mathcal{H}_A \otimes \mathcal{H}_B \cong \mathcal{H}_A^{(p)} = \mathcal{H}_A \oplus \cdots \oplus \mathcal{H}_A$ and $L(\mathcal{H}_A \otimes \mathcal{H}_B) \cong M_p(L(\mathcal{H}_A))$, block matrices (X_{ij}) acting by $(X_{ij})(h_j)_j = (\sum_j X_{ij} h_j)_i$; in block notation $\mathrm{tr}_B((X_{ij})) = X_{11} + \cdots + X_{pp}$ and $\mathrm{tr}_A((X_{ij})) = (\mathrm{tr}\, X_{ij})$.

Reformulated postulates. $1'$: the state of a system is a density matrix $\rho \in L(\mathcal{H}_A)$. $2'$: closed evolution is $\rho \mapsto U\rho U^*$. $3'$: a measurement system $\{M_\alpha\}$ with $\sum M_\alpha^* M_\alpha = I$ yields outcome α with probability $\mathrm{tr}(M_\alpha^* M_\alpha \rho)$. $4'$: systems with densities ρ_1, ρ_2 compose to $\rho_1 \otimes \rho_2$ on $\mathcal{H}_A \otimes \mathcal{H}_B$.

lemma *partial-trace-B-trace*:
fixes X :: *complex Matrix.mat*
assumes *Xsq*: $X \in$ *carrier-mat* $(m * p)$ $(m * p)$
and *p-pos*: $0 < p$
shows *Tr* (*partial-trace-B m p X*) = *Tr X*
⟨*proof*⟩

lemma *partial-trace-B-hermitian*:
fixes X :: *complex Matrix.mat*
assumes *Xsq*: $X \in$ *carrier-mat* $(m * p)$ $(m * p)$
and *herm*: *hermitian X*
shows *hermitian* (*partial-trace-B m p X*)
⟨*proof*⟩

lemma *partial-trace-B-is-density*:
fixes X :: *complex Matrix.mat*
assumes *dm*: *density-matrix* $(m * p)$ X
and *p-pos*: $0 < p$
shows *density-matrix m* (*partial-trace-B m p X*)
⟨*proof*⟩

Key identity: measurements in lab A only see $\rho^A = \mathrm{tr}_B\, \rho$.

theorem *partial-trace-measurement*:
fixes *meas* :: *complex Matrix.mat* **and** ϱ :: *complex Matrix.mat*
assumes *M-sq*: *meas* ∈ *carrier-mat m m*
and *dm*: *density-matrix* $(m * p)$ ϱ
and *p-pos*: $0 < p$
shows *Tr* (((*meas*† ∗ *meas*) ⨂ 1_m *p*) ∗ ϱ) =
Tr (*meas*† ∗ *meas* ∗ *partial-trace-B m p* ϱ)
⟨*proof*⟩

22 Day 17 (5 October 2011): measurement maps, quantum noise, completely positive maps

22.1 Measurement maps

Measuring ψ with $\{M_\alpha\}$ and keeping the outcome as an ensemble gives the density matrix $\sum_\alpha p_\alpha(\psi) |M_\alpha\psi/\|M_\alpha\psi\|\rangle\langle M_\alpha\psi/\|M_\alpha\psi\|| = \sum_\alpha M_\alpha(\psi\psi^*)M_\alpha^*$; for an ensemble with density ρ_1 one obtains $M(\rho_1) = \sum_\alpha M_\alpha \rho_1 M_\alpha^*$: a linear map on $L(\mathcal{H}_A)$ sending density matrices to density matrices.

Classical noise flips a bit with some probability; probability vectors evolve by stochastic matrices (non-negative entries, columns summing to one; doubly stochastic if rows do too). Quantum

noise: with an environment $\mathcal{H}_E$ in state $|\phi\rangle$, the system $\rho \otimes |\phi\rangle\langle\phi|$ evolves by a unitary $U = (U_{ij})$ on $\mathcal{H}_A \otimes \mathcal{H}_E$ and lab A sees

$$\operatorname{tr}_E\big(U(\rho \otimes |\phi\rangle\langle\phi|)U^*\big) = \sum_i W_i \rho W_i^*, \qquad W_i = \sum_k U_{ik}\alpha_k, \qquad \sum_i W_i^* W_i = I.$$

Measurement by an intruder and environmental noise are maps of the same form $\rho \mapsto \sum_\alpha X_\alpha \rho X_\alpha^*$ with $\sum X_\alpha^* X_\alpha = I$. The environment computation is *environment-model* in *Day17-Environment-Model*. Third way, axiomatic: a quantum event is a linear $\Phi : L(\mathcal{H}_A) \to L(\mathcal{H}_A)$.

Proposition (notes). Φ maps density matrices to density matrices iff Φ is positive ($\rho \geq 0 \Rightarrow \Phi(\rho) \geq 0$) and trace preserving ($\operatorname{tr}\Phi(X) = \operatorname{tr} X$); then $\Phi \otimes I_{L(\mathcal{H}_B)}$ also maps density matrices to density matrices for every $\mathcal{H}_B$.

Definition (notes). Φ is *completely positive* if $\Phi \otimes I_{L(\mathcal{H}_B)}$ is positive for every finite-dimensional $\mathcal{H}_B$.

Theorem (notes) (Choi–Kraus)**.** $\Phi : L(\mathcal{H}_A) \to L(\mathcal{H}_A)$ is completely positive and trace preserving iff $\Phi(X) = \sum_i E_i X E_i^*$ for matrices E_i with $\sum_i E_i^* E_i = I$.

The proposition is *density-preserving-iff-positive-tp* in *Day18-Positive-Maps*; complete positivity is defined below and the Choi–Kraus theorem is *choi-kraus-cptp* on Day 19.

definition *meas-map* :: *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
meas-map Ms ϱ ≡
foldr (λ*M acc*. *M* ∗ *ϱ* ∗ (*M*†) + *acc*) *Ms* (0*m* (*dim-row ϱ*) (*dim-col ϱ*))

lemma *meas-map-Nil* [*simp*]:
meas-map [] *ϱ* = 0*m* (*dim-row ϱ*) (*dim-col ϱ*)
⟨*proof*⟩

lemma *meas-map-Cons* [*simp*]:
meas-map (*M* # *Ms*) *ϱ* = *M* ∗ *ϱ* ∗ (*M*†) + *meas-map Ms ϱ*
⟨*proof*⟩

lemma *meas-map-carrier*:
assumes *Ms-c*: ∀ *M* ∈ *set Ms*. *M* ∈ *carrier-mat n n*
and *ϱ-c*: *ϱ* ∈ *carrier-mat n n*
shows *meas-map Ms ϱ* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *meas-map-psd*:
assumes *Ms-c*: ∀ *M* ∈ *set Ms*. *M* ∈ *carrier-mat n n*
and *ϱ-c*: *ϱ* ∈ *carrier-mat n n*
and *ϱ-psd*: *psd ϱ*
shows *psd* (*meas-map Ms ϱ*)
⟨*proof*⟩

lemma *foldr-MdagM-carrier*:
assumes *Ms-c*: ∀ *M* ∈ *set Ms*. *M* ∈ *carrier-mat n n*
shows *foldr* (λ*M acc*. (*M*†) ∗ *M* + *acc*) *Ms* (0*m n n*) ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *meas-map-trace-aux*:
assumes *Ms-c*: ∀ *M* ∈ *set Ms*. *M* ∈ *carrier-mat n n*
and *ϱ-c*: *ϱ* ∈ *carrier-mat n n*
shows *Tr* (*meas-map Ms ϱ*) =
Tr (*foldr* (λ*M acc*. (*M*†) ∗ *M* + *acc*) *Ms* (0*m n n*) ∗ *ϱ*)

⟨*proof*⟩

lemma *meas-map-density*:
assumes *ms*: *measurement-system n Ms*
and *dm*: *density-matrix n ϱ*
shows *density-matrix n* (*meas-map Ms ϱ*)
⟨*proof*⟩

22.2 Kraus maps, positive maps, complete positivity

A *Kraus map* $\rho \mapsto \sum_i W_i \rho W_i^\dagger$ is *kraus-map-dim d Ws*; the list *Ws* is a list of *Kraus operators* (*kraus-operators n d Ws*) when $\sum_i W_i^\dagger W_i = I_n$. Positive, trace-preserving and unital maps are defined on the carrier M_n. The amplification $\Phi^{(p)} = \Phi \otimes \mathrm{id}_p$ acts on block matrices by $(X_{ab}) \mapsto (\Phi(X_{ab}))$ (*amplification n d p Φ*), and Φ is p-positive when $\Phi^{(p)}$ is positive, completely positive when it is p-positive for all p. The main result of the subsection: Kraus maps are completely positive.

definition *kraus-map-dim* :: *nat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
kraus-map-dim d Ws ϱ ≡
foldr (λ*W acc*. *W* ∗ *ϱ* ∗ (*W*†) + *acc*) *Ws* (*0*$_m$ *d d*)

definition *kraus-map* :: *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
kraus-map Ws ϱ ≡ *kraus-map-dim* (*dim-row ϱ*) *Ws ϱ*

definition *kraus-operators* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat list* ⇒ *bool* **where**
kraus-operators n d Ws ≡
(∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*) ∧
foldr (λ*W acc*. (*W*†) ∗ *W* + *acc*) *Ws* (*0*$_m$ *n n*) = *1*$_m$ *n*

lemma *kraus-map-dim-Nil* [*simp*]:
kraus-map-dim d [] *ϱ* = *0*$_m$ *d d*
⟨*proof*⟩

lemma *kraus-map-dim-Cons* [*simp*]:
kraus-map-dim d (*W* # *Ws*) *ϱ* =
W ∗ *ϱ* ∗ (*W*†) + *kraus-map-dim d Ws ϱ*
⟨*proof*⟩

lemma *kraus-map-dim-carrier*:
assumes *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
and *ϱ-c*: *ϱ* ∈ *carrier-mat n n*
shows *kraus-map-dim d Ws ϱ* ∈ *carrier-mat d d*
⟨*proof*⟩

lemma *kraus-map-dim-psd*:
assumes *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
and *ϱ-c*: *ϱ* ∈ *carrier-mat n n*
and *ϱ-psd*: *psd ϱ*
shows *psd* (*kraus-map-dim d Ws ϱ*)
⟨*proof*⟩

definition *positive-map* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
positive-map n d Φ ≡
(∀ *A*. *A* ∈ *carrier-mat n n* ⟶ Φ *A* ∈ *carrier-mat d d*) ∧
(∀ *P*. *P* ∈ *carrier-mat n n* ⟶ *psd P* ⟶ *psd* (Φ *P*))

lemma *kraus-map-dim-positive-map*:

assumes *Ws*: *kraus-operators n d Ws*
shows *positive-map n d* (*kraus-map-dim d Ws*)
⟨*proof*⟩

definition *trace-preserving* :: *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
trace-preserving n Φ ≡
∀ *X*. *X* ∈ *carrier-mat n n* ⟶ *Tr* (Φ *X*) = *Tr X*

definition *unital-map* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
unital-map n d Φ ≡ Φ (*1*$_m$ *n*) = *1*$_m$ *d*

definition *block-of-mat* :: *nat* ⇒ *complex Matrix.mat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
block-of-mat n x a b ≡
Matrix.mat n n (λ(*r*,*c*). *x* $$ (*a*∗*n* + *r*, *b*∗*n* + *c*))

lemma *block-of-mat-carrier* [*simp*]:
block-of-mat n X a b ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *block-of-mat-index*:
assumes *r* < *n* **and** *c* < *n*
shows *block-of-mat n X a b* $$ (*r*,*c*) = *X* $$ (*a*∗*n* + *r*, *b*∗*n* + *c*)
⟨*proof*⟩

lemma *block-of-mat-single*:
assumes *X*: *X* ∈ *carrier-mat n n*
shows *block-of-mat n X 0 0* = *X*
⟨*proof*⟩

definition *amplification* :: *nat* ⇒ *nat* ⇒ *nat* ⇒
(*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
amplification n d p Φ *x* ≡
Matrix.mat (*p*∗*d*) (*p*∗*d*)
(λ(*i*,*j*). Φ (*block-of-mat n x* (*i div d*) (*j div d*)) $$
(*i mod d*, *j mod d*))

lemma *amplification-carrier* [*simp*]:
amplification n d p Φ *X* ∈ *carrier-mat* (*p*∗*d*) (*p*∗*d*)
⟨*proof*⟩

lemma *amplification-index*:
assumes *i* < *p*∗*d* **and** *j* < *p*∗*d*
shows *amplification n d p* Φ *X* $$ (*i*,*j*) =
Φ (*block-of-mat n X* (*i div d*) (*j div d*)) $$
(*i mod d*, *j mod d*)
⟨*proof*⟩

lemma *amplification-one*:
assumes *X*: *X* ∈ *carrier-mat n n*
and *range*: Φ *X* ∈ *carrier-mat d d*
shows *amplification n d 1* Φ *X* = Φ *X*
⟨*proof*⟩

lemma *tensor-id-kraus-carrier*:
assumes *W*: *W* ∈ *carrier-mat d n*
shows (*1*$_m$ *p* ⨂ *W*) ∈ *carrier-mat* (*p*∗*d*) (*p*∗*n*)
⟨*proof*⟩

lemma *tensor-id-kraus-list-carrier*:
assumes *Ws*: $\forall\, W \in set\ Ws.\ W \in carrier\text{-}mat\ d\ n$
shows $\forall\, L \in set\ (map\ (\lambda W.\ 1_m\ p \bigotimes W)\ Ws).$
$L \in carrier\text{-}mat\ (p{*}d)\ (p{*}n)$
⟨*proof*⟩

lemma *tensor-id-kraus-index*:
assumes *W*: $W \in carrier\text{-}mat\ d\ n$
and *r*: $r < p{*}d$
and *c*: $c < p{*}n$
and *dpos*: $0 < d$
and *npos*: $0 < n$
shows $(1_m\ p \bigotimes W)\ \$\$\ (r,c) =$
$(if\ r\ div\ d = c\ div\ n\ then\ W\ \$\$\ (r\ mod\ d,\ c\ mod\ n)\ else\ 0)$
⟨*proof*⟩

lemma *sum-block-delta*:
fixes $f :: nat \Rightarrow complex$
assumes *a*: $a < p$
and *npos*: $0 < n$
shows $(\sum t < p{*}n.\ if\ a = t\ div\ n\ then\ f\ (t\ mod\ n)\ else\ 0) =$
$(\sum r < n.\ f\ r)$
⟨*proof*⟩

lemma *sum-block-delta-index*:
fixes $h :: nat \Rightarrow complex$
assumes *a*: $a < p$
and *npos*: $0 < n$
shows $(\sum t < p{*}n.\ if\ a = t\ div\ n\ then\ h\ t\ else\ 0) =$
$(\sum r < n.\ h\ (a{*}n + r))$
⟨*proof*⟩

lemma *tensor-id-kraus-left-mult-index*:
assumes *W*: $W \in carrier\text{-}mat\ d\ n$
and *x*: $x \in carrier\text{-}mat\ (p{*}n)\ q$
and *i*: $i < p{*}d$
and *s*: $s < q$
and *dpos*: $0 < d$
and *npos*: $0 < n$
shows $((1_m\ p \bigotimes W) * x)\ \$\$\ (i,s) =$
$(\sum t < n.\ W\ \$\$\ (i\ mod\ d,\ t) * x\ \$\$\ ((i\ div\ d) * n + t,\ s))$
⟨*proof*⟩

lemma *tensor-id-kraus-right-mult-index*:
assumes *W*: $W \in carrier\text{-}mat\ d\ n$
and *y*: $y \in carrier\text{-}mat\ q\ (p{*}n)$
and *i*: $i < q$
and *j*: $j < p{*}d$
and *dpos*: $0 < d$
and *npos*: $0 < n$
shows $(y * (1_m\ p \bigotimes (W^\dagger)))\ \$\$\ (i,j) =$
$(\sum s < n.\ y\ \$\$\ (i,\ (j\ div\ d) * n + s) * (W^\dagger)\ \$\$\ (s,\ j\ mod\ d))$
⟨*proof*⟩

lemma *kraus-term-index*:
assumes *W*: $W \in carrier\text{-}mat\ d\ n$
and *B*: $B \in carrier\text{-}mat\ n\ n$
and *i*: $i < d$

and j: $j < d$
shows $(W * B * (W^\dagger))$ \$\$ $(i,j) =$
$(\sum s < n.\ (\sum t < n.\ W$ \$\$ $(i,t) * B$ \$\$ $(t,s)) * (W^\dagger)$ \$\$ $(s,j))$
⟨*proof*⟩

lemma *kraus-amplification-single*:
assumes W: $W \in$ *carrier-mat* $d\ n$
and x: $x \in$ *carrier-mat* $(p*n)\ (p*n)$
and *dpos*: $0 < d$
and *npos*: $0 < n$
shows *amplification* $n\ d\ p\ (\lambda\varrho.\ W * \varrho * (W^\dagger))\ x =$
$((1_m\ p \bigotimes W) * x) * (1_m\ p \bigotimes (W^\dagger))$
⟨*proof*⟩

lemma *amplification-zero-map*:
assumes *dpos*: $0 < d$
shows *amplification* $n\ d\ p\ (\lambda\varrho.\ 0_m\ d\ d)\ x = 0_m\ (p*d)\ (p*d)$
⟨*proof*⟩

lemma *amplification-add*:
assumes *dpos*: $0 < d$
and F: $\bigwedge a\ b.\ F\ (block\text{-}of\text{-}mat\ n\ x\ a\ b) \in carrier\text{-}mat\ d\ d$
and G: $\bigwedge a\ b.\ G\ (block\text{-}of\text{-}mat\ n\ x\ a\ b) \in carrier\text{-}mat\ d\ d$
shows *amplification* $n\ d\ p\ (\lambda\varrho.\ F\ \varrho + G\ \varrho)\ x =$
amplification $n\ d\ p\ F\ x +$ *amplification* $n\ d\ p\ G\ x$
⟨*proof*⟩

lemma *amplification-kraus-map-dim*:
assumes *Ws*: $\forall\, W \in set\ Ws.\ W \in carrier\text{-}mat\ d\ n$
and x: $x \in$ *carrier-mat* $(p*n)\ (p*n)$
and *dpos*: $0 < d$
and *npos*: $0 < n$
shows *amplification* $n\ d\ p\ (kraus\text{-}map\text{-}dim\ d\ Ws)\ x =$
kraus-map-dim $(p*d)\ (map\ (\lambda W.\ 1_m\ p \bigotimes W)\ Ws)\ x$
⟨*proof*⟩

definition *p-positive* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool*
where
p-positive $n\ d\ p\ \Phi \equiv$
$\forall\, X$ :: *complex Matrix.mat*. $X \in carrier\text{-}mat\ (p*n)\ (p*n) \longrightarrow psd\ X \longrightarrow$
psd (*amplification* $n\ d\ p\ \Phi\ X$)

definition *completely-positive* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool*
where
completely-positive $n\ d\ \Phi \equiv \forall\, p > 0.$ *p-positive* $n\ d\ p\ \Phi$

theorem *kraus-is-cp*:
assumes *Ws*: *kraus-operators* $n\ d\ Ws$
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *completely-positive* $n\ d$ (*kraus-map-dim* $d\ Ws$)
⟨*proof*⟩

23 Day 18 (7 October 2011): the theory of completely positive maps

Proposition (notes). Every $X \in M_n$ is $X = (P_1 - P_2) + i(P_3 - P_4)$ with $P_i \geq 0$: $\mathrm{span}(M_n^+) = M_n$.

(Write $X = H + iK$ with H, K Hermitian and split each spectral decomposition into its non-negative and negative eigenvalues.)

Definition (notes). A linear $\Phi : M_n \to M_d$ is *positive* if $P \geq 0$ implies $\Phi(P) \geq 0$; it is *p-positive* if $\Phi^{(p)} : M_p(M_n) \to M_p(M_d)$, $(X_{ij}) \mapsto (\Phi(X_{ij}))$, is positive, and *completely positive* if it is p-positive for every p.

Proposition (notes). A linear $\Phi : M_n \to M_d$ maps density matrices to density matrices iff it is positive and trace preserving.

Example (notes): the transpose $\Phi(A) = A^t$ on M_2 is positive but not 2-positive, since $\Phi^{(2)}$ sends the positive matrix $(E_{ij})_{i,j}$ to a matrix of determinant -1. The decomposition into positives is formalised below; the proposition is *density-preserving-iff-positive-tp* and the example *transpose-not-2-positive*, both in *Day18-Positive-Maps*.

lemma *diag-mat-of-real-hermitian*:
  $((\textit{diag-mat-of-real } D)^\dagger) = \textit{diag-mat-of-real } D$
⟨*proof*⟩

lemma *hermitian-decomp-psd*:
  fixes *H* :: *complex Matrix.mat*
  assumes *sq*: $H \in \textit{carrier-mat } n\ n$
    and *herm*: *hermitian H*
  shows $\exists$ *P1 P2* :: *complex Matrix.mat*.
    $P1 \in \textit{carrier-mat } n\ n \wedge P2 \in \textit{carrier-mat } n\ n \wedge$
    $\textit{psd } P1 \wedge \textit{psd } P2 \wedge$
    $H = P1 - P2$
⟨*proof*⟩

lemma *matrix-decomp-psd*:
  fixes *X* :: *complex Matrix.mat*
  assumes *sq*: $X \in \textit{carrier-mat } n\ n$
  shows $\exists$ *P1 P2 P3 P4* :: *complex Matrix.mat*.
    $(\forall i \in \{0..3\}.\ [P1,P2,P3,P4]\ !\ i \in \textit{carrier-mat } n\ n \wedge \textit{psd } ([P1,P2,P3,P4]\ !\ i)) \wedge$
    $X = (P1 - P2) + \mathrm{i} \cdot_m (P3 - P4)$
⟨*proof*⟩

24 Day 19 (10 October 2011): Choi's theorem

Theorem (notes) (Choi, 1975)**.** For linear $\Phi : M_n \to M_d$ the following are equivalent: (1) Φ is completely positive; (2) Φ is n-positive; (3) $P_\Phi = (\Phi(E_{ij}))_{i,j} \geq 0$ in $M_n(M_d)$; (4) there are $d \times n$ matrices $B_1, \ldots, B_K$ with $\Phi(X) = \sum_k B_k X B_k^*$.

Notes: (1) $\sum_k B_k X B_k^*$ is a *Choi–Kraus representation*; (2) P_Φ is the *Choi–Jamiołkowski matrix*; (3) $CP(M_n, M_d) \leftrightarrow M_n(M_d)^+$ via $\Phi \leftrightarrow P_\Phi$; (4) Φ is trace preserving iff $\mathrm{tr}(\Phi(E_{ij})) = \delta_{ij}$; (5) Φ is unital iff $\sum_i \Phi(E_{ii}) = I_d$.

Formally P_Φ is *choi-matrix n d* Φ (an $nd \times nd$ matrix built from the blocks $\Phi(E_{ij})$); *choi-theorem* is (1)⇔(3), *kraus-rep-of-psd-choi* is (3)⇒(4), and *kraus-is-cp* is (4)⇒(1). The step (2)⇒(3) is the

observation that $P_\Phi = \Phi^{(n)}((E_{ij}))$ with $(E_{ij}) \geq 0$ (*choi-matrix-as-amplification*). Linearity of Φ is assumed on the carrier M_n (additivity and homogeneity hypotheses). The trace-preserving case is *choi-kraus-cptp*; notes (4) and (5) in Choi-matrix form are *choi-trace-preserving-iff* and *choi-unital-iff* in theory Day19_Choi_TP_Unital.

definition *choi-matrix* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* **where**
choi-matrix n d Φ ≡
Matrix.mat (*n*∗*d*) (*n*∗*d*)
(λ(*r*,*c*). Φ (*mat-unit* (*r div d*) (*c div d*) *n n*) $$ (*r mod d*, *c mod d*))

notation *choi-matrix* (P_Φ)

definition *choi-seed* :: *nat* ⇒ *complex Matrix.mat* **where**
choi-seed n ≡
Matrix.mat (*n*∗*n*) (*n*∗*n*)
(λ(*r*,*c*). *mat-unit* (*r div n*) (*c div n*) *n n* $$ (*r mod n*, *c mod n*))

lemma *choi-seed-carrier* [*simp*]:
choi-seed n ∈ *carrier-mat* (*n*∗*n*) (*n*∗*n*)
⟨*proof*⟩

lemma *choi-seed-block*:
assumes *npos*: *n* > *0*
and *a*: *a* < *n*
and *b*: *b* < *n*
shows *block-of-mat n* (*choi-seed n*) *a b* = *mat-unit a b n n*
⟨*proof*⟩

lemma *choi-matrix-as-amplification*:
assumes *npos*: *n* > *0*
and *dpos*: *d* > *0*
shows *choi-matrix n d* Φ = *amplification n d n* Φ (*choi-seed n*)
⟨*proof*⟩

lemma *choi-psd-if-cp*:
assumes *npos*: *n* > *0*
and *dpos*: *d* > *0*
and *cp*: *completely-positive n d* Φ
shows *psd* (*choi-matrix n d* Φ)
⟨*proof*⟩

24.1 From a positive Choi matrix to a Kraus representation

The notes' proof of (3)⇒(4): write $P_\Phi = \sum_k v_k v_k^*$ with $v_k \in \mathbb{C}^{nd}$, cut each v_k into n blocks $w_1^k, \ldots, w_n^k \in \mathbb{C}^d$, and let $B_k = [w_1^k | \cdots | w_n^k]$; then $B_k E_{ij} B_k^* = w_i^k (w_j^k)^*$ and $\Phi(X) = \sum_{ij} x_{ij} \Phi(E_{ij}) = \sum_k B_k X B_k^*$ by linearity.

Complete positivity from a Kraus list without the trace-preservation constraint: the argument is the same as ⟦*kraus-operators ?n ?d ?Ws*; *0* < *?n*; *0* < *?d*⟧ ⟹ *completely-positive ?n ?d* (*kraus-map-dim ?d ?Ws*), which only uses the carrier hypothesis.

lemma *kraus-cp-no-tp*:
assumes *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
and *npos*: *0* < *n*
and *dpos*: *0* < *d*
shows *completely-positive n d* (*kraus-map-dim d Ws*)
⟨*proof*⟩

Linearity of the Kraus-sum map in its matrix argument.

lemma *kraus-map-dim-add*:
assumes *A-c*: $A \in$ *carrier-mat n n* **and** *B-c*: $B \in$ *carrier-mat n n*
and *Ws-c*: $\forall\, W \in$ *set Ws*. $W \in$ *carrier-mat d n*
shows *kraus-map-dim d Ws* $(A + B) =$
kraus-map-dim d Ws A + *kraus-map-dim d Ws B*
⟨*proof*⟩

lemma *kraus-map-dim-smult*:
assumes *A-c*: $A \in$ *carrier-mat n n*
and *Ws-c*: $\forall\, W \in$ *set Ws*. $W \in$ *carrier-mat d n*
shows *kraus-map-dim d Ws* $(c \cdot_m A) = c \cdot_m$ *kraus-map-dim d Ws A*
⟨*proof*⟩

lemma *kraus-map-dim-zero* [*simp*]:
assumes *Ws-c*: $\forall\, W \in$ *set Ws*. $W \in$ *carrier-mat d n*
shows *kraus-map-dim d Ws* $(0_m\ n\ n) = 0_m\ d\ d$
⟨*proof*⟩

Enumeration of $[0..<n] \times [0..<n]$ as a list of pairs.

definition *mat-unit-pairs* :: *nat* ⇒ (*nat* × *nat*) *list* **where**
mat-unit-pairs n ≡
concat (*map* (λ*a*. *map* (*Pair a*) [*0*..<*n*]) [*0*..<*n*])

lemma *mat-unit-pairs-set*:
set (*mat-unit-pairs n*) = {*p*. *fst p* < *n* ∧ *snd p* < *n*}
⟨*proof*⟩

lemma *mat-unit-pairs-distinct*: *distinct* (*mat-unit-pairs n*)
⟨*proof*⟩

Generic pair-indexed linear combination of $f\ (E_{ab})$ contributions, weighted by the entries of A.

primrec *pair-comb* ::
(*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* ⇒
nat ⇒ *nat* ⇒ (*nat* × *nat*) *list* ⇒ *complex Matrix.mat* **where**
pair-comb f A n d [] = $0_m\ d\ d$ |
pair-comb f A n d (*p* # *ps*) =
A $$ *p* ·$_m$ *f* (*mat-unit* (*fst p*) (*snd p*) *n n*) + *pair-comb f A n d ps*

lemma *pair-comb-carrier*:
assumes *f-car*: $\bigwedge a\ b.\ a < n \Longrightarrow b < n \Longrightarrow f$ (*mat-unit a b n n*) ∈ *carrier-mat d d*
and *ps-valid*: $\forall\, p \in$ *set ps*. *fst p* < *n* ∧ *snd p* < *n*
shows *pair-comb f A n d ps* ∈ *carrier-mat d d*
⟨*proof*⟩

lemma *pair-comb-entry*:
assumes *f-car*: $\bigwedge a\ b.\ a < n \Longrightarrow b < n \Longrightarrow f$ (*mat-unit a b n n*) ∈ *carrier-mat d d*
and *ps-valid*: $\forall\, p \in$ *set ps*. *fst p* < *n* ∧ *snd p* < *n*
and *r-lt*: *r* < *d* **and** *c-lt*: *c* < *d*
shows *pair-comb f A n d ps* $$ (*r*,*c*) =
($\sum p \leftarrow ps$. *A* $$ *p* ∗ *f* (*mat-unit* (*fst p*) (*snd p*) *n n*) $$ (*r*,*c*))
⟨*proof*⟩

For the identity-like choice $f = id$, the pair combination reconstructs A when *ps* enumerates all basis positions.

lemma *pair-comb-id-reconstructs*:
assumes *A-car*: *A* ∈ *carrier-mat n n*

shows *pair-comb* $(\lambda X.\ X)\ A\ n\ n\ (\textit{mat-unit-pairs}\ n) = A$
⟨*proof*⟩

For a linear map $f : M_n \to M_d$, applying f to A equals the pair combination of f evaluated on the basis.

lemma *linear-eq-pair-comb*:
fixes f :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *f-add*: $\bigwedge X\ Y.\ X \in \textit{carrier-mat}\ n\ n \Longrightarrow Y \in \textit{carrier-mat}\ n\ n \Longrightarrow$
$f\ (X + Y) = f\ X + f\ Y$
and *f-smult*: $\bigwedge c\ X.\ X \in \textit{carrier-mat}\ n\ n \Longrightarrow f\ (c \cdot_m X) = c \cdot_m f\ X$
and *f-car*: $\bigwedge X.\ X \in \textit{carrier-mat}\ n\ n \Longrightarrow f\ X \in \textit{carrier-mat}\ d\ d$
and *A-car*: $A \in \textit{carrier-mat}\ n\ n$
and *npos*: $n > 0$ **and** *dpos*: $d > 0$
shows $f\ A = \textit{pair-comb}\ f\ A\ n\ d\ (\textit{mat-unit-pairs}\ n)$
⟨*proof*⟩

Conjugation $W\ E_{ab}\ W^\dagger$ has entry $W_{ia} \cdot^\dagger W_{jb}$.

lemma *conjugation-mat-unit-entry*:
assumes *Wc*: $W \in \textit{carrier-mat}\ d\ n$
and *a-lt*: $a < n$ **and** *b-lt*: $b < n$
and *i-lt*: $i < d$ **and** *j-lt*: $j < d$
shows $(W * \textit{mat-unit}\ a\ b\ n\ n * (W^\dagger))\ \$\$\ (i, j) =$
$W\ \$\$\ (i, a) * \textit{cnj}\ (W\ \$\$\ (j, b))$
⟨*proof*⟩

Entry-wise formula for *kraus-map-dim* as a sum over the list.

lemma *kraus-map-dim-entry-sum-list*:
assumes *Ws-c*: $\forall\, W \in \textit{set}\ Ws.\ W \in \textit{carrier-mat}\ d\ n$
and *rho-c*: $\varrho \in \textit{carrier-mat}\ n\ n$
and *i-lt*: $i < d$ **and** *j-lt*: $j < d$
shows *kraus-map-dim* $d\ Ws\ \varrho\ \$\$\ (i, j) =$
sum-list $(\textit{map}\ (\lambda W.\ (W * \varrho * (W^\dagger))\ \$\$\ (i, j))\ Ws)$
⟨*proof*⟩

Kraus representation extracted from a psd Choi matrix: reshape each column of a psd factorization of P_Φ into a Kraus operator, and the resulting list agrees with Φ on all of M_n.

lemma *kraus-rep-of-psd-choi*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in \textit{carrier-mat}\ n\ n \Longrightarrow B \in \textit{carrier-mat}\ n\ n \Longrightarrow$
$\Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in \textit{carrier-mat}\ n\ n \Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall\, A.\ A \in \textit{carrier-mat}\ n\ n \longrightarrow \Phi\ A \in \textit{carrier-mat}\ d\ d$
and *npos*: $n > 0$
and *dpos*: $d > 0$
and *psdP*: *psd* (*choi-matrix* $n\ d\ \Phi$)
shows $\exists\, Ws$ :: *complex Matrix.mat list*.
length $Ws = n * d\ \wedge$
$(\forall\, W \in \textit{set}\ Ws.\ W \in \textit{carrier-mat}\ d\ n)\ \wedge$
$(\forall\, A.\ A \in \textit{carrier-mat}\ n\ n \longrightarrow \Phi\ A = \textit{kraus-map-dim}\ d\ Ws\ A)$
⟨*proof*⟩

theorem *choi-theorem*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in \textit{carrier-mat}\ n\ n \Longrightarrow B \in \textit{carrier-mat}\ n\ n \Longrightarrow$
$\Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in \textit{carrier-mat}\ n\ n \Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall\, A.\ A \in \textit{carrier-mat}\ n\ n \longrightarrow \Phi\ A \in \textit{carrier-mat}\ d\ d$

and *npos*: $n > 0$
and *dpos*: $d > 0$
shows *completely-positive n d* $\Phi \longleftrightarrow$
psd (*choi-matrix n d* Φ)
⟨*proof*⟩

Accumulated $\sum_i\ {W_i}^\dagger\ W_i$ over a Kraus list.

definition *sum-wdag-w* :: *nat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* **where**
sum-wdag-w n Ws ≡
foldr (λ*W acc*. (*W*†) ∗ *W* + *acc*) *Ws* (0$_m$ *n n*)

lemma *sum-wdag-w-Nil* [*simp*]: *sum-wdag-w n* [] = 0$_m$ *n n*
⟨*proof*⟩

lemma *sum-wdag-w-Cons*:
sum-wdag-w n (*W* # *Ws*) = (*W*†) ∗ *W* + *sum-wdag-w n Ws*
⟨*proof*⟩

lemma *sum-wdag-w-carrier*:
assumes ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
shows *sum-wdag-w n Ws* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *kraus-operators-altdef*:
kraus-operators n d Ws ⟷
(∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*) ∧ *sum-wdag-w n Ws* = 1$_m$ *n*
⟨*proof*⟩

lemma *Tr-kraus-map-dim-as-wdagw*:
fixes ϱ :: *complex Matrix.mat*
assumes *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
and *X-c*: ϱ ∈ *carrier-mat n n*
shows *Tr* (*kraus-map-dim d Ws* ϱ) = *Tr* (*sum-wdag-w n Ws* ∗ ϱ)
⟨*proof*⟩

The $\sum_i\ {W_i}^\dagger\ W_i$ matrix entry equals a sum-list of $cnj(W_{ika})\cdot W_{ikb}$ contributions (needed to recover 1_n from trace preservation).

lemma *sum-wdag-w-entry*:
assumes *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
and *a-lt*: *a* < *n* **and** *b-lt*: *b* < *n*
shows *sum-wdag-w n Ws* $$ (*a*, *b*) =
sum-list (*map* (λ*W*. ((*W*†) ∗ *W*) $$ (*a*, *b*)) *Ws*)
⟨*proof*⟩

Pairing with E_{ab} extracts the (b,a) entry under the trace.

lemma *Tr-mult-mat-unit*:
assumes *M-c*: *M* ∈ *carrier-mat n n*
and *a-lt*: *a* < *n* **and** *b-lt*: *b* < *n*
shows *Tr* (*M* ∗ *mat-unit a b n n*) = *M* $$ (*b*, *a*)
⟨*proof*⟩

theorem *choi-kraus-cptp*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *linear*: ⋀*A B*. *A* ∈ *carrier-mat n n* ⟹ *B* ∈ *carrier-mat n n* ⟹
Φ (*A* + *B*) = Φ *A* + Φ *B*
and *smult*: ⋀*c A*. *A* ∈ *carrier-mat n n* ⟹ Φ (*c* ·$_m$ *A*) = *c* ·$_m$ Φ *A*
and *range*: ∀ *A*. *A* ∈ *carrier-mat n n* ⟶ Φ *A* ∈ *carrier-mat d d*

and *npos*: $n > 0$
and *dpos*: $d > 0$
shows (*completely-positive n d* $\Phi \wedge$ *trace-preserving n* Φ) $\longleftrightarrow$
($\exists$ *Ws* :: *complex Matrix.mat list*.
kraus-operators n d Ws $\wedge$
($\forall$ *X*. *X* $\in$ *carrier-mat n n* $\longrightarrow$ Φ *X* = *kraus-map-dim d Ws X*))
⟨*proof*⟩

25 Day 20 (12 October 2011): positive functionals, adjoints, Choi rank

Proposition (notes). For linear $\phi : M_n \to \mathbb{C}$: ϕ positive $\Leftrightarrow$ ϕ completely positive $\Leftrightarrow$ $(\phi(E_{ij})) \in M_n^+$.

Proposition (notes). Let $\Phi : M_n \to M_d$ with Hilbert–Schmidt adjoint $\Phi^* : M_d \to M_n$ ($\langle Y|X\rangle = \mathrm{tr}(Y^*X)$). (1) If $\Phi(X) = \sum_i A_i X A_i^*$ then $\Phi^*(Y) = \sum_i A_i^* Y A_i$. (2) Φ is completely positive iff Φ^* is. (3) Φ is unital and completely positive iff Φ^* is completely positive and trace preserving.

Definition (notes). The *Choi rank* of a completely positive Φ is $\mathrm{cr}(\Phi) = \min\{L : \Phi(X) = \sum_{l=1}^{L} B_l X B_l^*\}$.

Theorem (notes) (Choi)**.** $\mathrm{cr}(\Phi) = \mathrm{rank}(P_\Phi)$.

The functional proposition, the adjoint proposition (for Kraus maps) and the definition of the Choi rank are formalised below; the rank theorem is *choi-rank-eq-rank* in *Day20-Choi-Rank* (its lemma $\mathrm{rank}((E_{ij})) = 1$ is not stated separately), and the bound $\mathrm{cr}(\Phi) \le nd$ is below.

25.1 The adjoint of a Kraus map

For $\Phi(X) = \sum_i A_i X A_i^\dagger$ with $A_i \in M_{d,n}$, the adjoint is $\Phi^*(Y) = \sum_i A_i^\dagger Y A_i$ (*kraus-adjoint-dim n As*; the output dimension n is kept explicit for the empty list).

definition *kraus-adjoint-dim* ::
nat $\Rightarrow$ *complex Matrix.mat list* $\Rightarrow$ *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat* **where**
kraus-adjoint-dim n As y $\equiv$
foldr (λA *acc*. ($A^\dagger$) $*$ *y* $*$ *A* + *acc*) *As* (0_m *n n*)

lemma *kraus-adjoint-dim-Nil* [*simp*]:
fixes *Y* :: *complex Matrix.mat*
shows *kraus-adjoint-dim n* [] *Y* = 0_m *n n*
⟨*proof*⟩

lemma *kraus-adjoint-dim-Cons* [*simp*]:
fixes *Y* :: *complex Matrix.mat*
shows *kraus-adjoint-dim n* (*A* # *As*) *Y* =
($A^\dagger$) $*$ *Y* $*$ *A* + *kraus-adjoint-dim n As Y*
⟨*proof*⟩

lemma *kraus-adjoint-dim-carrier*:
fixes *Y* :: *complex Matrix.mat*
assumes *As-c*: $\forall$ *A* $\in$ *set As*. *A* $\in$ *carrier-mat d n*
and *Y-c*: *Y* $\in$ *carrier-mat d d*
shows *kraus-adjoint-dim n As Y* $\in$ *carrier-mat n n*
⟨*proof*⟩

lemma *kraus-adjoint-dim-as-kraus-map-dim*:
fixes Y :: *complex Matrix.mat*
assumes *As-c*: $\forall A \in$ *set As*. $A \in$ *carrier-mat d n*
and *Y-c*: $Y \in$ *carrier-mat d d*
shows *kraus-adjoint-dim n As Y* =
kraus-map-dim n (*map* (λA. ($A^\dagger$)) *As*) *Y*
⟨*proof*⟩

The adjoint identity $\langle Y|\Phi(X)\rangle = \langle \Phi^*(Y)|X\rangle$, by induction on the Kraus list and cyclicity of the trace.

lemma *hs-inner-prod-left-add*:
assumes *d1r*: *dim-row Y1* = *dim-row X* **and** *d1c*: *dim-col Y1* = *dim-col X*
and *d2r*: *dim-row Y2* = *dim-row X* **and** *d2c*: *dim-col Y2* = *dim-col X*
shows ⟨⟨ *Y1* + *Y2* | *X* ⟩⟩ = ⟨⟨ *Y1* | *X* ⟩⟩ + ⟨⟨ *Y2* | *X* ⟩⟩
⟨*proof*⟩

lemma *hs-inner-prod-right-zero*:
⟨⟨ *y* | 0_m *m n* ⟩⟩ = *0*
⟨*proof*⟩

lemma *hs-inner-prod-left-zero*:
fixes *X* :: *complex Matrix.mat*
assumes *X-c*: *X* ∈ *carrier-mat m n*
shows ⟨⟨ 0_m *m n* | *X* ⟩⟩ = *0*
⟨*proof*⟩

lemma *hs-inner-prod-kraus-term*:
fixes *A X Y* :: *complex Matrix.mat*
assumes *A-c*: *A* ∈ *carrier-mat d n*
and *X-c*: *X* ∈ *carrier-mat n n*
and *Y-c*: *Y* ∈ *carrier-mat d d*
shows ⟨⟨ *Y* | *A* ∗ *X* ∗ ($A^\dagger$) ⟩⟩ = ⟨⟨ ($A^\dagger$) ∗ *Y* ∗ *A* | *X* ⟩⟩
⟨*proof*⟩

theorem *kraus-adjoint-correct*:
fixes *As* :: *complex Matrix.mat list*
and *X* :: *complex Matrix.mat*
and *Y* :: *complex Matrix.mat*
assumes *As-c*: $\forall A \in$ *set As*. *A* ∈ *carrier-mat d n*
and *X-c*: *X* ∈ *carrier-mat n n*
and *Y-c*: *Y* ∈ *carrier-mat d d*
shows ⟨⟨ *Y* | *kraus-map-dim d As X* ⟩⟩
= ⟨⟨ *kraus-adjoint-dim n As Y* | *X* ⟩⟩
⟨*proof*⟩

25.2 Complete positivity and the UCP/CPTP duality

Φ^* is the Kraus map of the adjoint operators, hence completely positive, and Φ is unital iff Φ^* is trace preserving: $\sum_i A_i A_i^* = I$ in both cases, using that $\mathrm{tr}(BY) = \mathrm{tr}(Y)$ for all Y forces $B = I$.

theorem *adjoint-cp-from-kraus*:
assumes *As-c*: $\forall A \in$ *set As*. *A* ∈ *carrier-mat d n*
and *npos*: *0* < *n*
and *dpos*: *0* < *d*
shows *completely-positive d n* (*kraus-adjoint-dim n As*)
⟨*proof*⟩

lemma *kraus-map-dim-one-eq-sum-wdag-w-dagger*:
fixes *As* :: *complex Matrix.mat list*
assumes *As-c*: $\forall A \in set\ As.\ A \in carrier\text{-}mat\ d\ n$
shows *kraus-map-dim d As* $(1_m\ n)$
$= sum\text{-}wdag\text{-}w\ d\ (map\ (\lambda A.\ (A^\dagger))\ As)$
⟨*proof*⟩

theorem *cp-unital-iff-adjoint-tp*:
assumes *As-c*: $\forall A \in set\ As.\ A \in carrier\text{-}mat\ d\ n$
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *unital-map n d* (*kraus-map-dim d As*)
$\longleftrightarrow$ *trace-preserving d* (*kraus-adjoint-dim n As*)
⟨*proof*⟩

25.3 Positive functionals are completely positive

The case $d = 1$ of Choi's theorem: for $P = (P_{ij}) \in M_r(M_n)^+$ and $v = (\alpha_i) \in \mathbb{C}^r$, $\langle v|(\phi(P_{ij}))v\rangle = \phi(\sum_{ij} \overline{\alpha_i}\alpha_j P_{ij}) = \phi(Y^*PY) \geq 0$ with Y the block column $(\alpha_i I)_i$.

lemma *phi-entry-double-sum*:
fixes φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *f-add*: $\bigwedge X\ Y.\ X \in carrier\text{-}mat\ n\ n \Longrightarrow Y \in carrier\text{-}mat\ n\ n$
$\Longrightarrow \varphi\ (X + Y) = \varphi\ X + \varphi\ Y$
and *f-smult*: $\bigwedge c\ X.\ X \in carrier\text{-}mat\ n\ n \Longrightarrow \varphi\ (c \cdot_m X) = c \cdot_m \varphi\ X$
and *f-car*: $\bigwedge X.\ X \in carrier\text{-}mat\ n\ n \Longrightarrow \varphi\ X \in carrier\text{-}mat\ d\ d$
and *M-c*: $M \in carrier\text{-}mat\ n\ n$
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
and *npos*: $0 < n$ **and** *dpos*: $0 < d$
shows $\varphi\ M\ \$\$\ (r,\ c) =$
$(\sum i{<}n.\ \sum j{<}n.\ M\ \$\$\ (i,\ j) * \varphi\ (mat\text{-}unit\ i\ j\ n\ n)\ \$\$\ (r,\ c))$
⟨*proof*⟩

theorem *positive-scalar-is-cp*:
fixes φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in carrier\text{-}mat\ n\ n \Longrightarrow B \in carrier\text{-}mat\ n\ n$
$\Longrightarrow \varphi\ (A + B) = \varphi\ A + \varphi\ B$
and *smult*: $\bigwedge c\ A.\ A \in carrier\text{-}mat\ n\ n \Longrightarrow \varphi\ (c \cdot_m A) = c \cdot_m \varphi\ A$
and *range*: $\forall A.\ A \in carrier\text{-}mat\ n\ n \longrightarrow \varphi\ A \in carrier\text{-}mat\ 1\ 1$
and *pos*: *positive-map n 1* φ
and *npos*: $0 < n$
shows *completely-positive n 1* φ
⟨*proof*⟩

25.4 Choi rank

choi-rank n d Φ is the least length of a Kraus list representing Φ on the carrier. The theorem $\mathrm{cr}(\Phi) = \mathrm{rank}(P_\Phi)$ is *choi-rank-eq-rank* in *Day20-Choi-Rank*; the bound $\mathrm{cr}(\Phi) \leq nd$ follows from the Kraus-rank bound of Day 21.

definition *choi-rank* ::
nat $\Rightarrow$ *nat* $\Rightarrow$ (*complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*) $\Rightarrow$ *nat* **where**
choi-rank n d $\Phi \equiv$
Inf { *length As* | *As* :: *complex Matrix.mat list*.
$(\forall A \in set\ As.\ A \in carrier\text{-}mat\ d\ n)$
$\wedge\ (\forall X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow \Phi\ X = kraus\text{-}map\text{-}dim\ d\ As\ X)$ }

26 Day 21 (14 October 2011): the Kraus rank bound; unitary freedom of representations

Corollary (notes). A completely positive $\Phi : M_n \to M_d$ has a Kraus representation $\Phi(X) = \sum_{l=1}^{K} A_l X A_l^*$ with $K \le nd$.

The notes then record a conjecture (every CP map is an average of n UCP maps of Choi rank $\le d$) and develop the uniqueness theory:

Lemma (notes). If $T : \mathbb{C}^r \to \mathbb{C}^m$ is one-to-one then T^* is onto.

Proposition (notes). If $v_1, \ldots, v_r \in \mathbb{C}^k$ are linearly independent then so are $\{v_i v_j^*\}_{i,j} \subseteq M_k$.

Lemma (notes). If $0 \le ww^* \le P$ then $w \in \mathrm{range}(P)$; hence for $P = \sum_l w_l w_l^* \ge 0$, $\mathrm{range}(P) = \mathrm{span}\{w_l\}$.

Theorem (notes). If $P = \sum_{j=1}^{r} v_j v_j^* = \sum_{l=1}^{m} w_l w_l^*$ with $\operatorname{rank} P = r$, there is $U = (\alpha_{ij})_{m \times r}$ with $w_i = \sum_j \alpha_{ij} v_j$ and $U^*U = I_r$.

Theorem (notes) (Choi 2)**.** If Φ is completely positive with $\mathrm{cr}(\Phi) = r$ and $\Phi(X) = \sum_{j=1}^{r} V_j X V_j^* = \sum_{l=1}^{m} W_l X W_l^*$, then $W_i = \sum_j \alpha_{ij} V_j$ for a unique $U = (\alpha_{ij})$ with $U^*U = I_r$.

Only the Kraus rank bound is formalised here. The kernel form of the lemma ($\ker A \subseteq \ker B$ when $0 \le B \le A$) is proved in theory Day23 and used there to prove Choi's Theorem 3 without the unitary-freedom theorem; the remaining statements are audit items D21-3, D21-5, D21-6.

corollary *kraus-rank-bound*:
  fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
  assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow B \in$ *carrier-mat n n*
    $\Longrightarrow \Phi\ (A + B) = \Phi\ A + \Phi\ B$
    and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
    and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow \Phi\ A \in$ *carrier-mat d d*
    and *cp*: *completely-positive n d* Φ
    and *npos*: $0 < n$
    and *dpos*: $0 < d$
  shows $\exists$ *As* :: *complex Matrix.mat list*.
    length As $\le n * d \wedge$
    $(\forall A \in$ *set As*. $A \in$ *carrier-mat d n*$) \wedge$
    $(\forall X.\ X \in$ *carrier-mat n n* $\longrightarrow \Phi\ X =$ *kraus-map-dim d As X*$)$
⟨*proof*⟩

Dimension bookkeeping for $T^\dagger$ (the onto statement is not formalised).

lemma *adjoint-image-in-dom*:
  fixes *T* :: *complex Matrix.mat*
  assumes *sq*: *T* ∈ *carrier-mat m r*
  shows ∀ *w* :: *complex Matrix.vec*. *dim-vec w = m*
    ⟶ *dim-vec* (*T*† $*_v$ *w*) = *r*
  ⟨*proof*⟩

$\mathrm{cr}(\Phi) \le nd$: the defining set of the Choi rank is non-empty.

corollary *choi-rank-le*:
  fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
  assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow B \in$ *carrier-mat n n*
    $\Longrightarrow \Phi\ (A + B) = \Phi\ A + \Phi\ B$
    and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
    and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow \Phi\ A \in$ *carrier-mat d d*
    and *cp*: *completely-positive n d* Φ

and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *choi-rank n d* $\Phi \leq n * d$
⟨*proof*⟩

27 Day 22 (17 October 2011): minimal Kraus representations, convex sets, UCP and CPTP

Proposition (notes). If Φ is completely positive, $\mathrm{cr}(\Phi) = r$ and $\Phi(X) = \sum_{i=1}^{r} V_i^* X V_i$, then $V_1, \ldots, V_r$ are linearly independent.

(Not formalised: on Day 23 linear independence of the Kraus operators is a hypothesis.)

Definition (notes). $C \subseteq V$ is *convex* if $tv_1 + (1-t)v_2 \in C$ for $v_1, v_2 \in C$, $0 \leq t \leq 1$; $v \in C$ is *extreme* if $v = \frac{1}{2}(v_1 + v_2)$ with $v_1, v_2 \in C$ forces $v = v_1 = v_2$. The *convex hull* of E is the set of convex combinations of points of E; a map between convex sets is *affine* if it preserves convex combinations.

Theorem (notes) (Krein–Milman)**.** A compact convex subset of a finite-dimensional real vector space is the convex hull of its extreme points.

Examples: $UCP(M_n, M_d)$, the unital completely positive maps, is convex ($P_{t\Phi_1+(1-t)\Phi_2} = tP_{\Phi_1} + (1-t)P_{\Phi_2} \geq 0$) and compact (closed; bounded because $0 \leq \Phi(E_{ii}) \leq I$ gives $|r_{ij}| \leq 1$ for the entries of P_Φ); $CPTP(M_d, M_n)$ is convex and compact, being the image of $UCP(M_n, M_d)$ under the affine bijection $\Gamma(\Phi) = \Phi^*$.

Theorem (notes) (Choi 3)**.** If $\Phi : M_n \to M_d$ is unital and completely positive, $\Phi(X) = \sum_{i=1}^{r} V_i^* X V_i$ with $r = \mathrm{cr}(\Phi)$, then Φ is an extreme point of $UCP(M_n, M_d)$ iff $\{V_i^* V_j : 1 \leq i, j \leq r\}$ is linearly independent in M_d.

Corollary (notes). If Φ is extreme then $\mathrm{cr}(\Phi) \leq d$. If $\Phi : M_d \to M_n$ is CPTP with $\Phi(X) = \sum_{i=1}^{r} V_i X V_i^*$, $r = \mathrm{cr}(\Phi)$, then Φ is extreme in $CPTP(M_d, M_n)$ iff $\{V_i^* V_j\}$ is linearly independent.

Convexity of UCP and CPTP is proved below; compactness and the affine bijection are not formalised; the corollary $\mathrm{cr}(\Phi) \leq d$ is *extreme-CPTP-choi-rank-le* in *Day22-Kraus-Independence*, and Choi's Theorem 3 is proved in theory Day23 in its CPTP form. Krein–Milman is available in HOL-Analysis (*Krein-Milman-Minkowski*).

27.1 Convex sets and extreme points

definition *convex-set-mat* :: *complex Matrix.mat set* ⇒ *bool* **where**
convex-set-mat C ≡
∀ *A* ∈ *C*. ∀ *B* ∈ *C*. ∀ *t* :: *real*. *0* ≤ *t* ∧ *t* ≤ *1* ⟶
complex-of-real t ·$_m$ *A*
+ (*1* − *complex-of-real t*) ·$_m$ *B* ∈ *C*

definition *extreme-point-mat* ::
complex Matrix.mat set ⇒ *complex Matrix.mat* ⇒ *bool* **where**
extreme-point-mat C v ≡
v ∈ *C* ∧
¬ (∃ *v1* ∈ *C*. ∃ *v2* ∈ *C*. *v1* ≠ *v* ∧ *v* ≠ *v2* ∧
v = (*1*/*2* :: *complex*) ·$_m$ *v1* + (*1*/*2* :: *complex*) ·$_m$ *v2*)

definition *affine-map-mat* ::
(*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**

affine-map-mat tmap ≡
  ∀ *A B*. ∀ *t* :: *real*. *0* ≤ *t* ∧ *t* ≤ *1* ⟶
    tmap (*complex-of-real t* ·$_m$ *A*
      + (*1* − *complex-of-real t*) ·$_m$ *B*) =
    complex-of-real t ·$_m$ *tmap A*
    + (*1* − *complex-of-real t*) ·$_m$ *tmap B*

27.2 The convex sets UCP and CPTP

definition *UCP* ::
  nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) *set* **where**
  UCP n d ≡ {Φ. *completely-positive n d* Φ ∧ *unital-map n d* Φ}

definition *CPTP* ::
  nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) *set* **where**
  CPTP n d ≡ {Φ. *completely-positive n d* Φ ∧ *trace-preserving n* Φ}

lemma *amplification-smult*:
  assumes *dpos*: *0* < *d*
    and *F-block*: ⋀*a b*. *F* (*block-of-mat n x a b*) ∈ *carrier-mat d d*
  shows *amplification n d p* (λϱ. *c* ·$_m$ *F* ϱ) *x*
      = *c* ·$_m$ *amplification n d p F x*
⟨*proof*⟩

UCP is convex. The range hypotheses are explicit because *completely-positive* does not constrain the values of Φ_i off the carrier.

lemma *UCP-convex*:
  fixes *t* :: *real*
  assumes *t-nn*: *0* ≤ *t*
    and *t-le*: *t* ≤ *1*
    and Φ*1-in*: Φ*1* ∈ *UCP n d*
    and Φ*2-in*: Φ*2* ∈ *UCP n d*
    and *r1*: ∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ*1 X* ∈ *carrier-mat d d*
    and *r2*: ∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ*2 X* ∈ *carrier-mat d d*
    and *dpos*: *0* < *d*
  shows (λ*X*. *complex-of-real t* ·$_m$ Φ*1 X*
      + (*1* − *complex-of-real t*) ·$_m$ Φ*2 X*) ∈ *UCP n d*
⟨*proof*⟩

CPTP is convex.

lemma *CPTP-convex*:
  fixes *t* :: *real*
  assumes *t-nn*: *0* ≤ *t*
    and *t-le*: *t* ≤ *1*
    and Φ*1-in*: Φ*1* ∈ *CPTP n d*
    and Φ*2-in*: Φ*2* ∈ *CPTP n d*
    and *r1*: ∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ*1 X* ∈ *carrier-mat d d*
    and *r2*: ∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ*2 X* ∈ *carrier-mat d d*
    and *dpos*: *0* < *d*
  shows (λ*X*. *complex-of-real t* ·$_m$ Φ*1 X*
      + (*1* − *complex-of-real t*) ·$_m$ Φ*2 X*) ∈ *CPTP n d*
⟨*proof*⟩

28 Day 29 (7 November 2011): classical error detection and correction

Days 23–28 are formalised in the theories Day23_ChoiExtremeUCP, Separable_Cone and Day24_Separability.

To transmit $v \in \mathbb{Z}_2^n$ through a channel that may flip bits (+1 in $\mathbb{Z}_2$), one encodes $v \mapsto \phi(v) \in \mathbb{Z}_2^{n+k}$, transmits, and decodes. Examples: repetition $v \mapsto (v, v)$ detects one error but corrects none and doubles the cost; the parity check $(a_1, \ldots, a_n) \mapsto (a_1, \ldots, a_n; a_1 + \cdots + a_n)$ detects one error cheaply; the Hamming $[7, 1, 3]$ code detects and corrects one error (theory Day29_Classical_Codes: the code as written detects one error but cannot correct it; the $[7, 4, 3]$ code corrects one).

Definition (notes). The *Hamming weight* $\|w\|$ of $w \in \mathbb{Z}_2^m$ is its number of non-zero entries; the weight of a code $S \subseteq \mathbb{Z}_2^m$ is $\min\{\|v - w\| : v \neq w \in S\}$, the least number of errors turning one code word into another; for a subspace code it is $\min\{\|v\| : 0 \neq v \in S\}$.

Decoding amounts to choosing a left inverse of ϕ (different left inverses correct differently); the best binary codes are cyclic codes, ideals in $\mathbb{Z}_2[x]/\langle x^m - 1\rangle$ (not formalised).

28.1 Binary strings, Hamming weight and distance

definition *hamming-weight* :: *nat list* ⇒ *nat* **where**
hamming-weight v ≡ *length* (*filter* (λ*b*. *b* ≠ *0*) *v*)

definition *hamming-dist* :: *nat list* ⇒ *nat list* ⇒ *nat* **where**
hamming-dist v w ≡ *hamming-weight* (*map2* (λ*a b*. (*a* + *b*) *mod 2*) *v w*)

lemma *hamming-weight-Nil* [*simp*]: *hamming-weight* [] = *0*
⟨*proof*⟩

lemma *hamming-weight-Cons*:
hamming-weight (*a* # *v*)
= (*if a* ≠ *0 then Suc* (*hamming-weight v*) *else hamming-weight v*)
⟨*proof*⟩

lemma *hamming-dist-Nil* [*simp*]:
hamming-dist [] [] = *0*
⟨*proof*⟩

lemma *hamming-dist-Cons*:
hamming-dist (*a* # *v*) (*b* # *w*)
= (*if* (*a* + *b*) *mod 2* ≠ *0*
then Suc (*hamming-dist v w*)
else hamming-dist v w)
⟨*proof*⟩

lemma *hamming-dist-symm*:
assumes *length v* = *length w*
shows *hamming-dist v w* = *hamming-dist w v*
⟨*proof*⟩

lemma *hamming-dist-zero*:
assumes *lenw*: *length v* = *length w*
and *bits-v*: ∀ *x* ∈ *set v*. *x* < *2*
and *bits-w*: ∀ *x* ∈ *set w*. *x* < *2*
shows *hamming-dist v w* = *0* ⟷ *v* = *w*

⟨*proof*⟩

lemma *hamming-dist-self* [*simp*]:
hamming-dist v v = 0
⟨*proof*⟩

28.2 Code distance

A code of distance at least $2t+1$ can correct t errors (recorded as the definition *corrects-errors*; the decoding theorem itself is not formalised).

definition *code-distance* :: *nat list set* ⇒ *nat* **where**
code-distance code ≡
if (∃ *v* ∈ *code*. ∃ *w* ∈ *code*. *v* ≠ *w*)
then Min {*hamming-dist v w* | *v w*. *v* ∈ *code* ∧ *w* ∈ *code* ∧ *v* ≠ *w*}
else 0

definition *corrects-errors* :: *nat list set* ⇒ *nat* ⇒ *bool* **where**
corrects-errors code t ≡ *code-distance code* ≥ *2*t + 1*

28.3 The parity-check and repetition codes

Parity check: append the sum of the bits; any single error changes the code word. Repetition doubles Hamming distances.

definition *parity-bit* :: *nat list* ⇒ *nat* **where**
parity-bit v ≡ *foldr* (λ*a b*. (*a* + *b*) *mod 2*) *v 0*

definition *parity-check-encode* :: *nat list* ⇒ *nat list* **where**
parity-check-encode v ≡ *v* @ [*parity-bit v*]

lemma *length-parity-check-encode* [*simp*]:
length (*parity-check-encode v*) = *length v* + *1*
⟨*proof*⟩

lemma *hamming-weight-pos-iff*:
0 < *hamming-weight v* ⟷ (∃ *x* ∈ *set v*. *x* ≠ *0*)
⟨*proof*⟩

lemma *hamming-dist-pos-iff*:
assumes *lenw*: *length v* = *length w*
and *bits-v*: ∀ *x* ∈ *set v*. *x* < *2*
and *bits-w*: ∀ *x* ∈ *set w*. *x* < *2*
shows *0* < *hamming-dist v w* ⟷ *v* ≠ *w*
⟨*proof*⟩

lemma *parity-bit-lt-2*: *parity-bit v* < *2*
⟨*proof*⟩

lemma *parity-check-encode-binary*:
assumes ∀ *x* ∈ *set v*. *x* < *2*
shows ∀ *x* ∈ *set* (*parity-check-encode v*). *x* < *2*
⟨*proof*⟩

lemma *parity-check-detects-one-error*:
assumes *lenE*: *length e* = *length v* + *1*
and *bits-v*: ∀ *x* ∈ *set v*. *x* < *2*
and *bits-e*: ∀ *x* ∈ *set e*. *x* < *2*

and *weight*: *hamming-weight e = 1*
shows *parity-check-encode v*
≠ map2 (λ*a b*. (*a* + *b*) *mod 2*) (*parity-check-encode v*) *e*
⟨*proof*⟩

definition *repetition-encode* :: *nat list* ⇒ *nat list* **where**
repetition-encode v ≡ *v* @ *v*

lemma *length-repetition-encode* [*simp*]:
length (*repetition-encode v*) = 2 ∗ *length v*
⟨*proof*⟩

lemma *hamming-dist-repetition-doubles*:
assumes *length v* = *length w*
shows *hamming-dist* (*repetition-encode v*) (*repetition-encode w*)
= 2 ∗ *hamming-dist v w*
⟨*proof*⟩

29 Day 30 (9 November 2011): error probabilities; the three-qubit bit-flip code

If a bit flips with probability p (independently), the repetition code on three bits leaves an error undetected with probability $3p^2(1-p)^4 + 3p^4(1-p)^2 + p^6$, the parity code with probability $\binom{4}{2}p^2(1-p)^2 + p^4$ (the former is smaller for $p < \frac{1}{2}$), and the majority-rule code $0 \mapsto 000$, $1 \mapsto 111$ decodes wrongly with probability $3p^2(1-p)+p^3$ (theory Day29_Classical_Codes; the comparison holds for all $0 < p < 1$).

Quantum error correction: one cannot clone, but basis vectors can be copied and measurements can decode. With $X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ the allowed errors are qubit flips $X \otimes I \otimes I$, $I \otimes X \otimes I$, ... The *three-qubit bit-flip code* encodes $a|0\rangle + b|1\rangle \mapsto a|000\rangle + b|111\rangle$ (two CNOTs). Decoding: (1) syndrome measurement with $P_0 = |000\rangle\langle 000| + |111\rangle\langle 111|$, $P_1 = |100\rangle\langle 100| + |011\rangle\langle 011|$, $P_2 = |010\rangle\langle 010| + |101\rangle\langle 101|$, $P_3 = |001\rangle\langle 001| + |110\rangle\langle 110|$, $\sum P_i^2 = I$, which leaves a state with at most one flip unchanged and reveals where the flip occurred; (2) recovery $R(Y) = P_0 Y P_0 + (X \otimes I \otimes I) P_1 Y P_1 (X \otimes I \otimes I) + \cdots$, a completely positive trace-preserving map with $R(|\psi_i\rangle\langle\psi_i|) = |\psi\rangle\langle\psi|$ after zero or one error. The code's recovery theorem is *bit-flip-code-recovery* in *Day30-BitFlip-Code* (syndrome projections, $\sum P_i = I$, R CPTP); the Pauli matrices and the bit-flip channel are below.

29.1 The Pauli matrices

The notes use two sign conventions for Y: $\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$ on Day 10 (the library's Y) and $\begin{pmatrix} 0 & i \\ -i & 0 \end{pmatrix}$ on Day 32. *Pauli-Y* follows Day 32; both are Hermitian unitary involutions, and the Pauli identities below carry the corresponding signs. *Pauli-X* and *Pauli-Z* are the library's *Quantum.X* and *Quantum.Z*.

definition *Pauli-X* :: *complex Matrix.mat* **where**
Pauli-X ≡ *Matrix.mat 2 2*
(λ(*i*,*j*). *if* (*i* = *0* ∧ *j* = *1*) ∨ (*i* = *1* ∧ *j* = *0*) *then 1 else 0*)

definition *Pauli-Y* :: *complex Matrix.mat* **where**
Pauli-Y ≡ *Matrix.mat 2 2*
(λ(*i*,*j*). *if i* = *0* ∧ *j* = *1 then* i

*else if i = 1 ∧ j = 0 then −*i *else 0*)

definition *Pauli-Z* :: *complex Matrix.mat* **where**
Pauli-Z ≡ *Matrix.mat 2 2*
(λ(*i*,*j*). *if i = 0 ∧ j = 0 then 1*
else if i = 1 ∧ j = 1 then −1 else 0)

lemma *Pauli-X-carrier* [*simp*]: *Pauli-X* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *Pauli-Y-carrier* [*simp*]: *Pauli-Y* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *Pauli-Z-carrier* [*simp*]: *Pauli-Z* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *Pauli-X-dagger*: *Pauli-X*† = *Pauli-X*
⟨*proof*⟩

lemma *Pauli-X-squared*: *Pauli-X* ∗ *Pauli-X* = 1_m *2*
⟨*proof*⟩

lemma *Pauli-X-unitary*: *unitary Pauli-X*
⟨*proof*⟩

lemma *Pauli-Y-dagger*: *Pauli-Y*† = *Pauli-Y*
⟨*proof*⟩

lemma *Pauli-Y-squared*: *Pauli-Y* ∗ *Pauli-Y* = 1_m *2*
⟨*proof*⟩

lemma *Pauli-Y-unitary*: *unitary Pauli-Y*
⟨*proof*⟩

lemma *Pauli-Z-dagger*: *Pauli-Z*† = *Pauli-Z*
⟨*proof*⟩

lemma *Pauli-Z-squared*: *Pauli-Z* ∗ *Pauli-Z* = 1_m *2*
⟨*proof*⟩

lemma *Pauli-Z-unitary*: *unitary Pauli-Z*
⟨*proof*⟩

29.2 Pauli magic (Day 32)

Proposition (notes). I, X, Y, Z are orthogonal in M_2 and all have 2-norm $\sqrt{2}$.

proposition *pauli-orthogonal*:
shows ⟨⟨ 1_m *2* | *Pauli-X* ⟩⟩ = *0*
and ⟨⟨ 1_m *2* | *Pauli-Y* ⟩⟩ = *0*
and ⟨⟨ 1_m *2* | *Pauli-Z* ⟩⟩ = *0*
and ⟨⟨ *Pauli-X* | *Pauli-Y* ⟩⟩ = *0*
and ⟨⟨ *Pauli-X* | *Pauli-Z* ⟩⟩ = *0*
and ⟨⟨ *Pauli-Y* | *Pauli-Z* ⟩⟩ = *0*
⟨*proof*⟩

proposition *pauli-norm*:
shows ⟨⟨ 1_m *2* | (1_m *2* :: *complex Matrix.mat*) ⟩⟩ = *2*

and ⟨⟨ *Pauli-X* | *Pauli-X* ⟩⟩ = *2*
and ⟨⟨ *Pauli-Y* | *Pauli-Y* ⟩⟩ = *2*
and ⟨⟨ *Pauli-Z* | *Pauli-Z* ⟩⟩ = *2*
⟨*proof*⟩

29.3 The bit-flip channel (Day 31)

Doing nothing to ψ while a flip occurs with probability p produces the ensemble $\{1-p, \psi\}, \{p, X\psi\}$, i.e. the error map $E(Y) = (1-p)Y + pXYX^*$, which is completely positive and trace preserving.

definition *bit-flip-channel* ::
real ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
bit-flip-channel p ϱ ≡
complex-of-real (1 − p) ·$_m$ *ϱ*
+ *complex-of-real p* ·$_m$ (*Pauli-X* ∗ *ϱ* ∗ (*Pauli-X*†))

lemma *bit-flip-channel-carrier*:
assumes *ϱ* ∈ *carrier-mat 2 2*
shows *bit-flip-channel p ϱ* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *bit-flip-channel-trace-preserving*:
fixes *p* :: *real*
assumes *ϱ-c*: *ϱ* ∈ *carrier-mat 2 2*
shows *Tr* (*bit-flip-channel p ϱ*) = *Tr ϱ*
⟨*proof*⟩

lemma *bit-flip-channel-psd*:
fixes *p* :: *real*
assumes *p-nn*: *0* ≤ *p*
and *p-le-1*: *p* ≤ *1*
and *ϱ-c*: *ϱ* ∈ *carrier-mat 2 2*
and *ϱ-psd*: *psd ϱ*
shows *psd* (*bit-flip-channel p ϱ*)
⟨*proof*⟩

30 Day 31 (11 November 2011): the bit-flip code as a channel; fidelity

Encoding $\tilde{\psi} = a|000\rangle + b|111\rangle$ and flipping each qubit independently with probability p gives the error map $E(Y) = (1-p)^3 Y + p(1-p)^2 \sum_j (X_j) Y (X_j)^* + \cdots + p^3 (X \otimes X \otimes X) Y (X \otimes X \otimes X)^*$, and with the recovery R of Day 30,

$$R \circ E(Y) = \big((1-p)^3 + 3p(1-p)^2\big) Y + \big(3p^2(1-p) + p^3\big)(X \otimes X \otimes X) Y (X \otimes X \otimes X).$$

The code helps iff $(1-p)^3 + 3p(1-p)^2 \geq 1 - p$ iff $p \leq \frac{1}{2}$ (theory Day31_Code_Fidelity, where the formula for $R \circ E$ holds for Y supported on the code space). Below: the encoding isometry.

definition *encode-mat* :: *complex Matrix.mat* **where**
encode-mat ≡ *Matrix.mat 8 2* (λ(*i*,*j*).
if i = *0* ∧ *j* = *0 then 1*
else if i = *7* ∧ *j* = *1 then 1*
else 0)

lemma *encode-mat-carrier* [*simp*]: *encode-mat* ∈ *carrier-mat 8 2*
⟨*proof*⟩

30.1 Fidelity

A state ψ perturbed to $\psi' = a\psi + b\psi^\perp$ is close to ψ when $|a|$ is close to 1, so the *fidelity* of states is $F = |\langle\psi'|\psi\rangle|$; for the density matrices $\rho = |\psi\rangle\langle\psi|$, $\rho' = |\psi'\rangle\langle\psi'|$ one has $\langle\rho|\rho'\rangle = \mathrm{Tr}(\rho\rho') = |\langle\psi'|\psi\rangle|^2$, so $F = \sqrt{\mathrm{Tr}(\rho\rho')}$. Error correction is compared by $F = \sqrt{\langle\psi|R \circ E(|\psi\rangle\langle\psi|)\psi\rangle}$, minimised or averaged over ψ: doing nothing gives $\min F = \sqrt{1-p}$, the bit-flip code gives $\min F = \sqrt{(1-p)^3 + 3p(1-p)^2}$ (theory Day31_Code_Fidelity). Formally *fidelity* ψ ϱ is $\sqrt{\Re\langle\psi|\rho\psi\rangle}$.

definition *fidelity* ::
complex Matrix.vec ⇒ *complex Matrix.mat* ⇒ *real* **where**
fidelity ψ $\varrho \equiv$ *sqrt* (*Re* ($\langle\psi$ | $\varrho *_v \psi\rangle$))

lemma *fidelity-def-eq*:
fidelity ψ ϱ = *sqrt* (*Re* ($\langle\psi$ | $\varrho *_v \psi\rangle$))
⟨*proof*⟩

lemma *fidelity-pure-states*:
fixes ψ ψ' :: *complex Matrix.vec*
assumes ψ-*qs*: *quantum-state n* ψ
and ψ'-*qs*: *quantum-state n* ψ'
shows *fidelity* ψ ((*mat-of-cols n* [ψ']) ∗ ((*mat-of-cols n* [ψ'])†))
= *cmod* ($\langle\psi$ | $\psi'\rangle$)
⟨*proof*⟩

lemma *fidelity-self*:
fixes ψ :: *complex Matrix.vec*
assumes ψ-*qs*: *quantum-state n* ψ
shows *fidelity* ψ ((*mat-of-cols n* [ψ]) ∗ ((*mat-of-cols n* [ψ])†)) = *1*
⟨*proof*⟩

lemma *fidelity-orthogonal*:
fixes ψ ψ' :: *complex Matrix.vec*
assumes ψ-*qs*: *quantum-state n* ψ
and ψ'-*qs*: *quantum-state n* ψ'
and *ortho*: $\langle\psi$ | $\psi'\rangle$ = *0*
shows *fidelity* ψ ((*mat-of-cols n* [ψ']) ∗ ((*mat-of-cols n* [ψ'])†)) = *0*
⟨*proof*⟩

31 Day 32 (14 November 2011): the phase-flip code, the Shor code, Pauli magic

31.1 The phase-flip code

With $Z|0\rangle = |0\rangle$, $Z|1\rangle = -|1\rangle$ and $|\pm\rangle = (|0\rangle \pm |1\rangle)/\sqrt{2}$ one has $|+\rangle \perp |-\rangle$, $Z|+\rangle = |-\rangle$, $Z|-\rangle = |+\rangle$: Z acts on $\{|+\rangle, |-\rangle\}$ as the bit flip acts on $\{|0\rangle, |1\rangle\}$. Encoding $|0\rangle \mapsto |++\!+\rangle$, $|1\rangle \mapsto |--\!-\rangle$ gives the three-qubit *phase-flip code*, which detects and corrects phase flips exactly as the bit-flip code corrects bit flips; with $\beta = Y|+\rangle$ (orthogonal to $|+\rangle$, $Y\beta = |+\rangle$) the same works for Y.

The *Shor code* encodes $|0\rangle \mapsto |0_L\rangle = 2^{-3/2}(|000\rangle + |111\rangle)^{\otimes 3}$ and $|1\rangle \mapsto |1_L\rangle = 2^{-3/2}(|000\rangle - |111\rangle)^{\otimes 3}$ in $\mathbb{C}^{2^9}$.

Theorem (notes) (Shor)**.** There is a completely positive trace-preserving $R : M_{2^9} \to M_{2^9}$

such that for every 2×2 unitary U, every position j, $\tilde{U} = I \otimes \cdots \otimes U \otimes \cdots \otimes I$, and every $\psi = a|0_L\rangle + b|1_L\rangle$, $R(\tilde{U}|\psi\rangle\langle\psi|\tilde{U}^*) = |\psi\rangle\langle\psi|$.

Proposition (notes). If $V \subseteq \mathbb{C}^n$ is a subspace and $U_1, \ldots, U_t$ are unitaries with $U_i V \perp U_j V$ for $i \neq j$, then there is a CPTP R with $R(U_i|\psi\rangle\langle\psi|U_i^*) = |\psi\rangle\langle\psi|$ for all $\psi \in V$ and all i (take $R(X) = \sum_i U_i^* P_i X P_i U_i$ with P_i the projection onto $U_i V$, $P_0 = I - \sum P_i$, $U_0 = I$).

The $3 \cdot 9 + 1$ *1-Pauli* operators $I \otimes \cdots \otimes U \otimes \cdots \otimes I$ (U a Pauli matrix) map $V = \mathrm{span}\{|0_L\rangle, |1_L\rangle\}$ to mutually orthogonal subspaces (up to the coincidences $Z_1 v = Z_2 v = Z_3 v$ etc.). The proposition is *recovery-single-error* in *Day32-Recovery-Proposition*, together with its linear-combination form *recovery-linear-combination* (Day 33); the Pauli expansion identity is *pauli-expansion-unitary* in *Day32-Pauli-Expansion*. The Shor code and Shor's theorem are *Day33-Shor-Code* (*shor-code-theorem*). As first stated, the orthogonality claim is false ($Z_1 \neq Z_2$ but $Z_1|0_L\rangle = Z_2|0_L\rangle$, *shor-notes-Z1-Z2*); Day 33 corrects it.

definition *ket-plus* :: *complex Matrix.vec* **where**
ket-plus ≡ (*1 / sqrt 2* :: *complex*) ·$_v$ (*unit-vec 2 0 + unit-vec 2 1*)

definition *ket-minus* :: *complex Matrix.vec* **where**
ket-minus ≡ (*1 / sqrt 2* :: *complex*) ·$_v$ (*unit-vec 2 0 − unit-vec 2 1*)

lemma *ket-plus-carrier* [*simp*]: *ket-plus* ∈ *carrier-vec 2*
⟨*proof*⟩

lemma *ket-minus-carrier* [*simp*]: *ket-minus* ∈ *carrier-vec 2*
⟨*proof*⟩

lemma *ket-plus-dim* [*simp*]: *dim-vec ket-plus = 2*
⟨*proof*⟩

lemma *ket-minus-dim* [*simp*]: *dim-vec ket-minus = 2*
⟨*proof*⟩

lemma *ket-plus-index-0*: *ket-plus* $ *0 = 1 / sqrt 2*
⟨*proof*⟩

lemma *ket-plus-index-1*: *ket-plus* $ *1 = 1 / sqrt 2*
⟨*proof*⟩

lemma *ket-minus-index-0*: *ket-minus* $ *0 = 1 / sqrt 2*
⟨*proof*⟩

lemma *ket-minus-index-1*: *ket-minus* $ *1 = − 1 / sqrt 2*
⟨*proof*⟩

lemma *ket-plus-unit*: ‖*ket-plus*‖ = *1*
⟨*proof*⟩

lemma *ket-minus-unit*: ‖*ket-minus*‖ = *1*
⟨*proof*⟩

lemma *ket-plus-minus-orthog*: ⟨*ket-plus* | *ket-minus*⟩ = *0*
⟨*proof*⟩

lemma *Pauli-Z-ket-plus*: *Pauli-Z* ∗$_v$ *ket-plus = ket-minus*
⟨*proof*⟩

lemma *Pauli-Z-ket-minus*: *Pauli-Z* $*_v$ *ket-minus* = *ket-plus*
⟨*proof*⟩

lemma *Pauli-X-ket-plus*: *Pauli-X* $*_v$ *ket-plus* = *ket-plus*
⟨*proof*⟩

lemma *Pauli-X-ket-minus*: *Pauli-X* $*_v$ *ket-minus* = − *ket-minus*
⟨*proof*⟩

The code words $|{+}{+}{+}\rangle$ and $|{-}{-}{-}\rangle$ of the three-qubit phase-flip code (an earlier version of this file called them *phase-code-zero*/*phase-code-one*; the nine-qubit Shor code words are not defined).

definition *phase-code-zero* :: *complex Matrix.vec* **where**
phase-code-zero ≡
let v = *unit-vec 2 0* + *unit-vec 2 1 in*
(*1* / (*2* ∗ *sqrt 2*) :: *complex*)
$\cdot_v$ *tensor-vec* (*tensor-vec v v*) *v*

definition *phase-code-one* :: *complex Matrix.vec* **where**
phase-code-one ≡
let v = *unit-vec 2 0* − *unit-vec 2 1 in*
(*1* / (*2* ∗ *sqrt 2*) :: *complex*)
$\cdot_v$ *tensor-vec* (*tensor-vec v v*) *v*

lemma *phase-code-zero-dim* [*simp*]: *dim-vec phase-code-zero* = *8*
⟨*proof*⟩

lemma *phase-code-one-dim* [*simp*]: *dim-vec phase-code-one* = *8*
⟨*proof*⟩

lemma *phase-code-zero-carrier* [*simp*]: *phase-code-zero* ∈ *carrier-vec 8*
⟨*proof*⟩

lemma *phase-code-one-carrier* [*simp*]: *phase-code-one* ∈ *carrier-vec 8*
⟨*proof*⟩

31.2 Projections

A *projection* on $\mathbb{C}^n$ is a Hermitian idempotent, the notion of Day 2; projections are the building blocks of syndrome measurements and recovery maps (Days 30–34). Day 2's theorem that such a P is the orthogonal projection onto its range follows the basic facts.

definition *projection* :: *nat* ⇒ *complex Matrix.mat* ⇒ *bool* **where**
projection n P ≡
P ∈ *carrier-mat n n* ∧ *hermitian P* ∧ *P* ∗ *P* = *P*

lemma *projection-carrier*:
projection n P ⟹ *P* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *projection-hermitian*:
projection n P ⟹ *hermitian P*
⟨*proof*⟩

lemma *projection-idempotent*:
projection n P ⟹ *P* ∗ *P* = *P*
⟨*proof*⟩

lemma *projection-zero*: *projection n* $(0_m\ n\ n)$
⟨*proof*⟩

lemma *projection-one*: *projection n* $(1_m\ n)$
⟨*proof*⟩

lemma *projection-psd*:
assumes *projection n P*
shows *psd P*
⟨*proof*⟩

The theorem of Day 2: a Hermitian idempotent P is the orthogonal projection onto its range, i.e. $Pv = v$ for v in the range and $Pw = 0$ for w orthogonal to the range.

lemma *projection-fixes-range*:
fixes *P* :: *complex Matrix.mat* **and** *x* :: *complex Matrix.vec*
assumes *P*: *projection n P* **and** *x*: *dim-vec x* = *n*
shows $P *_v (P *_v x) = P *_v x$
⟨*proof*⟩

lemma *projection-annihilates-orthogonal*:
fixes *P* :: *complex Matrix.mat* **and** *w* :: *complex Matrix.vec*
assumes *P*: *projection n P* **and** *w*: *dim-vec w* = *n*
and *orth*: $\bigwedge x$ :: *complex Matrix.vec*. *dim-vec x* = *n* $\Longrightarrow \langle P *_v x \mid w\rangle = 0$
shows $P *_v w = 0_v\ n$
⟨*proof*⟩

lemma *projection-complement-idempotent*:
assumes *projection n P*
shows $(1_m\ n - P) * (1_m\ n - P) = 1_m\ n - P$
⟨*proof*⟩

31.3 More on the repetition code

lemma *hamming-weight-append*:
hamming-weight (*xs* @ *ys*) = *hamming-weight xs* + *hamming-weight ys*
⟨*proof*⟩

lemma *hamming-weight-repetition-encode*:
hamming-weight (*repetition-encode v*) = 2 ∗ *hamming-weight v*
⟨*proof*⟩

lemma *repetition-encode-binary*:
assumes ∀ *x* ∈ *set v*. *x* < 2
shows ∀ *x* ∈ *set* (*repetition-encode v*). *x* < 2
⟨*proof*⟩

31.4 Algebra of projections; towards Days 33 and 34

Day 33 constructs the Shor recovery map from the projections onto the 22 orthogonal subspaces $V, X_1V, \ldots, X_9V, Y_1V, \ldots, Y_9V, Z_1V, Z_4V, Z_7V$ and proves, using $U = a_0I + a_1X + a_2Y + a_3Z$ with $\sum |a_k|^2 = 1$ for unitary U (Pauli magic), that it corrects every single-qubit unitary error. Day 34 gives the general theory:

Proposition (notes). For $V \subseteq \mathbb{C}^n$ with projection P and CPTP maps E (errors) and R (recovery), $R \circ E(|\psi\rangle\langle\psi|) = |\psi\rangle\langle\psi|$ for all $\psi \in V$ iff $R \circ E(PXP) = PXP$ for all X.

Theorem (notes) (1)**.** Let $E(X) = \sum_{i=1}^{r} E_i X E_i^*$ be CPTP. There is a CPTP R with $R \circ E(PXP) = PXP$ for all X iff $PE_i^* E_j P = \alpha_{ij} P$ for some scalars α_{ij}.

Theorem (notes) (2)**.** If R corrects E as in Theorem 1 and $G_i \in \mathrm{span}\{E_j\}$ with $\sum G_i^* G_i = I$, then R also corrects $G(X) = \sum_i G_i X G_i^*$.

Of the Knill–Laflamme results, the equivalence of Theorem 1's two readings (*recovery-states-iff-PXP*), the necessity of $PE_i^* E_j P = \alpha_{ij} P$ (*knill-laflamme-necessity*) and Theorem 2 (*knill-laflamme-closure*) are proved in *Day34-Knill-Laflamme*; the converse, the construction of R from the conditions, and the equivalence itself (*knill-laflamme*) are in *Day34-KL-Sufficiency*. The elementary algebra of orthogonal projections used by the proofs follows.

lemma *projection-complement*:
assumes *projection n P*
shows *projection n* (1_m *n* − *P*)
⟨*proof*⟩

Two projections are orthogonal if their products in both orders vanish.

definition *orthogonal-projections* ::
nat ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* ⇒ *bool* **where**
orthogonal-projections n P Q ≡
projection n P ∧ *projection n Q* ∧
P ∗ *Q* = 0_m *n n* ∧ *Q* ∗ *P* = 0_m *n n*

lemma *orthogonal-projections-carrier-left*:
orthogonal-projections n P Q ⟹ *P* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *orthogonal-projections-carrier-right*:
orthogonal-projections n P Q ⟹ *Q* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *orthogonal-projections-symm*:
orthogonal-projections n P Q ⟷ *orthogonal-projections n Q P*
⟨*proof*⟩

lemma *orthogonal-projections-sum*:
assumes *opq*: *orthogonal-projections n P Q*
shows *projection n* (*P* + *Q*)
⟨*proof*⟩

lemma *orthogonal-projections-decomp-id*:
assumes *orthogonal-projections n P Q*
shows *P* + *Q* + (1_m *n* − *P* − *Q*) = 1_m *n*
⟨*proof*⟩

32 Extensions beyond the notes

Nothing from here on is in Paulsen's notes. These are standard quantum-information facts proved with the machinery above: rank-one projections, the bit-flip, phase-flip, bit-phase-flip and unitary-conjugation channels and their membership in UCP and CPTP, composition of channels, Kraus lists, the maximally mixed state, and the Pauli product identities.

32.1 Rank-one projections

lemma *outer-sum-singleton-hermitian*:
fixes ψ :: *complex Matrix.vec*
shows $(\textit{outer-sum}\ n\ [\psi])^\dagger = \textit{outer-sum}\ n\ [\psi]$
⟨*proof*⟩

lemma *outer-sum-singleton-idempotent*:
fixes ψ :: *complex Matrix.vec*
assumes ψ-*qs*: *quantum-state n* ψ
shows $\textit{outer-sum}\ n\ [\psi] * \textit{outer-sum}\ n\ [\psi] = \textit{outer-sum}\ n\ [\psi]$
⟨*proof*⟩

lemma *rank1-projection*:
fixes ψ :: *complex Matrix.vec*
assumes ψ-*qs*: *quantum-state n* ψ
shows *projection n* (*outer-sum n* $[\psi]$)
⟨*proof*⟩

lemma *rank1-projection-acts-identity*:
fixes ψ :: *complex Matrix.vec*
assumes ψ-*qs*: *quantum-state n* ψ
shows $\textit{outer-sum}\ n\ [\psi] *_v \psi = \psi$
⟨*proof*⟩

lemma *rank1-projection-trace-inner*:
fixes ψ :: *complex Matrix.vec*
assumes ψ-*dim*: *dim-vec* $\psi = n$
shows $\textit{Tr}\ (\textit{outer-sum}\ n\ [\psi]) = \langle \psi \mid \psi \rangle$
⟨*proof*⟩

lemma *rank1-projection-trace-one*:
fixes ψ :: *complex Matrix.vec*
assumes ψ-*qs*: *quantum-state n* ψ
shows $\textit{Tr}\ (\textit{outer-sum}\ n\ [\psi]) = 1$
⟨*proof*⟩

32.2 The bit-flip channel at $p = 0$ and $p = 1$

lemma *bit-flip-channel-zero*:
assumes $\varrho \in \textit{carrier-mat}\ 2\ 2$
shows $\textit{bit-flip-channel}\ 0\ \varrho = \varrho$
⟨*proof*⟩

lemma *bit-flip-channel-one*:
assumes $\varrho \in \textit{carrier-mat}\ 2\ 2$
shows $\textit{bit-flip-channel}\ 1\ \varrho = \textit{Pauli-X} * \varrho * (\textit{Pauli-X}^\dagger)$
⟨*proof*⟩

32.3 The phase-flip channel

definition *phase-flip-channel* ::
real ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
phase-flip-channel p $\varrho \equiv$
$\quad \textit{complex-of-real}\ (1 - p) \cdot_m \varrho$
$\quad + \textit{complex-of-real}\ p \cdot_m (\textit{Pauli-Z} * \varrho * (\textit{Pauli-Z}^\dagger))$

lemma *phase-flip-channel-carrier*:
assumes $\varrho \in$ *carrier-mat 2 2*
shows *phase-flip-channel p* $\varrho \in$ *carrier-mat 2 2*
⟨*proof*⟩

lemma *phase-flip-channel-trace-preserving*:
fixes *p* :: *real*
assumes *ϱ-c*: $\varrho \in$ *carrier-mat 2 2*
shows *Tr* (*phase-flip-channel p* ϱ) = *Tr* ϱ
⟨*proof*⟩

lemma *phase-flip-channel-psd*:
fixes *p* :: *real*
assumes *p-nn*: $0 \leq p$
and *p-le-1*: $p \leq 1$
and *ϱ-c*: $\varrho \in$ *carrier-mat 2 2*
and *ϱ-psd*: *psd* ϱ
shows *psd* (*phase-flip-channel p* ϱ)
⟨*proof*⟩

32.4 Unitary-conjugation channels

For a unitary $U \in M_n$ and $0 \leq p \leq 1$, $\rho \mapsto (1-p)\rho + pU\rho U^\dagger$ is trace preserving and positive; the bit-flip and phase-flip channels are the instances $U = X$, $U = Z$.

definition *unitary-conj-channel* ::
complex Matrix.mat ⇒ *real* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
unitary-conj-channel U p $\varrho \equiv$
complex-of-real (1 − *p*) $\cdot_m$ ϱ
+ *complex-of-real p* $\cdot_m$ (*U* ∗ ϱ ∗ ($U^\dagger$))

lemma *unitary-conj-channel-carrier*:
assumes *U-c*: *U* ∈ *carrier-mat n n*
and *ϱ-c*: $\varrho \in$ *carrier-mat n n*
shows *unitary-conj-channel U p* $\varrho \in$ *carrier-mat n n*
⟨*proof*⟩

lemma *unitary-conj-channel-trace-preserving*:
fixes *p* :: *real*
assumes *U-c*: *U* ∈ *carrier-mat n n*
and *U-unit*: *unitary U*
and *ϱ-c*: $\varrho \in$ *carrier-mat n n*
shows *Tr* (*unitary-conj-channel U p* ϱ) = *Tr* ϱ
⟨*proof*⟩

lemma *unitary-conj-channel-psd*:
fixes *p* :: *real*
assumes *p-nn*: $0 \leq p$
and *p-le-1*: $p \leq 1$
and *U-c*: *U* ∈ *carrier-mat n n*
and *ϱ-c*: $\varrho \in$ *carrier-mat n n*
and *ϱ-psd*: *psd* ϱ
shows *psd* (*unitary-conj-channel U p* ϱ)
⟨*proof*⟩

lemma *bit-flip-eq-unitary-conj*:
bit-flip-channel p ϱ = *unitary-conj-channel Pauli-X p* ϱ

⟨*proof*⟩

lemma *phase-flip-eq-unitary-conj*:
phase-flip-channel p ϱ = unitary-conj-channel Pauli-Z p ϱ
⟨*proof*⟩

32.5 The bit-phase-flip channel and the identity channel

definition *bit-phase-flip-channel* ::
real ⇒ complex Matrix.mat ⇒ complex Matrix.mat **where**
bit-phase-flip-channel p ϱ ≡ unitary-conj-channel Pauli-Y p ϱ

lemma *bit-phase-flip-channel-carrier*:
assumes *ϱ ∈ carrier-mat 2 2*
shows *bit-phase-flip-channel p ϱ ∈ carrier-mat 2 2*
⟨*proof*⟩

lemma *bit-phase-flip-channel-trace-preserving*:
fixes *p* :: *real*
assumes *ϱ-c*: *ϱ ∈ carrier-mat 2 2*
shows *Tr* (*bit-phase-flip-channel p ϱ*) = *Tr ϱ*
⟨*proof*⟩

lemma *bit-phase-flip-channel-psd*:
fixes *p* :: *real*
assumes *p-nn*: *0 ≤ p*
and *p-le-1*: *p ≤ 1*
and *ϱ-c*: *ϱ ∈ carrier-mat 2 2*
and *ϱ-psd*: *psd ϱ*
shows *psd* (*bit-phase-flip-channel p ϱ*)
⟨*proof*⟩

definition *identity-channel* ::
complex Matrix.mat ⇒ complex Matrix.mat **where**
identity-channel ϱ ≡ ϱ

lemma *identity-channel-carrier*:
ϱ ∈ carrier-mat n n ⟹ identity-channel ϱ ∈ carrier-mat n n
⟨*proof*⟩

lemma *identity-channel-trace-preserving*:
ϱ ∈ carrier-mat n n ⟹ Tr (*identity-channel ϱ*) = *Tr ϱ*
⟨*proof*⟩

lemma *identity-channel-psd*:
ϱ ∈ carrier-mat n n ⟹ psd ϱ ⟹ psd (*identity-channel ϱ*)
⟨*proof*⟩

lemma *identity-channel-eq-unitary-conj-zero*:
assumes *ϱ-c*: *ϱ ∈ carrier-mat n n*
shows *identity-channel ϱ = unitary-conj-channel* (*1*$_m$ *n*) *0 ϱ*
⟨*proof*⟩

32.6 Composition of channels

lemma *compose-trace-preserving*:
assumes *F-tp*: *trace-preserving n F*
and *G-tp*: *trace-preserving n G*
and *F-range*: $\forall\, \varrho.\ \varrho \in$ *carrier-mat n n* $\longrightarrow$ *F* $\varrho \in$ *carrier-mat n n*
shows *trace-preserving n* $(\lambda\varrho.\ G\ (F\ \varrho))$
⟨*proof*⟩

lemma *identity-channel-compose-left*:
$(\lambda\varrho.$ *identity-channel* $(F\ \varrho)) = F$
⟨*proof*⟩

lemma *identity-channel-compose-right*:
$(\lambda\varrho.\ F$ (*identity-channel* $\varrho)) = F$
⟨*proof*⟩

lemma *compose-unital-map*:
assumes *F-un*: *unital-map n n F*
and *G-un*: *unital-map n n G*
shows *unital-map n n* $(\lambda\varrho.\ F\ (G\ \varrho))$
⟨*proof*⟩

lemma *identity-channel-unital*: *unital-map n n identity-channel*
⟨*proof*⟩

lemma *identity-channel-trace-preserving-full*:
trace-preserving n identity-channel
⟨*proof*⟩

lemma *identity-channel-completely-positive*:
assumes $0 < n$
shows *completely-positive n n identity-channel*
⟨*proof*⟩

32.7 Conjugation channels are completely positive

lemma *kraus-map-dim-is-cp*:
fixes *Ws* :: *complex Matrix.mat list*
assumes *Ws-c*: $\forall\, W \in$ *set Ws*. *W* $\in$ *carrier-mat d n*
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *completely-positive n d* (*kraus-map-dim d Ws*)
⟨*proof*⟩

definition *conjugation-channel* ::
complex Matrix.mat $\Rightarrow$ *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat* **where**
conjugation-channel U $\varrho \equiv U * \varrho * (U^\dagger)$

lemma *conjugation-channel-eq-kraus*:
assumes *U-c*: *U* $\in$ *carrier-mat d n*
and ϱ*-c*: $\varrho \in$ *carrier-mat n n*

shows *conjugation-channel U ϱ = kraus-map-dim d [U] ϱ*
⟨*proof*⟩

lemma *conjugation-channel-cp*:
assumes *U-c*: *U ∈ carrier-mat d n*
and *npos*: *0 < n*
and *dpos*: *0 < d*
shows *completely-positive n d* (*kraus-map-dim d* [*U*])
⟨*proof*⟩

lemma *identity-channel-UCP*:
assumes *0 < n*
shows *identity-channel ∈ UCP n n*
⟨*proof*⟩

lemma *identity-channel-CPTP*:
assumes *0 < n*
shows *identity-channel ∈ CPTP n n*
⟨*proof*⟩

lemma *map-conj-cp*:
assumes *U-c*: *U ∈ carrier-mat d n*
and *U-range*: *∀ ϱ. ϱ ∈ carrier-mat n n*
⟶ U ∗ ϱ ∗ (U†) ∈ carrier-mat d d
and *npos*: *0 < n*
and *dpos*: *0 < d*
shows *completely-positive n d* (*λϱ. U ∗ ϱ ∗ (U†)*)
⟨*proof*⟩

lemma *map-conj-unital*:
fixes *U* :: *complex Matrix.mat*
assumes *U-c*: *U ∈ carrier-mat n n*
and *U-unit*: *unitary U*
shows *U ∗ (1m n) ∗ (U†) = 1m n*
⟨*proof*⟩

32.8 Unitary-conjugation channels are UCP and CPTP

lemma *map-conj-in-UCP*:
fixes *U* :: *complex Matrix.mat*
assumes *U-c*: *U ∈ carrier-mat n n*
and *U-unit*: *unitary U*
and *npos*: *0 < n*
shows *(λϱ. U ∗ ϱ ∗ (U†)) ∈ UCP n n*
⟨*proof*⟩

lemma *unitary-conj-channel-in-UCP*:
fixes *U* :: *complex Matrix.mat* **and** *p* :: *real*
assumes *U-c*: *U ∈ carrier-mat n n*
and *U-unit*: *unitary U*
and *p-nn*: *0 ≤ p*
and *p-le*: *p ≤ 1*

and *npos*: $0 < n$
shows *unitary-conj-channel U p* $\in$ *UCP n n*
⟨*proof*⟩

lemma *unitary-conj-channel-in-CPTP*:
fixes *U* :: *complex Matrix.mat* **and** *p* :: *real*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
and *p-nn*: $0 \leq p$
and *p-le*: $p \leq 1$
and *npos*: $0 < n$
shows *unitary-conj-channel U p* $\in$ *CPTP n n*
⟨*proof*⟩

32.9 The Pauli channels are UCP and CPTP

lemma *bit-flip-channel-eq*:
bit-flip-channel p = *unitary-conj-channel Pauli-X p*
⟨*proof*⟩

lemma *phase-flip-channel-eq*:
phase-flip-channel p = *unitary-conj-channel Pauli-Z p*
⟨*proof*⟩

lemma *bit-phase-flip-channel-eq*:
bit-phase-flip-channel p = *unitary-conj-channel Pauli-Y p*
⟨*proof*⟩

lemma *bit-flip-channel-UCP*:
fixes *p* :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
shows *bit-flip-channel p* $\in$ *UCP 2 2*
⟨*proof*⟩

lemma *bit-flip-channel-CPTP*:
fixes *p* :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
shows *bit-flip-channel p* $\in$ *CPTP 2 2*
⟨*proof*⟩

lemma *phase-flip-channel-UCP*:
fixes *p* :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
shows *phase-flip-channel p* $\in$ *UCP 2 2*
⟨*proof*⟩

lemma *phase-flip-channel-CPTP*:
fixes *p* :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
shows *phase-flip-channel p* $\in$ *CPTP 2 2*
⟨*proof*⟩

lemma *bit-phase-flip-channel-UCP*:
fixes *p* :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
shows *bit-phase-flip-channel p* $\in$ *UCP 2 2*
⟨*proof*⟩

lemma *bit-phase-flip-channel-CPTP*:
fixes *p* :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
shows *bit-phase-flip-channel p* $\in$ *CPTP 2 2*
⟨*proof*⟩

32.10 Pauli matrices as Kraus operators

lemma *Pauli-X-kraus-operators*:
kraus-operators 2 2 [*Pauli-X*]
⟨*proof*⟩

lemma *Pauli-Y-kraus-operators*:
kraus-operators 2 2 [*Pauli-Y*]
⟨*proof*⟩

lemma *Pauli-Z-kraus-operators*:
kraus-operators 2 2 [*Pauli-Z*]
⟨*proof*⟩

32.11 Kraus maps of Kraus-operator lists are CPTP

lemma *kraus-is-trace-preserving*:
fixes *Ws* :: *complex Matrix.mat list*
assumes *Ws*: *kraus-operators n d Ws*
shows *trace-preserving n* (*kraus-map-dim d Ws*)
⟨*proof*⟩

lemma *kraus-is-cptp*:
fixes *Ws* :: *complex Matrix.mat list*
assumes *Ws*: *kraus-operators n d Ws*
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *kraus-map-dim d Ws* $\in$ *CPTP n d*
⟨*proof*⟩

lemma *Pauli-X-kraus-map-CPTP*:
kraus-map-dim 2 [*Pauli-X*] $\in$ *CPTP 2 2*
⟨*proof*⟩

lemma *Pauli-Y-kraus-map-CPTP*:
kraus-map-dim 2 [*Pauli-Y*] $\in$ *CPTP 2 2*
⟨*proof*⟩

lemma *Pauli-Z-kraus-map-CPTP*:
kraus-map-dim 2 [*Pauli-Z*] $\in$ *CPTP 2 2*
⟨*proof*⟩

lemma *identity-kraus-operators*:
kraus-operators n n [1_m *n*]
⟨*proof*⟩

lemma *identity-channel-eq-kraus-map-dim*:

fixes n :: *nat*
assumes ϱ-*c*: $\varrho \in$ *carrier-mat n n*
shows *identity-channel* ϱ = *kraus-map-dim n* [1_m *n*] ϱ
⟨*proof*⟩

32.12 Concatenation of Kraus lists

lemma *sum-wdag-w-append*:
fixes *Ws Vs* :: *complex Matrix.mat list*
assumes *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
and *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
shows *sum-wdag-w n* (*Ws* @ *Vs*)
= *sum-wdag-w n Ws* + *sum-wdag-w n Vs*
⟨*proof*⟩

lemma *kraus-map-dim-append*:
fixes *Ws Vs* :: *complex Matrix.mat list*
assumes *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
and *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and ϱ-*c*: $\varrho \in$ *carrier-mat n n*
shows *kraus-map-dim d* (*Ws* @ *Vs*) ϱ
= *kraus-map-dim d Ws* ϱ + *kraus-map-dim d Vs* ϱ
⟨*proof*⟩

32.13 Fixed points of Pauli channels

lemma *Pauli-X-conj-commute*:
fixes *M* :: *complex Matrix.mat*
assumes *M-c*: *M* ∈ *carrier-mat 2 2*
and *commute*: *Pauli-X* ∗ *M* = *M* ∗ *Pauli-X*
shows *Pauli-X* ∗ *M* ∗ (*Pauli-X*†) = *M*
⟨*proof*⟩

lemma *bit-flip-channel-fixed-point-commute*:
fixes *p* :: *real*
assumes *M-c*: *M* ∈ *carrier-mat 2 2*
and *commute*: *Pauli-X* ∗ *M* = *M* ∗ *Pauli-X*
shows *bit-flip-channel p M* = *M*
⟨*proof*⟩

lemma *Pauli-Z-conj-commute*:
fixes *M* :: *complex Matrix.mat*
assumes *M-c*: *M* ∈ *carrier-mat 2 2*
and *commute*: *Pauli-Z* ∗ *M* = *M* ∗ *Pauli-Z*
shows *Pauli-Z* ∗ *M* ∗ (*Pauli-Z*†) = *M*
⟨*proof*⟩

lemma *phase-flip-channel-fixed-point-commute*:
fixes *p* :: *real*
assumes *M-c*: *M* ∈ *carrier-mat 2 2*
and *commute*: *Pauli-Z* ∗ *M* = *M* ∗ *Pauli-Z*
shows *phase-flip-channel p M* = *M*
⟨*proof*⟩

lemma *unitary-conj-channel-identity-op*:
fixes p :: *real*
assumes ϱ-*c*: $\varrho \in$ *carrier-mat n n*
shows *unitary-conj-channel* $(1_m\ n)\ p\ \varrho = \varrho$
⟨*proof*⟩

lemma *unitary-conj-channel-identity-op-eq*:
fixes p :: *real*
shows $(\lambda\varrho.$ *if* $\varrho \in$ *carrier-mat n n*
then unitary-conj-channel $(1_m\ n)\ p\ \varrho$ *else identity-channel* $\varrho)$
$=$ *identity-channel*
⟨*proof*⟩

32.14 The maximally mixed state

definition *maximally-mixed-state* :: *nat* ⇒ *complex Matrix.mat* **where**
maximally-mixed-state n ≡ *complex-of-real* $(1\ /$ *real n*$)\ \cdot_m\ (1_m\ n)$

lemma *maximally-mixed-state-carrier* [*simp*]:
maximally-mixed-state n ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *Tr-one-mat*:
Tr $(1_m\ n$ :: *complex Matrix.mat*$) =$ *of-nat n*
⟨*proof*⟩

lemma *maximally-mixed-state-trace*:
assumes *npos*: $0 < n$
shows *Tr* (*maximally-mixed-state n*) $= 1$
⟨*proof*⟩

lemma *maximally-mixed-state-psd*:
psd (*maximally-mixed-state n*)
⟨*proof*⟩

lemma *maximally-mixed-state-density-matrix*:
assumes *npos*: $0 < n$
shows *density-matrix n* (*maximally-mixed-state n*)
⟨*proof*⟩

32.15 Fixed points of unitary-conjugation channels

lemma *unitary-conj-commute*:
fixes M :: *complex Matrix.mat*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
and *M-c*: $M \in$ *carrier-mat n n*
and *commute*: $U * M = M * U$
shows $U * M * (U^\dagger) = M$
⟨*proof*⟩

lemma *unitary-conj-channel-fixed-point-commute*:
fixes p :: *real*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
and *M-c*: $M \in$ *carrier-mat n n*

and *commute*: $U * M = M * U$
shows *unitary-conj-channel U p M* = *M*
⟨*proof*⟩

lemma *maximally-mixed-state-commutes*:
fixes *U* :: *complex Matrix.mat*
assumes *U-c*: $U \in$ *carrier-mat n n*
shows $U *$ *maximally-mixed-state n* = *maximally-mixed-state n* $* U$
⟨*proof*⟩

lemma *unitary-conj-channel-fixes-max-mixed*:
fixes *p* :: *real*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
shows *unitary-conj-channel U p* (*maximally-mixed-state n*)
= *maximally-mixed-state n*
⟨*proof*⟩

lemma *bit-flip-channel-fixes-max-mixed*:
fixes *p* :: *real*
shows *bit-flip-channel p* (*maximally-mixed-state 2*)
= *maximally-mixed-state 2*
⟨*proof*⟩

lemma *phase-flip-channel-fixes-max-mixed*:
fixes *p* :: *real*
shows *phase-flip-channel p* (*maximally-mixed-state 2*)
= *maximally-mixed-state 2*
⟨*proof*⟩

lemma *bit-phase-flip-channel-fixes-max-mixed*:
fixes *p* :: *real*
shows *bit-phase-flip-channel p* (*maximally-mixed-state 2*)
= *maximally-mixed-state 2*
⟨*proof*⟩

32.16 Channels preserve density matrices

lemma *unitary-conj-channel-preserves-density-matrix*:
fixes *p* :: *real*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
and *p-nn*: $0 \leq p$
and *p-le*: $p \leq 1$
and *dm*: *density-matrix n* ϱ
shows *density-matrix n* (*unitary-conj-channel U p* ϱ)
⟨*proof*⟩

lemma *bit-flip-channel-preserves-density-matrix*:
fixes *p* :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
and *dm*: *density-matrix 2* ϱ
shows *density-matrix 2* (*bit-flip-channel p* ϱ)
⟨*proof*⟩

lemma *phase-flip-channel-preserves-density-matrix*:
fixes p :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
and *dm*: *density-matrix 2* ϱ
shows *density-matrix 2* (*phase-flip-channel p* ϱ)
⟨*proof*⟩

lemma *bit-phase-flip-channel-preserves-density-matrix*:
fixes p :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
and *dm*: *density-matrix 2* ϱ
shows *density-matrix 2* (*bit-phase-flip-channel p* ϱ)
⟨*proof*⟩

32.17 Pauli product identities

With the Day 32 convention for Y: $XY = -iZ$, $YZ = -iX$, $ZX = -iY$.

lemma *Pauli-X-times-Pauli-Y*: *Pauli-X* $*$ *Pauli-Y* $= -\mathrm{i} \cdot_m$ *Pauli-Z*
⟨*proof*⟩

lemma *Pauli-Y-times-Pauli-Z*: *Pauli-Y* $*$ *Pauli-Z* $= -\mathrm{i} \cdot_m$ *Pauli-X*
⟨*proof*⟩

lemma *Pauli-Z-times-Pauli-X*: *Pauli-Z* $*$ *Pauli-X* $= -\mathrm{i} \cdot_m$ *Pauli-Y*
⟨*proof*⟩

end
theory *QC-Prelims*
imports *Day23-ChoiExtremeUCP*
begin

33 Small matrix and vector lemmas shared by the gap-closure theories

Index case splits, trace facts, entrywise vector sums, rank-one identities and rectangular *foldr* sums. Nothing here is specific to a lecture; the theories *Day12-Counting-Probability*, *Day18-Positive-Maps*, *Day22-Kraus-Independence*, *Day28-Horodecki-Peres*, *Day30-BitFlip-Code* and *Day32-Pauli-Expansion* build on it.

lemma *cnj-mult-self-cmod*: *cnj z* $*$ *z* $=$ *complex-of-real* $((cmod\ z)^2)$
⟨*proof*⟩

lemma *Re-cnj-mult-self*: *Re* $(cnj\ z * z) = (cmod\ z)^2$
⟨*proof*⟩

lemma *idx4*: $k < (4::nat) \Longrightarrow k = 0 \vee k = 1 \vee k = 2 \vee k = 3$
⟨*proof*⟩

lemma *mult-index-lt*:
fixes $i\ j\ n\ m$ :: *nat*
assumes $i < n$ **and** $j < m$
shows $i * m + j < n * m$
⟨*proof*⟩

lemma *Tr-hermitian-Im-zero*:

assumes *Pc*: $P \in$ *carrier-mat n n* **and** *herm*: *hermitian P*
shows *Im* (*Tr P*) = *0*
⟨*proof*⟩

lemma *Tr-hermitian-real*:
assumes *Pc*: $P \in$ *carrier-mat n n* **and** *herm*: *hermitian P*
shows *Tr P* = *complex-of-real* (*Re* (*Tr P*))
⟨*proof*⟩

lemma *Tr-minus*:
fixes *x y* :: *complex Matrix.mat*
assumes *dim-row x* = *dim-row y* **and** *dim-col x* = *dim-col y*
shows *Tr* (*x* − *y*) = *Tr x* − *Tr y*
⟨*proof*⟩

A positive semidefinite matrix of trace zero is zero (the notes: "given a non-zero $P \geq 0$, $\mathrm{Tr}(P) \neq 0$").

lemma *psd-Tr-zero-imp-zero*:
assumes *Pc*: $P \in$ *carrier-mat n n* **and** *pos*: *psd P* **and** *tr0*: *Tr P* = *0*
shows *P* = 0_m *n n*
⟨*proof*⟩

lemma *smult-smult-mat*:
fixes *A* :: *complex Matrix.mat*
shows *a* $\cdot_m$ (*b* $\cdot_m$ *A*) = (*a* ∗ *b*) $\cdot_m$ *A*
⟨*proof*⟩

lemma *zero-smult-mat-carrier*:
fixes *A* :: *complex Matrix.mat*
assumes *A* ∈ *carrier-mat m k*
shows *0* $\cdot_m$ *A* = 0_m *m k*
⟨*proof*⟩

Finite sums of vectors, entrywise.

definition *vec-sum* :: *nat* ⇒ *'c set* ⇒ (*'c* ⇒ *complex Matrix.vec*) ⇒ *complex Matrix.vec* **where**
vec-sum n Js w = *Matrix.vec n* (λ*i*. *sum* (λ*J*. *w J* $ *i*) *Js*)

lemma *vec-sum-dim* [*simp*]: *dim-vec* (*vec-sum n Js w*) = *n*
⟨*proof*⟩

lemma *vec-sum-index* [*simp*]: *i* < *n* ⟹ *vec-sum n Js w* $ *i* = *sum* (λ*J*. *w J* $ *i*) *Js*
⟨*proof*⟩

lemma *vec-sum-cong*:
assumes ⋀*J*. *J* ∈ *Js* ⟹ *w J* = *w' J*
shows *vec-sum n Js w* = *vec-sum n Js w'*
⟨*proof*⟩

lemma *mult-mat-vec-vec-sum*:
assumes *Ac*: *A* ∈ *carrier-mat m n* **and** *fin*: *finite Js*
and *wd*: ⋀*J*. *J* ∈ *Js* ⟹ *dim-vec* (*w J*) = *n*
shows *A* ∗$_v$ *vec-sum n Js w* = *vec-sum m Js* (λ*J*. *A* ∗$_v$ *w J*)
⟨*proof*⟩

lemma *inner-prod-vec-sum-right*:
assumes *ud*: *dim-vec u* = *n*
and *wd*: ⋀*J*. *J* ∈ *Js* ⟹ *dim-vec* (*w J*) = *n*
shows ⟨*u* | *vec-sum n Js w*⟩ = *sum* (λ*J*. ⟨*u* | *w J*⟩) *Js*

⟨*proof*⟩

lemma *mult-mat-vec-smult-vec*:
fixes A :: *complex Matrix.mat* **and** v :: *complex Matrix.vec*
assumes *Ac*: $A \in$ *carrier-mat m n* **and** *vd*: *dim-vec* $v = n$
shows $A *_v (c \cdot_v v) = c \cdot_v (A *_v v)$
⟨*proof*⟩

lemma *inner-prod-unit-vec-right*:
assumes *ud*: *dim-vec* $u = n$ **and** *j*: $j < n$
shows $\langle u \mid$ *unit-vec n j*$\rangle =$ *cnj* $(u \$ j)$
⟨*proof*⟩

lemma *outer-prod-self-psd*:
assumes *dim-vec* $v = n$
shows *psd* $(v \oslash v)$
⟨*proof*⟩

Conjugating a rank-one matrix: $W|\varphi\rangle\langle\varphi|W^* = |W\varphi\rangle\langle W\varphi|$.

lemma *conj-outer-prod*:
fixes W :: *complex Matrix.mat* **and** φ :: *complex Matrix.vec*
assumes *Wc*: $W \in$ *carrier-mat m n* **and** φ*d*: *dim-vec* $\varphi = n$
shows $W * (\varphi \oslash \varphi) * (W^\dagger) = (W *_v \varphi) \oslash (W *_v \varphi)$
⟨*proof*⟩

lemma *outer-prod-zero-vec*: $(0_v\ n$ :: *complex Matrix.vec*$) \oslash (0_v\ n) = 0_m\ n\ n$
⟨*proof*⟩

lemma *idx8*: $k <$ (*8*::*nat*) $\Longrightarrow k = 0 \vee k = 1 \vee k = 2 \vee k = 3 \vee k = 4 \vee k = 5 \vee k = 6 \vee k = 7$
⟨*proof*⟩

lemma *s8*: $\{0..<$(*8*::*nat*)$\} = \{0,1,2,3,4,5,6,7\}$ ⟨*proof*⟩

lemma *dagger-mult-diag-inner*:
fixes Y :: *complex Matrix.mat*
assumes *Yc*: $Y \in$ *carrier-mat p q* **and** *j*: $j < q$
shows $((Y^\dagger) * Y)$ \$\$ $(j,j) = \langle$*Matrix.col Y j* $\mid$ *Matrix.col Y j*$\rangle$
⟨*proof*⟩

lemma *foldr-add-mat-carrier-rect*:
assumes $\forall x \in$ *set xs*. *f x* $\in$ *carrier-mat p q*
shows *foldr* $(\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ p\ q) \in$ *carrier-mat p q*
⟨*proof*⟩

lemma *foldr-add-mat-index-rect*:
assumes $\forall x \in$ *set xs*. *f x* $\in$ *carrier-mat p q* **and** $i < p$ **and** $j < q$
shows *foldr* $(\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ p\ q)$ \$\$ $(i,j) = (\sum x \leftarrow xs.\ f\ x$ \$\$ $(i,j))$
⟨*proof*⟩

end
theory *Day12-Counting-Probability*
imports *QC-Prelims*
begin

34 Day 12: $H^{\otimes N}$ on $|0\cdots 0\rangle$ and the counting probability (D12-1, D12-4)

N-fold tensor powers (last factor least significant, matching $(\otimes)$).

primrec *tensor-pow-mat* :: *complex Matrix.mat* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
  tensor-pow-mat A 0 = 1$_m$ *1*
| *tensor-pow-mat A* (*Suc k*) = *Tensor.tensor-mat* (*tensor-pow-mat A k*) *A*

primrec *tensor-pow-vec* :: *complex Matrix.vec* ⇒ *nat* ⇒ *complex Matrix.vec* **where**
  tensor-pow-vec v 0 = Matrix.vec 1 (λ-. *1*)
| *tensor-pow-vec v* (*Suc k*) = *tensor-vec* (*tensor-pow-vec v k*) *v*

lemma *tensor-pow-mat-carrier*:
  assumes *A* ∈ *carrier-mat 2 2*
  shows *tensor-pow-mat A k* ∈ *carrier-mat* (*2^k*) (*2^k*)
  ⟨*proof*⟩

lemma *tensor-pow-vec-dim*:
  assumes *dim-vec v = 2*
  shows *dim-vec* (*tensor-pow-vec v k*) = *2^k*
  ⟨*proof*⟩

lemma *tensor-pow-mat-vec*:
  assumes *Ac*: *A* ∈ *carrier-mat 2 2* **and** *vd*: *dim-vec v = 2*
  shows *tensor-pow-mat A k* ∗$_v$ *tensor-pow-vec v k = tensor-pow-vec* (*A* ∗$_v$ *v*) *k*
⟨*proof*⟩

lemma *tensor-pow-vec-const*:
  tensor-pow-vec (*Matrix.vec 2* (λ-. *c*)) *k = Matrix.vec* (*2^k*) (λ-. *c^k*)
⟨*proof*⟩

definition *uniform-vec* :: *nat* ⇒ *complex Matrix.vec* **where**
  uniform-vec N = Matrix.vec (*2^N*) (λ-. *complex-of-real* (*1 / sqrt 2*) ^ *N*)

lemma *uniform-vec-dim* [*simp*]: *dim-vec* (*uniform-vec N*) = *2^N*
  ⟨*proof*⟩

lemma *hadamard-e0-const*: *Hadamard* ∗$_v$ *unit-vec 2 0 = Matrix.vec 2* (λ-. *complex-of-real* (*1 / sqrt 2*))
⟨*proof*⟩

theorem *hadamard-uniform-N*:
  tensor-pow-mat Hadamard N ∗$_v$ *tensor-pow-vec* (*unit-vec 2 0*) *N = uniform-vec N*
⟨*proof*⟩

lemma *inner-prod-unit-vec-2*:
  assumes *k < 2*
  shows ⟨*unit-vec 2 0* | *unit-vec 2 k*⟩ = (*if k = 0 then 1 else 0*)
⟨*proof*⟩

lemma *inner-prod-uniform-unit*:
  assumes *J < 2^N*
  shows ⟨*uniform-vec N* | *unit-vec* (*2^N*) *J*⟩ = *complex-of-real* (*1 / sqrt 2*) ^ *N*
  ⟨*proof*⟩

lemma *half-sqrt2-sq*: (*1 / sqrt 2* :: *real*) ∗ (*1 / sqrt 2*) = *1 / 2*
⟨*proof*⟩

lemma *uniform-coeff-sq*:
*complex-of-real (1 / sqrt 2) ^ N * complex-of-real (1 / sqrt 2) ^ N = complex-of-real ((1/2) ^ N)*
⟨*proof*⟩

lemma *uniform-vec-self*: ⟨*uniform-vec N* | *uniform-vec N*⟩ = *1*
⟨*proof*⟩

lemma *uniform-tensor-e0-as-sum*:
tensor-vec (uniform-vec N) (unit-vec 2 0) =
vec-sum (2^(N+1)) {J. J < 2^N}
(λJ. complex-of-real (1 / sqrt 2) ^ N ·$_v$ tensor-vec (unit-vec (2^N) J) (unit-vec 2 0))
⟨*proof*⟩

lemma *uniform-e0-state*: *quantum-state (2^(N+1)) (tensor-vec (uniform-vec N) (unit-vec 2 0))*
⟨*proof*⟩

The notes' computation: with $x = H^{\otimes N}|0\cdots0\rangle$ and $\gamma = x \otimes e_0$, $p_0(U_f(x \otimes e_0)) = (M/2^N)^2$ where $M = \#\{J : f(J) = 0\}$.

theorem *counting-probability*:
fixes *f* :: *nat* ⇒ *nat* **and** *U* :: *complex Matrix.mat*
assumes *f-range*: ∀ *J* < *2^N*. *f J* < *2*
and *Uc*: *U* ∈ *carrier-mat (2^(N+1)) (2^(N+1))*
and *orac*: ∀ *J* < *2^N*. ∀ *i* < *2*.
U *$_v$ (tensor-vec (unit-vec (2^N) J) (unit-vec 2 i)) =*
tensor-vec (unit-vec (2^N) J) (unit-vec 2 ((i + f J) mod 2))
shows *meas-prob (tensor-vec (uniform-vec N) (unit-vec 2 0) ⊘ tensor-vec (uniform-vec N) (unit-vec 2 0))*
*(U *$_v$ tensor-vec (uniform-vec N) (unit-vec 2 0))*
= (real (count-zeros f N) / 2^N)^2
⟨*proof*⟩

end
theory *Day17-Environment-Model*
imports *Day22-Minimal-Kraus*
begin

Day 17, the environment model of noise: a state ρ of lab A meets an environment in the state $|\varphi\rangle$, the pair evolves by a unitary U on $\mathbb{C}^m \otimes \mathbb{C}^p$, and lab A sees $\mathrm{tr}_E(U(\rho \otimes |\varphi\rangle\langle\varphi|)U^*) = \sum_i W_i \rho W_i^*$ with $\sum_i W_i^* W_i = I$. The index of $\mathbb{C}^m \otimes \mathbb{C}^p$ is $a\,p + i$ (lab index a, environment index i), and W_i is the matrix with entries $\sum_k U_{(a,i),(c,k)}\varphi_k$, i.e. the notes' $W_i = \sum_k U_{ik}\alpha_k$ for the blocks U_{ik} of U.

definition *env-kraus* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.vec* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
*env-kraus m p U φ i = Matrix.mat m m (λ(a,c). ∑k<p. U $$ (a * p + i, c * p + k) * φ $ k)*

lemma *env-kraus-carrier* [*simp*]: *env-kraus m p U φ i ∈ carrier-mat m m*
⟨*proof*⟩

lemma *env-kraus-dims* [*simp*]: *dim-row (env-kraus m p U φ i) = m dim-col (env-kraus m p U φ i) = m*
⟨*proof*⟩

lemma *env-kraus-index*: *a* < *m* ⟹ *c* < *m* ⟹ *env-kraus m p U φ i $$ (a,c) = (∑k<p. U $$ (a * p + i, c * p + k) * φ $ k)*
⟨*proof*⟩

lemma *sum-split-mp*:
fixes $f :: nat \Rightarrow 'a$::*comm-monoid-add*
shows $(\sum x < m * p.\ f\ x) = (\sum c{<}m.\ \sum k{<}p.\ f\ (c * p + k))$
⟨*proof*⟩

lemma *idx-mp*:
fixes $c\ k :: nat$
assumes *c*: $c < m$ **and** *k*: $k < p$
shows $c * p + k < m * p\ (c * p + k)\ div\ p = c\ (c * p + k)\ mod\ p = k$
⟨*proof*⟩

lemma *tensor-outer-index*:
fixes ϱ :: *complex Matrix.mat* **and** φ :: *complex Matrix.vec*
assumes ϱ: $\varrho \in$ *carrier-mat m m* **and** φ: *dim-vec* $\varphi = p$
and *c*: $c < m$ **and** *d*: $d < m$ **and** *k*: $k < p$ **and** *l*: $l < p$
shows *Tensor.tensor-mat* $\varrho\ (\varphi \oslash \varphi)$ \$\$ $(c * p + k,\ d * p + l) = \varrho$ \$\$ $(c,d) * (\varphi$ \$ $k * cnj\ (\varphi$ \$ $l))$
⟨*proof*⟩

The entries of $U(\rho \otimes |\varphi\rangle\langle\varphi|)U^*$.

lemma *env-entry*:
fixes $U\ \varrho$ :: *complex Matrix.mat* **and** φ :: *complex Matrix.vec*
assumes *U*: $U \in$ *carrier-mat* $(m * p)\ (m * p)$ **and** ϱ: $\varrho \in$ *carrier-mat m m* **and** φ: *dim-vec* $\varphi = p$
and *x*: $x < m * p$ **and** *y*: $y < m * p$
shows $(U *$ *Tensor.tensor-mat* $\varrho\ (\varphi \oslash \varphi) * (U^\dagger))$ \$\$ (x,y)
$= (\sum d{<}m.\ \sum c{<}m.\ (\sum k{<}p.\ U$ \$\$ $(x,\ c * p + k) * \varphi$ \$ $k) * \varrho$ \$\$ (c,d)
$* cnj\ (\sum l{<}p.\ U$ \$\$ $(y,\ d * p + l) * \varphi$ \$ $l))$
⟨*proof*⟩

theorem *environment-model*:
fixes $U\ \varrho$ :: *complex Matrix.mat* **and** φ :: *complex Matrix.vec*
assumes *U*: $U \in$ *carrier-mat* $(m * p)\ (m * p)$ **and** *Uu*: *unitary U*
and φ: *dim-vec* $\varphi = p$ **and** $\varphi 1$: $\langle\varphi|\varphi\rangle = 1$ **and** ϱ: $\varrho \in$ *carrier-mat m m*
defines $Ws \equiv$ *map* (*env-kraus m p U* φ) $[0..{<}p]$
shows *partial-trace-B m p* $(U *$ *Tensor.tensor-mat* $\varrho\ (\varphi \oslash \varphi) * (U^\dagger)) =$ *kraus-map-dim m Ws* ϱ
and *kraus-operators m m Ws*
⟨*proof*⟩

end
theory *Day18-Positive-Maps*
imports *QC-Prelims*
begin

35 Days 17–18: densities to densities iff positive and trace-preserving (D17-4, D18-2)

theorem *density-preserving-iff-positive-tp*:
fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow B \in$ *carrier-mat n n* $\Longrightarrow$
$\Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow \Phi\ A \in$ *carrier-mat d d*
and *npos*: $0 < n$
shows $(\forall \varrho.$ *density-matrix n* $\varrho \longrightarrow$ *density-matrix d* $(\Phi\ \varrho)) \longleftrightarrow$
(*positive-map n d* $\Phi \wedge$ *trace-preserving n* Φ)
⟨*proof*⟩

36 Day 18: the transpose is positive but not 2-positive (D18-4)

lemma *transpose-positive-map*: *positive-map n n* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*)
⟨*proof*⟩

The notes' witness: $X = (E_{ij}) \in M_2(M_2)$ is positive (it is $|w\rangle\langle w|$ for $w = e_0 \otimes e_0 + e_1 \otimes e_1$) while $\Phi^{(2)}(X)$ has the eigenvector $(0, 1, -1, 0)$ with eigenvalue -1 (the notes compute $\det \Phi^{(2)}(X) = -1$ instead).

lemma *choi-seed-2-outer*: *choi-seed 2 = outer-sum 4* [*Matrix.vec 4* (λ*i*. *if i = 0 ∨ i = 3 then 1 else 0*)]
⟨*proof*⟩

lemma *choi-seed-2-psd*: *psd* (*choi-seed 2*)
⟨*proof*⟩

lemma *transpose-not-2-positive*:
¬ *p-positive 2 2 2* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*)
⟨*proof*⟩

corollary *transpose-not-completely-positive*:
¬ *completely-positive 2 2* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*)
⟨*proof*⟩

end
theory *Day19-Choi-TP-Unital*
imports *Day25-Arveson-Correspondence*
begin

Day 19, notes (4) and (5) after Choi's theorem: write $P_\Phi = (R_{ij})$ with $R_{ij} = \Phi(E_{ij}) \in M_d$. Then Φ is trace preserving iff $\operatorname{Tr} R_{ij} = \delta_{ij}$, and unital iff $\sum_i R_{ii} = I_d$; with *choi-theorem* this characterises CPTP and UCP maps by their Choi matrix. Linearity on M_n is *lin-on n d* (*carrier-mat n n*) Φ.

lemma *mat-unit-carrier′* [*simp*]: *E⟦i,j ; n,n⟧ ∈ carrier-mat n n*
⟨*proof*⟩

lemma *choi-block*:
assumes *range*: ∀ *A*. *A ∈ carrier-mat n n ⟶ Φ A ∈ carrier-mat d d* **and** *i*: *i < n* **and** *j*: *j < n*
shows *block-of-mat d* (*choi-matrix n d Φ*) *i j = Φ* (*E⟦i,j ; n,n⟧*)
⟨*proof*⟩

lemma *msum-cong*: (⋀*x*. *x ∈ set xs ⟹ g x = h x*) *⟹ msum r c g xs = msum r c h xs*
⟨*proof*⟩

lemma *Tr-msum*:
assumes ∀ *x ∈ set xs*. *g x ∈ carrier-mat d d*
shows *Tr* (*msum d d g xs*) = (∑ *x*←*xs*. *Tr* (*g x*))
⟨*proof*⟩

lemma *unit-expansion*:
assumes *A*: *A ∈ carrier-mat n n*
shows *A = msum n n* (λ(*i*,*j*). *A* $$ (*i*,*j*) ·$_m$ *E⟦i,j ; n,n⟧*) (*List.product* [*0*..<*n*] [*0*..<*n*])
⟨*proof*⟩

lemma *id-expansion*: *1*$_m$ *n = msum n n* (λ*i*. *E⟦i,i ; n,n⟧*) [*0*..<*n*]
⟨*proof*⟩

lemma *lin-expansion*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *A*: *A* ∈ *carrier-mat n n*
shows Φ *A* = *msum d d* (λ(*i*,*j*). *A* $$ (*i*,*j*) ·$_m$ Φ (*E*⟦*i*,*j* ; *n*,*n*⟧)) (*List.product* [*0*..<*n*] [*0*..<*n*])
⟨*proof*⟩

D19-3, note (4): trace preservation in Choi-matrix form.

theorem *choi-trace-preserving-iff*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ
shows *trace-preserving n* Φ ⟷
(∀ *i* < *n*. ∀ *j* < *n*. *Tr* (*block-of-mat d* (*choi-matrix n d* Φ) *i j*) = (*if i* = *j then 1 else 0*))
⟨*proof*⟩

D19-3, note (5): unitality in Choi-matrix form, $\sum_i R_{ii} = I_d$.

theorem *choi-unital-iff*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ
shows *unital-map n d* Φ ⟷ *msum d d* (λ*i*. *block-of-mat d* (*choi-matrix n d* Φ) *i i*) [*0*..<*n*] = *1*$_m$ *d*
⟨*proof*⟩

lemma *lin-on-choi-hyps*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ
shows ⋀*A B*. *A* ∈ *carrier-mat n n* ⟹ *B* ∈ *carrier-mat n n* ⟹ Φ (*A* + *B*) = Φ *A* + Φ *B*
and ⋀*c A*. *A* ∈ *carrier-mat n n* ⟹ Φ (*c* ·$_m$ *A*) = *c* ·$_m$ Φ *A*
and ∀ *A*. *A* ∈ *carrier-mat n n* ⟶ Φ *A* ∈ *carrier-mat d d*
⟨*proof*⟩

Notes (4) and (5) with Choi's theorem: CPTP iff $P_\Phi \geq 0$ and $\operatorname{Tr} R_{ij} = \delta_{ij}$; UCP iff $P_\Phi \geq 0$ and $\sum_i R_{ii} = I_d$.

corollary *choi-cptp-iff*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *n*: *n* > *0* **and** *d*: *d* > *0*
shows *completely-positive n d* Φ ∧ *trace-preserving n* Φ ⟷
psd (*choi-matrix n d* Φ) ∧
(∀ *i* < *n*. ∀ *j* < *n*. *Tr* (*block-of-mat d* (*choi-matrix n d* Φ) *i j*) = (*if i* = *j then 1 else 0*))
⟨*proof*⟩

corollary *choi-ucp-iff*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *n*: *n* > *0* **and** *d*: *d* > *0*
shows *completely-positive n d* Φ ∧ *unital-map n d* Φ ⟷
psd (*choi-matrix n d* Φ) ∧ *msum d d* (λ*i*. *block-of-mat d* (*choi-matrix n d* Φ) *i i*) [*0*..<*n*] = *1*$_m$ *d*
⟨*proof*⟩

end
theory *Day20-Choi-Rank*
imports *Day34-Knill-Laflamme Day22-Kraus-Independence*
begin

37 Day 20: Choi's rank theorem $\operatorname{cr}(\Phi) = \operatorname{rank} P_\Phi$ (D20-4)

Rank is JNF's *vec-space.rank*, the dimension of the column space.

abbreviation *mat-rank* :: *nat* ⇒ *complex Matrix.mat* ⇒ *nat* **where**
mat-rank m A ≡ *vec-space.rank m A*

37.1 $\operatorname{rank}(\sum_{l<L} w_l w_l^*) \leq L$

lemma *rank-outer-prod-le-1*:
assumes *vd*: *dim-vec v* = *m*
shows *mat-rank m* (*v* ⊘ *v*) ≤ *1*

⟨*proof*⟩

lemma *rank-foldr-outer-le*:
assumes *vsd*: $\forall\, v \in set\ vs.\ dim\text{-}vec\ v = m$
shows $mat\text{-}rank\ m\ (foldr\ (\lambda v\ acc.\ v \oslash v + acc)\ vs\ (0_m\ m\ m)) \leq length\ vs$
⟨*proof*⟩

lemma *rank-outer-sum-le*:
assumes *vsd*: $\forall\, v \in set\ vs.\ dim\text{-}vec\ v = m$
shows $mat\text{-}rank\ m\ (outer\text{-}sum\ m\ vs) \leq length\ vs$
⟨*proof*⟩

37.2 Independent vectors inside the column space bound the rank from below

lemma *mult-mat-vec-in-span*:
assumes *Ac*: $A \in carrier\text{-}mat\ m\ k$ **and** *xc*: $x \in carrier\text{-}vec\ k$
shows $A *_v x \in module.span\ class\text{-}ring\ (module\text{-}vec\ TYPE(complex)\ m)\ (set\ (cols\ A))$
⟨*proof*⟩

lemma *rank-ge-indep-in-colspace*:
assumes *Ac*: $A \in carrier\text{-}mat\ m\ k$ **and** *li*: *lin-indep-vecs m ws*
and *insp*: $\bigwedge w.\ w \in set\ ws \Longrightarrow w \in module.span\ class\text{-}ring\ (module\text{-}vec\ TYPE(complex)\ m)\ (set\ (cols\ A))$
shows $length\ ws \leq mat\text{-}rank\ m\ A$
⟨*proof*⟩

37.3 The theorem

lemma *choi-rank-attained*:
fixes $\Phi :: complex\ Matrix.mat \Rightarrow complex\ Matrix.mat$
assumes *linear*: $\bigwedge A\ B.\ A \in carrier\text{-}mat\ n\ n \Longrightarrow B \in carrier\text{-}mat\ n\ n \Longrightarrow \Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in carrier\text{-}mat\ n\ n \Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall\, A.\ A \in carrier\text{-}mat\ n\ n \longrightarrow \Phi\ A \in carrier\text{-}mat\ d\ d$
and *cp*: *completely-positive n d* Φ
and *npos*: $0 < n$ **and** *dpos*: $0 < d$
shows $\exists\, As :: complex\ Matrix.mat\ list.\ length\ As = choi\text{-}rank\ n\ d\ \Phi \wedge$
$(\forall\, A \in set\ As.\ A \in carrier\text{-}mat\ d\ n) \wedge$
$(\forall\, X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow \Phi\ X = kraus\text{-}map\text{-}dim\ d\ As\ X)$
⟨*proof*⟩

theorem *choi-rank-eq-rank*:
fixes $\Phi :: complex\ Matrix.mat \Rightarrow complex\ Matrix.mat$
assumes *linear*: $\bigwedge A\ B.\ A \in carrier\text{-}mat\ n\ n \Longrightarrow B \in carrier\text{-}mat\ n\ n \Longrightarrow \Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in carrier\text{-}mat\ n\ n \Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall\, A.\ A \in carrier\text{-}mat\ n\ n \longrightarrow \Phi\ A \in carrier\text{-}mat\ d\ d$
and *cp*: *completely-positive n d* Φ
and *npos*: $0 < n$ **and** *dpos*: $0 < d$
shows $choi\text{-}rank\ n\ d\ \Phi = mat\text{-}rank\ (n{*}d)\ (choi\text{-}matrix\ n\ d\ \Phi)$
⟨*proof*⟩

end
theory *Day21-Unitary-Freedom*
imports *QC-Prelims*
begin

38 Day 21: unitary freedom of rank-one decompositions and Kraus representations (D21-3, D21-5, D21-6)

38.1 The Gram matrix of an independent list is invertible; dual vectors

lemma *gram-det-nonzero*:
  assumes *li*: *lin-indep-vecs n vs*
  shows *det* $((\textit{mat-of-cols}\ n\ \textit{vs})^\dagger * \textit{mat-of-cols}\ n\ \textit{vs}) \neq 0$
⟨*proof*⟩

lemma *gram-inverse*:
  assumes *li*: *lin-indep-vecs n vs*
  obtains B **where** $B \in \textit{carrier-mat}\ (\textit{length vs})\ (\textit{length vs})$
    and $((\textit{mat-of-cols}\ n\ \textit{vs})^\dagger * \textit{mat-of-cols}\ n\ \textit{vs}) * B = 1_m\ (\textit{length vs})$
    and $B * ((\textit{mat-of-cols}\ n\ \textit{vs})^\dagger * \textit{mat-of-cols}\ n\ \textit{vs}) = 1_m\ (\textit{length vs})$
    and $B^\dagger = B$
⟨*proof*⟩

Dual vectors (the notes' "T^* is onto"): for independent $v_1, \ldots, v_k$ there are $\varphi_1, \ldots, \varphi_k$ with $\langle \varphi_i | v_j \rangle = \delta_{ij}$.

lemma *dual-vectors-exist*:
  assumes *li*: *lin-indep-vecs n vs*
  shows $\exists \varphi s :: \textit{complex Matrix.vec list}.\ \textit{length}\ \varphi s = \textit{length vs} \wedge$
    $(\forall \varphi \in \textit{set}\ \varphi s.\ \textit{dim-vec}\ \varphi = n) \wedge$
    $(\forall i < \textit{length vs}.\ \forall j < \textit{length vs}.\ \langle \varphi s\ !\ i \mid \textit{vs}\ !\ j \rangle = (\textit{if } i = j \textit{ then 1 else 0}))$
⟨*proof*⟩

38.2 D21-3: independent vectors give independent rank-one matrices

definition *outer-prods* :: *complex Matrix.vec list* ⇒ *complex Matrix.mat list* **where**
  outer-prods vs = *map* $(\lambda k.\ \textit{vs}\ !\ (k\ \textit{div length vs}) \oslash \textit{vs}\ !\ (k\ \textit{mod length vs}))\ [0..<\textit{length vs} * \textit{length vs}]$

lemma *length-outer-prods* [*simp*]: *length* (*outer-prods vs*) = *length vs* ∗ *length vs*
  ⟨*proof*⟩

lemma *outer-prods-nth*:
  assumes $k <$ *length vs* ∗ *length vs*
  shows *outer-prods vs* ! *k* = *vs* ! (*k div length vs*) ⊘ *vs* ! (*k mod length vs*)
  ⟨*proof*⟩

lemma *inner-prod-cnj-delta*:
  assumes *dim-vec* $\varphi = n$ **and** *dim-vec* $v = n$ **and** $\langle \varphi \mid v \rangle = (\textit{if } P \textit{ then 1 else 0})$
  shows $\langle v \mid \varphi \rangle = (\textit{if } P \textit{ then 1 else 0})$
  ⟨*proof*⟩

proposition *outer-prods-lin-indep*:
  assumes *li*: *lin-indep-vecs n vs*
  shows *mat-lin-indep-complex n* (*outer-prods vs*)
⟨*proof*⟩

38.3 Isometric mixing of columns preserves VV^*

lemma *dagger-transpose-mult*:
  assumes *Uc*: $U \in \textit{carrier-mat}\ m\ r$ **and** *UU*: $(U^\dagger) * U = 1_m\ r$
  shows $\textit{transpose-mat}\ U * ((\textit{transpose-mat}\ U)^\dagger) = 1_m\ r$
⟨*proof*⟩

lemma *isometry-mix-outer*:

assumes *Vc*: $V \in$ *carrier-mat n r* **and** *Uc*: $U \in$ *carrier-mat m r* **and** *UU*: $(U^\dagger) * U = 1_m\ r$
shows $(V * \textit{transpose-mat}\ U) * ((V * \textit{transpose-mat}\ U)^\dagger) = V * (V^\dagger)$
⟨*proof*⟩

lemma *dagger-mult-zero-imp-zero*:
fixes Y :: *complex Matrix.mat*
assumes *Yc*: $Y \in$ *carrier-mat p q* **and** *YY*: $(Y^\dagger) * Y = 0_m\ q\ q$
shows $Y = 0_m\ p\ q$
⟨*proof*⟩

38.4 D21-5: two rank-one decompositions are related by an isometry

If $P = \sum_{j<r} v_j v_j^* = \sum_{l<m} w_l w_l^*$ with $v_1, \ldots, v_r$ linearly independent (the notes assume rank $P = r$, which is equivalent), then $w_l = \sum_j \alpha_{lj} v_j$ with $U = (\alpha_{lj})$ an $m \times r$ isometry, $U^*U = I_r$. Matrix form: $W = VU^t$.

theorem *rank-one-decomp-isometry*:
assumes *li*: *lin-indep-vecs n vs*
and *wsd*: $\forall\, w \in$ *set ws. dim-vec w = n*
and *same*: *outer-sum n vs = outer-sum n ws*
shows $\exists\, U$ :: *complex Matrix.mat. U* $\in$ *carrier-mat* (*length ws*) (*length vs*) $\wedge$
$(U^\dagger) * U = 1_m$ (*length vs*) $\wedge$
*mat-of-cols n ws = mat-of-cols n vs * transpose-mat U*
⟨*proof*⟩

corollary *rank-one-decomp-isometry-cols*:
assumes *li*: *lin-indep-vecs n vs*
and *wsd*: $\forall\, w \in$ *set ws. dim-vec w = n*
and *same*: *outer-sum n vs = outer-sum n ws*
shows $\exists\, U$ :: *complex Matrix.mat. U* $\in$ *carrier-mat* (*length ws*) (*length vs*) $\wedge$
$(U^\dagger) * U = 1_m$ (*length vs*) $\wedge$
$(\forall\, l <$ *length ws. ws* ! $l =$ *mat-of-cols n vs* $*_v$ *Matrix.row U l*)
⟨*proof*⟩

38.5 D21-6 (Choi 2): unitary freedom of Kraus representations

lemma *vec-of-kraus-mat-of-cols-index*:
assumes *Vs-c*: $\forall\, V \in$ *set Vs. V* $\in$ *carrier-mat d n* **and** *dpos*: $0 < d$
and *i*: $i < d$ **and** *a*: $a < n$ **and** *k*: $k <$ *length Vs*
shows *mat-of-cols* $(n*d)$ (*map* (*vec-of-kraus d n*) *Vs*) $\$\$\ (a*d + i,\ k) =$ *Vs* ! $k\ \$\$\ (i,\ a)$
⟨*proof*⟩

lemma *lin-indep-complex-mat-vec-of-kraus*:
assumes *li*: *lin-indep-complex-mat d n Vs* **and** *dpos*: $0 < d$
shows *lin-indep-vecs* $(n*d)$ (*map* (*vec-of-kraus d n*) *Vs*)
⟨*proof*⟩

Choi's Theorem 2: if $\Phi(X) = \sum_{j<r} V_j X V_j^* = \sum_{l<m} W_l X W_l^*$ with $V_1, \ldots, V_r$ linearly independent (the notes: $r = \mathrm{cr}(\Phi)$), then $W_l = \sum_j \alpha_{lj} V_j$ with $U = (\alpha_{lj})$ an isometry, $U^*U = I_r$.

theorem *choi-2-unitary-freedom*:
assumes *npos*: $0 < n$ **and** *dpos*: $0 < d$
and *Ws-c*: $\forall\, W \in$ *set Ws. W* $\in$ *carrier-mat d n*
and *li*: *lin-indep-complex-mat d n Vs*
and *same*: $\forall\, X.\ X \in$ *carrier-mat n n* $\longrightarrow$ *kraus-map-dim d Vs X = kraus-map-dim d Ws X*
shows $\exists\, U$ :: *complex Matrix.mat. U* $\in$ *carrier-mat* (*length Ws*) (*length Vs*) $\wedge$
$(U^\dagger) * U = 1_m$ (*length Vs*) $\wedge$
$(\forall\, l <$ *length Ws. Ws* ! $l =$
foldr $(\lambda j\ acc.\ U\ \$\$\ (l,j) \cdot_m (Vs\ !\ j) + acc)\ [0..<\textit{length Vs}]\ (0_m\ d\ n))$

⟨*proof*⟩

end
theory *Day22-Kraus-Independence*
imports *QC-Prelims Jordan-Normal-Form.DL-Rank*
begin

39 Day 22: linear independence bounds the number of Kraus operators (D22-6)

A linearly independent list of vectors in $\mathbb{C}^n$ has at most n members. The bridge is to JNF's *vec-space*: kernel-triviality of *mat-of-cols* is JNF's *lin-indpt* (via *vec-space.lin-depE*), and *li-le-dim* bounds the cardinality.

lemma *lin-indep-vecs-distinct*:
assumes *li*: *lin-indep-vecs n vs*
shows *distinct vs*
⟨*proof*⟩

lemma *lin-indep-vecs-length-le*:
assumes *li*: *lin-indep-vecs n vs*
shows *length vs* ≤ *n*
⟨*proof*⟩

Flattening a matrix into a vector carries *mat-lin-indep-complex* to *lin-indep-vecs*, so a linearly independent family in M_n has at most n^2 members.

definition *flatten-mat* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.vec* **where**
flatten-mat n B = *Matrix.vec* (*n*∗*n*) (λ*k*. *B* $$ (*k div n*, *k mod n*))

lemma *flatten-mat-dim* [*simp*]: *dim-vec* (*flatten-mat n B*) = *n*∗*n*
⟨*proof*⟩

lemma *flatten-mat-index*:
assumes *i* < *n* **and** *j* < *n*
shows *flatten-mat n B* $ (*i*∗*n* + *j*) = *B* $$ (*i*,*j*)
⟨*proof*⟩

lemma *mat-lin-indep-complex-flatten*:
assumes *li*: *mat-lin-indep-complex n Bs*
shows *lin-indep-vecs* (*n*∗*n*) (*map* (*flatten-mat n*) *Bs*)
⟨*proof*⟩

lemma *mat-lin-indep-complex-length-le*:
assumes *mat-lin-indep-complex n Bs*
shows *length Bs* ≤ *n*∗*n*
⟨*proof*⟩

Corollary to Choi's Theorem 3 (D22-6): an extreme CPTP map $M_n \to M_d$ with a minimal Kraus representation $(V_i)_{i<r}$ has $r^2 \le n^2$, i.e. $\mathrm{cr}(\Phi) \le n$.

lemma *length-kraus-products*: *length* (*kraus-products Vs*) = *length Vs* ∗ *length Vs*
⟨*proof*⟩

corollary *extreme-CPTP-kraus-length-le*:
assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
and Φ-*in*: Φ ∈ *CPTP n d*
and *rep*: *cptp-kraus-rep n d* Φ *Vs*

and *minimal*: *lin-indep-complex-mat d n Vs*
and *ext*: *extreme-CPTP-carrier n d* Φ
shows *length Vs* ≤ *n*
⟨*proof*⟩

corollary *extreme-CPTP-choi-rank-le*:
assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
and Φ*-in*: Φ ∈ *CPTP n d*
and *rep*: *cptp-kraus-rep n d* Φ *Vs*
and *minimal*: *lin-indep-complex-mat d n Vs*
and *ext*: *extreme-CPTP-carrier n d* Φ
shows *choi-rank n d* Φ ≤ *n*
⟨*proof*⟩

end
theory *Day22-Minimal-Kraus*
imports *Day32-Recovery-Proposition*
begin

40 Day 22: a minimal Kraus representation is linearly independent (D22-1)

40.1 Mixing a Kraus list by an isometry does not change the map

definition *kraus-mix* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat list* **where**
kraus-mix d n U Vs =
map (λ*l*. *foldr* (λ*j acc*. *U* $$ (*l*,*j*) ·$_m$ (*Vs* ! *j*) + *acc*) [*0*..<*length Vs*] (*0*$_m$ *d n*)) [*0*..<*dim-row U*]

lemma *kraus-mix-length* [*simp*]: *length* (*kraus-mix d n U Vs*) = *dim-row U*
⟨*proof*⟩

lemma *kraus-mix-nth*:
assumes *l* < *dim-row U*
shows *kraus-mix d n U Vs* ! *l* = *foldr* (λ*j acc*. *U* $$ (*l*,*j*) ·$_m$ (*Vs* ! *j*) + *acc*) [*0*..<*length Vs*] (*0*$_m$ *d n*)
⟨*proof*⟩

lemma *kraus-mix-carrier*:
assumes ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
shows ∀ *W* ∈ *set* (*kraus-mix d n U Vs*). *W* ∈ *carrier-mat d n*
⟨*proof*⟩

lemma *kraus-mix-entry*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n* **and** *l*: *l* < *dim-row U* **and** *i*: *i* < *d* **and** *a*: *a* < *n*
shows *kraus-mix d n U Vs* ! *l* $$ (*i*,*a*) = (∑ *j* = *0*..<*length Vs*. *U* $$ (*l*,*j*) ∗ *Vs* ! *j* $$ (*i*,*a*))
⟨*proof*⟩

lemma *kraus-mix-mat-of-cols*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n* **and** *Uc*: *U* ∈ *carrier-mat m* (*length Vs*) **and** *dpos*: *0* < *d*
shows *mat-of-cols* (*n*∗*d*) (*map* (*vec-of-kraus d n*) (*kraus-mix d n U Vs*))
= *mat-of-cols* (*n*∗*d*) (*map* (*vec-of-kraus d n*) *Vs*) ∗ *transpose-mat U*
⟨*proof*⟩

lemma *kraus-mix-same-map*:
assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
and *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*

and *Uc*: $U \in$ *carrier-mat m* (*length Vs*) **and** *UU*: $(U^\dagger) * U = 1_m$ (*length Vs*)
and *Xc*: $M \in$ *carrier-mat n n*
shows *kraus-map-dim d* (*kraus-mix d n U Vs*) *M* = *kraus-map-dim d Vs M*
⟨*proof*⟩

40.2 A unitary with a prescribed first row

lemma *mat-mult-entry*:
fixes *A B* :: *complex Matrix.mat*
assumes *A* ∈ *carrier-mat r s B* ∈ *carrier-mat s t k* < *r j* < *t*
shows (*A* ∗ *B*) $$ (*k*,*j*) = (∑ *l*<*s*. *A* $$ (*k*,*l*) ∗ *B* $$ (*l*,*j*))
⟨*proof*⟩

lemma *transpose-unitary-isometry*:
assumes *Mc*: *M* ∈ *carrier-mat r r* **and** *MM*: $M * (M^\dagger) = 1_m\ r$
shows $((\textit{transpose-mat}\ M)^\dagger) * \textit{transpose-mat}\ M = 1_m\ r$
⟨*proof*⟩

lemma *isometry-with-first-row*:
assumes *ud*: *dim-vec u* = *r* **and** *un*: ‖*u*‖ = *1*
shows ∃ *U* :: *complex Matrix.mat*. *U* ∈ *carrier-mat r r* ∧ $(U^\dagger) * U = 1_m\ r$ ∧ (∀ *j* < *r*. *U* $$ (*0*,*j*) = *u* $ *j*)
⟨*proof*⟩

40.3 D22-1

lemma *kraus-map-dim-zero-head*:
assumes *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n* **and** *Xc*: *M* ∈ *carrier-mat n n*
shows *kraus-map-dim d* (0_m *d n* # *Ws*) *M* = *kraus-map-dim d Ws M*
⟨*proof*⟩

If $\Phi(X) = \sum_{j<r} V_j X V_j^*$ with $r = \mathrm{cr}(\Phi)$, then $V_0, \ldots, V_{r-1}$ are linearly independent: a dependency $\sum_j c_j V_j = 0$ would allow a unitary change of representation whose first Kraus operator is 0, giving a representation with $r - 1$ terms.

theorem *minimal-kraus-rep-lin-indep*:
assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
and *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *minimal*: *choi-rank n d* (*kraus-map-dim d Vs*) = *length Vs*
shows *lin-indep-complex-mat d n Vs*
⟨*proof*⟩

end
theory *Day29-Classical-Codes*
imports *Quantum-Computing11*
begin

Days 29–30: classical codes over $\mathbb{Z}_2$. Words are lists over $\{0, 1\}$; all finite checks are decided by *code-simp* (proof-producing evaluation, no oracle).

definition *bin-words* :: *nat* ⇒ *nat list set* **where**
bin-words n = *set* (*List.n-lists n* [*0*, *1*])

definition *xor-word* :: *nat list* ⇒ *nat list* ⇒ *nat list* **where**
xor-word v w = *map2* (λ*a b*. (*a* + *b*) *mod 2*) *v w*

40.4 The notes' "Hamming [7,1,3]" code does not correct one error

The notes encode $(a_1,\ldots,a_7) \mapsto (a_1,\ldots,a_7;\ a_1{+}a_2{+}a_3{+}a_4;\ a_1{+}a_2{+}a_5{+}a_6;\ a_1{+}a_3{+}a_5{+}a_7)$ and claim it detects and corrects one error. It detects one error, but two codewords are at distance 2: the word e_4 is at distance 1 from the codewords of 0 and of e_4, so no decoder corrects every single error.

definition *notes-hamming-encode* :: *nat list* ⇒ *nat list* **where**
notes-hamming-encode a = *a* @ [(*a*!*0* + *a*!*1* + *a*!*2* + *a*!*3*) *mod 2*, (*a*!*0* + *a*!*1* + *a*!*4* + *a*!*5*) *mod 2*, (*a*!*0* + *a*!*2* + *a*!*4* + *a*!*6*) *mod 2*]

definition *notes-hamming-code* :: *nat list set* **where**
notes-hamming-code = *notes-hamming-encode* ‘ *bin-words 7*

definition *flip-at* :: *nat* ⇒ *nat list* ⇒ *nat list* **where**
flip-at i w = *w*[*i* := (*w*!*i* + *1*) *mod 2*]

lemma *notes-hamming-code-iff*:
w ∈ *notes-hamming-code* ⟷ *take 7 w* ∈ *bin-words 7* ∧ *w* = *notes-hamming-encode* (*take 7 w*)
⟨*proof*⟩

lemma *bin-words-iff*: *w* ∈ *bin-words n* ⟷ *length w* = *n* ∧ *set w* ⊆ {*0*, *1*}
⟨*proof*⟩

lemma *bin-nth-lt2*: *set v* ⊆ {*0*,*1*} ⟹ *k* < *length v* ⟹ *v* ! *k* < (*2*::*nat*)
⟨*proof*⟩

lemma *xor-word-length* [*simp*]: *length* (*xor-word v w*) = *min* (*length v*) (*length w*)
⟨*proof*⟩

lemma *xor-word-nth*: *k* < *length v* ⟹ *k* < *length w* ⟹ *xor-word v w* ! *k* = (*v* ! *k* + *w* ! *k*) *mod 2*
⟨*proof*⟩

lemma *xor-word-cancel*:
assumes *length c* = *length v length c* = *length w set v* ⊆ {*0*,*1*} *set w* ⊆ {*0*,*1*}
shows *xor-word c v* = *xor-word c w* ⟷ *v* = *w*
⟨*proof*⟩

lemma *mod2-eq-iff*: (*p*::*nat*) *mod 2* = *q mod 2* ⟷ (*even p* ⟷ *even q*)
⟨*proof*⟩

The encoding is linear over $\mathbb{Z}_2$.

lemma *notes-hamming-encode-xor*:
assumes *length a* = *7 length b* = *7*
shows *notes-hamming-encode* (*xor-word a b*) = *xor-word* (*notes-hamming-encode a*) (*notes-hamming-encode b*)
⟨*proof*⟩

lemma *flip-at-xor*:
assumes *set c* ⊆ {*0*,*1*} *i* < *length c*
shows *flip-at i c* = *xor-word c* ((*replicate* (*length c*) *0*)[*i* := *1*])
⟨*proof*⟩

theorem *notes-hamming-detects-one-error*:
∀ *c* ∈ *notes-hamming-code*. ∀ *i* < *10*. *flip-at i c* ∉ *notes-hamming-code*
⟨*proof*⟩

theorem *notes-hamming-ambiguous-word*:

defines $w \equiv [0,0,0,1,0,0,0,0,0,0]$
shows *notes-hamming-encode* (*replicate 7 0*) ∈ *notes-hamming-code*
and *notes-hamming-encode* $[0,0,0,1,0,0,0]$ ∈ *notes-hamming-code*
and *notes-hamming-encode* (*replicate 7 0*) ≠ *notes-hamming-encode* $[0,0,0,1,0,0,0]$
and *hamming-dist* (*notes-hamming-encode* (*replicate 7 0*)) *w* = *1*
and *hamming-dist* (*notes-hamming-encode* $[0,0,0,1,0,0,0]$) *w* = *1*
⟨*proof*⟩

theorem *notes-hamming-not-correcting*: ¬ *corrects-errors notes-hamming-code 1*
⟨*proof*⟩

40.5 The Hamming [7,4,3] code corrects one error

definition *hamming74-encode* :: *nat list* ⇒ *nat list* **where**
hamming74-encode a = *a* @ [(*a*!*0* + *a*!*1* + *a*!*3*) *mod 2*, (*a*!*0* + *a*!*2* + *a*!*3*) *mod 2*, (*a*!*1* + *a*!*2* + *a*!*3*) *mod 2*]

definition *hamming74-code* :: *nat list set* **where**
hamming74-code = *hamming74-encode* ‘ *bin-words 4*

lemma *ex1-in-set*: (∃!*c*. *c* ∈ *set xs* ∧ *P c*) ⟷ *card* (*set* (*filter P xs*)) = *1*
⟨*proof*⟩

theorem *hamming74-unique-decoding*:
∀ *w* ∈ *bin-words 7*. ∃!*c*. *c* ∈ *hamming74-code* ∧ *hamming-dist c w* ≤ *1*
⟨*proof*⟩

theorem *hamming74-corrects-one-error*: *corrects-errors hamming74-code 1*
⟨*proof*⟩

40.6 Probabilities of undetected and uncorrected errors (Day 30)

Each bit flips independently with probability p; an error pattern e of weight k on n bits has probability $p^k(1-p)^{n-k}$. An error is undetected when it turns the sent codeword into another codeword.

definition *word-prob* :: *real* ⇒ *nat* ⇒ *nat list* ⇒ *real* **where**
word-prob p n e = *p* ^ *hamming-weight e* ∗ (*1* − *p*) ^ (*n* − *hamming-weight e*)

definition *err-prob* :: *real* ⇒ *nat* ⇒ *nat list set* ⇒ *real* **where**
err-prob p n ES = (∑ *e* ∈ *ES*. *word-prob p n e*)

lemma *err-prob-list*: *distinct es* ⟹ *err-prob p n* (*set es*) = (∑ *e*←*es*. *word-prob p n e*)
⟨*proof*⟩

definition *rep-code* :: *nat list set* **where** *rep-code* = *repetition-encode* ‘ *bin-words 3*
definition *par-code* :: *nat list set* **where** *par-code* = *parity-check-encode* ‘ *bin-words 3*

lemma *rep-code-list*: *rep-code* = *set* (*map repetition-encode* (*List.n-lists 3* [*0*, *1*]))
⟨*proof*⟩

lemma *par-code-list*: *par-code* = *set* (*map parity-check-encode* (*List.n-lists 3* [*0*, *1*]))
⟨*proof*⟩

definition *undetected* :: *nat list set* ⇒ *nat* ⇒ *nat list* ⇒ *nat list set* **where**
undetected C n c = {*e* ∈ *bin-words n*. *e* ≠ *replicate n 0* ∧ *xor-word c e* ∈ *C*}

lemma *set-filter-bin*: {*e* ∈ *bin-words n*. *P e*} = *set* (*filter P* (*List.n-lists n* [*0*, *1*]))

⟨*proof*⟩

Repetition on three bits: $3p^2(1-p)^4 + 3p^4(1-p)^2 + p^6$.

theorem *repetition-undetected-prob*:
assumes *c*: *c* ∈ *rep-code*
shows *err-prob p 6* (*undetected rep-code 6 c*) = *3 * p^2 * (1 - p)^4 + 3 * p^4 * (1 - p)^2 + p^6*
⟨*proof*⟩

Parity on three bits: $\binom{4}{2}p^2(1-p)^2 + p^4$.

theorem *parity-undetected-prob*:
assumes *c*: *c* ∈ *par-code*
shows *err-prob p 4* (*undetected par-code 4 c*) = *6 * p^2 * (1 - p)^2 + p^4*
⟨*proof*⟩

The notes compare the two for $p < \frac{1}{2}$; the inequality in fact holds for all $0 < p < 1$.

theorem *repetition-better-than-parity*:
assumes *p0*: *0 < p* **and** *p1*: *p < 1*
shows *3 * p^2 * (1 - p)^4 + 3 * p^4 * (1 - p)^2 + p^6 < 6 * p^2 * (1 - p)^2 + (p::real)^4*
⟨*proof*⟩

corollary *repetition-better-than-parity-notes*:
*0 < p ⟹ p < 1/2 ⟹ 3 * p^2 * (1 - p)^4 + 3 * p^4 * (1 - p)^2 + p^6 < 6 * p^2 * (1 - p)^2 + (p::real)^4*
⟨*proof*⟩

Majority rule $0 \mapsto 000$, $1 \mapsto 111$: decoding fails with probability $3p^2(1-p) + p^3$.

definition *maj-decode* :: *nat list* ⇒ *nat* **where**
maj-decode w = (*if 2 ≤ sum-list w then 1 else 0*)

theorem *majority-uncorrected-prob*:
assumes *b*: *b* ∈ {*0*, *1*}
shows *err-prob p 3* {*e* ∈ *bin-words 3*. *maj-decode* (*xor-word* (*replicate 3 b*) *e*) ≠ *b*}
= *3 * p^2 * (1 - p) + p^3*
⟨*proof*⟩

end
theory *Day30-BitFlip-Code*
imports *QC-Prelims*
begin

41 Day 30: the three-qubit bit-flip code and its recovery map (D30-3, D30-4)

The syndrome projections $P_0 = |000\rangle\langle 000| + |111\rangle\langle 111|$, …, as diagonal 0/1 matrices ($|abc\rangle \leftrightarrow e_{4a+2b+c}$), and the single flips $X \otimes I \otimes I$, $I \otimes X \otimes I$, $I \otimes I \otimes X$ as permutation matrices.

definition *syndrome-proj* :: *nat set* ⇒ *complex Matrix.mat* **where**
syndrome-proj K = *Matrix.mat 8 8* (λ(*i*,*j*). *if i = j ∧ i ∈ K then 1 else 0*)

abbreviation P_0 ≡ *syndrome-proj* {*0*, *7*}
abbreviation P_1 ≡ *syndrome-proj* {*4*, *3*}
abbreviation P_2 ≡ *syndrome-proj* {*2*, *5*}
abbreviation P_3 ≡ *syndrome-proj* {*1*, *6*}

definition *flip-bit-mat* :: *nat* ⇒ *complex Matrix.mat* **where**

flip-bit-mat b = Matrix.mat 8 8 (λ(i,j). if j = (if i div 2^b mod 2 = 0 then i + 2^b else i − 2^b) then 1 else 0)

abbreviation $X_1 \equiv$ *flip-bit-mat 2*
abbreviation $X_2 \equiv$ *flip-bit-mat 1*
abbreviation $X_3 \equiv$ *flip-bit-mat 0*

lemma *syndrome-proj-carrier* [*simp*]: *syndrome-proj K ∈ carrier-mat 8 8*
⟨*proof*⟩

lemma *flip-bit-mat-carrier* [*simp*]: *flip-bit-mat b ∈ carrier-mat 8 8*
⟨*proof*⟩

lemma *syndrome-proj-dims* [*simp*]: *dim-row (syndrome-proj K) = 8 dim-col (syndrome-proj K) = 8*
⟨*proof*⟩

lemma *flip-bit-mat-dims* [*simp*]: *dim-row (flip-bit-mat b) = 8 dim-col (flip-bit-mat b) = 8*
⟨*proof*⟩

lemma *encode-mat-dims* [*simp*]: *dim-row encode-mat = 8 dim-col encode-mat = 2*
⟨*proof*⟩

lemma *flip-proj-carrier* [*simp*]: *flip-bit-mat b * syndrome-proj K ∈ carrier-mat 8 8*
⟨*proof*⟩

lemma *b3-cases*: $b < (3::nat) \Longrightarrow b = 0 \vee b = 1 \vee b = 2$ ⟨*proof*⟩

Entry of an 8×8 product, kept as an abstract sum: *simp* unrolls $\sum k{<}8$ into eight terms and then splits every *if*, which does not terminate in practice.

lemma *mat-mult-entry-8*:
fixes *A B* :: *complex Matrix.mat*
assumes *A ∈ carrier-mat 8 8 B ∈ carrier-mat 8 8 i < 8 j < 8*
shows *(A * B) $$ (i,j) = (∑k<8. A $$ (i,k) * B $$ (k,j))*
⟨*proof*⟩

lemma *syndrome-proj-index*:
assumes *i < 8 j < 8*
shows *syndrome-proj K $$ (i,j) = (if i = j ∧ i ∈ K then 1 else 0)*
⟨*proof*⟩

lemma *syndrome-proj-two*:
assumes *a*: *a < 8* **and** *b*: *b < 8* **and** *ab*: *a ≠ b*
shows *syndrome-proj {a, b} = unit-vec 8 a ⊘ unit-vec 8 a + unit-vec 8 b ⊘ unit-vec 8 b*
⟨*proof*⟩

lemma *syndrome-proj-as-outer*:
P_0 = unit-vec 8 0 ⊘ unit-vec 8 0 + unit-vec 8 7 ⊘ unit-vec 8 7
P_1 = unit-vec 8 4 ⊘ unit-vec 8 4 + unit-vec 8 3 ⊘ unit-vec 8 3
P_2 = unit-vec 8 2 ⊘ unit-vec 8 2 + unit-vec 8 5 ⊘ unit-vec 8 5
P_3 = unit-vec 8 1 ⊘ unit-vec 8 1 + unit-vec 8 6 ⊘ unit-vec 8 6
⟨*proof*⟩

The flips are the notes' tensors.

lemma *tensor3-index*:
fixes *A B C* :: *complex Matrix.mat*
assumes *A*: *A ∈ carrier-mat 2 2* **and** *B*: *B ∈ carrier-mat 2 2* **and** *C*: *C ∈ carrier-mat 2 2*
and *i*: *i < 8* **and** *j*: *j < 8*
shows *Tensor.tensor-mat (Tensor.tensor-mat A B) C $$ (i,j)*

$= A \$\$ (i \mathbin{div} 4, j \mathbin{div} 4) * B \$\$ (i \mathbin{div} 2 \bmod 2, j \mathbin{div} 2 \bmod 2) * C \$\$ (i \bmod 2, j \bmod 2)$
⟨*proof*⟩

lemma *flips-as-tensors*:
$X_1 = Tensor.tensor\text{-}mat\ (Tensor.tensor\text{-}mat\ Pauli\text{-}X\ (1_m\ 2))\ (1_m\ 2)$
$X_2 = Tensor.tensor\text{-}mat\ (Tensor.tensor\text{-}mat\ (1_m\ 2)\ Pauli\text{-}X)\ (1_m\ 2)$
$X_3 = Tensor.tensor\text{-}mat\ (Tensor.tensor\text{-}mat\ (1_m\ 2)\ (1_m\ 2))\ Pauli\text{-}X$
⟨*proof*⟩

lemma *flip-bit-mat-mult-vec*:
assumes *b*: $b < 3$ **and** *vd*: $dim\text{-}vec\ v = 8$ **and** *i*: $i < 8$
shows $(flip\text{-}bit\text{-}mat\ b *_v v) \$ i = v \$ (if\ i \mathbin{div} 2\hat{}b \bmod 2 = 0\ then\ i + 2\hat{}b\ else\ i - 2\hat{}b)$
⟨*proof*⟩

lemma *syndrome-proj-mult-vec*:
assumes *vd*: $dim\text{-}vec\ v = 8$ **and** *i*: $i < 8$
shows $(syndrome\text{-}proj\ K *_v v) \$ i = (if\ i \in K\ then\ v \$ i\ else\ 0)$
⟨*proof*⟩

lemma *syndrome-proj-hermitian*: $(syndrome\text{-}proj\ K)^\dagger = syndrome\text{-}proj\ K$
⟨*proof*⟩

lemma *syndrome-proj-idem*: $syndrome\text{-}proj\ K * syndrome\text{-}proj\ K = syndrome\text{-}proj\ K$
⟨*proof*⟩

lemma *syndrome-proj-sum*:
$P_0 + P_1 + P_2 + P_3 = 1_m\ 8$
⟨*proof*⟩

The flip *flip-bit-mat b* permutes the basis by the involution *flip-tgt b*.

definition *flip-tgt* :: *nat* ⇒ *nat* ⇒ *nat* **where**
$flip\text{-}tgt\ b\ i = (if\ i \mathbin{div} 2\hat{}b \bmod 2 = 0\ then\ i + 2\hat{}b\ else\ i - 2\hat{}b)$

lemma *flip-tgt-lt*:
assumes *b*: $b < 3$ **and** *i*: $i < 8$ **shows** $flip\text{-}tgt\ b\ i < 8$
⟨*proof*⟩

lemma *flip-tgt-invol*:
assumes *b*: $b < 3$ **and** *i*: $i < 8$ **shows** $flip\text{-}tgt\ b\ (flip\text{-}tgt\ b\ i) = i$
⟨*proof*⟩

lemma *flip-bit-mat-index*:
assumes *i*: $i < 8$ **and** *j*: $j < 8$
shows $flip\text{-}bit\text{-}mat\ b \$\$ (i,j) = (if\ j = flip\text{-}tgt\ b\ i\ then\ 1\ else\ 0)$
⟨*proof*⟩

lemma *flip-tgt-iff*:
assumes *b*: $b < 3$ **and** *i*: $i < 8$ **and** *j*: $j < 8$
shows $i = flip\text{-}tgt\ b\ j \longleftrightarrow j = flip\text{-}tgt\ b\ i$
⟨*proof*⟩

lemma *flip-bit-mat-hermitian*:
assumes *b*: $b < 3$
shows $(flip\text{-}bit\text{-}mat\ b)^\dagger = flip\text{-}bit\text{-}mat\ b$
⟨*proof*⟩

lemma *flip-bit-mat-squared*:
assumes *b*: $b < 3$

shows *flip-bit-mat b ∗ flip-bit-mat b = 1*$_m$ *8*
⟨*proof*⟩

Kraus operators of the recovery map $R(Y) = P_0 Y P_0 + \sum_k X_k P_k Y P_k X_k$.

definition *recovery-ops* :: *complex Matrix.mat list* **where**
recovery-ops = [P_0, $X_1 * P_1$, $X_2 * P_2$, $X_3 * P_3$]

definition *recovery-map* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
recovery-map = *kraus-map-dim 8 recovery-ops*

lemma *flip-proj-dag-mult*:
assumes *b*: *b* < *3*
shows ((*flip-bit-mat b ∗ syndrome-proj K*)†) ∗ (*flip-bit-mat b ∗ syndrome-proj K*) = *syndrome-proj K*
⟨*proof*⟩

theorem *recovery-ops-kraus*: *kraus-operators 8 8 recovery-ops*
⟨*proof*⟩

corollary *recovery-map-CPTP*: *recovery-map* ∈ *CPTP 8 8*
⟨*proof*⟩

The encoded state $\psi = a|000\rangle + b|111\rangle$ and its single-flip corruptions.

lemma *encode-mat-mult-vec*:
assumes *xd*: *dim-vec x = 2* **and** *i*: *i* < *8*
shows (*encode-mat* ∗$_v$ *x*) $ *i* = (*if i = 0 then x $ 0 else if i = 7 then x $ 1 else 0*)
⟨*proof*⟩

lemma *encode-mat-dim* [*simp*]: *dim-vec* (*encode-mat* ∗$_v$ *x*) = *8* ⟨*proof*⟩

lemma *recovery-kraus-term*:
fixes *φ* :: *complex Matrix.vec*
assumes *φd*: *dim-vec φ = 8*
shows *recovery-map* (*φ* ⊘ *φ*) =
(*P*$_0$ ∗$_v$ *φ*) ⊘ (*P*$_0$ ∗$_v$ *φ*) + ((*X*$_1$ ∗$_v$ (*P*$_1$ ∗$_v$ *φ*)) ⊘ (*X*$_1$ ∗$_v$ (*P*$_1$ ∗$_v$ *φ*))
+ ((*X*$_2$ ∗$_v$ (*P*$_2$ ∗$_v$ *φ*)) ⊘ (*X*$_2$ ∗$_v$ (*P*$_2$ ∗$_v$ *φ*))
+ ((*X*$_3$ ∗$_v$ (*P*$_3$ ∗$_v$ *φ*)) ⊘ (*X*$_3$ ∗$_v$ (*P*$_3$ ∗$_v$ *φ*)) + *0*$_m$ *8 8*)))
⟨*proof*⟩

theorem *bit-flip-code-recovery*:
fixes *x* :: *complex Matrix.vec*
assumes *xd*: *dim-vec x = 2*
defines *ψ* ≡ *encode-mat* ∗$_v$ *x*
shows *recovery-map* (*ψ* ⊘ *ψ*) = *ψ* ⊘ *ψ*
and *recovery-map* ((*X*$_1$ ∗$_v$ *ψ*) ⊘ (*X*$_1$ ∗$_v$ *ψ*)) = *ψ* ⊘ *ψ*
and *recovery-map* ((*X*$_2$ ∗$_v$ *ψ*) ⊘ (*X*$_2$ ∗$_v$ *ψ*)) = *ψ* ⊘ *ψ*
and *recovery-map* ((*X*$_3$ ∗$_v$ *ψ*) ⊘ (*X*$_3$ ∗$_v$ *ψ*)) = *ψ* ⊘ *ψ*
⟨*proof*⟩

end
theory *Day31-Code-Fidelity*
imports *Day30-BitFlip-Code*
begin

Day 31: the three-qubit code against independent bit flips. Each qubit flips with probability p, so $E(Y) = (1-p)^3 Y + p(1-p)^2 \sum_i X_i Y X_i + p^2(1-p) \sum_{i<j} X_i X_j Y X_j X_i + p^3\, X_1X_2X_3 Y X_3X_2X_1$. On an encoded state the recovery R leaves $((1-p)^3 + 3p(1-p)^2)|\tilde\psi\rangle\langle\tilde\psi| + (3p^2(1-p) + p^3)|X_1X_2X_3\tilde\psi\rangle\langle X_1X_2X_3\tilde\psi|$. The notes state this formula for a general Y; it holds for Y supported on the code space (for $Y = |100\rangle\langle 100|$ and $p = 0$ one gets $R(Y) = |000\rangle\langle 000| \neq Y$).

definition *three-bit-channel* :: *real* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
three-bit-channel p M =
complex-of-real ((1 − *p*)^3) ·$_m$ *M*
+ *complex-of-real* (*p* ∗ (1 − *p*)^2) ·$_m$ (X_1 ∗ *M* ∗ (X_1†))
+ *complex-of-real* (*p* ∗ (1 − *p*)^2) ·$_m$ (X_2 ∗ *M* ∗ (X_2†))
+ *complex-of-real* (*p* ∗ (1 − *p*)^2) ·$_m$ (X_3 ∗ *M* ∗ (X_3†))
+ *complex-of-real* (*p*^2 ∗ (1 − *p*)) ·$_m$ ((X_2 ∗ X_3) ∗ *M* ∗ ((X_2 ∗ X_3)†))
+ *complex-of-real* (*p*^2 ∗ (1 − *p*)) ·$_m$ ((X_1 ∗ X_3) ∗ *M* ∗ ((X_1 ∗ X_3)†))
+ *complex-of-real* (*p*^2 ∗ (1 − *p*)) ·$_m$ ((X_1 ∗ X_2) ∗ *M* ∗ ((X_1 ∗ X_2)†))
+ *complex-of-real* (*p*^3) ·$_m$ ((X_1 ∗ X_2 ∗ X_3) ∗ *M* ∗ ((X_1 ∗ X_2 ∗ X_3)†))

definition *swap2* :: *complex Matrix.vec* ⇒ *complex Matrix.vec* **where**
swap2 x = *Matrix.vec 2* (λ*i*. *x* $ (1 − *i*))

lemma *swap2-dim* [*simp*]: *dim-vec* (*swap2 x*) = 2 ⟨*proof*⟩

lemma *flips-on-codeword*:
assumes *x*: *dim-vec x* = 2
defines ψ ≡ *encode-mat* ∗$_v$ *x* **and** φ ≡ *encode-mat* ∗$_v$ *swap2 x*
shows (X_1 ∗ X_2 ∗ X_3) ∗$_v$ ψ = φ
and (X_2 ∗ X_3) ∗$_v$ ψ = X_1 ∗$_v$ φ
and (X_1 ∗ X_3) ∗$_v$ ψ = X_2 ∗$_v$ φ
and (X_1 ∗ X_2) ∗$_v$ ψ = X_3 ∗$_v$ φ
⟨*proof*⟩

41.1 Errors followed by recovery

lemma *recovery-ops-carrier'*: ∀ *W* ∈ *set recovery-ops*. *W* ∈ *carrier-mat 8 8*
⟨*proof*⟩

lemma *recovery-map-add*: *A* ∈ *carrier-mat 8 8* ⟹ *B* ∈ *carrier-mat 8 8* ⟹ *recovery-map* (*A* + *B*) = *recovery-map A* + *recovery-map B*
⟨*proof*⟩

lemma *recovery-map-smult*: *A* ∈ *carrier-mat 8 8* ⟹ *recovery-map* (*c* ·$_m$ *A*) = *c* ·$_m$ *recovery-map A*
⟨*proof*⟩

lemma *comb8*:
fixes *A B* :: *complex Matrix.mat*
assumes *A*: *A* ∈ *carrier-mat n n* **and** *B*: *B* ∈ *carrier-mat n n*
shows *complex-of-real a* ·$_m$ *A* + *complex-of-real b* ·$_m$ *A* + *complex-of-real b* ·$_m$ *A* + *complex-of-real b* ·$_m$ *A*
+ *complex-of-real c* ·$_m$ *B* + *complex-of-real c* ·$_m$ *B* + *complex-of-real c* ·$_m$ *B* + *complex-of-real d* ·$_m$ *B*
= *complex-of-real* (*a* + 3 ∗ *b*) ·$_m$ *A* + *complex-of-real* (3 ∗ *c* + *d*) ·$_m$ *B*
⟨*proof*⟩

lemma *outer8*: *dim-vec v* = 8 ⟹ *v* ⊘ *v* ∈ *carrier-mat 8 8*
⟨*proof*⟩

D31-2: errors followed by recovery on an encoded state.

theorem *three-bit-code-recovery*:
assumes *x*: *dim-vec x* = 2
defines ψ ≡ *encode-mat* ∗$_v$ *x* **and** φ ≡ *encode-mat* ∗$_v$ *swap2 x*
shows *recovery-map* (*three-bit-channel p* (ψ ⊘ ψ))
= *complex-of-real* ((1 − *p*)^3 + 3 ∗ *p* ∗ (1 − *p*)^2) ·$_m$ (ψ ⊘ ψ)
+ *complex-of-real* (3 ∗ *p*^2 ∗ (1 − *p*) + *p*^3) ·$_m$ (φ ⊘ φ)
⟨*proof*⟩

The code is better than doing nothing iff $(1-p)^3 + 3p(1-p)^2 \geq 1-p$ iff $p \leq \frac{1}{2}$.

theorem *three-bit-code-better-iff*:
assumes *p0*: $0 \leq p$ **and** *p1*: $p < 1$
shows $(1 - p) \leq (1 - p)\hat{}3 + 3 * p * (1 - p)\hat{}2 \longleftrightarrow p \leq (1/2 :: real)$
⟨*proof*⟩

41.2 Minimum fidelities (D31-4)

lemma *smult-mat-mult-vec8*:
fixes $A :: complex\ Matrix.mat$
assumes *dim-col* $A =$ *dim-vec* v
shows $(k \cdot_m A) *_v v = k \cdot_v (A *_v v)$
⟨*proof*⟩

lemma *quad-two-outer*:
fixes $\psi\ \varphi :: complex\ Matrix.vec$
assumes ψd: *dim-vec* $\psi = n$ **and** φd: *dim-vec* $\varphi = n$
shows $\langle \psi \mid (a \cdot_m (\psi \oslash \psi) + b \cdot_m (\varphi \oslash \varphi)) *_v \psi \rangle$
$= a * (\langle \psi|\psi \rangle * \langle \psi|\psi \rangle) + b * (\langle \varphi|\psi \rangle * \langle \psi|\varphi \rangle)$
⟨*proof*⟩

lemma *encode-inner*:
assumes *u*: *dim-vec* $u = 2$ **and** *v*: *dim-vec* $v = 2$
shows $\langle encode\text{-}mat *_v u \mid encode\text{-}mat *_v v \rangle = \langle u|v \rangle$
⟨*proof*⟩

theorem *code-min-fidelity*:
assumes *p0*: $0 \leq p$ **and** *p1*: $p \leq 1$
defines *c1* $\equiv (1 - p)\hat{}3 + 3 * p * (1 - p)\hat{}2$
shows $\forall x.$ *dim-vec* $x = 2 \longrightarrow \langle x|x \rangle = 1 \longrightarrow$
sqrt c1 $\leq$ *fidelity* (*encode-mat* $*_v$ x) (*recovery-map* (*three-bit-channel p* ((*encode-mat* $*_v$ x) $\oslash$ (*encode-mat* $*_v$ x))))
and *fidelity* (*encode-mat* $*_v$ *unit-vec 2 0*)
(*recovery-map* (*three-bit-channel p* ((*encode-mat* $*_v$ *unit-vec 2 0*) $\oslash$ (*encode-mat* $*_v$ *unit-vec 2 0*)))) = *sqrt c1*
⟨*proof*⟩

theorem *nothing-min-fidelity*:
assumes *p0*: $0 \leq p$ **and** *p1*: $p \leq 1$
shows $\forall \psi.$ *dim-vec* $\psi = 2 \longrightarrow \langle \psi|\psi \rangle = 1 \longrightarrow$ *sqrt* $(1 - p) \leq$ *fidelity* ψ (*bit-flip-channel p* ($\psi \oslash \psi$))
and *fidelity* (*unit-vec 2 0*) (*bit-flip-channel p* (*unit-vec 2 0* $\oslash$ *unit-vec 2 0*)) = *sqrt* $(1 - p)$
⟨*proof*⟩

end
theory *Day32-Pauli-Expansion*
imports *QC-Prelims*
begin

42 Day 32: the Pauli expansion identity for unitaries (D32-7)

lemma *cmod-parallelogram*:
fixes $a\ b :: complex$
shows $(cmod\ (a + b))^2 + (cmod\ (a - b))^2 = 2 * (cmod\ a)^2 + 2 * (cmod\ b)^2$
⟨*proof*⟩

proposition *pauli-expansion-unitary*:
fixes $U :: complex\ Matrix.mat$ **and** $a0\ a1\ a2\ a3 :: complex$

assumes *Uu*: *unitary U*
and *U-eq*: $U = a0 \cdot_m 1_m\ 2 + a1 \cdot_m Pauli\text{-}X + a2 \cdot_m Pauli\text{-}Y + a3 \cdot_m Pauli\text{-}Z$
shows $(cmod\ a0)^2 + (cmod\ a1)^2 + (cmod\ a2)^2 + (cmod\ a3)^2 = 1$
⟨*proof*⟩

end
theory *Day32-Recovery-Proposition*
imports *Day21-Unitary-Freedom*
begin

43 Days 32–33: composition of Kraus maps and the general recovery proposition (D32-4, D33-2)

43.1 Algebra of *foldr*-sums of matrices

lemma *foldr-add-mat-mult-left*:
fixes $f :: 'a \Rightarrow complex\ Matrix.mat$
assumes *Ac*: $A \in carrier\text{-}mat\ m\ n$ **and** *fc*: $\forall x \in set\ xs.\ f\ x \in carrier\text{-}mat\ n\ p$
shows $A * foldr\ (\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ n\ p) = foldr\ (\lambda x\ acc.\ A * f\ x + acc)\ xs\ (0_m\ m\ p)$
⟨*proof*⟩

lemma *foldr-add-mat-mult-right*:
fixes $f :: 'a \Rightarrow complex\ Matrix.mat$
assumes *Bc*: $B \in carrier\text{-}mat\ n\ p$ **and** *fc*: $\forall x \in set\ xs.\ f\ x \in carrier\text{-}mat\ m\ n$
shows $foldr\ (\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ m\ n) * B = foldr\ (\lambda x\ acc.\ f\ x * B + acc)\ xs\ (0_m\ m\ p)$
⟨*proof*⟩

lemma *foldr-add-mat-carrier'*:
fixes $f :: 'a \Rightarrow complex\ Matrix.mat$
assumes $\forall x \in set\ xs.\ f\ x \in carrier\text{-}mat\ m\ n$
shows $foldr\ (\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ m\ n) \in carrier\text{-}mat\ m\ n$
⟨*proof*⟩

lemma *foldr-add-mat-dagger*:
fixes $f :: 'a \Rightarrow complex\ Matrix.mat$
assumes $\forall x \in set\ xs.\ f\ x \in carrier\text{-}mat\ m\ n$
shows $(foldr\ (\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ m\ n))^\dagger = foldr\ (\lambda x\ acc.\ (f\ x)^\dagger + acc)\ xs\ (0_m\ n\ m)$
⟨*proof*⟩

lemma *foldr-add-mat-cong*:
fixes $f\ g :: 'a \Rightarrow complex\ Matrix.mat$
assumes $\bigwedge x.\ x \in set\ xs \Longrightarrow f\ x = g\ x$
shows $foldr\ (\lambda x\ acc.\ f\ x + acc)\ xs\ z = foldr\ (\lambda x\ acc.\ g\ x + acc)\ xs\ z$
⟨*proof*⟩

lemma *foldr-add-mat-zero*:
$foldr\ (\lambda x\ acc.\ (0_m\ m\ n :: complex\ Matrix.mat) + acc)\ xs\ (0_m\ m\ n) = 0_m\ m\ n$
⟨*proof*⟩

lemma *foldr-add-mat-delta*:
fixes $f :: nat \Rightarrow complex\ Matrix.mat$
assumes *fc*: $\forall x \in set\ xs.\ f\ x \in carrier\text{-}mat\ m\ n$ **and** *d*: *distinct xs* **and** *i*: $i \in set\ xs$
shows $foldr\ (\lambda x\ acc.\ (if\ x = i\ then\ f\ x\ else\ 0_m\ m\ n) + acc)\ xs\ (0_m\ m\ n) = f\ i$
⟨*proof*⟩

lemma *foldr-add-mat-smult-pull*:
fixes $f :: 'a \Rightarrow complex\ Matrix.mat$ **and** $c :: 'a \Rightarrow complex$

assumes *fc*: $\forall\, x \in set\ xs.\ f\ x = c\ x \cdot_m M$ **and** *Mc*: $M \in carrier\text{-}mat\ m\ n$
shows $foldr\ (\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ m\ n) = sum\text{-}list\ (map\ c\ xs) \cdot_m M$
⟨*proof*⟩

lemma *foldr-list-as-index*:
fixes $g :: complex\ Matrix.mat \Rightarrow complex\ Matrix.mat$
shows $foldr\ (\lambda A\ acc.\ g\ A + acc)\ As\ z = foldr\ (\lambda l\ acc.\ g\ (As\ !\ l) + acc)\ [0..<length\ As]\ z$
⟨*proof*⟩

43.2 Composition of Kraus maps

definition $kraus\text{-}compose :: complex\ Matrix.mat\ list \Rightarrow complex\ Matrix.mat\ list \Rightarrow complex\ Matrix.mat\ list$ **where**
$kraus\text{-}compose\ As\ Es = concat\ (map\ (\lambda E.\ map\ (\lambda A.\ A * E)\ As)\ Es)$

lemma *kraus-compose-Nil* [*simp*]: $kraus\text{-}compose\ As\ [] = []$
⟨*proof*⟩

lemma *kraus-compose-Cons*: $kraus\text{-}compose\ As\ (E \#\ Es) = map\ (\lambda A.\ A * E)\ As\ @\ kraus\text{-}compose\ As\ Es$
⟨*proof*⟩

lemma *length-kraus-compose* [*simp*]: $length\ (kraus\text{-}compose\ As\ Es) = length\ Es * length\ As$
⟨*proof*⟩

lemma *kraus-compose-nth*:
assumes *i*: $i < length\ Es$ **and** *l*: $l < length\ As$
shows $kraus\text{-}compose\ As\ Es\ !\ (i * length\ As + l) = As\ !\ l * Es\ !\ i$
⟨*proof*⟩

lemma *kraus-compose-carrier*:
assumes $\forall\, A \in set\ As.\ A \in carrier\text{-}mat\ m\ d$ **and** $\forall\, E \in set\ Es.\ E \in carrier\text{-}mat\ d\ n$
shows $\forall\, W \in set\ (kraus\text{-}compose\ As\ Es).\ W \in carrier\text{-}mat\ m\ n$
⟨*proof*⟩

lemma *kraus-map-dim-map-mult-right*:
assumes *As-c*: $\forall\, A \in set\ As.\ A \in carrier\text{-}mat\ m\ d$ **and** *Ec*: $E \in carrier\text{-}mat\ d\ n$ **and** *Xc*: $M \in carrier\text{-}mat\ n\ n$
shows $kraus\text{-}map\text{-}dim\ m\ (map\ (\lambda A.\ A * E)\ As)\ M = kraus\text{-}map\text{-}dim\ m\ As\ (E * M * (E^\dagger))$
⟨*proof*⟩

lemma *kraus-map-dim-compose*:
assumes *As-c*: $\forall\, A \in set\ As.\ A \in carrier\text{-}mat\ m\ d$ **and** *Es-c*: $\forall\, E \in set\ Es.\ E \in carrier\text{-}mat\ d\ n$
and *Xc*: $M \in carrier\text{-}mat\ n\ n$
shows $kraus\text{-}map\text{-}dim\ m\ (kraus\text{-}compose\ As\ Es)\ M = kraus\text{-}map\text{-}dim\ m\ As\ (kraus\text{-}map\text{-}dim\ d\ Es\ M)$
⟨*proof*⟩

lemma *sum-wdag-w-map-mult-right*:
assumes *As-c*: $\forall\, A \in set\ As.\ A \in carrier\text{-}mat\ m\ d$ **and** *Ec*: $E \in carrier\text{-}mat\ d\ n$
shows $sum\text{-}wdag\text{-}w\ n\ (map\ (\lambda A.\ A * E)\ As) = (E^\dagger) * sum\text{-}wdag\text{-}w\ d\ As * E$
⟨*proof*⟩

lemma *sum-wdag-w-compose*:
assumes *As-c*: $\forall\, A \in set\ As.\ A \in carrier\text{-}mat\ m\ d$ **and** *Es-c*: $\forall\, E \in set\ Es.\ E \in carrier\text{-}mat\ d\ n$
and *As-tp*: $sum\text{-}wdag\text{-}w\ d\ As = 1_m\ d$
shows $sum\text{-}wdag\text{-}w\ n\ (kraus\text{-}compose\ As\ Es) = sum\text{-}wdag\text{-}w\ n\ Es$
⟨*proof*⟩

lemma *kraus-operators-compose*:
assumes *As*: *kraus-operators d m As* **and** *Es*: *kraus-operators n d Es*
shows *kraus-operators n m* (*kraus-compose As Es*)
⟨*proof*⟩

43.3 The general recovery proposition (D32-4) and its linear-combination form (D33-2)

43.4 Matrix sums acting on vectors

lemma *smult-zero-vec* [*simp*]: (*c* :: *complex*) ·$_v$ 0_v *n* = 0_v *n*
⟨*proof*⟩

lemma *zero-mat-mult-vec*:
assumes *dim-vec w* = *n* **shows** (0_m *m n* :: *complex Matrix.mat*) *∗*$_v$ *w* = 0_v *m*
⟨*proof*⟩

lemma *mult-mat-vec-zero-vec*:
assumes *Ac*: *A* ∈ *carrier-mat m n* **shows** *A ∗*$_v$ (0_v *n* :: *complex Matrix.vec*) = 0_v *m*
⟨*proof*⟩

lemma *foldr-add-mat-mult-vec-zero*:
fixes *f* :: *'a* ⇒ *complex Matrix.mat*
assumes *fc*: ∀ *x* ∈ *set xs*. *f x* ∈ *carrier-mat n n* **and** *wd*: *dim-vec w* = *n*
and *z*: ⋀*x*. *x* ∈ *set xs* ⟹ *f x ∗*$_v$ *w* = 0_v *n*
shows *foldr* (λ*x acc*. *f x* + *acc*) *xs* (0_m *n n*) *∗*$_v$ *w* = 0_v *n*
⟨*proof*⟩

lemma *foldr-add-mat-mult-vec-delta*:
fixes *f* :: *nat* ⇒ *complex Matrix.mat*
assumes *fc*: ∀ *x* ∈ *set xs*. *f x* ∈ *carrier-mat n n* **and** *wd*: *dim-vec w* = *n*
and *d*: *distinct xs* **and** *j*: *j* ∈ *set xs*
and *del*: ⋀*x*. *x* ∈ *set xs* ⟹ *f x ∗*$_v$ *w* = (*if x* = *j then w else* 0_v *n*)
shows *foldr* (λ*x acc*. *f x* + *acc*) *xs* (0_m *n n*) *∗*$_v$ *w* = *w*
⟨*proof*⟩

lemma *vec-sum-delta*:
assumes *fin*: *finite Js* **and** *j*: *j* ∈ *Js* **and** *wd*: ⋀*i*. *i* ∈ *Js* ⟹ *dim-vec* (*w i*) = *n*
shows *vec-sum n Js* (λ*i*. *if i* = *j then w i else* 0_v *n*) = *w j*
⟨*proof*⟩

Let $V = \operatorname{ran} P$ for a projection P and let $U_0, \dots, U_{t-1}$ be unitaries with $U_i V \perp U_j V$ for $i \neq j$, i.e. $P U_i^* U_j P = 0$. With $P_i = U_i P U_i^*$, $Q = I - \sum_i P_i$ and the Kraus operators $P U_i^*$ (the notes write $U_i^* P_i = P U_i^*$) the map $R(X) = QXQ + \sum_i P U_i^* X U_i P$ is CPTP and recovers every $\psi \in V$ from any $\varphi = \sum_i a_i U_i \psi$: $R(|\varphi\rangle\langle\varphi|) = (\sum_i |a_i|^2)|\psi\rangle\langle\psi|$.

definition *image-proj-sum* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* **where**
image-proj-sum n P Us = *foldr* (λ*i acc*. *Us* ! *i ∗ P ∗* ((*Us* ! *i*)†) + *acc*) [*0..<length Us*] (0_m *n n*)

definition *recovery-ops* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat list* **where**
recovery-ops n P Us = (1$_m$ *n* − *image-proj-sum n P Us*) # *map* (λ*i*. *P ∗* ((*Us* ! *i*)†)) [*0..<length Us*]

The hypotheses of the proposition, bundled: P a projection and $P U_i^* U_j P = \delta_{ij} P$. For unitaries U_i the case $i = j$ is automatic (*recovery-setting-unitaries*); the weaker form also covers the rescaled errors of the Knill–Laflamme theorem.

definition *recovery-setting* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat list* ⇒ *bool* **where**

recovery-setting n P Us $\longleftrightarrow$ *projection n P* $\wedge$ $(\forall\, U \in set\ Us.\ U \in carrier\text{-}mat\ n\ n)$
$\wedge\ (\forall\, i < length\ Us.\ \forall\, j < length\ Us.\ P * (((Us\ !\ i)^\dagger) * Us\ !\ j) * P = (if\ i = j\ then\ P\ else\ 0_m\ n\ n))$

lemma *recovery-settingD*:
assumes *recovery-setting n P Us*
shows *projection n P* $\forall\, U \in set\ Us.\ U \in carrier\text{-}mat\ n\ n$
$\forall\, i < length\ Us.\ \forall\, j < length\ Us.\ i \neq j \longrightarrow P * (((Us\ !\ i)^\dagger) * Us\ !\ j) * P = 0_m\ n\ n$
$\forall\, i < length\ Us.\ P * (((Us\ !\ i)^\dagger) * Us\ !\ i) * P = P$
⟨*proof*⟩

lemma *recovery-setting-unitaries*:
assumes *P*: *projection n P* **and** *Us*: $\forall\, U \in set\ Us.\ U \in carrier\text{-}mat\ n\ n \wedge unitary\ U$
and *orth*: $\forall\, i < length\ Us.\ \forall\, j < length\ Us.\ i \neq j \longrightarrow P * (((Us\ !\ i)^\dagger) * Us\ !\ j) * P = 0_m\ n\ n$
shows *recovery-setting n P Us*
⟨*proof*⟩

lemma
assumes *S*: *recovery-setting n P Us*
shows *rec-Pc*: $P \in carrier\text{-}mat\ n\ n$ **and** *rec-Ph*: $P^\dagger = P$ **and** *rec-PP*: $P * P = P$
⟨*proof*⟩

lemma
assumes *S*: *recovery-setting n P Us*
shows *rec-Uc*: $i < length\ Us \Longrightarrow Us\ !\ i \in carrier\text{-}mat\ n\ n$
and *rec-PUUP*: $i < length\ Us \Longrightarrow P * (((Us\ !\ i)^\dagger) * Us\ !\ i) * P = P$
⟨*proof*⟩

lemma *rec-Pi-c*:
assumes *S*: *recovery-setting n P Us* **and** *i*: $i < length\ Us$
shows $Us\ !\ i * P * ((Us\ !\ i)^\dagger) \in carrier\text{-}mat\ n\ n$
⟨*proof*⟩

lemma *rec-Pi-mult*:
assumes *S*: *recovery-setting n P Us* **and** *i*: $i < length\ Us$ **and** *j*: $j < length\ Us$
shows $(Us\ !\ i * P * ((Us\ !\ i)^\dagger)) * (Us\ !\ j * P * ((Us\ !\ j)^\dagger))$
$= (if\ i = j\ then\ Us\ !\ i * P * ((Us\ !\ i)^\dagger)\ else\ 0_m\ n\ n)$
⟨*proof*⟩

lemma *image-proj-sum-carrier*:
assumes *S*: *recovery-setting n P Us*
shows *image-proj-sum n P Us* $\in carrier\text{-}mat\ n\ n$
⟨*proof*⟩

lemma *image-proj-sum-hermitian*:
assumes *S*: *recovery-setting n P Us*
shows $(image\text{-}proj\text{-}sum\ n\ P\ Us)^\dagger = image\text{-}proj\text{-}sum\ n\ P\ Us$
⟨*proof*⟩

lemma *image-proj-sum-idem*:
assumes *S*: *recovery-setting n P Us*
shows *image-proj-sum n P Us* $*$ *image-proj-sum n P Us* $=$ *image-proj-sum n P Us*
⟨*proof*⟩

lemma *image-proj-sum-projection*:
assumes *S*: *recovery-setting n P Us*
shows *projection n* (*image-proj-sum n P Us*)
⟨*proof*⟩

lemma *dagger-one-minus*:
assumes *Sc*: $M \in$ *carrier-mat n n* **and** *Sh*: $M^\dagger = M$
shows $(1_m\ n - M)^\dagger = 1_m\ n - M$
⟨*proof*⟩

lemma *recovery-ops-carrier*:
assumes *S*: *recovery-setting n P Us*
shows $\forall\, W \in$ *set* (*recovery-ops n P Us*). $W \in$ *carrier-mat n n*
⟨*proof*⟩

theorem *recovery-ops-kraus-operators*:
assumes *S*: *recovery-setting n P Us*
shows *kraus-operators n n* (*recovery-ops n P Us*)
⟨*proof*⟩

corollary *recovery-map-CPTP′*:
assumes *S*: *recovery-setting n P Us* **and** *npos*: $0 < n$
shows *kraus-map-dim n* (*recovery-ops n P Us*) $\in$ *CPTP n n*
⟨*proof*⟩

Action of the Kraus operators on $\varphi = \sum_i a_i U_i \psi$, $\psi \in V$.

lemma *rec-kraus-on-image*:
assumes *S*: *recovery-setting n P Us*
and ψd: *dim-vec* $\psi = n$ **and** ψV: $P *_v \psi = \psi$ **and** *i*: $i <$ *length Us* **and** *j*: $j <$ *length Us*
shows $(P * ((Us\ !\ i)^\dagger)) *_v (Us\ !\ j *_v \psi) =$ (*if* $i = j$ *then* ψ *else* $0_v\ n$)
⟨*proof*⟩

lemma *rec-Q-on-image*:
assumes *S*: *recovery-setting n P Us*
and ψd: *dim-vec* $\psi = n$ **and** ψV: $P *_v \psi = \psi$ **and** *j*: $j <$ *length Us*
shows $(1_m\ n -$ *image-proj-sum n P Us*$) *_v (Us\ !\ j *_v \psi) = 0_v\ n$
⟨*proof*⟩

theorem *recovery-linear-combination*:
fixes *a* :: *nat* ⇒ *complex*
assumes *S*: *recovery-setting n P Us* **and** ψd: *dim-vec* $\psi = n$ **and** ψV: $P *_v \psi = \psi$
defines $\varphi \equiv$ *vec-sum n* $\{i.\ i <$ *length Us*$\}$ $(\lambda i.\ a\ i \cdot_v (Us\ !\ i *_v \psi))$
shows *kraus-map-dim n* (*recovery-ops n P Us*) $(\varphi \oslash \varphi)$
$=$ *complex-of-real* $(\sum i <$ *length Us*. $(cmod\ (a\ i))^2) \cdot_m (\psi \oslash \psi)$
⟨*proof*⟩

corollary *recovery-single-error*:
assumes *S*: *recovery-setting n P Us*
and ψd: *dim-vec* $\psi = n$ **and** ψV: $P *_v \psi = \psi$ **and** *j*: $j <$ *length Us*
shows *kraus-map-dim n* (*recovery-ops n P Us*) $((Us\ !\ j *_v \psi) \oslash (Us\ !\ j *_v \psi)) = \psi \oslash \psi$
⟨*proof*⟩

end
theory *Day32-Y-Code*
imports *Day33-Shor-Code*
begin

Day 32: the code for Y flips. With $\beta = Y|+\rangle$ one has $|+\rangle \perp \beta$, $Y|+\rangle = \beta$, $Y\beta = |+\rangle$, and the code $|0\rangle \mapsto |++\!+\rangle$, $|1\rangle \mapsto |\beta\beta\beta\rangle$ corrects one Y flip exactly as the bit-flip code corrects one X flip.

definition *ket-beta* :: *complex Matrix.vec* **where** *ket-beta* = i $\cdot_v$ *ket-minus*

definition *ub* :: *bool* ⇒ *complex Matrix.vec* **where** *ub s* = (*if s then ket-plus else ket-beta*)

lemma *ket-beta-dim* [*simp*]: *dim-vec ket-beta* = *2* ⟨*proof*⟩
lemma *ket-plus-dim* [*simp*]: *dim-vec ket-plus* = *2* ⟨*proof*⟩
lemma *ub-dim* [*simp*]: *dim-vec* (*ub s*) = *2* ⟨*proof*⟩

lemma *ket-beta-index*: *ket-beta* $ *0* = i / *sqrt 2 ket-beta* $ *1* = − i / *sqrt 2*
⟨*proof*⟩

lemma *ket-plus-index*: *ket-plus* $ *0* = *1* / *sqrt 2 ket-plus* $ *1* = *1* / *sqrt 2*
⟨*proof*⟩

lemma *Y-plus*: *Pauli-Y* ∗$_v$ *ket-plus* = *ket-beta*
⟨*proof*⟩

lemma *Y-beta*: *Pauli-Y* ∗$_v$ *ket-beta* = *ket-plus*
⟨*proof*⟩

lemma *Y-ub*: *Pauli-Y* ∗$_v$ *ub s* = *ub* (¬ *s*)
⟨*proof*⟩

lemma *inner2*: *dim-vec u* = *2* ⟹ *dim-vec v* = *2* ⟹ ⟨*u*|*v*⟩ = *cnj* (*u* $ *0*) ∗ *v* $ *0* + *cnj* (*u* $ *1*) ∗ *v* $ *1*
⟨*proof*⟩

lemma *ub-inner*: ⟨*ub s* | *ub t*⟩ = (*if s* = *t then 1 else 0*)
⟨*proof*⟩

theorem *ket-beta-props*: ⟨*ket-plus* | *ket-beta*⟩ = *0 Pauli-Y* ∗$_v$ *ket-plus* = *ket-beta Pauli-Y* ∗$_v$ *ket-beta* = *ket-plus*
⟨*proof*⟩

43.5 Codes with orthogonal error images

For a code with orthonormal basis $v_1, \ldots, v_k$ (the columns of V, projection VV^*) and unitary errors whose images of the basis are mutually orthogonal, the recovery proposition gives one CPTP map correcting each error on every code state Vc.

lemma *proj-of-orthonormal*:
assumes *orth*: *orthonormal-list n vs*
defines *V* ≡ *mat-of-cols n vs*
shows *projection n* (*V* ∗ (*V*†))
⟨*proof*⟩

lemma *proj-fixes-code*:
assumes *orth*: *orthonormal-list n vs* **and** *c*: *dim-vec c* = *length vs*
defines *V* ≡ *mat-of-cols n vs*
shows (*V* ∗ (*V*†)) ∗$_v$ (*V* ∗$_v$ *c*) = *V* ∗$_v$ *c*
⟨*proof*⟩

theorem *orthogonal-images-recovery*:
assumes *orth*: *orthonormal-list n vs*
and *Us*: ∀ *U* ∈ *set Us*. *U* ∈ *carrier-mat n n* ∧ *unitary U*
and *img*: ∀ *i* < *length Us*. ∀ *j* < *length Us*. *i* ≠ *j* ⟶
(∀ *a* < *length vs*. ∀ *b* < *length vs*. ⟨*Us* ! *i* ∗$_v$ *vs* ! *a* | *Us* ! *j* ∗$_v$ *vs* ! *b*⟩ = *0*)
defines *V* ≡ *mat-of-cols n vs*
shows *recovery-setting n* (*V* ∗ (*V*†)) *Us*
⟨*proof*⟩

corollary *orthogonal-images-correct*:
assumes *orth*: *orthonormal-list n vs*
and *Us*: $\forall\, U \in$ *set Us*. $U \in$ *carrier-mat n n* $\wedge$ *unitary U*
and *img*: $\forall\, i <$ *length Us*. $\forall\, j <$ *length Us*. $i \neq j \longrightarrow$
$(\forall\, a <$ *length vs*. $\forall\, b <$ *length vs*. $\langle Us\ !\ i *_v vs\ !\ a \mid Us\ !\ j *_v vs\ !\ b\rangle = 0)$
shows $\exists$ *Rs*. *kraus-operators n n Rs* $\wedge$
$(\forall\, (c ::$ *complex Matrix.vec*$)\ j$. *dim-vec c* $=$ *length vs* $\longrightarrow$ $j <$ *length Us* $\longrightarrow$
kraus-map-dim n Rs (*Us* ! $j *$ ((*mat-of-cols n vs* $*_v$ *c*) $\oslash$ (*mat-of-cols n vs* $*_v$ *c*)) $*$ ((*Us* ! $j)^\dagger$))
$=$ (*mat-of-cols n vs* $*_v$ *c*) $\oslash$ (*mat-of-cols n vs* $*_v$ *c*))
⟨*proof*⟩

43.6 The Y-flip code

definition *tv2* :: *bool* $\Rightarrow$ *bool* $\Rightarrow$ *bool* $\Rightarrow$ *complex Matrix.vec* **where**
tv2 a b c = *tensor-vec* (*tensor-vec* (*ub a*) (*ub b*)) (*ub c*)

definition *y-errors* :: *complex Matrix.mat list* **where**
y-errors = [1_m *8*, *bop 0 Pauli-Y*, *bop 1 Pauli-Y*, *bop 2 Pauli-Y*]

definition *y-codewords* :: *complex Matrix.vec list* **where**
y-codewords = [*tv2 True True True*, *tv2 False False False*]

lemma *tv2-dim* [*simp*]: *dim-vec* (*tv2 a b c*) = *8*
⟨*proof*⟩

lemma *tv2-inner*: $\langle$*tv2 a b c* $\mid$ *tv2 a′ b′ c′*$\rangle$ = (*if* $a = a' \wedge b = b' \wedge c = c'$ *then 1 else 0*)
⟨*proof*⟩

lemma *y-actions*:
bop 0 Pauli-Y $*_v$ *tv2 a b c* = *tv2* ($\neg$ *a*) *b c*
bop 1 Pauli-Y $*_v$ *tv2 a b c* = *tv2 a* ($\neg$ *b*) *c*
bop 2 Pauli-Y $*_v$ *tv2 a b c* = *tv2 a b* ($\neg$ *c*)
⟨*proof*⟩

lemma *y-codewords-orthonormal*: *orthonormal-list 8 y-codewords*
⟨*proof*⟩

lemma *y-errors-unitary*: $\forall\, U \in$ *set y-errors*. $U \in$ *carrier-mat 8 8* $\wedge$ *unitary U*
⟨*proof*⟩

lemma *y-images-orthogonal*:
$\forall\, i <$ *length y-errors*. $\forall\, j <$ *length y-errors*. $i \neq j \longrightarrow$
$(\forall\, a <$ *length y-codewords*. $\forall\, b <$ *length y-codewords*.
$\langle$*y-errors* ! $i *_v$ *y-codewords* ! $a \mid$ *y-errors* ! $j *_v$ *y-codewords* ! $b\rangle = 0)$
⟨*proof*⟩

D32-2: one CPTP map corrects every single Y flip on every state $a|++\rangle + b|\beta\beta\beta\rangle$.

theorem *y-code-corrects*:
$\exists$ *Rs*. *kraus-operators 8 8 Rs* $\wedge$
$(\forall\, c\ j$. *dim-vec c* $=$ *2* $\longrightarrow$ $j <$ *4* $\longrightarrow$
kraus-map-dim 8 Rs (*y-errors* ! $j *$ ((*mat-of-cols 8 y-codewords* $*_v$ *c*) $\oslash$ (*mat-of-cols 8 y-codewords* $*_v$ *c*)) $*$ ((*y-errors* ! $j)^\dagger$))
$=$ (*mat-of-cols 8 y-codewords* $*_v$ *c*) $\oslash$ (*mat-of-cols 8 y-codewords* $*_v$ *c*))
⟨*proof*⟩

end

theory *Day33-Shor-Code*
imports *Day32-Recovery-Proposition Day32-Pauli-Expansion Day30-BitFlip-Code*
begin

Days 32–33: the nine-qubit Shor code. The 2^9-dimensional space is never handled entrywise: nine qubits are three blocks of three, every single-qubit operator acts inside one block, and all inner products factor over the blocks (*tensor-inner-prod*).

43.7 Tensor products of matrices

lemma *tensor-index-c*:
fixes *A B* :: *complex Matrix.mat*
assumes *A* ∈ *carrier-mat r c B* ∈ *carrier-mat p q i* < *r* ∗ *p j* < *c* ∗ *q* 0 < *c* 0 < *q*
shows *Tensor.tensor-mat A B* $$ (*i*,*j*) = *A* $$ (*i div p*, *j div q*) ∗ *B* $$ (*i mod p*, *j mod q*)
⟨*proof*⟩

lemma *tensor-one-left*:
fixes *A* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat m m* **and** *m*: 0 < *m*
shows *Tensor.tensor-mat* (1$_m$ 1) *A* = *A*
⟨*proof*⟩

lemma *tensor-one-right*:
fixes *A* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat m m* **and** *m*: 0 < *m*
shows *Tensor.tensor-mat A* (1$_m$ 1) = *A*
⟨*proof*⟩

lemma *tensor-mat-add-right*:
fixes *A B C* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat r c* **and** *Bc*: *B* ∈ *carrier-mat p q* **and** *Cc*: *C* ∈ *carrier-mat p q*
and *c*: 0 < *c* **and** *q*: 0 < *q*
shows *Tensor.tensor-mat A* (*B* + *C*) = *Tensor.tensor-mat A B* + *Tensor.tensor-mat A C*
⟨*proof*⟩

lemma *tensor-mat-add-left*:
fixes *A B C* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat r c* **and** *Bc*: *B* ∈ *carrier-mat r c* **and** *Cc*: *C* ∈ *carrier-mat p q*
and *c*: 0 < *c* **and** *q*: 0 < *q*
shows *Tensor.tensor-mat* (*A* + *B*) *C* = *Tensor.tensor-mat A C* + *Tensor.tensor-mat B C*
⟨*proof*⟩

lemma *tensor-mat-smult-right*:
fixes *A B* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat r c* **and** *Bc*: *B* ∈ *carrier-mat p q* **and** *c*: 0 < *c* **and** *q*: 0 < *q*
shows *Tensor.tensor-mat A* (*k* ·$_m$ *B*) = *k* ·$_m$ *Tensor.tensor-mat A B*
⟨*proof*⟩

lemma *tensor-mat-smult-left*:
fixes *A B* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat r c* **and** *Bc*: *B* ∈ *carrier-mat p q* **and** *c*: 0 < *c* **and** *q*: 0 < *q*
shows *Tensor.tensor-mat* (*k* ·$_m$ *A*) *B* = *k* ·$_m$ *Tensor.tensor-mat A B*
⟨*proof*⟩

43.8 Single-qubit operators on nine qubits, and on a block of three

$\tilde{U} = I \otimes \cdots \otimes I \otimes U \otimes I \otimes \cdots \otimes I$ with U in position i ($i = 0, \ldots, 8$).

definition *qubit-op* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
qubit-op i U = *Tensor.tensor-mat* (*Tensor.tensor-mat* (*1*$_m$ (*2*^*i*)) *U*) (*1*$_m$ (*2*^(*8*−*i*)))

definition *bop* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
bop s U = *Tensor.tensor-mat* (*Tensor.tensor-mat* (*1*$_m$ (*2*^*s*)) *U*) (*1*$_m$ (*2*^(*2*−*s*)))

definition *blk* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
blk b M = *Tensor.tensor-mat* (*Tensor.tensor-mat* (*1*$_m$ (*8*^*b*)) *M*) (*1*$_m$ (*8*^(*2*−*b*)))

lemma *qubit-op-carrier*:
assumes *U*: *U* ∈ *carrier-mat 2 2* **and** *i*: *i* < *9*
shows *qubit-op i U* ∈ *carrier-mat 512 512*
⟨*proof*⟩

lemma *qubit-op-blk*:
assumes *b*: *b* < *3* **and** *s*: *s* < *3*
shows *qubit-op* (*3* ∗ *b* + *s*) *U* = *blk b* (*bop s U*)
⟨*proof*⟩

lemma *qubit-op-blk-div*:
assumes *i*: *i* < *9*
shows *qubit-op i U* = *blk* (*i div 3*) (*bop* (*i mod 3*) *U*)
⟨*proof*⟩

lemma *bop-cases*:
fixes *U* :: *complex Matrix.mat*
assumes *U*: *U* ∈ *carrier-mat 2 2*
shows *bop 0 U* = *Tensor.tensor-mat* (*Tensor.tensor-mat U* (*1*$_m$ *2*)) (*1*$_m$ *2*)
and *bop 1 U* = *Tensor.tensor-mat* (*Tensor.tensor-mat* (*1*$_m$ *2*) *U*) (*1*$_m$ *2*)
and *bop 2 U* = *Tensor.tensor-mat* (*Tensor.tensor-mat* (*1*$_m$ *2*) (*1*$_m$ *2*)) *U*
⟨*proof*⟩

lemma *bop-carrier*:
assumes *U*: *U* ∈ *carrier-mat 2 2* **and** *s*: *s* < *3*
shows *bop s U* ∈ *carrier-mat 8 8*
⟨*proof*⟩

definition *tv3* :: *complex Matrix.vec* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.vec* **where**
tv3 x y z = *tensor-vec* (*tensor-vec x y*) *z*

lemma *tv3-dim*: *dim-vec x* = *8* ⟹ *dim-vec y* = *8* ⟹ *dim-vec z* = *8* ⟹ *dim-vec* (*tv3 x y z*) = *512*
⟨*proof*⟩

lemma *blk-tv3*:
assumes *M*: *M* ∈ *carrier-mat 8 8* **and** *x*: *dim-vec x* = *8* **and** *y*: *dim-vec y* = *8* **and** *z*: *dim-vec z* = *8*
shows *blk 0 M* ∗$_v$ *tv3 x y z* = *tv3* (*M* ∗$_v$ *x*) *y z*
and *blk 1 M* ∗$_v$ *tv3 x y z* = *tv3 x* (*M* ∗$_v$ *y*) *z*
and *blk 2 M* ∗$_v$ *tv3 x y z* = *tv3 x y* (*M* ∗$_v$ *z*)
⟨*proof*⟩

43.9 The block vectors $|m\rangle \pm |7-m\rangle$

In a block of three qubits, *gvec p 0* is $|000\rangle \pm |111\rangle$; a bit flip moves it to *gvec p m* with $m \in \{4, 2, 1\}$. These eight vectors are mutually orthogonal.

definition *pm* :: *bool* ⇒ *complex* **where** *pm p* = (*if p then 1 else* −*1*)

definition *gvec* :: *bool* ⇒ *nat* ⇒ *complex Matrix.vec* **where**
gvec p m = *unit-vec 8 m* + *pm p* ·$_v$ *unit-vec 8* (7 − *m*)

lemma *gvec-dim* [*simp*]: *dim-vec* (*gvec p m*) = *8*
⟨*proof*⟩

lemma *gvec-index*:
assumes *r*: *r* < *8* **and** *m*: *m* < *8*
shows *gvec p m* $ *r* = (*if r* = *m then 1 else 0*) + *pm p* ∗ (*if r* = 7 − *m then 1 else 0*)
⟨*proof*⟩

lemma *mult-gvec0*:
assumes *M*: *M* ∈ *carrier-mat 8 8* **and** *r*: *r* < *8*
shows (*M* ∗$_v$ *gvec p 0*) $ *r* = *M* $$ (*r*,*0*) + *pm p* ∗ *M* $$ (*r*,7)
⟨*proof*⟩

lemma *bop-entry-cases*:
assumes *U*: *U* ∈ *carrier-mat 2 2* **and** *r*: *r* < *8* **and** *k*: *k* < *8*
shows *bop 0 U* $$ (*r*,*k*) = *U* $$ (*r div 4*, *k div 4*) ∗ *1*$_m$ *2* $$ (*r div 2 mod 2*, *k div 2 mod 2*) ∗ *1*$_m$ *2* $$ (*r mod 2*, *k mod 2*)
and *bop 1 U* $$ (*r*,*k*) = *1*$_m$ *2* $$ (*r div 4*, *k div 4*) ∗ *U* $$ (*r div 2 mod 2*, *k div 2 mod 2*) ∗ *1*$_m$ *2* $$ (*r mod 2*, *k mod 2*)
and *bop 2 U* $$ (*r*,*k*) = *1*$_m$ *2* $$ (*r div 4*, *k div 4*) ∗ *1*$_m$ *2* $$ (*r div 2 mod 2*, *k div 2 mod 2*) ∗ *U* $$ (*r mod 2*, *k mod 2*)
⟨*proof*⟩

lemma *bop-X-gvec*:
assumes *s*: *s* < *3*
shows *bop s Pauli-X* ∗$_v$ *gvec p 0* = *gvec p* (*2*^(*2*−*s*))
⟨*proof*⟩

lemma *bop-Y-gvec*:
assumes *s*: *s* < *3*
shows *bop s Pauli-Y* ∗$_v$ *gvec p 0* = (−i) ·$_v$ *gvec* (¬ *p*) (*2*^(*2*−*s*))
⟨*proof*⟩

lemma *bop-Z-gvec*:
assumes *s*: *s* < *3*
shows *bop s Pauli-Z* ∗$_v$ *gvec p 0* = *gvec* (¬ *p*) *0*
⟨*proof*⟩

lemma *inner-two-units*:
assumes *a*: *a* < *n* **and** *b*: *b* < *n* **and** *c*: *c* < *n* **and** *d*: *d* < *n*
shows ⟨*unit-vec n a* + *α* ·$_v$ *unit-vec n b* | *unit-vec n c* + *β* ·$_v$ *unit-vec n d*⟩
= (*if a* = *c then 1 else 0*) + *β* ∗ (*if a* = *d then 1 else 0*)
+ *cnj α* ∗ (*if b* = *c then 1 else 0*) + *cnj α* ∗ *β* ∗ (*if b* = *d then 1 else 0*)
⟨*proof*⟩

lemma *gvec-inner*:
assumes *m*: *m* ∈ {*0*,*1*,*2*,*4*} **and** *m*′: *m*′ ∈ {*0*,*1*,*2*,*4*}
shows ⟨*gvec p m* | *gvec p*′ *m*′⟩ = (*if m* = *m*′ ∧ *p* = *p*′ *then 2 else 0*)
⟨*proof*⟩

43.10 Code words and the images of the errors

definition *gv* :: *bool* × *nat* ⇒ *complex Matrix.vec* **where** *gv x* = *gvec* (*fst x*) (*snd x*)

A label lists the block vectors of a product state, one per block.

definition *lab-vec* :: (*bool* × *nat*) *list* ⇒ *complex Matrix.vec* **where**
lab-vec l = *tv3* (*gv* (*l* ! *0*)) (*gv* (*l* ! *1*)) (*gv* (*l* ! *2*))

definition *valid-label* :: (*bool* × *nat*) *list* ⇒ *bool* **where**
valid-label l ⟷ *length l* = *3* ∧ (∀ *x* ∈ *set l*. *snd x* ∈ {*0*,*1*,*2*,*4*})

definition *sk* :: *complex* **where** *sk* = *complex-of-real* (*1* / *sqrt 8*)

definition *shor-cw* :: *bool* ⇒ *complex Matrix.vec* **where**
shor-cw p = *sk* ·$_v$ *lab-vec* (*replicate 3* (*p*, *0*))

The notes' code words $|0_L\rangle, |1_L\rangle$.

lemma *shor-cw-notes*:
shor-cw True = *complex-of-real* (*1* / *sqrt* (*2*^*3*)) ·$_v$
tensor-vec (*tensor-vec* (*unit-vec 8 0* + *unit-vec 8 7*) (*unit-vec 8 0* + *unit-vec 8 7*)) (*unit-vec 8 0* + *unit-vec 8 7*)
shor-cw False = *complex-of-real* (*1* / *sqrt* (*2*^*3*)) ·$_v$
tensor-vec (*tensor-vec* (*unit-vec 8 0* − *unit-vec 8 7*) (*unit-vec 8 0* − *unit-vec 8 7*)) (*unit-vec 8 0* − *unit-vec 8 7*)
⟨*proof*⟩

lemma *gv-dim* [*simp*]: *dim-vec* (*gv x*) = *8*
⟨*proof*⟩

lemma *lab-vec-dim* [*simp*]: *dim-vec* (*lab-vec l*) = *512*
⟨*proof*⟩

lemma *shor-cw-dim* [*simp*]: *dim-vec* (*shor-cw p*) = *512*
⟨*proof*⟩

lemma *tv3-smult*:
assumes *dim-vec x* = *8 dim-vec y* = *8 dim-vec z* = *8*
shows *tv3* (*k* ·$_v$ *x*) *y z* = *k* ·$_v$ *tv3 x y z tv3 x* (*k* ·$_v$ *y*) *z* = *k* ·$_v$ *tv3 x y z*
tv3 x y (*k* ·$_v$ *z*) = *k* ·$_v$ *tv3 x y z*
⟨*proof*⟩

lemma *qubit-op-tl*:
assumes *i*: *i* < *9* **and** *U*: *U* ∈ *carrier-mat 2 2*
and *act*: *bop* (*i mod 3*) *U* ∗$_v$ *gvec p 0* = *κ* ·$_v$ *gvec q m*
shows *qubit-op i U* ∗$_v$ *lab-vec* (*replicate 3* (*p*, *0*)) = *κ* ·$_v$ *lab-vec* ((*replicate 3* (*p*, *0*))[*i div 3* := (*q*, *m*)])
⟨*proof*⟩

lemma *qubit-op-X-tl*:
i < *9* ⟹ *qubit-op i Pauli-X* ∗$_v$ *lab-vec* (*replicate 3* (*p*, *0*))
= *lab-vec* ((*replicate 3* (*p*, *0*))[*i div 3* := (*p*, *2*^(*2* − *i mod 3*))])
⟨*proof*⟩

lemma *qubit-op-Y-tl*:
i < *9* ⟹ *qubit-op i Pauli-Y* ∗$_v$ *lab-vec* (*replicate 3* (*p*, *0*))
= (−i) ·$_v$ *lab-vec* ((*replicate 3* (*p*, *0*))[*i div 3* := (¬ *p*, *2*^(*2* − *i mod 3*))])
⟨*proof*⟩

lemma *qubit-op-Z-tl*:
i < *9* ⟹ *qubit-op i Pauli-Z* ∗$_v$ *lab-vec* (*replicate 3* (*p*, *0*))
= *lab-vec* ((*replicate 3* (*p*, *0*))[*i div 3* := (¬ *p*, *0*)])
⟨*proof*⟩

lemma *lab-vec-inner*:
assumes *l*: *valid-label l* **and** *l'*: *valid-label l'*
shows ⟨*lab-vec l* | *lab-vec l'*⟩ = (*if l* = *l' then 8 else 0*)
⟨*proof*⟩

43.11 The 22 correctable errors

I, $X_1, \ldots, X_9$, $Y_1, \ldots, Y_9$, Z_1, Z_4, Z_7 (positions counted from 0 here).

definition *shor-errors* :: *complex Matrix.mat list* **where**
shor-errors = 1_m *512* # *map* (λ*i*. *qubit-op i Pauli-X*) [*0*..<*9*] @ *map* (λ*i*. *qubit-op i Pauli-Y*) [*0*..<*9*]
@ *map* (λ*b*. *qubit-op* (*3* ∗ *b*) *Pauli-Z*) [*0*..<*3*]

definition *shor-labels* :: *bool* ⇒ (*bool* × *nat*) *list list* **where**
shor-labels p = *replicate 3* (*p*, *0*)
map (λ*i*. (*replicate 3* (*p*, *0*))[*i div 3* := (*p*, *2*^(*2* − *i mod 3*))]) [*0*..<*9*]
@ *map* (λ*i*. (*replicate 3* (*p*, *0*))[*i div 3* := (¬ *p*, *2*^(*2* − *i mod 3*))]) [*0*..<*9*]
@ *map* (λ*b*. (*replicate 3* (*p*, *0*))[*b* := (¬ *p*, *0*)]) [*0*..<*3*]

definition *shor-kappa* :: *complex list* **where**
shor-kappa = *1* # *replicate 9 1* @ *replicate 9* (−i) @ *replicate 3 1*

lemma *shor-lengths* [*simp*]: *length shor-errors* = *22 length* (*shor-labels p*) = *22 length shor-kappa* = *22*
⟨*proof*⟩

lemma *shor-errors-cases*:
assumes *j*: *j* < *22*
obtains *j* = *0 shor-errors* ! *j* = 1_m *512 shor-labels p* ! *j* = *replicate 3* (*p*, *0*) *shor-kappa* ! *j* = *1*
| *i* **where** *i* < *9 j* = *Suc i shor-errors* ! *j* = *qubit-op i Pauli-X*
shor-labels p ! *j* = (*replicate 3* (*p*, *0*))[*i div 3* := (*p*, *2*^(*2* − *i mod 3*))] *shor-kappa* ! *j* = *1*
| *i* **where** *i* < *9 j* = *10* + *i shor-errors* ! *j* = *qubit-op i Pauli-Y*
shor-labels p ! *j* = (*replicate 3* (*p*, *0*))[*i div 3* := (¬ *p*, *2*^(*2* − *i mod 3*))] *shor-kappa* ! *j* = −i
| *b* **where** *b* < *3 j* = *19* + *b shor-errors* ! *j* = *qubit-op* (*3* ∗ *b*) *Pauli-Z*
shor-labels p ! *j* = (*replicate 3* (*p*, *0*))[*b* := (¬ *p*, *0*)] *shor-kappa* ! *j* = *1*
⟨*proof*⟩

lemma *shor-errors-carrier*: *j* < *22* ⟹ *shor-errors* ! *j* ∈ *carrier-mat 512 512*
⟨*proof*⟩

lemma *shor-errors-action*:
assumes *j*: *j* < *22*
shows *shor-errors* ! *j* ∗$_v$ *shor-cw p* = (*shor-kappa* ! *j* ∗ *sk*) ·$_v$ *lab-vec* (*shor-labels p* ! *j*)
⟨*proof*⟩

lemma *shor-labels-valid*: *j* < *22* ⟹ *valid-label* (*shor-labels p* ! *j*)
⟨*proof*⟩

lemma *shor-labels-distinct*: *distinct* (*shor-labels True* @ *shor-labels False*)
⟨*proof*⟩

lemma *shor-labels-differ*:
assumes *i*: *i* < *22* **and** *j*: *j* < *22* **and** *ij*: *i* ≠ *j*
shows *shor-labels p* ! *i* ≠ *shor-labels p'* ! *j*
⟨*proof*⟩

lemma *shor-errors-orth*:
assumes *i*: *i* < *22* **and** *j*: *j* < *22* **and** *ij*: *i* ≠ *j*
shows ⟨*shor-errors* ! *i* ∗$_v$ *shor-cw p* | *shor-errors* ! *j* ∗$_v$ *shor-cw p'*⟩ = *0*

⟨*proof*⟩

lemma *sk-sq*: *cnj sk* ∗ *sk* = *1* / *8*
⟨*proof*⟩

lemma *shor-cw-inner*: ⟨*shor-cw p* | *shor-cw p'*⟩ = (*if p* = *p' then 1 else 0*)
⟨*proof*⟩

43.12 The code space and its projection

definition *shor-V* :: *complex Matrix.mat* **where** *shor-V* = *mat-of-cols 512* [*shor-cw True*, *shor-cw False*]

definition *shor-P* :: *complex Matrix.mat* **where** *shor-P* = *shor-V* ∗ (*shor-V*†)

definition *shor-state* :: *complex* ⇒ *complex* ⇒ *complex Matrix.vec* **where**
shor-state α β = α ·$_v$ *shor-cw True* + β ·$_v$ *shor-cw False*

lemma *shor-state-dim* [*simp*]: *dim-vec* (*shor-state* α β) = *512*
⟨*proof*⟩

lemma *shor-V-carrier*: *shor-V* ∈ *carrier-mat 512 2*
⟨*proof*⟩

lemma *shor-orthonormal*: *orthonormal-list 512* [*shor-cw True*, *shor-cw False*]
⟨*proof*⟩

lemma *shor-VdV*: (*shor-V*†) ∗ *shor-V* = *1*$_m$ *2*
⟨*proof*⟩

lemma *mat-vec-unit-col*:
fixes *M* :: *complex Matrix.mat*
assumes *M*: *M* ∈ *carrier-mat n k* **and** *a*: *a* < *k*
shows *M* ∗$_v$ *unit-vec k a* = *Matrix.col M a*
⟨*proof*⟩

lemma *shor-V-col*: *a* < *2* ⟹ *Matrix.col shor-V a* = [*shor-cw True*, *shor-cw False*] ! *a*
⟨*proof*⟩

lemma *shor-V-unit*: *shor-V* ∗$_v$ *unit-vec 2 0* = *shor-cw True shor-V* ∗$_v$ *unit-vec 2 1* = *shor-cw False*
⟨*proof*⟩

lemma *shor-P-projection*: *projection 512 shor-P*
⟨*proof*⟩

lemma *shor-P-cw*: *shor-P* ∗$_v$ *shor-cw p* = *shor-cw p*
⟨*proof*⟩

lemma *shor-P-fixes*: *shor-P* ∗$_v$ *shor-state* α β = *shor-state* α β
⟨*proof*⟩

43.13 Proposition (Day 33): orthogonal images, and $Z_1 v = Z_2 v = Z_3 v$

lemma *gram-entry*:
fixes *A B* :: *complex Matrix.mat*
assumes *A*: *A* ∈ *carrier-mat n k* **and** *B*: *B* ∈ *carrier-mat n k* **and** *a*: *a* < *k* **and** *b*: *b* < *k*
shows ((*A*†) ∗ *B*) $$ (*a*,*b*) = ⟨*Matrix.col A a* | *Matrix.col B b*⟩
⟨*proof*⟩

lemma *sandwich-VVd*:
fixes V W :: *complex Matrix.mat*
assumes V: $V \in$ *carrier-mat n k* **and** W: $W \in$ *carrier-mat n n*
shows $(V * (V^\dagger)) * W * (V * (V^\dagger)) = V * ((V^\dagger) * W * V) * (V^\dagger)$
⟨*proof*⟩

lemma *qubit-op-unitary*: *unitary U* ⟹ *unitary* (*qubit-op i U*)
⟨*proof*⟩

lemma *shor-errors-unitary*: $U \in$ *set shor-errors* ⟹ $U \in$ *carrier-mat 512 512* ∧ *unitary U*
⟨*proof*⟩

theorem *shor-orthogonal-images*:
assumes *i*: $i < 22$ **and** *j*: $j < 22$ **and** *ij*: $i \neq j$
shows *shor-P* $*$ (((*shor-errors* ! *i*)†) $*$ *shor-errors* ! *j*) $*$ *shor-P* = 0_m *512 512*
⟨*proof*⟩

lemma *shor-recovery-setting*: *recovery-setting 512 shor-P shor-errors*
⟨*proof*⟩

lemma *qubit-op-Z-cw*:
assumes *i*: $i < 9$
shows *qubit-op i Pauli-Z* $*_v$ *shor-cw p* = *qubit-op* (*3* $*$ (*i div 3*)) *Pauli-Z* $*_v$ *shor-cw p*
⟨*proof*⟩

theorem *shor-Z-coincide*:
assumes *i*: $i < 9$ **and** *j*: $j < 9$ **and** *ij*: *i div 3* = *j div 3*
shows *qubit-op i Pauli-Z* $*_v$ *shor-state* α β = *qubit-op j Pauli-Z* $*_v$ *shor-state* α β
⟨*proof*⟩

The Day 32 proposition as first stated (distinct 1-Paulis map V to orthogonal subspaces) is false: $Z_1 \neq Z_2$, yet $Z_1|0_L\rangle = Z_2|0_L\rangle \neq 0$. The notes correct it on Day 33.

lemma *shor-notes-Z1-Z2*:
qubit-op 0 Pauli-Z ≠ *qubit-op 1 Pauli-Z*
⟨*qubit-op 0 Pauli-Z* $*_v$ *shor-cw True* | *qubit-op 1 Pauli-Z* $*_v$ *shor-cw True*⟩ = *1*
⟨*proof*⟩

43.14 Shor's theorem

lemma *pauli-decomp*:
fixes U :: *complex Matrix.mat*
assumes U: $U \in$ *carrier-mat 2 2*
shows U = ((*U* $$ (*0,0*) + *U* $$ (*1,1*)) / *2*) $\cdot_m$ 1_m *2* + ((*U* $$ (*0,1*) + *U* $$ (*1,0*)) / *2*) $\cdot_m$ *Pauli-X*
+ (−i $*$ (*U* $$ (*0,1*) − *U* $$ (*1,0*)) / *2*) $\cdot_m$ *Pauli-Y* + ((*U* $$ (*0,0*) − *U* $$ (*1,1*)) / *2*) $\cdot_m$ *Pauli-Z*
(**is** U = *?R*)
⟨*proof*⟩

lemma *qubit-op-add*:
assumes A: $A \in$ *carrier-mat 2 2* **and** B: $B \in$ *carrier-mat 2 2*
shows *qubit-op i* (*A* + *B*) = *qubit-op i A* + *qubit-op i B*
⟨*proof*⟩

lemma *qubit-op-smult*:
assumes A: $A \in$ *carrier-mat 2 2*
shows *qubit-op i* (*k* $\cdot_m$ *A*) = *k* $\cdot_m$ *qubit-op i A*
⟨*proof*⟩

lemma *qubit-op-one*:
assumes *i*: $i < 9$
shows *qubit-op i* $(1_m\ 2) = 1_m\ 512$
⟨*proof*⟩

lemma *smult-mat-mult-vec-c*:
fixes A :: *complex Matrix.mat*
assumes *dim-col* A = *dim-vec* v
shows $(k \cdot_m A) *_v v = k \cdot_v (A *_v v)$
⟨*proof*⟩

lemma *shor-errors-nth*:
assumes *i*: $i < 9$ **and** *b*: $b < 3$
shows *shor-errors* ! 0 = 1_m 512 *shor-errors* ! *Suc i* = *qubit-op i Pauli-X*
shor-errors ! (10 + *i*) = *qubit-op i Pauli-Y shor-errors* ! (19 + *b*) = *qubit-op* (3 ∗ *b*) *Pauli-Z*
⟨*proof*⟩

Shor: one CPTP recovery map R corrects every single-qubit unitary error on every qubit, for every state $\psi = a|0_L\rangle + b|1_L\rangle$ of the code space.

theorem *shor-code-theorem*:
∃ *Rs. kraus-operators 512 512 Rs* ∧
(∀ *U i* α β. *U* ∈ *carrier-mat 2 2* ⟶ *unitary U* ⟶ *i* < 9 ⟶
kraus-map-dim 512 Rs (*qubit-op i U* ∗ (*shor-state* α β ⊘ *shor-state* α β) ∗ ((*qubit-op i U*)†))
= *shor-state* α β ⊘ *shor-state* α β)
⟨*proof*⟩

end
theory *Day34-Knill-Laflamme*
imports *Day22-Minimal-Kraus*
begin

44 Day 34: the Knill–Laflamme conditions (D34-1, D34-2, D34-3)

44.1 Vectors in the range of a projection; rank-one decompositions inside V

lemma *projection-fixed-of-orth-kernel*:
assumes *P*: *projection n P* **and** *vd*: *dim-vec v* = *n*
and *orth*: ⋀*x*. *dim-vec x* = *n* ⟹ *P* $*_v$ *x* = 0_v *n* ⟹ ⟨*v* | *x*⟩ = 0
shows *P* $*_v$ *v* = *v*
⟨*proof*⟩

lemma *outer-sum-Nil*: *outer-sum n* [] = 0_m *n n*
⟨*proof*⟩

lemma *outer-sum-singleton-eq*:
assumes *vd*: *dim-vec v* = *n*
shows *outer-sum n* [*v*] = *v* ⊘ *v*
⟨*proof*⟩

lemma *outer-sum-as-foldr*:
assumes ∀ *v* ∈ *set vs*. *dim-vec v* = *n*
shows *outer-sum n vs* = *foldr* (λ*v acc*. *v* ⊘ *v* + *acc*) *vs* (0_m *n n*)
⟨*proof*⟩

For $X \geq 0$ the matrix PXP is a sum of $|\psi_l\rangle\langle\psi_l|$ with $\psi_l \in V$ (the notes: “non-zero eigenvectors

of PXP are all in V").

lemma *psd-PXP-decomp-in-range*:
assumes *P*: *projection n P* **and** *Xc*: $M \in$ *carrier-mat n n* **and** *Xpsd*: *psd M*
shows $\exists$ *vs* :: *complex Matrix.vec list*. ($\forall$ *v* $\in$ *set vs*. *dim-vec v* = *n* $\wedge$ *P* $*_v$ *v* = *v*) $\wedge$
P * *M* * *P* = *foldr* (λ*v acc*. *v* $\oslash$ *v* + *acc*) *vs* (0_m *n n*)
⟨*proof*⟩

44.2 Linearity facts for the composite map $\mathcal{R} \circ \mathbb{E}$

definition *RE* :: *nat* $\Rightarrow$ *complex Matrix.mat list* $\Rightarrow$ *complex Matrix.mat list* $\Rightarrow$ *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat* **where**
RE n As Es M = *kraus-map-dim n As* (*kraus-map-dim n Es M*)

lemma *RE-carrier*: **assumes** *As-c*: $\forall$ *A* $\in$ *set As*. *A* $\in$ *carrier-mat n n* **and** *Es-c*: $\forall$ *E* $\in$ *set Es*. *E* $\in$ *carrier-mat n n* **and** *M*: *M* $\in$ *carrier-mat n n* **shows** *RE n As Es M* $\in$ *carrier-mat n n*
⟨*proof*⟩

lemma *RE-add*: **assumes** *As-c*: $\forall$ *A* $\in$ *set As*. *A* $\in$ *carrier-mat n n* **and** *Es-c*: $\forall$ *E* $\in$ *set Es*. *E* $\in$ *carrier-mat n n* **and** *M*: *M* $\in$ *carrier-mat n n* **and** *N*: *N* $\in$ *carrier-mat n n* **shows** *RE n As Es* (*M* + *N*) = *RE n As Es M* + *RE n As Es N*
⟨*proof*⟩

lemma *RE-smult*: **assumes** *As-c*: $\forall$ *A* $\in$ *set As*. *A* $\in$ *carrier-mat n n* **and** *Es-c*: $\forall$ *E* $\in$ *set Es*. *E* $\in$ *carrier-mat n n* **and** *M*: *M* $\in$ *carrier-mat n n* **shows** *RE n As Es* (*c* $\cdot_m$ *M*) = *c* $\cdot_m$ *RE n As Es M*
⟨*proof*⟩

lemma *RE-minus*:
assumes *As-c*: $\forall$ *A* $\in$ *set As*. *A* $\in$ *carrier-mat n n* **and** *Es-c*: $\forall$ *E* $\in$ *set Es*. *E* $\in$ *carrier-mat n n*
and *Xc*: *M* $\in$ *carrier-mat n n* **and** *Yc*: *N* $\in$ *carrier-mat n n*
shows *RE n As Es* (*M* − *N*) = *RE n As Es M* − *RE n As Es N*
⟨*proof*⟩

lemma *RE-foldr*:
assumes *As-c*: $\forall$ *A* $\in$ *set As*. *A* $\in$ *carrier-mat n n* **and** *Es-c*: $\forall$ *E* $\in$ *set Es*. *E* $\in$ *carrier-mat n n* **and** $\forall$ *v* $\in$ *set vs*. *dim-vec v* = *n*
shows *RE n As Es* (*foldr* (λ*v acc*. *v* $\oslash$ *v* + *acc*) *vs* (0_m *n n*)) = *foldr* (λ*v acc*. *RE n As Es* (*v* $\oslash$ *v*) + *acc*) *vs* (0_m *n n*)
⟨*proof*⟩

D34-1: $\mathcal{R} \circ \mathbb{E}$ fixes $|\psi\rangle\langle\psi|$ for all $\psi \in V$ iff it fixes PXP for all X.

proposition *recovery-states-iff-PXP*:
assumes *As-c*: $\forall$ *A* $\in$ *set As*. *A* $\in$ *carrier-mat n n* **and** *Es-c*: $\forall$ *E* $\in$ *set Es*. *E* $\in$ *carrier-mat n n* **and** *P*: *projection n P*
shows ($\forall \psi$. *dim-vec* ψ = *n* $\longrightarrow$ *P* $*_v$ ψ = ψ $\longrightarrow$ *RE n As Es* ($\psi \oslash \psi$) = $\psi \oslash \psi$)
$\longleftrightarrow$ ($\forall$ *X*. *X* $\in$ *carrier-mat n n* $\longrightarrow$ *RE n As Es* (*P* * *X* * *P*) = *P* * *X* * *P*)
⟨*proof*⟩

44.3 Knill–Laflamme, Theorem 1, necessity: $PE_i^* E_j P = \alpha_{ij} P$

lemma *lin-indep-complex-mat-single*:
assumes *Pc*: *P* $\in$ *carrier-mat d n* **and** *Pnz*: *P* $\neq$ 0_m *d n*
shows *lin-indep-complex-mat d n* [*P*]
⟨*proof*⟩

The core of the necessity proof: by Choi's Theorem 2 applied to the two Kraus representations $\{P\}$ and $\{A_l E_i P\}$ of $X \mapsto PXP$, every $A_l E_i P$ is a multiple of P.

lemma *kl-kraus-scalars*:

assumes *P*: *projection n P* **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *npos*: $0 < n$
and *As-c*: $\forall A \in set\ As.\ A \in carrier\text{-}mat\ n\ n$ **and** *Es-c*: $\forall E \in set\ Es.\ E \in carrier\text{-}mat\ n\ n$
and *corr*: $\forall X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow$
kraus-map-dim n As (*kraus-map-dim n Es* $(P * X * P)) = P * X * P$
shows $\exists \beta :: nat \Rightarrow nat \Rightarrow complex.\ \forall l < length\ As.\ \forall i < length\ Es.$
$As\ !\ l * Es\ !\ i * P = \beta\ l\ i \cdot_m P$
⟨*proof*⟩

theorem *knill-laflamme-necessity*:
assumes *P*: *projection n P* **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *npos*: $0 < n$
and *As*: *kraus-operators n n As* **and** *Es*: *kraus-operators n n Es*
and *corr*: $\forall X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow$
kraus-map-dim n As (*kraus-map-dim n Es* $(P * X * P)) = P * X * P$
shows $\exists \alpha :: nat \Rightarrow nat \Rightarrow complex.\ \forall i < length\ Es.\ \forall j < length\ Es.$
$P * (((Es\ !\ i)^\dagger) * Es\ !\ j) * P = \alpha\ i\ j \cdot_m P$
⟨*proof*⟩

44.4 Kraus lists consisting of multiples of one operator

lemma *kraus-map-dim-scalar-list*:
assumes *Pc*: $P \in carrier\text{-}mat\ n\ n$ **and** *Xc*: $M \in carrier\text{-}mat\ n\ n$
shows *kraus-map-dim n* (*map* $(\lambda\gamma.\ \gamma \cdot_m P)$ *gs*) *M*
$= sum\text{-}list\ (map\ (\lambda\gamma.\ cnj\ \gamma * \gamma)\ gs) \cdot_m (P * M * (P^\dagger))$
⟨*proof*⟩

lemma *sum-wdag-w-scalar-list*:
assumes *Pc*: $P \in carrier\text{-}mat\ n\ n$
shows *sum-wdag-w n* (*map* $(\lambda\gamma.\ \gamma \cdot_m P)$ *gs*) = *sum-list* (*map* $(\lambda\gamma.\ cnj\ \gamma * \gamma)$ *gs*) $\cdot_m ((P^\dagger) * P)$
⟨*proof*⟩

lemma *smult-eq-self-imp-one*:
fixes *P* :: *complex Matrix.mat*
assumes *Pc*: $P \in carrier\text{-}mat\ n\ n$ **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *eq*: $s \cdot_m P = P$
shows $s = 1$
⟨*proof*⟩

44.5 Theorem 2

If $\mathcal{R} \circ \mathbb{E}$ fixes PXP and $G_i = \sum_j g_{ij} E_j$ with $\sum_i G_i^* G_i = I$, then $\mathcal{R} \circ \mathbb{G}$ fixes PXP as well (same $\mathcal{R}$).

theorem *knill-laflamme-closure*:
assumes *P*: *projection n P* **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *npos*: $0 < n$
and *As*: *kraus-operators n n As*
and *Es-c*: $\forall E \in set\ Es.\ E \in carrier\text{-}mat\ n\ n$
and *corr*: $\forall X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow$
kraus-map-dim n As (*kraus-map-dim n Es* $(P * X * P)) = P * X * P$
and *Gc*: $G \in carrier\text{-}mat\ t\ (length\ Es)$
and *Gs-tp*: *sum-wdag-w n* (*kraus-mix n n G Es*) $= 1_m\ n$
shows $\forall X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow$
kraus-map-dim n As (*kraus-map-dim n* (*kraus-mix n n G Es*) $(P * X * P)) = P * X * P$
⟨*proof*⟩

end
theory *Day34-KL-Sufficiency*
imports *Day34-Knill-Laflamme*
begin

D34-2, sufficiency: if $PE_i^*E_jP = \alpha_{ij}P$ then a recovery channel exists. Diagonalise $(\alpha_{ij}) = u\,\mathrm{diag}(d)\,u^*$; the errors $F_k = \sum_i u_{ik}E_i$ give the same channel and satisfy $PF_k^*F_lP = \delta_{kl}d_kP$, so the $F_k/\sqrt{d_k}$ with $d_k > 0$ satisfy the hypotheses of the recovery proposition (*recovery-setting*).

lemma *smult-cancel-nonzero*:
fixes P :: *complex Matrix.mat*
assumes *Pc*: $P \in$ *carrier-mat n n* **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *eq*: $c \cdot_m P = d \cdot_m P$
shows $c = d$
⟨*proof*⟩

lemma *one-smult-mat'*: $(1{::}complex) \cdot_m M = M$
⟨*proof*⟩

lemma *smult-mat-mult-vec'*:
fixes A :: *complex Matrix.mat*
assumes *dim-col* A = *dim-vec* v
shows $(k \cdot_m A) *_v v = k \cdot_v (A *_v v)$
⟨*proof*⟩

lemma *outer-prod-smult-vec*:
fixes v :: *complex Matrix.vec*
shows $(s \cdot_v v) \oslash (s \cdot_v v) = (s * cnj\ s) \cdot_m (v \oslash v)$
⟨*proof*⟩

lemma *kraus-map-dim-foldr-add*:
assumes *Ws-c*: $\forall W \in set\ Ws.\ W \in carrier\text{-}mat\ n\ n$ **and** *fc*: $\forall k \in set\ xs.\ f\ k \in carrier\text{-}mat\ n\ n$
shows *kraus-map-dim n Ws* (*foldr* $(\lambda k\ acc.\ f\ k + acc)$ *xs* $(0_m\ n\ n)$)
$=$ *foldr* $(\lambda k\ acc.$ *kraus-map-dim n Ws* $(f\ k) + acc)$ *xs* $(0_m\ n\ n)$
⟨*proof*⟩

lemma *kraus-map-dim-outer-as-foldr*:
assumes *Ws-c*: $\forall W \in set\ Ws.\ W \in carrier\text{-}mat\ n\ n$ **and** ψd: *dim-vec* $\psi = n$
shows *kraus-map-dim n Ws* $(\psi \oslash \psi)$
$=$ *foldr* $(\lambda k\ acc.\ (Ws\ !\ k *_v \psi) \oslash (Ws\ !\ k *_v \psi) + acc)$ $[0..<length\ Ws]$ $(0_m\ n\ n)$
⟨*proof*⟩

lemma *foldr-smult-sandwich*:
fixes V :: *nat* ⇒ *complex Matrix.mat*
assumes *Mc*: $M \in carrier\text{-}mat\ n\ n$ **and** *Nc*: $N \in carrier\text{-}mat\ n\ n$ **and** *Pc*: $P \in carrier\text{-}mat\ n\ n$
and *Vc*: $\forall j < r.\ V\ j \in carrier\text{-}mat\ n\ n$ **and** *h*: $\forall j < r.\ M * V\ j * N = \beta\ j \cdot_m P$
shows $M *$ *foldr* $(\lambda j\ acc.\ b\ j \cdot_m V\ j + acc)$ $[0..<r]$ $(0_m\ n\ n) * N = (\sum j{<}r.\ b\ j * \beta\ j) \cdot_m P$
⟨*proof*⟩

lemma *foldr-smult-dagger*:
fixes V :: *nat* ⇒ *complex Matrix.mat*
assumes *Vc*: $\forall j < r.\ V\ j \in carrier\text{-}mat\ n\ n$
shows (*foldr* $(\lambda j\ acc.\ b\ j \cdot_m V\ j + acc)$ $[0..<r]$ $(0_m\ n\ n))^\dagger$
$=$ *foldr* $(\lambda j\ acc.\ cnj\ (b\ j) \cdot_m ((V\ j)^\dagger) + acc)$ $[0..<r]$ $(0_m\ n\ n)$
⟨*proof*⟩

The bracket $P\,(\sum_i a_iE_i)^*(\sum_j b_jE_j)\,P$ under the Knill–Laflamme conditions.

lemma *kl-mix-bracket*:
assumes *Es-c*: $\forall E \in set\ Es.\ E \in carrier\text{-}mat\ n\ n$ **and** *Pc*: $P \in carrier\text{-}mat\ n\ n$
and *kl*: $\forall i < length\ Es.\ \forall j < length\ Es.\ P * (((Es\ !\ i)^\dagger) * Es\ !\ j) * P = \alpha\ i\ j \cdot_m P$
and *r*: $r = length\ Es$
shows $P *$ $(((foldr\ (\lambda i\ acc.\ a\ i \cdot_m (Es\ !\ i) + acc)\ [0..<r]\ (0_m\ n\ n))^\dagger)$
$*$ *foldr* $(\lambda j\ acc.\ b\ j \cdot_m (Es\ !\ j) + acc)$ $[0..<r]$ $(0_m\ n\ n)) * P$
$= (\sum j{<}r.\ b\ j * (\sum i{<}r.\ cnj\ (a\ i) * \alpha\ i\ j)) \cdot_m P$

⟨*proof*⟩

lemma *dagger-sandwich*:
assumes *Pc*: $P \in$ *carrier-mat n n* **and** *Ph*: $P^\dagger = P$ **and** *Ac*: $A \in$ *carrier-mat n n* **and** *Bc*: $B \in$ *carrier-mat n n*
shows $(P * ((A^\dagger) * B) * P)^\dagger = P * ((B^\dagger) * A) * P$
⟨*proof*⟩

lemma *smult-bracket*:
assumes *Pc*: $P \in$ *carrier-mat n n* **and** *Ac*: $A \in$ *carrier-mat n n* **and** *Bc*: $B \in$ *carrier-mat n n*
shows $P * (((x \cdot_m A)^\dagger) * (y \cdot_m B)) * P = (cnj\ x * y) \cdot_m (P * ((A^\dagger) * B) * P)$
⟨*proof*⟩

theorem *knill-laflamme-sufficiency*:
assumes *P*: *projection n P* **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *npos*: $0 < n$
and *Es*: *kraus-operators n n Es*
and *kl*: $\forall\, i < length\ Es.\ \forall\, j < length\ Es.\ P * (((Es\ !\ i)^\dagger) * Es\ !\ j) * P = \alpha\ i\ j \cdot_m P$
shows $\exists\, As.$ *kraus-operators n n As* $\wedge$
$(\forall\, X.\ X \in$ *carrier-mat n n* $\longrightarrow$ *kraus-map-dim n As* (*kraus-map-dim n Es* $(P * X * P)) = P * X * P)$
⟨*proof*⟩

D34-2 (Theorem 1): the code is correctable iff $PE_i^* E_j P = \alpha_{ij} P$.

theorem *knill-laflamme*:
assumes *P*: *projection n P* **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *npos*: $0 < n$
and *Es*: *kraus-operators n n Es*
shows ($\exists\, As.$ *kraus-operators n n As* $\wedge$
$(\forall\, X.\ X \in$ *carrier-mat n n* $\longrightarrow$ *kraus-map-dim n As* (*kraus-map-dim n Es* $(P * X * P)) = P * X * P)$)
$\longleftrightarrow$ $(\exists\, \alpha :: nat \Rightarrow nat \Rightarrow complex.\ \forall\, i < length\ Es.\ \forall\, j < length\ Es.$
$P * (((Es\ !\ i)^\dagger) * Es\ !\ j) * P = \alpha\ i\ j \cdot_m P)$
⟨*proof*⟩

end

Part III

Lectures 23–26: extreme points, operator systems and Arveson's theorem

theory *Day23-ChoiExtremeUCP*
imports
Quantum-Computing11
begin

45 Day 23 (19 October 2011): Choi's Theorem 3, extreme points of UCP and CPTP

Day 22 stated and Day 23 proves:

Theorem (notes) (Choi 3)**.** If $\Phi : M_n \to M_d$ is unital and completely positive, $\Phi(X) = \sum_{i=1}^r V_i^* X V_i$ with $r = \mathrm{cr}(\Phi)$, then Φ is extreme in $UCP(M_n, M_d)$ if and only if $\{V_i^* V_j : 1 \leq$

$i, j \leq r\} \subseteq M_d$ is linearly independent.

Corollary (notes). If $\Phi : M_d \to M_n$ is completely positive and trace preserving, $\Phi(X) = \sum_{i=1}^{r} V_i X V_i^*$ with $r = \mathrm{cr}(\Phi)$, then Φ is extreme in $CPTP(M_d, M_n)$ iff $\{V_i^* V_j\}$ is linearly independent (apply the theorem to Φ^*).

The notes' proof. ($\Rightarrow$) If $\sum_{ij} \lambda_{ij} V_i^* V_j = 0$ it suffices to treat Hermitian $\mu = (\mu_{ij})$ with $-I \leq \mu \leq I$: writing $I \pm \mu = (\alpha_{ij})^*(\alpha_{ij})$ and $W_i = \sum_j \alpha_{ij} V_j$ shows that $\psi_\pm(X) = \sum_i V_i^* X V_i \pm \sum_{ij} \mu_{ij} V_i^* X V_j$ are UCP with $\Phi = \frac{1}{2}(\psi_+ + \psi_-)$, so extremality gives $\psi_+ = \Phi$, and Choi 2 (unitary freedom) forces $I + \mu = U^* U = I$, i.e. $\mu = 0$. ($\Leftarrow$) If $\Phi = \frac{1}{2}(\psi_1 + \psi_2)$ with $\psi_k(X) = \sum_p W_p^* X W_p$, Choi 2 gives $W_p/\sqrt{2} = \sum_j \alpha_{pj} V_j$ with $U^* U = I_r$; unitality of ψ_1 and linear independence of the $V_i^* V_j$ force $U_1^* U_1 = \frac{1}{2} I_r$, whence $\psi_1 = \Phi = \psi_2$. Examples: $X \mapsto U^* X U$ is extreme; $\frac{1}{2}(U_1^* X U_1 + U_2^* X U_2)$, $X \mapsto \mathrm{tr}(X) I_d$ and $X \mapsto \mathrm{diag}(X)$ are not.

What is formalised here is the corollary, i.e. the CPTP form, for a fixed Kraus list *Vs* of linearly independent operators (the consequence of $r = \mathrm{cr}(\Phi)$ proved on Day 22): extremality (in the carrier sense *extreme-CPTP-carrier*) holds if and only if the products $V_j^\dagger V_i$ are linearly independent over $\mathbb{C}$ (*choi-theorem-3-cptp-iff*). The direction ($\Rightarrow$) follows the notes' perturbation argument (*kraus-quad-map*, *small-eps-psd*); the direction ($\Leftarrow$) replaces Choi 2 by a CP Radon–Nikodym theorem proved below from the Moore–Penrose pseudoinverse of the Choi matrix. Unitary freedom itself is *choi-2-unitary-freedom* in *Day21-Unitary-Freedom*, proved afterwards and independently of this theory; the examples are in theory Day23_Extremality_Examples (the UCP form is not formalised).

45.1 Extreme UCP maps

An *extreme UCP map* is one that cannot be written as a proper convex combination of two distinct UCP maps. Following Day-22's *extreme-point-mat*, we use the half-half mid-point formulation; this is equivalent to the standard definition for convex sets.

definition *extreme-UCP* ::
  nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool*
where
  extreme-UCP n d Φ ≡
    Φ ∈ *UCP n d* ∧
    ¬ (∃ Φ₁ ∈ *UCP n d*. ∃ Φ₂ ∈ *UCP n d*.
        Φ₁ ≠ Φ ∧ Φ ≠ Φ₂ ∧
        (∀ *X*. *X* ∈ *carrier-mat n n*
            ⟶ Φ *X* = (*1/2* :: *complex*) ·ₘ Φ₁ *X*
                + (*1/2* :: *complex*) ·ₘ Φ₂ *X*))

lemma *extreme-UCP-member*:
  assumes *extreme-UCP n d* Φ
  shows Φ ∈ *UCP n d*
  ⟨*proof*⟩

Trivial unfolding: an extreme UCP map satisfies no non-trivial mid-point decomposition.

lemma *extreme-UCP-no-split*:
  assumes *ext*: *extreme-UCP n d* Φ
    and Φ*1-in*: Φ₁ ∈ *UCP n d*
    and Φ*2-in*: Φ₂ ∈ *UCP n d*
    and Φ*1-neq*: Φ₁ ≠ Φ
    and Φ*2-neq*: Φ ≠ Φ₂
  shows ∃ *X* ∈ *carrier-mat n n*.
        Φ *X* ≠ (*1/2* :: *complex*) ·ₘ Φ₁ *X*
            + (*1/2* :: *complex*) ·ₘ Φ₂ *X*
  ⟨*proof*⟩

45.2 UCP-Kraus representation

A *UCP Kraus representation* for a map $\Phi : M_n \to M_d$ is a list Vs of $d \times n$ matrices such that

- each $V \in Vs$ is $d \times n$;
- the unitality identity $\sum_i V_i\ V_i^\dagger = I_d$ holds;
- $\Phi\ X = \sum_i V_i\ X\ V_i^\dagger$ for every $X \in M_n$.

Note that the standard *kraus-operators* predicate of Day 17 uses the *trace-preserving* identity $\sum V_i^\dagger\ V_i = I_n$; the *unital* form $\sum V_i\ V_i^\dagger = I_d$ is the Heisenberg-dual condition appropriate for *UCP*.

definition *ucp-kraus-rep* ::
  nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒
    complex Matrix.mat list ⇒ *bool*
where
  ucp-kraus-rep n d Φ Vs ≡
    (∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*) ∧
    foldr (λ*V acc*. *V* ∗ (*V*†) + *acc*) *Vs* (*0*$_m$ *d d*) = *1*$_m$ *d* ∧
    (∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ *X* = *kraus-map-dim d Vs X*)

The *products set* of a Kraus list Vs is the list of $n \times n$ matrices $V_j^\dagger\ V_i$ for all pairs of indices (i,j).

definition *kraus-products* ::
  complex Matrix.mat list ⇒ *complex Matrix.mat list*
where
  kraus-products Vs ≡
    concat (*map* (λ*i*. *map* (λ*j*. (*Vs*!*j*)† ∗ *Vs*!*i*) [*0*..<*length Vs*])
          [*0*..<*length Vs*])

Real linear independence for a list of complex matrices. A list Bs is independent over $\mathbb{R}$ iff the only $\mathbb{R}$-coefficients making $\sum_k c_k\ B_k = 0$ are all zero. We use the real-coefficient form because Choi's perturbation argument constructs real combinations of hermitian generators.

definition *mat-lin-indep-real* ::
  nat ⇒ *complex Matrix.mat list* ⇒ *bool*
where
  mat-lin-indep-real n Bs ≡
    (∀ *B* ∈ *set Bs*. *B* ∈ *carrier-mat n n*) ∧
    (∀ *c* :: *nat* ⇒ *real*.
      foldr (λ*k acc*. *complex-of-real* (*c k*) ·$_m$ (*Bs*!*k*) + *acc*)
          [*0*..<*length Bs*] (*0*$_m$ *n n*)
        = *0*$_m$ *n n*
      ⟶ (∀ *k* < *length Bs*. *c k* = *0*))

Complex linear independence: the classical Choi statement uses **complex**-coefficient linear independence. This is strictly stronger than *mat-lin-indep-real*.

definition *mat-lin-indep-complex* ::
  nat ⇒ *complex Matrix.mat list* ⇒ *bool*
where
  mat-lin-indep-complex n Bs ≡
    (∀ *B* ∈ *set Bs*. *B* ∈ *carrier-mat n n*) ∧
    (∀ *c* :: *nat* ⇒ *complex*.
      foldr (λ*k acc*. *c k* ·$_m$ (*Bs*!*k*) + *acc*)
          [*0*..<*length Bs*] (*0*$_m$ *n n*)
        = *0*$_m$ *n n*

$\longrightarrow (\forall k < \mathit{length}\ \mathit{Bs}.\ c\ k = 0))$

Complex linear independence implies real.

lemma *mat-lin-indep-complex-imp-real*:
assumes *mat-lin-indep-complex n Bs*
shows *mat-lin-indep-real n Bs*
⟨*proof*⟩

45.3 Unital condition from the Kraus list

If *Ws* is a list of $d \times n$ matrices, then evaluating the Kraus map at the identity $1_m\ n$ gives exactly the sum $\sum\ W\ W^\dagger$. Combined with unitality, this gives the $\sum\ W\ W^\dagger = 1_m\ d$ identity that distinguishes UCP from plain CP.

lemma *kraus-map-dim-one*:
assumes $\forall\, W \in \mathit{set}\ \mathit{Ws}.\ W \in \mathit{carrier\text{-}mat}\ d\ n$
shows *kraus-map-dim d Ws* $(1_m\ n)$
$= \mathit{foldr}\ (\lambda W\ \mathit{acc}.\ W * (W^\dagger) + \mathit{acc})\ \mathit{Ws}\ (0_m\ d\ d)$
⟨*proof*⟩

45.4 UCP-Kraus existence

Every linear UCP map admits a UCP Kraus representation of length $n\ d$. Linearity, scalar-multiplication and range hypotheses match those of *kraus-rep-of-psd-choi*; the unital condition $\sum\ V\ V^\dagger = 1_m\ d$ is supplied by $\Phi\ (1_m\ n) = 1_m\ d$ + *kraus-map-dim-one*.

lemma *ucp-has-kraus-rep*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in \mathit{carrier\text{-}mat}\ n\ n \Longrightarrow B \in \mathit{carrier\text{-}mat}\ n\ n \Longrightarrow$
$\Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in \mathit{carrier\text{-}mat}\ n\ n \Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall\, A.\ A \in \mathit{carrier\text{-}mat}\ n\ n \longrightarrow \Phi\ A \in \mathit{carrier\text{-}mat}\ d\ d$
and *npos*: $0 < n$ **and** *dpos*: $0 < d$
and *ucp*: $\Phi \in \mathit{UCP}\ n\ d$
shows $\exists\ \mathit{Vs}$ :: *complex Matrix.mat list*.
ucp-kraus-rep n d Φ *Vs* $\wedge$ *length Vs* $= n * d$
⟨*proof*⟩

45.5 Sanity facts about *kraus-products*

lemma *kraus-products-carrier*:
assumes $\forall\, V \in \mathit{set}\ \mathit{Vs}.\ V \in \mathit{carrier\text{-}mat}\ d\ n$
shows $\forall\, B \in \mathit{set}\ (\mathit{kraus\text{-}products}\ \mathit{Vs}).\ B \in \mathit{carrier\text{-}mat}\ n\ n$
⟨*proof*⟩

45.6 Quadratic Kraus map (perturbation primitive)

For a Kraus list *Vs* of $d \times n$ matrices and a coefficient matrix $T \in M_k$ (where k = *length Vs*), the *quadratic Kraus map* is (*kraus-quad-map d Vs T*) $X = \sum_{\{i,j\}}\ T_{\{ij\}}\ V_i\ X\ V_j{}^\dagger$. When $T = I_k$ this reduces to the ordinary Kraus map *kraus-map-dim d Vs* $X = \sum\ V_i\ X\ V_i{}^\dagger$; for PSD T the result is again a Kraus map (Lemma A below). The perturbations $T_\pm = I \pm \varepsilon\ A$ for small ε and hermitian A will provide the *splittings* witnessing non-extremity.

definition *kraus-quad-row* ::
nat ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒
complex Matrix.mat ⇒ *nat* ⇒ *complex Matrix.mat*
where
kraus-quad-row d Vs Tm rho i ≡

foldr (λj *acc. Tm* \$\$ (i, j) $\cdot_m$ (*Vs*!i $*$ *rho* $*$ ((*Vs*!j)†)) $+$ *acc*)
[0..<*length Vs*] (0_m *d d*)

definition *kraus-quad-map* ::
nat $\Rightarrow$ *complex Matrix.mat list* $\Rightarrow$ *complex Matrix.mat* $\Rightarrow$
complex Matrix.mat $\Rightarrow$ *complex Matrix.mat*
where
kraus-quad-map d Vs Tm rho $\equiv$
foldr (λi *acc. kraus-quad-row d Vs Tm rho i* $+$ *acc*)
[0..<*length Vs*] (0_m *d d*)

Generic carrier-preservation lemma for accumulating folds over [0..<N]: if each per-index term lies in M_d, so does the fold.

lemma *foldr-add-carrier*:
fixes f :: $'a \Rightarrow$ *complex Matrix.mat*
assumes $\forall\, x \in$ *set xs*. $f\ x \in$ *carrier-mat d d*
shows *foldr* (λx *acc*. $f\ x +$ *acc*) *xs* (0_m *d d*) $\in$ *carrier-mat d d*
⟨*proof*⟩

lemma *foldr-upt-add-carrier*:
fixes f :: *nat* $\Rightarrow$ *complex Matrix.mat*
assumes $\forall\, k < N$. $f\ k \in$ *carrier-mat d d*
shows *foldr* (λk *acc*. $f\ k +$ *acc*) [0..<N] (0_m *d d*) $\in$ *carrier-mat d d*
⟨*proof*⟩

lemma *kraus-quad-row-carrier*:
assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
and *rho-c*: *rho* $\in$ *carrier-mat n n*
and *i-lt*: $i <$ *length Vs*
shows *kraus-quad-row d Vs Tm rho i* $\in$ *carrier-mat d d*
⟨*proof*⟩

lemma *kraus-quad-map-carrier*:
assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
and *rho-c*: *rho* $\in$ *carrier-mat n n*
shows *kraus-quad-map d Vs Tm rho* $\in$ *carrier-mat d d*
⟨*proof*⟩

Per-entry index formula for the accumulating fold (with arbitrary base).

lemma *foldr-upt-add-acc-index*:
fixes f :: *nat* $\Rightarrow$ *complex Matrix.mat*
assumes *carr*: $\forall\, k < N$. $f\ k \in$ *carrier-mat d d*
and *z-c*: $z \in$ *carrier-mat d d*
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
shows *foldr* (λk *acc*. $f\ k +$ *acc*) [0..<N] z \$\$ (r, c)
$= (\sum k = 0..<N.\ f\ k$ \$\$ $(r, c)) + z$ \$\$ (r, c)
⟨*proof*⟩

lemma *foldr-upt-add-index*:
fixes f :: *nat* $\Rightarrow$ *complex Matrix.mat*
assumes *carr*: $\forall\, k < N$. $f\ k \in$ *carrier-mat d d*
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
shows *foldr* (λk *acc*. $f\ k +$ *acc*) [0..<N] (0_m *d d*) \$\$ (r, c)
$= (\sum k = 0..<N.\ f\ k$ \$\$ $(r, c))$
⟨*proof*⟩

Per-entry formula for *kraus-quad-row*.

lemma *kraus-quad-row-index*:

assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *rho-c*: $rho \in carrier\text{-}mat\ n\ n$
and *i-lt*: $i < length\ Vs$
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
shows *kraus-quad-row d Vs Tm rho i* \$\$ $(r,\ c)$
$= (\sum j = 0..<length\ Vs.$
Tm \$\$ $(i,\ j) * (Vs!i * rho * ((Vs!j)^\dagger))$ \$\$ $(r,\ c))$
⟨*proof*⟩

Per-entry formula for *kraus-quad-map*: double sum over indices.

lemma *kraus-quad-map-index*:
assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *rho-c*: $rho \in carrier\text{-}mat\ n\ n$
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
shows *kraus-quad-map d Vs Tm rho* \$\$ $(r,\ c)$
$= (\sum i = 0..<length\ Vs.$
$\sum j = 0..<length\ Vs.$
Tm \$\$ $(i,\ j) * (Vs!i * rho * ((Vs!j)^\dagger))$ \$\$ $(r,\ c))$
⟨*proof*⟩

Carrier-preservation in a different dimension pair.

lemma *foldr-add-carrier-dn*:
fixes $f :: 'a \Rightarrow complex\ Matrix.mat$
assumes $\forall\, x \in set\ xs.\ f\ x \in carrier\text{-}mat\ d\ n$
shows $foldr\ (\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ d\ n) \in carrier\text{-}mat\ d\ n$
⟨*proof*⟩

Linear combination of Kraus operators with complex coefficients. *kraus-combo d n Vs u* is $\sum_i u_i\ V_i$, a $d \times n$ matrix.

definition *kraus-combo* ::
$nat \Rightarrow nat \Rightarrow complex\ Matrix.mat\ list \Rightarrow complex\ Matrix.vec \Rightarrow$
$complex\ Matrix.mat$
where
kraus-combo d n Vs u ≡
$foldr\ (\lambda i\ acc.\ u$ \$ $i \cdot_m (Vs!i) + acc)\ [0..<length\ Vs]\ (0_m\ d\ n)$

lemma *kraus-combo-carrier*:
assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
shows *kraus-combo d n Vs u* $\in carrier\text{-}mat\ d\ n$
⟨*proof*⟩

Per-entry formula for *kraus-combo*: foldr index lemma in d×n.

lemma *foldr-upt-add-acc-index-dn*:
fixes $f :: nat \Rightarrow complex\ Matrix.mat$
assumes *carr*: $\forall\, k < N.\ f\ k \in carrier\text{-}mat\ d\ n$
and *z-c*: $z \in carrier\text{-}mat\ d\ n$
and *r-lt*: $r < d$ **and** *c-lt*: $c < n$
shows $foldr\ (\lambda k\ acc.\ f\ k + acc)\ [0..<N]\ z$ \$\$ $(r,\ c)$
$= (\sum k = 0..<N.\ f\ k$ \$\$ $(r,\ c)) + z$ \$\$ $(r,\ c)$
⟨*proof*⟩

lemma *kraus-combo-index*:
assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *r-lt*: $r < d$ **and** *c-lt*: $c < n$
shows *kraus-combo d n Vs u* \$\$ $(r,\ c)$
$= (\sum i = 0..<length\ Vs.\ u$ \$ $i * (Vs!i)$ \$\$ $(r,\ c))$
⟨*proof*⟩

A direct per-entry formula for matrix products.

lemma *mat-mult-index-sum*:
assumes *A-c*: $A \in$ *carrier-mat p q*
and *B-c*: $B \in$ *carrier-mat q s*
and *r-lt*: $r < p$ **and** *c-lt*: $c < s$
shows $(A * B)$ \$\$ $(r, c) = (\sum k = 0..<q.\ A$ \$\$ $(r, k) * B$ \$\$ $(k, c))$
⟨*proof*⟩

Per-entry formula for $W *$ *rho*, where $W =$ *kraus-combo d n Vs u*.

lemma *kraus-combo-times-rho-index*:
assumes *Vs-c*: $\forall V \in$ *set Vs*. $V \in$ *carrier-mat d n*
and *rho-c*: *rho* $\in$ *carrier-mat n n*
and *r-lt*: $r < d$ **and** *k-lt*: $k < n$
shows (*kraus-combo d n Vs u* $*$ *rho*) \$\$ (r, k)
$= (\sum i = 0..<$*length Vs*. *u* \$ *i* $*$ (*Vs*!*i* $*$ *rho*) \$\$ $(r, k))$
⟨*proof*⟩

The full quadratic per-entry formula: $(W\ rho\ W^\dagger)$ \$\$ $(r,c) = \sum_{\{i,j\}} u_i\ |u_j|\ (V_i\ rho\ {V_j}^\dagger)$ \$\$ (r,c).

lemma *kraus-combo-quad-index*:
assumes *Vs-c*: $\forall V \in$ *set Vs*. $V \in$ *carrier-mat d n*
and *rho-c*: *rho* $\in$ *carrier-mat n n*
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
shows (*kraus-combo d n Vs u* $*$ *rho* $*$ ((*kraus-combo d n Vs u*)†)) \$\$ (r, c)
$= (\sum i = 0..<$*length Vs*. $\sum j = 0..<$*length Vs*.
u \$ *i* $*$ *cnj* (*u* \$ *j*) $*$ (*Vs*!*i* $*$ *rho* $*$ ((*Vs*!*j*)†)) \$\$ $(r, c))$
⟨*proof*⟩

Linearity of *kraus-quad-map* in the coefficient matrix (needed for the rank-1 decomposition below).

lemma *kraus-quad-map-add-Tm*:
assumes *Vs-c*: $\forall V \in$ *set Vs*. $V \in$ *carrier-mat d n*
and *rho-c*: *rho* $\in$ *carrier-mat n n*
and *T1-c*: *T1* $\in$ *carrier-mat* (*length Vs*) (*length Vs*)
and *T2-c*: *T2* $\in$ *carrier-mat* (*length Vs*) (*length Vs*)
shows *kraus-quad-map d Vs* (*T1* + *T2*) *rho*
$=$ *kraus-quad-map d Vs T1 rho* + *kraus-quad-map d Vs T2 rho*
⟨*proof*⟩

Carrier of *outer-sum k vs*.

lemma *outer-sum-carrier*:
outer-sum k vs $\in$ *carrier-mat k k*
⟨*proof*⟩

Cons-recursion for *outer-sum*: adds a rank-1 piece.

lemma *outer-sum-Cons-split*:
assumes $\forall v \in$ *set* (*u* # *us*). *dim-vec v* $= k$
shows *outer-sum k* (*u* # *us*)
$=$ *outer-sum k* [*u*] + *outer-sum k us*
⟨*proof*⟩

Rank-1 identity: *kraus-quad-map d Vs* (*outer-sum k* [*u*]) *rho* $= W\ rho\ W^\dagger$ where $W =$ *kraus-combo d n Vs u*.

lemma *kraus-quad-map-outer-singleton*:
assumes *Vs-c*: $\forall V \in$ *set Vs*. $V \in$ *carrier-mat d n*
and *rho-c*: *rho* $\in$ *carrier-mat n n*

and *u-dim*: *dim-vec u = length Vs*
shows *kraus-quad-map d Vs* (*outer-sum* (*length Vs*) [*u*]) *rho*
= *kraus-combo d n Vs u* ∗ *rho* ∗ ((*kraus-combo d n Vs u*)†)
⟨*proof*⟩

Vanishing of the quadratic map when the coefficient matrix is zero.

lemma *kraus-quad-map-zero-Tm*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *rho-c*: *rho* ∈ *carrier-mat n n*
shows *kraus-quad-map d Vs* (0$_m$ (*length Vs*) (*length Vs*)) *rho* = 0$_m$ *d d*
⟨*proof*⟩

Sum-decomposition: a PSD-style coefficient matrix *outer-sum k us* gives *kraus-quad-map* as a kraus_map_dim of the combos.

lemma *kraus-quad-map-outer-sum-eq-kraus-map-dim*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *rho-c*: *rho* ∈ *carrier-mat n n*
and *us-dim*: ∀ *v* ∈ *set us*. *dim-vec v = length Vs*
shows *kraus-quad-map d Vs* (*outer-sum* (*length Vs*) *us*) *rho*
= *kraus-map-dim d* (*map* (*kraus-combo d n Vs*) *us*) *rho*
⟨*proof*⟩

45.7 Lemma A: PSD coefficient matrix gives a CP perturbation

Since *amplification* only evaluates Φ on $n \times n$ blocks, two maps that agree on *carrier-mat n n* yield the same amplification, hence the same notion of complete positivity.

lemma *amplification-cong*:
fixes *Phi Psi* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Xm* :: *complex Matrix.mat*
assumes *eq*: ⋀*r*. *r* ∈ *carrier-mat n n* ⟹ *Phi r = Psi r*
shows *amplification n d p Phi Xm = amplification n d p Psi Xm*
⟨*proof*⟩

lemma *completely-positive-cong*:
fixes *Phi Psi* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes ⋀*r*. *r* ∈ *carrier-mat n n* ⟹ *Phi r = Psi r*
shows *completely-positive n d Phi* ⟷ *completely-positive n d Psi*
⟨*proof*⟩

Lemma A: if T is PSD then *kraus-quad-map d Vs T* is completely positive.

Proof sketch. Use ⟦*?P* ∈ *carrier-mat ?n ?n*; *?P* ≥$_M$ *0*⟧ ⟹ ∃ *vs*. *length vs = ?n* ∧ (∀ *v*∈*set vs*. *dim-vec v = ?n*) ∧ *outer-sum ?n vs = ?P* to write T = *outer-sum k us*. Then by ⟦∀ *V*∈*set ?Vs*. *V* ∈ *carrier-mat ?d ?n*; *?rho* ∈ *carrier-mat ?n ?n*; ∀ *v*∈*set ?us*. *dim-vec v = length ?Vs*⟧ ⟹ *kraus-quad-map ?d ?Vs* (*outer-sum* (*length ?Vs*) *?us*) *?rho* = *kraus-map-dim ?d* (*map* (*kraus-combo ?d ?n ?Vs*) *?us*) *?rho*, *kraus-quad-map d Vs T* agrees on *carrier-mat n n* with *kraus-map-dim d Ws*, where *Ws* := *map* (*kraus-combo d n Vs*) *us*. Apply ⟦∀ *W*∈*set ?Ws*. *W* ∈ *carrier-mat ?d ?n*; *0* < *?n*; *0* < *?d*⟧ ⟹ *completely-positive ?n ?d* (*kraus-map-dim ?d ?Ws*).

lemma *kraus-perturbation-cp*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *Tm-c*: *Tm* ∈ *carrier-mat* (*length Vs*) (*length Vs*)
and *Tm-psd*: *psd Tm*
and *npos*: *0* < *n* **and** *dpos*: *0* < *d*
shows *completely-positive n d* (*kraus-quad-map d Vs Tm*)
⟨*proof*⟩

45.8 Small-ε PSD perturbation of the identity

For any hermitian $A : k \times k$, the matrix $1_m\ k + \delta \cdot_m A$ is PSD whenever $|\delta|$ is small enough. The bound uses the entry-wise sum $S := \sum_{\{i,j\}} |A_{\{ij\}}|$, taking $\varepsilon := 1/(1 + S)$; this avoids any operator-norm computation and just uses Cauchy-Schwarz on entries.

Bilinearity of the inner product in the right slot.

lemma *inner-prod-add-right*:
fixes *u v w* :: *complex Matrix.vec*
assumes *ud*: *dim-vec u* = *k* **and** *vd*: *dim-vec v* = *k* **and** *wd*: *dim-vec w* = *k*
shows $\langle u \mid v + w\rangle = \langle u \mid v\rangle + \langle u \mid w\rangle$
⟨*proof*⟩

lemma *inner-prod-smult-right*:
fixes *u v* :: *complex Matrix.vec*
assumes *ud*: *dim-vec u* = *k* **and** *vd*: *dim-vec v* = *k*
shows $\langle u \mid c \cdot_v v\rangle = c * \langle u \mid v\rangle$
⟨*proof*⟩

The hermitian sum.

lemma *hermitian-add*:
assumes *Aherm*: *hermitian A* **and** *Bherm*: *hermitian B*
and *Ac*: $A \in$ *carrier-mat k k* **and** *Bc*: $B \in$ *carrier-mat k k*
shows *hermitian* $(A + B)$
⟨*proof*⟩

lemma *hermitian-one-mat*:
hermitian $(1_m\ k ::$ *complex Matrix.mat*$)$
⟨*proof*⟩

lemma *hermitian-smult-real*:
fixes *A* :: *complex Matrix.mat*
assumes *hermitian A A* $\in$ *carrier-mat k k*
shows *hermitian* (*complex-of-real d* $\cdot_m$ *A*)
⟨*proof*⟩

Entry-wise bound on the quadratic form of a hermitian matrix.

lemma *quad-form-bound-by-entry-sum*:
fixes *A* :: *complex Matrix.mat* **and** *x* :: *complex Matrix.vec*
assumes *A-c*: $A \in$ *carrier-mat k k* **and** *x-d*: *dim-vec x* = *k*
shows *cmod* $(\langle x \mid A *_v x\rangle)$
$\leq (\sum i = 0..<k.\ \sum j = 0..<k.\ cmod\ (A\ \$\$\ (i,j))) * Re\ \langle x \mid x\rangle$
⟨*proof*⟩

The main lemma: small ε keeps the perturbation PSD.

lemma *small-eps-psd*:
fixes *A* :: *complex Matrix.mat*
assumes *A-c*: $A \in$ *carrier-mat k k* **and** *herm*: *hermitian A*
shows $\exists \varepsilon > 0.\ \forall d ::$ *real*. $|d| \leq \varepsilon \longrightarrow$
psd $(1_m\ k +$ *complex-of-real d* $\cdot_m$ *A*$)$
⟨*proof*⟩

45.9 Symmetric / anti-hermitian split of the coefficient matrix

The *dual* linear combination *kraus-pair-sum n Vs A* $= \sum_{\{i,j\}} A_{\{ij\}}\ V_j{}^\dagger\ V_i$ is an $n \times n$ matrix. The dependence assumption ¬ *mat-lin-indep-real n* (*kraus-products Vs*) says that some real-coefficient A makes this sum vanish.

definition *kraus-pair-row* ::
  nat ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒
    nat ⇒ *complex Matrix.mat*
where
  kraus-pair-row n Vs Amat i ≡
    foldr (λ*j acc*. *Amat* $$ (*i*, *j*) ·$_m$ ((*Vs*!*j*)† ∗ *Vs*!*i*) + *acc*)
      [*0*..<*length Vs*] (*0*$_m$ *n n*)

definition *kraus-pair-sum* ::
  nat ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒
    complex Matrix.mat
where
  kraus-pair-sum n Vs Amat ≡
    foldr (λ*i acc*. *kraus-pair-row n Vs Amat i* + *acc*)
      [*0*..<*length Vs*] (*0*$_m$ *n n*)

lemma *kraus-pair-row-carrier*:
  assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
    and *i-lt*: *i* < *length Vs*
  shows *kraus-pair-row n Vs Amat i* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *kraus-pair-sum-carrier*:
  assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
  shows *kraus-pair-sum n Vs Amat* ∈ *carrier-mat n n*
⟨*proof*⟩

Per-entry index for the dual sum.

lemma *kraus-pair-row-index*:
  assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
    and *i-lt*: *i* < *length Vs*
    and *r-lt*: *r* < *n* **and** *c-lt*: *c* < *n*
  shows *kraus-pair-row n Vs Amat i* $$ (*r*, *c*)
      = (∑ *j* = *0*..<*length Vs*.
          Amat $$ (*i*, *j*) ∗ ((*Vs*!*j*)† ∗ *Vs*!*i*) $$ (*r*, *c*))
⟨*proof*⟩

lemma *kraus-pair-sum-index*:
  assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
    and *r-lt*: *r* < *n* **and** *c-lt*: *c* < *n*
  shows *kraus-pair-sum n Vs Amat* $$ (*r*, *c*)
      = (∑ *i* = *0*..<*length Vs*. ∑ *j* = *0*..<*length Vs*.
          Amat $$ (*i*, *j*) ∗ ((*Vs*!*j*)† ∗ *Vs*!*i*) $$ (*r*, *c*))
⟨*proof*⟩

Linearity in the coefficient matrix.

lemma *kraus-pair-sum-add-A*:
  assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
    and *A1-c*: *A1* ∈ *carrier-mat* (*length Vs*) (*length Vs*)
    and *A2-c*: *A2* ∈ *carrier-mat* (*length Vs*) (*length Vs*)
  shows *kraus-pair-sum n Vs* (*A1* + *A2*)
      = *kraus-pair-sum n Vs A1* + *kraus-pair-sum n Vs A2*
⟨*proof*⟩

lemma *kraus-pair-sum-smult-A*:
  assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
    and *Amat-c*: *Amat* ∈ *carrier-mat* (*length Vs*) (*length Vs*)
  shows *kraus-pair-sum n Vs* (*c* ·$_m$ *Amat*)

$= c \cdot_m$ *kraus-pair-sum n Vs Amat*
⟨*proof*⟩

Dagger identity: taking the dagger of *kraus-pair-sum n Vs A* swaps and conjugates the coefficient indices.

lemma *kraus-pair-sum-dagger*:
assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
and *Amat-c*: *Amat* $\in$ *carrier-mat* (*length Vs*) (*length Vs*)
shows (*kraus-pair-sum n Vs Amat*)†
$=$ *kraus-pair-sum n Vs*
(*Matrix.mat* (*length Vs*) (*length Vs*)
(λ(*i*,*j*). *cnj* (*Amat* $$ (*j*, *i*))))
⟨*proof*⟩

Existence of a hermitian witness: if *A-orig* (with real entries) gives *kraus-pair-sum n Vs A-orig* $=$ *0* and *A-orig* $\neq$ *0*, then either the symmetric or the anti-hermitian part yields a non-zero hermitian matrix with the same property.

lemma *sym-or-antisym-witness*:
fixes *A-orig* :: *complex Matrix.mat*
assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
and *A-c*: *A-orig* $\in$ *carrier-mat* (*length Vs*) (*length Vs*)
and *A-real*: $\bigwedge i\ j.\ i <$ *length Vs* $\Longrightarrow$ $j <$ *length Vs* $\Longrightarrow$
cnj (*A-orig* $$ (*i*, *j*)) = *A-orig* $$ (*i*, *j*)
and *A-zero*: *kraus-pair-sum n Vs A-orig* $= 0_m$ *n n*
and *A-nontriv*: *A-orig* $\neq 0_m$ (*length Vs*) (*length Vs*)
shows $\exists\, H.\ H \in$ *carrier-mat* (*length Vs*) (*length Vs*)
$\wedge$ *hermitian H*
$\wedge$ $H \neq 0_m$ (*length Vs*) (*length Vs*)
$\wedge$ *kraus-pair-sum n Vs H* $= 0_m$ *n n*
⟨*proof*⟩

45.10 Bridge: ¬ *mat-lin-indep-real* gives a real *A-orig* witness

Generic concat-nth for uniform-length rows.

lemma *concat-uniform-length*:
assumes $\forall\, xs \in$ *set L*. *length xs* $=$ *m*
shows *length* (*concat L*) $=$ *length L* $*$ *m*
⟨*proof*⟩

lemma *concat-uniform-nth*:
assumes *uniform*: $\forall\, xs \in$ *set L*. *length xs* $=$ *m*
and *i-lt*: $i <$ *length L*
and *j-lt*: $j < m$
shows *concat L* ! ($i * m + j$) $=$ *L* ! *i* ! *j*
⟨*proof*⟩

kraus-products Vs has length k^2 with *kraus-products Vs* ! (i*$k + j$) $= (V_j)^\dagger\ V_i$.

lemma *length-kraus-products*:
length (*kraus-products Vs*) $=$ *length Vs* $*$ *length Vs*
⟨*proof*⟩

lemma *kraus-products-nth*:
assumes *i-lt*: $i <$ *length Vs* **and** *j-lt*: $j <$ *length Vs*
shows *kraus-products Vs* ! ($i *$ *length Vs* $+ j$) $= (Vs!j)^\dagger *$ *Vs*!*i*
⟨*proof*⟩

Per-entry of the foldr that mat_lin_indep_real builds.

lemma *foldr-smult-nth-index*:
fixes $c :: nat \Rightarrow real$ **and** $Bs :: complex\ Matrix.mat\ list$
assumes *Bs-c*: $\forall B \in set\ Bs.\ B \in carrier\text{-}mat\ n\ n$
and *r-lt*: $r < n$ **and** *c-lt*: $c1 < n$
shows $foldr\ (\lambda k\ acc.\ complex\text{-}of\text{-}real\ (c\ k) \cdot_m (Bs\ !\ k) + acc)$
$[0..<length\ Bs]\ (0_m\ n\ n)\ \$\$\ (r,\ c1)$
$= (\sum k = 0..<length\ Bs.$
$complex\text{-}of\text{-}real\ (c\ k) * (Bs\ !\ k)\ \$\$\ (r,\ c1))$
⟨*proof*⟩

Reindex $\sum_{\{}\kappa = 0..<n{\cdot}k\}$ via $\kappa = i{*}k + j$.

lemma *sum-split-prod-atLeastLessThan*:
fixes $f :: nat \Rightarrow 'a{::}comm\text{-}monoid\text{-}add$
shows $(\sum \kappa = 0..<N * k.\ f\ \kappa) = (\sum i = 0..<N.\ \sum j = 0..<k.\ f\ (i * k + j))$
⟨*proof*⟩

The main bridge: a non-trivial real dependence yields a real, non-zero matrix *A-orig* for which *kraus-pair-sum* vanishes.

lemma *mat-lin-indep-real-neg-witness*:
assumes *Vs-c*: $\forall V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *npos*: $0 < n$
and *dep*: $\neg\ mat\text{-}lin\text{-}indep\text{-}real\ n\ (kraus\text{-}products\ Vs)$
shows $\exists A\text{-}orig :: complex\ Matrix.mat.$
$A\text{-}orig \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
$\wedge\ (\forall i\ j.\ i < length\ Vs \longrightarrow j < length\ Vs \longrightarrow$
$cnj\ (A\text{-}orig\ \$\$\ (i,\ j)) = A\text{-}orig\ \$\$\ (i,\ j))$
$\wedge\ A\text{-}orig \neq 0_m\ (length\ Vs)\ (length\ Vs)$
$\wedge\ kraus\text{-}pair\text{-}sum\ n\ Vs\ A\text{-}orig = 0_m\ n\ n$
⟨*proof*⟩

The same with complex coefficients: a non-trivial complex relation among the products $V_j^\dagger V_i$ gives a non-zero complex coefficient matrix with vanishing pair sum, and then (closure of the family under adjoints) a non-zero *Hermitian* one.

lemma *mat-lin-indep-complex-neg-witness*:
assumes *Vs-c*: $\forall V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *npos*: $0 < n$
and *dep*: $\neg\ mat\text{-}lin\text{-}indep\text{-}complex\ n\ (kraus\text{-}products\ Vs)$
shows $\exists A\text{-}orig :: complex\ Matrix.mat.$
$A\text{-}orig \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
$\wedge\ A\text{-}orig \neq 0_m\ (length\ Vs)\ (length\ Vs)$
$\wedge\ kraus\text{-}pair\text{-}sum\ n\ Vs\ A\text{-}orig = 0_m\ n\ n$
⟨*proof*⟩

lemma *herm-witness-of-complex*:
fixes *A-orig* :: *complex Matrix.mat*
assumes *Vs-c*: $\forall V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *A-c*: $A\text{-}orig \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
and *A-zero*: $kraus\text{-}pair\text{-}sum\ n\ Vs\ A\text{-}orig = 0_m\ n\ n$
and *A-nontriv*: $A\text{-}orig \neq 0_m\ (length\ Vs)\ (length\ Vs)$
shows $\exists H.\ H \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
$\wedge\ hermitian\ H$
$\wedge\ H \neq 0_m\ (length\ Vs)\ (length\ Vs)$
$\wedge\ kraus\text{-}pair\text{-}sum\ n\ Vs\ H = 0_m\ n\ n$
⟨*proof*⟩

45.11 Auxiliary lemmas for the assembly

Scalar-multiplication linearity of *kraus-quad-map* in T.

lemma *kraus-quad-map-smult-Tm*:
  assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
    and *rho-c*: *rho* $\in$ *carrier-mat n n*
    and *Tm-c*: *Tm* $\in$ *carrier-mat* (*length Vs*) (*length Vs*)
  shows *kraus-quad-map d Vs* (*c* $\cdot_m$ *Tm*) *rho*
      = *c* $\cdot_m$ *kraus-quad-map d Vs Tm rho*
⟨*proof*⟩

The identity-coefficient case: *kraus-quad-map* with $1_m\ k$ reduces to the ordinary *kraus-map-dim*.

lemma *kraus-quad-map-one-eq-kraus-map-dim*:
  assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
    and *rho-c*: *rho* $\in$ *carrier-mat n n*
  shows *kraus-quad-map d Vs* (1_m (*length Vs*)) *rho* = *kraus-map-dim d Vs rho*
⟨*proof*⟩

The pair-sum at the identity coefficient is exactly *sum-wdag-w*.

lemma *kraus-pair-sum-one-eq-sum-wdag-w*:
  assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
  shows *kraus-pair-sum n Vs* (1_m (*length Vs*)) = *sum-wdag-w n Vs*
⟨*proof*⟩

45.12 Minimality and the distinctness lemma (Lemma B)

Linear independence of a complex-matrix list (over $\mathbb{C}$). This is the *minimality* assumption needed to make the reverse direction of Choi's Theorem 3 go through: without it, perturbations of a non-minimal Kraus list can produce the same map.

definition *lin-indep-complex-mat* ::
  nat ⇒ *nat* ⇒ *complex Matrix.mat list* ⇒ *bool*
where
  lin-indep-complex-mat d n Ws ≡
    (∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*) ∧
    (∀ *c* :: *nat* ⇒ *complex*.
      foldr (λ*k acc*. *c k* $\cdot_m$ (*Ws*!*k*) + *acc*) [*0*..<*length Ws*] (*0*$_m$ *d n*)
        = *0*$_m$ *d n*
      ⟶ (∀ *k* < *length Ws*. *c k* = *0*))

Lemma B: under minimality of *Vs*, a non-zero coefficient matrix gives a non-vanishing quadratic Kraus map for some ϱ.

The witness ϱ := *mat-unit p q n n* (a single 1 at (p,q)) makes $(V_i\ \varrho\ V_j{}^\dagger)$ \$\$ $(r,c) = V_i$ \$\$ $(r,p)\ |V_j$ \$\$ $(c,q)|$, so universal vanishing of *kraus-quad-map* on all (p,q,r,c) combined with linear independence of *Vs* (applied twice, separated by conjugation) forces $A = 0$.

A convenient form of minimality: if the per-entry sum vanishes for all (r,p), then all coefficients are zero.

lemma *lin-indep-zero-combo*:
  assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
    and *minimal*: *lin-indep-complex-mat d n Vs*
    and *coeff-kills*: $\bigwedge r\ p.\ r < d \Longrightarrow p < n \Longrightarrow$
                  ($\sum i = 0..<$*length Vs*. *c i* ∗ (*Vs*!*i*) \$\$ (*r*, *p*)) = *0*
  shows ∀ *i* < *length Vs*. *c i* = *0*
⟨*proof*⟩

The per-entry computation for the rank-1 ϱ = *mat-unit p q n n*.

lemma *kraus-quad-map-mat-unit-index*:
  assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
    and *i-lt*: $i <$ *length Vs* **and** *j-lt*: $j <$ *length Vs*
    and *p-lt*: $p < n$ **and** *q-lt*: $q < n$
    and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
  shows (*Vs*!i $*$ *mat-unit p q n n* $*$ ((*Vs*!j)†)) $$ (r, c)
      = (*Vs*!i) $$ (r, p) $*$ *cnj* ((*Vs*!j) $$ (c, q))
⟨*proof*⟩

Lemma B. Under minimality, a non-zero coefficient matrix gives a non-vanishing quadratic Kraus map for some rank-1 ϱ.

lemma *kraus-quad-map-nonzero-witness*:
  assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
    and *minimal*: *lin-indep-complex-mat d n Vs*
    and *A-c*: $A \in$ *carrier-mat* (*length Vs*) (*length Vs*)
    and *A-nz*: $A \neq 0_m$ (*length Vs*) (*length Vs*)
    and *npos*: $0 < n$ **and** *dpos*: $0 < d$
  shows $\exists\, \rho \in$ *carrier-mat n n*. *kraus-quad-map d Vs A rho* $\neq 0_m$ *d d*
⟨*proof*⟩

Trace-preservation lemma for *kraus-quad-map*.

The proof shows *Tr* (*kraus-quad-map d Vs T* ϱ) = *Tr* (*kraus-pair-sum n Vs T* $*$ ϱ) by per-entry sum manipulation and one application of $[\![$*dim-col ?A* = *dim-row ?B*; *dim-row ?A* = *dim-col ?B*$]\!] \Longrightarrow$ *Tr* (*?A* $*$ *?B*) = *Tr* (*?B* $*$ *?A*) to flip $V_i\ \varrho\ {V_j}^\dagger$ into ${V_j}^\dagger\ V_i\ \varrho$; the TP conclusion follows when the pair-sum equals 1_m *n*.

lemma *Tr-kraus-quad-map-eq*:
  assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
    and *Tmat-c*: *Tmat* $\in$ *carrier-mat* (*length Vs*) (*length Vs*)
    and *rho-c*: *rho* $\in$ *carrier-mat n n*
  shows *Tr* (*kraus-quad-map d Vs Tmat rho*)
      = *Tr* (*kraus-pair-sum n Vs Tmat* $*$ *rho*)
⟨*proof*⟩

lemma *kraus-quad-map-is-TP*:
  assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
    and *Tmat-c*: *Tmat* $\in$ *carrier-mat* (*length Vs*) (*length Vs*)
    and *Tmat-pair-sum*: *kraus-pair-sum n Vs Tmat* $= 1_m$ *n*
    and *npos*: $0 < n$
  shows *trace-preserving n* (*kraus-quad-map d Vs Tmat*)
  ⟨*proof*⟩

45.13 CPTP-Kraus representation and extreme CPTP maps

A *cptp-kraus-rep n d* Φ *Vs* says *Vs* is a Kraus operator list (trace-preserving) and Φ equals the corresponding *kraus-map-dim* on every $X \in$ *carrier-mat n n*.

definition *cptp-kraus-rep* ::
  nat $\Rightarrow$ *nat* $\Rightarrow$ (*complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*) $\Rightarrow$
    complex Matrix.mat list $\Rightarrow$ *bool*
where
  cptp-kraus-rep n d Φ *Vs* $\equiv$
    kraus-operators n d Vs $\wedge$
    ($\forall\, X$. $X \in$ *carrier-mat n n* $\longrightarrow$ Φ X = *kraus-map-dim d Vs X*)

Extreme points of $CPTP(n,d)$: analog of *extreme-UCP*.

definition *extreme-CPTP* ::
  nat $\Rightarrow$ *nat* $\Rightarrow$ (*complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*) $\Rightarrow$ *bool*

where
extreme-CPTP n d Φ ≡
Φ ∈ CPTP n d ∧
¬ (∃ Φ₁ ∈ CPTP n d. ∃ Φ₂ ∈ CPTP n d.
Φ₁ ≠ Φ ∧ Φ ≠ Φ₂ ∧
(∀ X. X ∈ carrier-mat n n
⟶ Φ X = (1/2 :: complex) ·ₘ Φ₁ X
+ (1/2 :: complex) ·ₘ Φ₂ X))

45.14 Vectorisation of Kraus operators

The *vectorisation* of a $d \times n$ matrix V is the length-$n{*}d$ vector with r-th entry $V(r \bmod d,\ r \operatorname{div} d)$. This is the column-stacking isomorphism $M_{\{d \times n\}} \cong \mathbb{C}^{\{nd\}}$ used in the Choi/Stinespring picture.

definition *vec-of-kraus :: nat ⇒ nat ⇒ complex Matrix.mat ⇒ complex Matrix.vec*
where
*vec-of-kraus d n V = Matrix.vec (n*d) (λr. V $$ (r mod d, r div d))*

lemma *vec-of-kraus-dim* [*simp*]: *dim-vec (vec-of-kraus d n V) = n*d*
⟨*proof*⟩

lemma *vec-of-kraus-index*:
assumes *r-lt*: *r < n*d*
shows *vec-of-kraus d n V $ r = V $$ (r mod d, r div d)*
⟨*proof*⟩

For $i < d$ and $a < n$, indexing the vectorisation at the *combined* index $a{*}d + i$ recovers the matrix entry $V(i,\ a)$.

lemma *vec-of-kraus-combined-index*:
assumes *V-c*: *V ∈ carrier-mat d n*
and *a-lt*: *a < n* **and** *i-lt*: *i < d* **and** *dpos*: *0 < d*
shows *vec-of-kraus d n V $ (a*d + i) = V $$ (i, a)*
⟨*proof*⟩

45.15 Choi matrix of a Kraus map

Key structural identity: the Choi matrix of a Kraus map $\Phi(X) = \sum_i V_i\ X\ {V_i}^\dagger$ equals the outer sum $\sum_i v_i\ {v_i}^\dagger$ where $v_i =$ *vec-of-kraus*(V_i). In particular, its range is the linear span of the vectorisations of *Vs*.

lemma *kraus-map-dim-mat-unit-entry*:
assumes *Vs-c*: *∀ V ∈ set Vs. V ∈ carrier-mat d n*
and *a-lt*: *a < n* **and** *b-lt*: *b < n*
and *i-lt*: *i < d* **and** *j-lt*: *j < d*
shows *(kraus-map-dim d Vs (mat-unit a b n n)) $$ (i, j)*
*= (∑ k<length Vs. Vs!k $$ (i, a) * cnj (Vs!k $$ (j, b)))*
⟨*proof*⟩

lemma *choi-matrix-kraus-eq*:
assumes *Vs-c*: *∀ V ∈ set Vs. V ∈ carrier-mat d n*
and *npos*: *0 < n* **and** *dpos*: *0 < d*
shows *choi-matrix n d (kraus-map-dim d Vs)*
*= outer-sum (n*d) (map (vec-of-kraus d n) Vs)*
⟨*proof*⟩

45.16 Choi matrix is linear in the map

The Choi matrix is a linear function of the underlying map (evaluated on the basis *mat-unit*'s). In particular, $P_{\{}\Phi - \Psi\} = P_\Phi - P_\Psi$.

lemma *choi-matrix-carrier* [*simp*]:
choi-matrix n d $\Phi \in$ *carrier-mat* (*n*∗*d*) (*n*∗*d*)
⟨*proof*⟩

lemma *choi-matrix-diff*:
fixes *Phi Psi* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *Phi-range*: $\bigwedge X.$ *X* ∈ *carrier-mat n n* ⟹ *Phi X* ∈ *carrier-mat d d*
and *Psi-range*: $\bigwedge X.$ *X* ∈ *carrier-mat n n* ⟹ *Psi X* ∈ *carrier-mat d d*
and *npos*: *0* < *n* **and** *dpos*: *0* < *d*
shows *choi-matrix n d* (λ*X*. *Phi X* − *Psi X*)
= *choi-matrix n d Phi* − *choi-matrix n d Psi*
⟨*proof*⟩

The Choi matrix agrees on maps that agree on the carrier.

lemma *choi-matrix-cong*:
assumes *agree*: $\bigwedge X.$ *X* ∈ *carrier-mat n n* ⟹ *Phi X* = *Psi X*
shows *choi-matrix n d Phi* = *choi-matrix n d Psi*
⟨*proof*⟩

45.17 The Choi matrix of a *kraus-quad-map*

The Choi matrix of $X \mapsto \sum_{\{} ij\}$ $T_{\{} ij\}$ V_i *X* $V_j{}^\dagger$ equals *V T* $V^\dagger$ where *V* = *mat-of-cols* (*n*∗*d*) (*map vec-of-kraus Vs*).

This is the bridge from the classical dominated-PSD parametrisation back to a Kraus quadratic form.

definition *kraus-vec-mat* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat*
where *kraus-vec-mat d n Vs* = *mat-of-cols* (*n*∗*d*) (*map* (*vec-of-kraus d n*) *Vs*)

lemma *kraus-vec-mat-carrier* [*simp*]:
kraus-vec-mat d n Vs ∈ *carrier-mat* (*n*∗*d*) (*length Vs*)
⟨*proof*⟩

lemma *kraus-vec-mat-index*:
assumes *r-lt*: *r* < *n*∗*d* **and** *k-lt*: *k* < *length Vs*
shows *kraus-vec-mat d n Vs* $$ (*r*, *k*) = *vec-of-kraus d n* (*Vs*!*k*) $ *r*
⟨*proof*⟩

lemma *kraus-vec-mat-dagger-index*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *k-lt*: *k* < *length Vs* **and** *c-lt*: *c* < *n*∗*d*
shows (*kraus-vec-mat d n Vs*)† $$ (*k*, *c*) = *cnj* (*vec-of-kraus d n* (*Vs*!*k*) $ *c*)
⟨*proof*⟩

The Choi matrix of *kraus-quad-map d Vs T* is the sandwich *V T* $V^\dagger$, where *V* = *kraus-vec-mat d n Vs*.

lemma *choi-matrix-kraus-quad-eq*:
fixes *Tmat* :: *complex Matrix.mat*
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *T-c*: *Tmat* ∈ *carrier-mat* (*length Vs*) (*length Vs*)
and *npos*: *0* < *n* **and** *dpos*: *0* < *d*
shows *choi-matrix n d* (*kraus-quad-map d Vs Tmat*)
= (*kraus-vec-mat d n Vs*) ∗ *Tmat* ∗ ((*kraus-vec-mat d n Vs*)†)

⟨*proof*⟩

45.18 CP Radon–Nikodym — the dominated-PSD kernel

Dominated PSD parametrisation. This is the genuinely classical fact behind CP Radon–Nikodym: any PSD matrix B dominated by the outer sum $\sum\ v_i\ {v_i}^\dagger$ can be sandwiched as $V\ T\ V^\dagger$ for some PSD T, where $V = \textit{mat-of-cols}\ k\ \textit{vs}$.

Geometric content: $\textit{range}(B) \subseteq \textit{range}(\textit{outer-sum}\ k\ \textit{vs})$ (equivalent to B being Loewner-below outer_sum k vs after PSD), and that range is spanned by the columns of V. Pulling B back through that span gives T.

Proved below as *dominated-outer-sum-param* through the Moore–Penrose pseudoinverse of the outer sum (no axioms).

Loewner–kernel inclusion. If $A \geq B \geq 0$ in Loewner order, then every vector annihilated by A is also annihilated by B. Used to derive the range inclusion $\textit{range}(B) \subseteq \textit{range}(A)$.

Proof structure:

1. $\langle x \mid A\ x\rangle = 0$ from $A\ x = 0$.
2. Linearity of the inner product: $\langle x \mid A\ x\rangle = \langle x \mid B\ x\rangle + \langle x \mid (A-B)\ x\rangle$.
3. Both summands have non-negative real part (PSD); the sum is 0, so each is 0.
4. Hermiticity of B makes $\langle x \mid B\ x\rangle$ real, so it equals 0.
5. Decompose $B = N\ N^\dagger$ via $[\![?P \in \textit{carrier-mat}\ ?n\ ?n;\ ?P \geq_M 0]\!] \Longrightarrow \exists\, \textit{vs}.\ \textit{length}\ \textit{vs} = ?n \wedge (\forall\, v{\in}\textit{set}\ \textit{vs}.\ \textit{dim-vec}\ v = ?n) \wedge \textit{outer-sum}\ ?n\ \textit{vs} = ?P$, then $\langle x \mid B\ x\rangle = \langle N^\dagger\ x \mid N^\dagger\ x\rangle = 0$ forces $N^\dagger\ x = 0$, hence $B\ x = N\ (N^\dagger\ x) = 0$.

lemma *loewner-ker-subset*:
  fixes $A\ B$ :: *complex Matrix.mat* **and** x :: *complex Matrix.vec*
  assumes *A-c*: $A \in \textit{carrier-mat}\ k\ k$
    and *B-c*: $B \in \textit{carrier-mat}\ k\ k$
    and *B-psd*: *psd* B
    and *diff-psd*: *psd* $(A - B)$
    and *x-dim*: $\textit{dim-vec}\ x = k$
    and *Ax-zero*: $A *_v x = 0_v\ k$
  shows $B *_v x = 0_v\ k$
⟨*proof*⟩

PSD congruence. If P is PSD and M is any matrix of compatible shape, then $M^\dagger\ P\ M$ is PSD. This is the standard "sandwich" fact, generalising $?M \in \textit{carrier-mat}\ ?n\ ?m \Longrightarrow ?M^\dagger * ?M \geq_M 0$ (the case $P = 1_m$).

lemma *psd-conjugate*:
  fixes P :: *complex Matrix.mat* **and** M :: *complex Matrix.mat*
  assumes *P-c*: $P \in \textit{carrier-mat}\ k\ k$
    and *P-psd*: *psd* P
    and *M-c*: $M \in \textit{carrier-mat}\ k\ n$
  shows *psd* $(M^\dagger * P * M)$
⟨*proof*⟩

45.18.1 Diagonal pseudo-inverse list and its identity

Pseudo-inverse of a real list (interpreted as a diagonal). Zero entries are kept at zero; non-zero entries are reciprocated. This is the diagonal Moore–Penrose pseudoinverse.

definition *pinv-list* :: *real list* ⇒ *real list* **where**
pinv-list D = *map* (λ*d*. *if d* = *0 then 0 else 1*/*d*) *D*

lemma *pinv-list-length* [*simp*]: *length* (*pinv-list D*) = *length D*
⟨*proof*⟩

lemma *pinv-list-index*:
assumes *i* < *length D*
shows *pinv-list D* ! *i* = (*if D* ! *i* = *0 then 0 else 1* / (*D* ! *i*))
⟨*proof*⟩

lemma *pinv-list-nonneg*:
assumes ⋀*i*. *i* < *length D* ⟹ *0* ≤ *D* ! *i*
shows ⋀*i*. *i* < *length D* ⟹ *0* ≤ *pinv-list D* ! *i*
⟨*proof*⟩

The central identity: $D \cdot D^{+} \cdot D = D$ entrywise on the diagonal. For $D_i = 0$ the left-hand side is $0 \cdot 0 \cdot 0 = 0 = D_i$; for $D_i \neq 0$ it is $D_i \cdot (1/D_i) \cdot D_i = D_i$. This is the core diagonal pseudoinverse identity that lifts to operator level through unitary conjugation.

lemma *diag-pinv-diag-eq*:
fixes *D* :: *real list*
shows *diag-mat-of-real D*
∗ *diag-mat-of-real* (*pinv-list D*)
∗ *diag-mat-of-real D*
= *diag-mat-of-real D*
⟨*proof*⟩

Pseudoinverse identity at operator level. Given a hermitian spectral decomposition $A = U\ D\ U^{\dagger}$, define the Moore–Penrose pseudoinverse via $A^{+} = U\ D^{+}\ U^{\dagger}$. Then $A\ A^{+}\ A = A$. This is the unitary-conjugation lift of *diag-mat-of-real ?D* ∗ *diag-mat-of-real* (*pinv-list ?D*) ∗ *diag-mat-of-real ?D* = *diag-mat-of-real ?D*.

lemma *A-Aplus-A-eq*:
fixes *U* :: *complex Matrix.mat* **and** *D* :: *real list*
assumes *Uc*: *U* ∈ *carrier-mat k k*
and *UU*: *unitary U*
and *Dlen*: *length D* = *k*
shows (*U* ∗ *diag-mat-of-real D* ∗ (*U*†))
∗ (*U* ∗ *diag-mat-of-real* (*pinv-list D*) ∗ (*U*†))
∗ (*U* ∗ *diag-mat-of-real D* ∗ (*U*†))
= *U* ∗ *diag-mat-of-real D* ∗ (*U*†)
⟨*proof*⟩

Two square matrices agreeing on every vector are equal.

lemma *mat-eq-of-mult-vec-eq*:
fixes *M N* :: *complex Matrix.mat*
assumes *M-c*: *M* ∈ *carrier-mat k k*
and *N-c*: *N* ∈ *carrier-mat k k*
and *eq*: ⋀*x*. *x* ∈ *carrier-vec k* ⟹ *M* ∗*v x* = *N* ∗*v x*
shows *M* = *N*
⟨*proof*⟩

Loewner-implied pseudoinverse identity, proved at operator level: with $Q = A^{+}\ A$ one has $A\ Q = A$, so $x - Q\ x \in ker\ A \subseteq ker\ B$, whence $B = B\ Q$; taking adjoints gives $B = A\ A^{+}\ B$.

lemma *A-Aplus-B-eq-B*:
fixes *U* :: *complex Matrix.mat* **and** *D* :: *real list* **and** *B* :: *complex Matrix.mat*
assumes *Uc*: *U* ∈ *carrier-mat k k*

and *UU*: *unitary U*
and *Dlen*: *length D = k*
and *Dnn*: $\bigwedge i.\ i < k \Longrightarrow 0 \leq D\ !\ i$
and *B-c*: *B ∈ carrier-mat k k*
and *B-psd*: *psd B*
and *dom-psd*: *psd ((U * diag-mat-of-real D * (U†)) − B)*
shows *(U * diag-mat-of-real D * (U†))*
** (U * diag-mat-of-real (pinv-list D) * (U†)) * B = B*
⟨*proof*⟩

Spectral decomposition of a psd matrix with non-negative diagonal.

Dominated psd matrices are parametrised through the dominating frame: if $0 \leq B \leq V\ V^\dagger$ then $B = V\ T\ V^\dagger$ with $T = V^\dagger\ A^+\ B\ A^+\ V \geq 0$, $A = V\ V^\dagger$, A^+ the Moore–Penrose pseudoinverse.

lemma *dominated-outer-sum-param*:
fixes *vs* :: *complex Matrix.vec list*
and *B* :: *complex Matrix.mat*
assumes *vs-dim*: ∀ *v* ∈ *set vs. dim-vec v = k*
and *k-pos*: *0 < k*
and *B-c*: *B ∈ carrier-mat k k*
and *B-psd*: *psd B*
and *B-dom*: *psd (outer-sum k vs − B)*
shows ∃ *T. T ∈ carrier-mat (length vs) (length vs) ∧ psd T ∧*
*B = (mat-of-cols k vs) * T * ((mat-of-cols k vs)†)*
⟨*proof*⟩

45.19 Linearity of *kraus-quad-map* in its matrix argument

Per-entry computation shows *kraus-quad-map d Vs Tm* is linear in its third (matrix) argument: $\varrho \mapsto \sum_{\{ij\}}\ T_{\{ij\}}\ V_i\ \varrho\ {V_j}^\dagger$.

lemma *kraus-quad-map-add-rho*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs. V ∈ carrier-mat d n*
and *rho1-c*: *rho1 ∈ carrier-mat n n*
and *rho2-c*: *rho2 ∈ carrier-mat n n*
shows *kraus-quad-map d Vs Tm (rho1 + rho2)*
= kraus-quad-map d Vs Tm rho1 + kraus-quad-map d Vs Tm rho2
⟨*proof*⟩

lemma *kraus-quad-map-smult-rho*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs. V ∈ carrier-mat d n*
and *rho-c*: *rho ∈ carrier-mat n n*
shows *kraus-quad-map d Vs Tm (c ·m rho)*
= c ·m kraus-quad-map d Vs Tm rho
⟨*proof*⟩

45.20 Choi matrix determines a linear CP map on carrier

If two maps that are linear on carrier have the same Choi matrix and the same target carrier, they agree pointwise on carrier.

lemma *map-eq-from-choi-when-linear*:
fixes *Phi Psi* :: *complex Matrix.mat ⇒ complex Matrix.mat*
assumes *Phi-add*: $\bigwedge X\ Y.\ X \in$ *carrier-mat n n* $\Longrightarrow Y \in$ *carrier-mat n n*
$\Longrightarrow$ *Phi (X + Y) = Phi X + Phi Y*
and *Phi-smult*: $\bigwedge c\ X.\ X \in$ *carrier-mat n n* $\Longrightarrow$ *Phi (c ·m X) = c ·m Phi X*
and *Phi-car*: $\bigwedge X.\ X \in$ *carrier-mat n n* $\Longrightarrow$ *Phi X ∈ carrier-mat d d*
and *Psi-add*: $\bigwedge X\ Y.\ X \in$ *carrier-mat n n* $\Longrightarrow Y \in$ *carrier-mat n n*

$\Longrightarrow$ *Psi* (*X* + *Y*) = *Psi X* + *Psi Y*
and *Psi-smult*: $\bigwedge$*c X*. *X* ∈ *carrier-mat n n* $\Longrightarrow$ *Psi* (*c* ·$_m$ *X*) = *c* ·$_m$ *Psi X*
and *Psi-car*: $\bigwedge$*X*. *X* ∈ *carrier-mat n n* $\Longrightarrow$ *Psi X* ∈ *carrier-mat d d*
and *choi-eq*: *choi-matrix n d Phi* = *choi-matrix n d Psi*
and *npos*: *n* > *0* **and** *dpos*: *d* > *0*
and *A-c*: *A* ∈ *carrier-mat n n*
shows *Phi A* = *Psi A*
⟨*proof*⟩

45.21 CP Radon–Nikodym

CP Radon–Nikodym (Choi level). At the Choi-matrix level, the parametrisation falls out from ⟦∀ *v*∈*set ?vs*. *dim-vec v* = *?k*; *0* < *?k*; *?B* ∈ *carrier-mat ?k ?k*; *?B* ≥$_M$ *0*; *outer-sum ?k ?vs* − *?B* ≥$_M$ *0*⟧ $\Longrightarrow$ ∃ *T*. *T* ∈ *carrier-mat* (*length ?vs*) (*length ?vs*) ∧ *T* ≥$_M$ *0* ∧ *?B* = *mat-of-cols ?k ?vs* ∗ *T* ∗ *mat-of-cols ?k ?vs*† composed with the structural identities ⟦∀ *V*∈*set ?Vs*. *V* ∈ *carrier-mat ?d ?n*; *0* < *?n*; *0* < *?d*⟧ $\Longrightarrow$ P_Φ *?n ?d* (*kraus-map-dim ?d ?Vs*) = *outer-sum* (*?n* ∗ *?d*) (*map* (*vec-of-kraus ?d ?n*) *?Vs*) and ⟦∀ *V*∈*set ?Vs*. *V* ∈ *carrier-mat ?d ?n*; *?Tmat* ∈ *carrier-mat* (*length ?Vs*) (*length ?Vs*); *0* < *?n*; *0* < *?d*⟧ $\Longrightarrow$ P_Φ *?n ?d* (*kraus-quad-map ?d ?Vs ?Tmat*) = *kraus-vec-mat ?d ?n ?Vs* ∗ *?Tmat* ∗ *kraus-vec-mat ?d ?n ?Vs*†. This proves what is needed at the matrix level; passing to the ∀ *X*-quantified form requires linearity of Ψ, which is the project's *completely-positive* does *not* include (see the lemma after).

lemma *cp-radon-nikodym-choi*:
fixes *Phi Psi* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list*
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *Phi-kraus*: $\bigwedge$*X*. *X* ∈ *carrier-mat n n* $\Longrightarrow$ *Phi X* = *kraus-map-dim d Vs X*
and *Psi-cp*: *completely-positive n d Psi*
and *Psi-range*: $\bigwedge$*X*. *X* ∈ *carrier-mat n n* $\Longrightarrow$ *Psi X* ∈ *carrier-mat d d*
and *diff-cp*: *completely-positive n d* (λ*X*. *Phi X* − *Psi X*)
and *npos*: *0* < *n*
and *dpos*: *0* < *d*
shows ∃ *T*. *T* ∈ *carrier-mat* (*length Vs*) (*length Vs*) ∧ *psd T* ∧
choi-matrix n d Psi = *choi-matrix n d* (*kraus-quad-map d Vs T*)
⟨*proof*⟩

CP Radon–Nikodym (operator level). With Ψ linear on *carrier-mat n n*, the Choi-level parametrisation gives the pointwise equality $\Psi(X) = \sum\ T_{\{ij\}}\ V_i\ X\ {V_j}^\dagger$.
Linearity of Ψ is supplied as an explicit hypothesis: the project's *completely-positive* predicate does *not* imply linearity (it constrains only positivity under amplification). This corresponds to the classical fact that "CP map" standardly includes linearity; we make it explicit so the consumer can supply it.

Operator-level CP Radon–Nikodym, with linearity assumption. This version is fully discharged via ⟦∀ *V*∈*set ?Vs*. *V* ∈ *carrier-mat ?d ?n*; $\bigwedge$*X*. *X* ∈ *carrier-mat ?n ?n* $\Longrightarrow$ *?Phi X* = *kraus-map-dim ?d ?Vs X*; *completely-positive ?n ?d ?Psi*; $\bigwedge$*X*. *X* ∈ *carrier-mat ?n ?n* $\Longrightarrow$ *?Psi X* ∈ *carrier-mat ?d ?d*; *completely-positive ?n ?d* (λ*X*. *?Phi X* − *?Psi X*); *0* < *?n*; *0* < *?d*⟧ $\Longrightarrow$ ∃ *T*. *T* ∈ *carrier-mat* (*length ?Vs*) (*length ?Vs*) ∧ *T* ≥$_M$ *0* ∧ P_Φ *?n ?d ?Psi* = P_Φ *?n ?d* (*kraus-quad-map ?d ?Vs T*) + ⟦$\bigwedge$*X Y*. ⟦*X* ∈ *carrier-mat ?n ?n*; *Y* ∈ *carrier-mat ?n ?n*⟧ $\Longrightarrow$ *?Phi* (*X* + *Y*) = *?Phi X* + *?Phi Y*; $\bigwedge$*c X*. *X* ∈ *carrier-mat ?n ?n* $\Longrightarrow$ *?Phi* (*c* ·$_m$ *X*) = *c* ·$_m$ *?Phi X*; $\bigwedge$*X*. *X* ∈ *carrier-mat ?n ?n* $\Longrightarrow$ *?Phi X* ∈ *carrier-mat ?d ?d*; $\bigwedge$*X Y*. ⟦*X* ∈ *carrier-mat ?n ?n*; *Y* ∈ *carrier-mat ?n ?n*⟧ $\Longrightarrow$ *?Psi* (*X* + *Y*) = *?Psi X* + *?Psi Y*; $\bigwedge$*c X*. *X* ∈ *carrier-mat ?n ?n* $\Longrightarrow$ *?Psi* (*c* ·$_m$ *X*) = *c* ·$_m$ *?Psi X*; $\bigwedge$*X*. *X* ∈ *carrier-mat ?n ?n* $\Longrightarrow$ *?Psi X* ∈ *carrier-mat ?d ?d*; P_Φ *?n ?d ?Phi* = P_Φ *?n ?d ?Psi*; *0* < *?n*; *0* < *?d*; *?A* ∈ *carrier-mat ?n ?n*⟧ $\Longrightarrow$ *?Phi ?A* = *?Psi ?A*, which rest on ⟦∀ *v*∈*set ?vs*. *dim-vec v* = *?k*; *0* < *?k*; *?B* ∈

carrier-mat ?k ?k; *?B* $\geq_M$ *0*; *outer-sum ?k ?vs − ?B* $\geq_M$ *0*$\rrbracket \Longrightarrow \exists\, T.\ T \in$ *carrier-mat* (*length ?vs*) (*length ?vs*) $\wedge$ *T* $\geq_M$ *0* $\wedge$ *?B = mat-of-cols ?k ?vs* $*$ *T* $*$ *mat-of-cols ?k ?vs*†. It requires *Psi* to be linear on *carrier-mat n n*, which is not implied by the project's *completely-positive* predicate.

lemma *cp-radon-nikodym-linear*:
  fixes *Phi Psi* :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
    and *Vs* :: *complex Matrix.mat list*
  assumes *Vs-c*: $\forall$ *V* $\in$ *set Vs. V* $\in$ *carrier-mat d n*
      and *Phi-kraus*: $\bigwedge$*X. X* $\in$ *carrier-mat n n* $\Longrightarrow$ *Phi X = kraus-map-dim d Vs X*
      and *Psi-cp*: *completely-positive n d Psi*
      and *Psi-range*: $\bigwedge$*X. X* $\in$ *carrier-mat n n* $\Longrightarrow$ *Psi X* $\in$ *carrier-mat d d*
      and *Psi-add*: $\bigwedge$*X Y. X* $\in$ *carrier-mat n n* $\Longrightarrow$ *Y* $\in$ *carrier-mat n n*
                  $\Longrightarrow$ *Psi* (*X* + *Y*) = *Psi X* + *Psi Y*
      and *Psi-smult*: $\bigwedge$*c X. X* $\in$ *carrier-mat n n* $\Longrightarrow$ *Psi* (*c* $\cdot_m$ *X*) = *c* $\cdot_m$ *Psi X*
      and *diff-cp*: *completely-positive n d* (λ*X. Phi X* − *Psi X*)
      and *npos*: *0* < *n*
      and *dpos*: *0* < *d*
  shows $\exists$ *T. T* $\in$ *carrier-mat* (*length Vs*) (*length Vs*) $\wedge$ *psd T* $\wedge$
          ($\forall$ *X. X* $\in$ *carrier-mat n n* $\longrightarrow$ *Psi X = kraus-quad-map d Vs T X*)
⟨*proof*⟩

The use site *choi-theorem-3-cptp-forward-complex* obtains linearity of its Ψ through the strengthened *cptp-pair-mid* predicate (which carries *lin-on-carrier* for both midpoint witnesses) and dispatches to $[\![\forall$ *V*$\in$*set ?Vs. V* $\in$ *carrier-mat ?d ?n*; $\bigwedge$*X. X* $\in$ *carrier-mat ?n ?n* $\Longrightarrow$ *?Phi X = kraus-map-dim ?d ?Vs X*; *completely-positive ?n ?d ?Psi*; $\bigwedge$*X. X* $\in$ *carrier-mat ?n ?n* $\Longrightarrow$ *?Psi X* $\in$ *carrier-mat ?d ?d*; $\bigwedge$*X Y.* $[\![$*X* $\in$ *carrier-mat ?n ?n*; *Y* $\in$ *carrier-mat ?n ?n*$]\!] \Longrightarrow$ *?Psi* (*X* + *Y*) = *?Psi X* + *?Psi Y*; $\bigwedge$*c X. X* $\in$ *carrier-mat ?n ?n* $\Longrightarrow$ *?Psi* (*c* $\cdot_m$ *X*) = *c* $\cdot_m$ *?Psi X*; *completely-positive ?n ?d* (λ*X. ?Phi X* − *?Psi X*); *0* < *?n*; *0* < *?d*$]\!] \Longrightarrow \exists\, T.\ T \in$ *carrier-mat* (*length ?Vs*) (*length ?Vs*) $\wedge$ *T* $\geq_M$ *0* $\wedge$ ($\forall$ *X. X* $\in$ *carrier-mat ?n ?n* $\longrightarrow$ *?Psi X = kraus-quad-map ?d ?Vs T X*). No operator-level *cp-radon-nikodym* without linearity is needed any more.

45.22 Choi's Theorem 3 — reverse direction

Reverse direction of Choi's Theorem 3 (CPTP form). Under minimality of the Kraus list *Vs*, the non-linear-independence of the products $\{V_j^\dagger\ V_i\}$ forces Φ to fail to be an extreme point of *CPTP n d*.

The witness uses the explicit perturbation $\Phi_\pm(\varrho) := \sum(I_{\{ij\}} \pm \varepsilon\ H_{\{ij\}})\ V_i\ \varrho\ V_j^\dagger$ for a hermitian *H* (from $[\![\forall$ *V*$\in$*set ?Vs. V* $\in$ *carrier-mat ?d ?n*; *?A-orig* $\in$ *carrier-mat* (*length ?Vs*) (*length ?Vs*); $\bigwedge$*i j.* $[\![$*i* < *length ?Vs*; *j* < *length ?Vs*$]\!] \Longrightarrow$ *cnj* (*?A-orig* $$ (*i*, *j*)) = *?A-orig* $$ (*i*, *j*); *kraus-pair-sum ?n ?Vs ?A-orig = 0*$_m$ *?n ?n*; *?A-orig* $\neq$ *0*$_m$ (*length ?Vs*) (*length ?Vs*)$]\!]$ $\Longrightarrow \exists\, H.\ H \in$ *carrier-mat* (*length ?Vs*) (*length ?Vs*) $\wedge$ *hermitian H* $\wedge$ *H* $\neq$ *0*$_m$ (*length ?Vs*) (*length ?Vs*) $\wedge$ *kraus-pair-sum ?n ?Vs H = 0*$_m$ *?n ?n*) and a small ε (from $[\![$*?A* $\in$ *carrier-mat ?k ?k*; *hermitian ?A*$]\!] \Longrightarrow \exists\varepsilon{>}0.\ \forall\, d.\ |d| \leq \varepsilon \longrightarrow$ *1*$_m$ *?k* + *complex-of-real d* $\cdot_m$ *?A* $\geq_M$ *0*). Distinctness uses $[\![\forall$ *V*$\in$*set ?Vs. V* $\in$ *carrier-mat ?d ?n*; *lin-indep-complex-mat ?d ?n ?Vs*; *?A* $\in$ *carrier-mat* (*length ?Vs*) (*length ?Vs*); *?A* $\neq$ *0*$_m$ (*length ?Vs*) (*length ?Vs*); *0* < *?n*; *0* < *?d*$]\!]$ $\Longrightarrow \exists$ *rho*$\in$*carrier-mat ?n ?n. kraus-quad-map ?d ?Vs ?A rho* $\neq$ *0*$_m$ *?d ?d* (Lemma B).

theorem *choi-theorem-3-cptp-reverse*:
  fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
    and *Vs* :: *complex Matrix.mat list*
  assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
      and Φ*-in*: $\Phi \in$ *CPTP n d*
      and *rep*: *cptp-kraus-rep n d* Φ *Vs*

and *minimal*: *lin-indep-complex-mat d n Vs*
and *dep*: ¬ *mat-lin-indep-real n* (*kraus-products Vs*)
shows ¬ *extreme-CPTP n d* Φ
⟨*proof*⟩

Caveat on *extreme-CPTP* **(forward shadow).** Because both *completely-positive* and *trace-preserving* only constrain behaviour on *carrier-mat n n*, the raw function inequality $\Phi_1 \neq \Phi$ in *extreme-CPTP* is trivially satisfiable. We therefore introduce a *carrier*-point non-extremity notion below.

To keep *extreme-CPTP-carrier*'s definition manageable for automated proofs, we factor it through a helper predicate *cptp-pair-mid*: the data of a CPTP midpoint pair witnessing non-extremity (range, linearity on carrier, off-equal at some carrier-point, and the midpoint identity). Unfolding only the outer definition keeps the goal small.

definition *lin-on-carrier* ::
nat ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool*
where
lin-on-carrier n Φ ≡
(∀ *X Y*. *X* ∈ *carrier-mat n n* ⟶ *Y* ∈ *carrier-mat n n*
⟶ Φ (*X* + *Y*) = Φ *X* + Φ *Y*) ∧
(∀ *c X*. *X* ∈ *carrier-mat n n* ⟶ Φ (*c* ·$_m$ *X*) = *c* ·$_m$ Φ *X*)

lemma *lin-on-carrier-add*:
assumes *lin-on-carrier n* Φ *X* ∈ *carrier-mat n n Y* ∈ *carrier-mat n n*
shows Φ (*X* + *Y*) = Φ *X* + Φ *Y*
⟨*proof*⟩

lemma *lin-on-carrier-smult*:
assumes *lin-on-carrier n* Φ *X* ∈ *carrier-mat n n*
shows Φ (*c* ·$_m$ *X*) = *c* ·$_m$ Φ *X*
⟨*proof*⟩

lemma *lin-on-carrierI*:
assumes ⋀*X Y*. *X* ∈ *carrier-mat n n* ⟹ *Y* ∈ *carrier-mat n n*
⟹ Φ (*X* + *Y*) = Φ *X* + Φ *Y*
and ⋀*c X*. *X* ∈ *carrier-mat n n* ⟹ Φ (*c* ·$_m$ *X*) = *c* ·$_m$ Φ *X*
shows *lin-on-carrier n* Φ
⟨*proof*⟩

definition *cptp-pair-mid* ::
nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*)
⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*)
⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool*
where
cptp-pair-mid n d Φ Φ$_1$ Φ$_2$ ≡
Φ$_1$ ∈ *CPTP n d* ∧ Φ$_2$ ∈ *CPTP n d* ∧
(∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ$_1$ *X* ∈ *carrier-mat d d*) ∧
(∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ$_2$ *X* ∈ *carrier-mat d d*) ∧
lin-on-carrier n Φ$_1$ ∧ *lin-on-carrier n* Φ$_2$ ∧
(∃ *X*$_1$ ∈ *carrier-mat n n*. Φ$_1$ *X*$_1$ ≠ Φ *X*$_1$) ∧
(∃ *X*$_2$ ∈ *carrier-mat n n*. Φ *X*$_2$ ≠ Φ$_2$ *X*$_2$) ∧
(∀ *X*. *X* ∈ *carrier-mat n n* ⟶
Φ *X* = (*1/2* :: *complex*) ·$_m$ Φ$_1$ *X* + (*1/2* :: *complex*) ·$_m$ Φ$_2$ *X*)

definition *extreme-CPTP-carrier* ::
nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool*
where

extreme-CPTP-carrier n d Φ ≡
Φ ∈ CPTP n d ∧ ¬ (∃ Φ₁ Φ₂. cptp-pair-mid n d Φ Φ₁ Φ₂)

The same construction as in ⟦*0 < ?n*; *0 < ?d*; *?Φ ∈ CPTP ?n ?d*; *cptp-kraus-rep ?n ?d ?Φ ?Vs*; *lin-indep-complex-mat ?d ?n ?Vs*; ¬ *mat-lin-indep-real ?n* (*kraus-products ?Vs*)⟧ ⟹ ¬ *extreme-CPTP ?n ?d ?Φ* yields the stronger fact that Φ fails the *extreme-CPTP-carrier* test: the perturbed maps *Phip*, *Phim* are CPTP, have range *carrier-mat d d*, and disagree with Φ at the carrier-point *rho*.

lemma *not-extreme-CPTP-carrier-of-herm-witness*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list* **and** *H* :: *complex Matrix.mat*
assumes *npos*: *0 < n* **and** *dpos*: *0 < d*
and Φ-*in*: *Φ ∈ CPTP n d*
and *rep*: *cptp-kraus-rep n d Φ Vs*
and *minimal*: *lin-indep-complex-mat d n Vs*
and *H-c*: *H ∈ carrier-mat* (*length Vs*) (*length Vs*)
and *H-herm*: *hermitian H*
and *H-nz*: *H ≠ 0*$_m$ (*length Vs*) (*length Vs*)
and *H-pair*: *kraus-pair-sum n Vs H = 0*$_m$ *n n*
shows ¬ *extreme-CPTP-carrier n d Φ*
⟨*proof*⟩

Immediate contrapositive: extreme-in-carrier-sense forces linear independence of the products.

theorem *choi-theorem-3-cptp-reverse-carrier*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list*
assumes *npos*: *0 < n* **and** *dpos*: *0 < d*
and Φ-*in*: *Φ ∈ CPTP n d*
and *rep*: *cptp-kraus-rep n d Φ Vs*
and *minimal*: *lin-indep-complex-mat d n Vs*
and *dep*: ¬ *mat-lin-indep-real n* (*kraus-products Vs*)
shows ¬ *extreme-CPTP-carrier n d Φ*
⟨*proof*⟩

theorem *choi-theorem-3-cptp-reverse-carrier-complex*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list*
assumes *npos*: *0 < n* **and** *dpos*: *0 < d*
and Φ-*in*: *Φ ∈ CPTP n d*
and *rep*: *cptp-kraus-rep n d Φ Vs*
and *minimal*: *lin-indep-complex-mat d n Vs*
and *dep*: ¬ *mat-lin-indep-complex n* (*kraus-products Vs*)
shows ¬ *extreme-CPTP-carrier n d Φ*
⟨*proof*⟩

corollary *choi-theorem-3-cptp-carrier-forward*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list*
assumes *npos*: *0 < n* **and** *dpos*: *0 < d*
and Φ-*in*: *Φ ∈ CPTP n d*
and *rep*: *cptp-kraus-rep n d Φ Vs*
and *minimal*: *lin-indep-complex-mat d n Vs*
and *ext*: *extreme-CPTP-carrier n d Φ*
shows *mat-lin-indep-real n* (*kraus-products Vs*)
⟨*proof*⟩

45.23 Trace pairing duality and TP-converse

If $Tr\ (M * \varrho) = Tr\ (N * \varrho)$ for every ϱ in carrier, then $M = N$. Used to invert $[\![\forall\ V{\in}set\ ?Vs.\ V \in carrier\text{-}mat\ ?d\ ?n;\ ?Tmat \in carrier\text{-}mat\ (length\ ?Vs)\ (length\ ?Vs);\ kraus\text{-}pair\text{-}sum\ ?n\ ?Vs\ ?Tmat = 1_m\ ?n;\ 0 < ?n]\!] \Longrightarrow trace\text{-}preserving\ ?n\ (kraus\text{-}quad\text{-}map\ ?d\ ?Vs\ ?Tmat)$.

lemma *Tr-mat-unit-mult*:
  fixes M :: *complex Matrix.mat*
  assumes *M-c*: $M \in carrier\text{-}mat\ n\ n$
    and *r-lt*: $r < n$ **and** *s-lt*: $s < n$
  shows $Tr\ (M * mat\text{-}unit\ s\ r\ n\ n) = M\ \$\$\ (r,\ s)$
⟨*proof*⟩

lemma *mat-trace-pairing-unique*:
  fixes $M\ N$ :: *complex Matrix.mat*
  assumes *M-c*: $M \in carrier\text{-}mat\ n\ n$
    and *N-c*: $N \in carrier\text{-}mat\ n\ n$
    and *pair-eq*: $\bigwedge rho.\ rho \in carrier\text{-}mat\ n\ n \Longrightarrow Tr\ (M * rho) = Tr\ (N * rho)$
  shows $M = N$
⟨*proof*⟩

45.24 Choi's Theorem 3 — forward direction

Important caveat on *extreme-CPTP***.** The current definition uses raw function inequality $\Phi_1 \neq \Phi$, but both *completely-positive* and *trace-preserving* only constrain behaviour on *carrier-mat n n*. Hence for *any* CPTP Φ one can construct Φ_1 that agrees with Φ on the carrier but disagrees on (say) a *1×1* matrix; this Φ_1 is itself in CPTP, so the negation of *extreme-CPTP* is vacuously satisfied. Consequently *extreme-CPTP* **as currently defined is always false**, and Choi's Theorem 3 as stated (an iff with *mat-lin-indep-real*) does not hold.

The fix is to use a *carrier*-point notion of inequality (defined above as *extreme-CPTP-carrier*).

The forward direction targets the *carrier*-flavoured version: *mat-lin-indep-complex* $\Longrightarrow$ *extreme-CPTP-carrier*. Proof uses $[\![\forall\ V{\in}set\ ?Vs.\ V \in carrier\text{-}mat\ ?d\ ?n;\ \bigwedge X.\ X \in carrier\text{-}mat\ ?n\ ?n \Longrightarrow ?Phi\ X = kraus\text{-}map\text{-}dim\ ?d\ ?Vs\ X;\ completely\text{-}positive\ ?n\ ?d\ ?Psi;\ \bigwedge X.\ X \in carrier\text{-}mat\ ?n\ ?n \Longrightarrow ?Psi\ X \in carrier\text{-}mat\ ?d\ ?d;\ \bigwedge X\ Y.\ [\![X \in carrier\text{-}mat\ ?n\ ?n;\ Y \in carrier\text{-}mat\ ?n\ ?n]\!] \Longrightarrow ?Psi\ (X + Y) = ?Psi\ X + ?Psi\ Y;\ \bigwedge c\ X.\ X \in carrier\text{-}mat\ ?n\ ?n \Longrightarrow ?Psi\ (c \cdot_m X) = c \cdot_m ?Psi\ X;\ completely\text{-}positive\ ?n\ ?d\ (\lambda X.\ ?Phi\ X - ?Psi\ X);\ 0 < ?n;\ 0 < ?d]\!] \Longrightarrow \exists T.\ T \in carrier\text{-}mat\ (length\ ?Vs)\ (length\ ?Vs) \wedge T \geq_M 0 \wedge (\forall X.\ X \in carrier\text{-}mat\ ?n\ ?n \longrightarrow ?Psi\ X = kraus\text{-}quad\text{-}map\ ?d\ ?Vs\ T\ X)$, obtaining linearity of Ψ from the *lin-on-carrier* conjuncts carried by *cptp-pair-mid*.

theorem *choi-theorem-3-cptp-forward-complex*:
  fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
    and *Vs* :: *complex Matrix.mat list*
  assumes *npos*: $0 < n$ **and** *dpos*: $0 < d$
    and Φ-*in*: $\Phi \in CPTP\ n\ d$
    and *rep*: *cptp-kraus-rep n d* Φ *Vs*
    and *lin-indep*: *mat-lin-indep-complex n* (*kraus-products Vs*)
  shows *extreme-CPTP-carrier n d* Φ
⟨*proof*⟩

45.25 Choi's Theorem 3 (statement)

NOTE on conventions. Choi (1975, Theorem 3) gives an extreme-point characterisation in terms of *linear independence of certain products of Kraus operators*. Which products appear depends on which of the two natural normalisations the underlying convex set uses:

- **CPTP** ($\sum\ V_i{}^\dagger\ V_i = I_n$): products are $\{V_j{}^\dagger\ V_i\}$ in M_n. The perturbation $\Phi_\varepsilon(X) = \sum(\delta_{\{}ij\} + \varepsilon\ a_{\{}ij\})\ V_i\ X\ V_j{}^\dagger$ stays trace-preserving iff $\sum\ a_{\{}ij\}\ V_j{}^\dagger\ V_i = 0$.
- **UCP** ($\sum\ V_i\ V_i{}^\dagger = I_d$): products are $\{V_i\ V_j{}^\dagger\}$ in M_d. Same perturbation stays unital iff $\sum\ a_{\{}ij\}\ V_i\ V_j{}^\dagger = 0$.

The two are interchanged by the Hilbert–Schmidt adjoint $\Phi \mapsto \Phi^*$; this provides a UCP $\leftrightarrow$ CPTP bijection.

We state both versions, parametrised by a fixed Kraus representation; the "any/every" character follows from the unitary-freedom of Kraus reps (Day 22 / "Kraus uniqueness"), which is not yet formalised in this development.

Reverse direction (extreme $\Rightarrow$ lin. indep.): contrapositive. For the CPTP version: if $\sum\ a_{\{}ij\}\ V_j{}^\dagger\ V_i = 0$ non-trivially, define Φ_ε as above; for small hermitian a, both $\Phi_{\pm\varepsilon}$ are CPTP and distinct, giving a non-trivial midpoint of Φ.

Forward direction (lin. indep. $\Rightarrow$ extreme): requires Radon–Nikodym for CP maps (every CP $\Psi \leq_{\{} cp\}\ \lambda\Phi$ has $\Psi(X) = \sum\ T_{\{}ij\}\ V_i\ X\ V_j{}^\dagger$ for a PSD matrix T) which is itself a substantial development.

The CPTP form of Choi's Theorem 3 (carrier-flavoured). Φ is extreme in CPTP iff the products $\{V_j{}^\dagger\ V_i\}$ are linearly independent in M_n.

The project's *extreme-CPTP* uses a raw function inequality $\Phi_1 \neq \Phi$; since *completely-positive* and *trace-preserving* only constrain behaviour on *carrier-mat n n*, this is technically vacuous (every CPTP map admits a CPTP Φ_1 that agrees with it on carrier but disagrees off carrier). The theorem is therefore stated in terms of *extreme-CPTP-carrier*, the carrier-point variant.

The first biconditional below uses different lin-indep notions on each direction (the historical form); *choi-theorem-3-cptp-iff* states the genuine iff:

- *extreme-CPTP-carrier* $\Longrightarrow$ *mat-lin-indep-real* (contrapositive of ⟦*0 < ?n*; *0 < ?d*; *?Φ ∈ CPTP ?n ?d*; *cptp-kraus-rep ?n ?d ?Φ ?Vs*; *lin-indep-complex-mat ?d ?n ?Vs*; ¬ *mat-lin-indep-real ?n* (*kraus-products ?Vs*)⟧ $\Longrightarrow$ ¬ *extreme-CPTP-carrier ?n ?d ?Φ*);
- *mat-lin-indep-complex* $\Longrightarrow$ *extreme-CPTP-carrier* (the strengthened Choi forward direction ⟦*0 < ?n*; *0 < ?d*; *?Φ ∈ CPTP ?n ?d*; *cptp-kraus-rep ?n ?d ?Φ ?Vs*; *mat-lin-indep-complex ?n* (*kraus-products ?Vs*)⟧ $\Longrightarrow$ *extreme-CPTP-carrier ?n ?d ?Φ*, modulo CP Radon–Nikodym).

  The complex-coefficient witness *herm-witness-of-complex* bridges the two notions, so the direction "extreme $\Rightarrow$ independent" also holds over $\mathbb{C}$.

theorem *choi-theorem-3-cptp*:
  fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
    and *Vs* :: *complex Matrix.mat list*
  assumes *npos*: *0 < n* **and** *dpos*: *0 < d*
      and Φ-*in*: *Φ ∈ CPTP n d*
      and *rep*: *cptp-kraus-rep n d Φ Vs*
      and *minimal*: *lin-indep-complex-mat d n Vs*
  shows (*extreme-CPTP-carrier n d Φ*
          ⟶ *mat-lin-indep-real n* (*kraus-products Vs*))
      ∧ (*mat-lin-indep-complex n* (*kraus-products Vs*)
          ⟶ *extreme-CPTP-carrier n d Φ*)
⟨*proof*⟩

With the complex-coefficient witness, the two directions use the same notion of linear independence: Choi's Theorem 3 in the form of the notes' corollary.

theorem *choi-theorem-3-cptp-iff*:

fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list*
assumes *npos*: $0 < n$ **and** *dpos*: $0 < d$
and Φ-*in*: $\Phi \in$ *CPTP n d*
and *rep*: *cptp-kraus-rep n d* Φ *Vs*
and *minimal*: *lin-indep-complex-mat d n Vs*
shows *extreme-CPTP-carrier n d* $\Phi \longleftrightarrow$ *mat-lin-indep-complex n* (*kraus-products Vs*)
⟨*proof*⟩

The UCP form of Choi's Theorem 3. In the UCP / Heisenberg convention, the relevant products live in M_d with the *opposite* order $\{V_i\ {V_j}^\dagger\}$.

The project's *kraus-products* uses the CPTP-side convention $\{{V_j}^\dagger\ V_i\}$, and there is no UCP-side carrier-extreme notion or analogue of ⟦*0 < ?n*; *0 < ?d*; *?*$\Phi \in$ *CPTP ?n ?d*; *cptp-kraus-rep ?n ?d ?*Φ *?Vs*; *lin-indep-complex-mat ?d ?n ?Vs*; ¬ *mat-lin-indep-real ?n* (*kraus-products ?Vs*)⟧ $\Longrightarrow$ ¬ *extreme-CPTP-carrier ?n ?d ?*Φ in this development. A proper UCP form of Choi's theorem requires either:

- a parallel *kraus-products-ucp* ($V_i\ {V_j}^\dagger$), an *extreme-UCP-carrier* predicate, and full UCP-side analogues of the carrier-flavoured lemmas; or
- a Hilbert–Schmidt duality between *extreme-UCP* and *extreme-CPTP* that transports the CPTP theorem above.

  We leave both routes for future work and state no UCP form here.

45.26 Sanity / easy facts about extreme UCP

If a UCP map has a non-trivial mid-point representation by two distinct UCP maps, it is not extreme *by definition*.

lemma *not-extreme-UCP-if-midpoint*:
assumes $\Phi_1 \in$ *UCP n d* $\Phi_2 \in$ *UCP n d*
and $\Phi_1 \neq \Phi$ **and** $\Phi \neq \Phi_2$
and $\Phi \in$ *UCP n d*
and $\bigwedge X.\ X \in$ *carrier-mat n n*
$\Longrightarrow \Phi\ X = (1/2 :: \mathit{complex}) \cdot_m \Phi_1\ X$
$+ (1/2 :: \mathit{complex}) \cdot_m \Phi_2\ X$
shows ¬ *extreme-UCP n d* Φ
⟨*proof*⟩

The contrapositive: every extreme UCP map, when written as a mid-point of two UCP maps, must have those maps coincide with itself on every $X \in$ *carrier-mat n n*.

lemma *extreme-UCP-midpoint-collapses*:
assumes *ext*: *extreme-UCP n d* Φ
and Φ*1-in*: $\Phi_1 \in$ *UCP n d*
and Φ*2-in*: $\Phi_2 \in$ *UCP n d*
and *mid*: $\bigwedge X.\ X \in$ *carrier-mat n n*
$\Longrightarrow \Phi\ X = (1/2 :: \mathit{complex}) \cdot_m \Phi_1\ X$
$+ (1/2 :: \mathit{complex}) \cdot_m \Phi_2\ X$
shows $\Phi_1 = \Phi \vee \Phi = \Phi_2$
⟨*proof*⟩

This closes the Day-23 stub. The full content of Choi's Theorem 3 awaits a Kraus-list formalisation; the definitions collected above will plug into that proof once available.

end

theory *Day23-Extremality-Examples*
  imports *Day22-Kraus-Independence Day25-Arveson-Correspondence*
begin

Day 23, examples after Choi's Theorem 3, in the CPTP form and with the Kraus convention $\Phi(X) = \sum_i W_i X W_i^*$ of the formalisation (the notes write $U^* X U$; replace U by U^*).

45.27 Linear dependence tests

lemma *foldr-smult-index*:
  assumes *Bs*: $\forall\, B \in$ *set Bs*. $B \in$ *carrier-mat n m* **and** *r*: $r < n$ **and** *s*: $s < m$
  shows *foldr* (λk *acc*. *c k* $\cdot_m$ (*Bs* ! *k*) + *acc*) [*0*..<*length Bs*] (0_m *n m*) $$ (*r*,*s*)
    = ($\sum$ *k*<*length Bs*. *c k* ∗ *Bs* ! *k* $$ (*r*,*s*))
⟨*proof*⟩

lemma *not-indep-of-dup*:
  assumes *a*: $a <$ *length Bs* **and** *b*: $b <$ *length Bs* **and** *ab*: $a \neq b$ **and** *eq*: *Bs* ! *a* = *Bs* ! *b*
  shows ¬ *mat-lin-indep-complex n Bs*
⟨*proof*⟩

lemma *not-indep-of-zero*:
  assumes *a*: $a <$ *length Bs* **and** *z*: *Bs* ! *a* = 0_m *n n*
  shows ¬ *mat-lin-indep-complex n Bs*
⟨*proof*⟩

lemma *single-lin-indep*:
  assumes *W*: $W \in$ *carrier-mat d n* **and** *r*: $r < d$ **and** *s*: $s < n$ **and** *nz*: *W* $$ (*r*,*s*) $\neq$ *0*
  shows *lin-indep-complex-mat d n* [*W*]
  ⟨*proof*⟩

lemma *single-mat-lin-indep*:
  assumes *B*: $B \in$ *carrier-mat n n* **and** *r*: $r < n$ **and** *s*: $s < n$ **and** *nz*: *B* $$ (*r*,*s*) $\neq$ *0*
  shows *mat-lin-indep-complex n* [*B*]
  ⟨*proof*⟩

lemma *nonzero-entry*:
  fixes *A* :: *complex Matrix.mat*
  assumes *A*: $A \in$ *carrier-mat m k* **and** *nz*: $A \neq 0_m$ *m k*
  obtains *r s* **where** $r < m$ $s < k$ *A* $$ (*r*,*s*) $\neq$ *0*
⟨*proof*⟩

lemma *mat-unit-carrier-sq* [*simp*]: $E⟦i,j$; $n,n⟧ \in$ *carrier-mat n n*
  ⟨*proof*⟩

lemma *one-smult-mat'*: (*1*::*complex*) $\cdot_m$ *A* = *A*
  ⟨*proof*⟩

lemma *dagger-smult-mat*: (*c* $\cdot_m$ *A*)† = *cnj c* $\cdot_m$ ($A^\dagger$)
  ⟨*proof*⟩

45.28 Example 1: unitary conjugation is extreme

theorem *unitary-conjugation-extreme*:
  assumes *U*: $U \in$ *carrier-mat n n* **and** *Uu*: *unitary U* **and** *n*: $0 < n$
  shows *extreme-CPTP-carrier n n* (*kraus-map-dim n* [*U*])
⟨*proof*⟩

45.29 Example 2: an average of two unitary conjugations

definition *avg2* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat list* **where**
avg2 U1 U2 = [(*1 / csqrt 2*) ·$_m$ *U1*, (*1 / csqrt 2*) ·$_m$ *U2*]

lemma *half-sq*: *cnj* (*1 / csqrt 2*) ∗ (*1 / csqrt 2*) = *1/2*
⟨*proof*⟩

lemma *smult-prod*:
fixes *A B* :: *complex Matrix.mat*
assumes *A* ∈ *carrier-mat m k B* ∈ *carrier-mat k l*
shows (*a* ·$_m$ *A*) ∗ (*b* ·$_m$ *B*) = (*a* ∗ *b*) ·$_m$ (*A* ∗ *B*)
⟨*proof*⟩

lemma *smult-triple*:
fixes *A X B* :: *complex Matrix.mat*
assumes *A* ∈ *carrier-mat m k X* ∈ *carrier-mat k k B* ∈ *carrier-mat k l*
shows (*a* ·$_m$ *A*) ∗ *X* ∗ (*b* ·$_m$ *B*) = (*a* ∗ *b*) ·$_m$ (*A* ∗ *X* ∗ *B*)
⟨*proof*⟩

lemma *avg2-products*:
assumes *U1*: *U1* ∈ *carrier-mat n n unitary U1* **and** *U2*: *U2* ∈ *carrier-mat n n unitary U2*
shows (*avg2 U1 U2* ! *0*)† ∗ *avg2 U1 U2* ! *0* = (*1/2*) ·$_m$ *1*$_m$ *n*
and (*avg2 U1 U2* ! *1*)† ∗ *avg2 U1 U2* ! *1* = (*1/2*) ·$_m$ *1*$_m$ *n*
⟨*proof*⟩

lemma *avg2-kraus*:
assumes *U1*: *U1* ∈ *carrier-mat n n unitary U1* **and** *U2*: *U2* ∈ *carrier-mat n n unitary U2*
shows *kraus-operators n n* (*avg2 U1 U2*)
⟨*proof*⟩

lemma *avg2-map*:
assumes *U1*: *U1* ∈ *carrier-mat n n* **and** *U2*: *U2* ∈ *carrier-mat n n* **and** *M*: *M* ∈ *carrier-mat n n*
shows *kraus-map-dim n* (*avg2 U1 U2*) *M* = (*1/2*) ·$_m$ (*U1* ∗ *M* ∗ (*U1*†) + *U2* ∗ *M* ∗ (*U2*†))
⟨*proof*⟩

The notes' claim needs U_1, U_2 linearly independent: then the products $V_i^* V_j$ contain $\frac{1}{2}I$ twice and Φ is not extreme.

theorem *avg2-not-extreme*:
assumes *U1*: *U1* ∈ *carrier-mat n n unitary U1* **and** *U2*: *U2* ∈ *carrier-mat n n unitary U2*
and *n*: *0* < *n* **and** *ind*: *lin-indep-complex-mat n n* (*avg2 U1 U2*)
shows ¬ *extreme-CPTP-carrier n n* (*kraus-map-dim n* (*avg2 U1 U2*))
⟨*proof*⟩

Without that hypothesis the claim fails: for $U_2 = \lambda U_1$ with $|\lambda| = 1$ the map is $X \mapsto U_1 X U_1^*$, which is extreme.

theorem *avg2-proportional-extreme*:
assumes *U1*: *U1* ∈ *carrier-mat n n unitary U1* **and** *lam*: *cmod z* = *1* **and** *n*: *0* < *n*
shows *extreme-CPTP-carrier n n* (*kraus-map-dim n* (*avg2 U1* (*z* ·$_m$ *U1*)))
⟨*proof*⟩

45.30 Example 4: the diagonal map

lemma *unit-mult-left-entry*:
assumes *Bm*: *Bm* ∈ *carrier-mat n m* **and** *r*: *r* < *p* **and** *k*: *k* < *m* **and** *j*: *j* < *n*
shows (*E*⟦*i,j* ; *p,n*⟧ ∗ *Bm*) $$ (*r,k*) = (*if r* = *i then Bm* $$ (*j,k*) *else 0*)
⟨*proof*⟩

lemma *unit-mult-right-entry*:
assumes *Cm*: *Cm* ∈ *carrier-mat p n* **and** *r*: *r* < *p* **and** *s*: *s* < *m* **and** *i*: *i* < *n*
shows (*Cm* ∗ *E*⟦*i*,*j* ; *n*,*m*⟧) $$ (*r*,*s*) = (*if s* = *j then Cm* $$ (*r*,*i*) *else* 0)
⟨*proof*⟩

definition *diag-kraus* :: *nat* ⇒ *complex Matrix.mat list* **where**
diag-kraus n = *map* (λ*i*. *E*⟦*i*,*i* ; *n*,*n*⟧) [0..<*n*]

lemma *diag-kraus-carrier*: ∀ *W* ∈ *set* (*diag-kraus n*). *W* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *diag-kraus-map*:
assumes *M*: *M* ∈ *carrier-mat n n*
shows *kraus-map-dim n* (*diag-kraus n*) *M* = *Matrix.mat n n* (λ(*r*,*s*). *if r* = *s then M* $$ (*r*,*r*) *else* 0)
⟨*proof*⟩

lemma *diag-kraus-operators*: *kraus-operators n n* (*diag-kraus n*)
⟨*proof*⟩

lemma *diag-kraus-lin-indep*: *lin-indep-complex-mat n n* (*diag-kraus n*)
⟨*proof*⟩

theorem *diag-not-extreme*:
assumes *n*: 2 ≤ *n*
shows ¬ *extreme-CPTP-carrier n n* (*kraus-map-dim n* (*diag-kraus n*))
⟨*proof*⟩

45.31 Example 3: $X \mapsto \mathrm{Tr}(X)\, I_d$

As written, $\Phi(X) = \mathrm{Tr}(X) I_d$ is neither unital ($\Phi(I_n) = n I_d$) nor trace preserving ($\mathrm{Tr}\,\Phi(X) = d\,\mathrm{Tr}\,X$) unless $n = 1$ resp. $d = 1$. The CPTP normalisation is $\Phi(X) = \mathrm{Tr}(X) I_d/d$, with the nd Kraus operators $E_{ab}/\sqrt{d}$; these are linearly independent, so $\mathrm{cr}(\Phi) = nd > n$ for $d \geq 2$ and Φ is not extreme (D22-6).

definition *tr-kraus* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat list* **where**
tr-kraus n d = *map* (λ(*a*,*b*). *complex-of-real* (1 / *sqrt* (*real d*)) ·$_m$ *E*⟦*a*,*b* ; *d*,*n*⟧) (*List.product* [0..<*d*] [0..<*n*])

lemma *tr-kraus-carrier*: ∀ *W* ∈ *set* (*tr-kraus n d*). *W* ∈ *carrier-mat d n*
⟨*proof*⟩

lemma *tr-kraus-length*: *length* (*tr-kraus n d*) = *d* ∗ *n*
⟨*proof*⟩

lemma *tr-scalar*:
assumes *d*: 0 < *d*
shows *cnj* (*complex-of-real* (1 / *sqrt* (*real d*))) ∗ *complex-of-real* (1 / *sqrt* (*real d*)) = 1 / *of-nat d*
and *complex-of-real* (1 / *sqrt* (*real d*)) ∗ *cnj* (*complex-of-real* (1 / *sqrt* (*real d*))) = 1 / *of-nat d*
⟨*proof*⟩

lemma *tr-kraus-operators*:
assumes *d*: 0 < *d*
shows *kraus-operators n d* (*tr-kraus n d*)
⟨*proof*⟩

lemma *tr-kraus-map*:
assumes *d*: 0 < *d* **and** *M*: *M* ∈ *carrier-mat n n*
shows *kraus-map-dim d* (*tr-kraus n d*) *M* = (*Tr M* / *of-nat d*) ·$_m$ 1$_m$ *d*

⟨*proof*⟩

lemma *tr-kraus-lin-indep*:
assumes *d*: $0 < d$ **and** *n*: $0 < n$
shows *lin-indep-complex-mat d n* (*tr-kraus n d*)
⟨*proof*⟩

theorem *tr-map-not-extreme*:
assumes *n*: $0 < n$ **and** *d*: $2 \leq d$
shows ¬ *extreme-CPTP-carrier n d* (*kraus-map-dim d* (*tr-kraus n d*))
⟨*proof*⟩

end
theory *Day24-Operator-Systems*
imports *Day17-Environment-Model*
begin

Day 24: operator systems $S \subseteq M_n$ (subspaces containing I and closed under adjoints), the matrices $M_p(S)$ over them, and completely positive maps on S. An element of $M_p(M_n)$ is a $pn \times pn$ matrix whose (a, b) block is *block-of-mat n X a b*; the ampliation $\Phi^{(p)}$ is *amplification n d p* Φ (Day 19).

definition *operator-system* :: *nat* ⇒ *complex Matrix.mat set* ⇒ *bool* **where**
operator-system n OS ⟷ *OS* ⊆ *carrier-mat n n* ∧ 1_m *n* ∈ *OS* ∧ (∀ *A* ∈ *OS*. $A^\dagger$ ∈ *OS*)
∧ (∀ *A* ∈ *OS*. ∀ *B* ∈ *OS*. *A* + *B* ∈ *OS*) ∧ (∀ *A* ∈ *OS*. ∀ *c*. *c* ·$_m$ *A* ∈ *OS*)

definition *Mp* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat set* ⇒ *complex Matrix.mat set* **where**
Mp p n OS = {*A* ∈ *carrier-mat* (*p* ∗ *n*) (*p* ∗ *n*). ∀ *a* < *p*. ∀ *b* < *p*. *block-of-mat n A a b* ∈ *OS*}

definition *lin-on* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
lin-on n d OS Φ ⟷ (∀ *A* ∈ *OS*. Φ *A* ∈ *carrier-mat d d*) ∧ (∀ *A* ∈ *OS*. ∀ *B* ∈ *OS*. Φ (*A* + *B*) = Φ *A* + Φ *B*)
∧ (∀ *A* ∈ *OS*. ∀ *c*. Φ (*c* ·$_m$ *A*) = *c* ·$_m$ Φ *A*)

definition *p-positive-on* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
p-positive-on n d p OS Φ ⟷ (∀ *A* ∈ *Mp p n OS*. *psd A* ⟶ *psd* (*amplification n d p* Φ *A*))

definition *cp-on* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
cp-on n d OS Φ ⟷ (∀ *p* > 0. *p-positive-on n d p OS* Φ)

lemma *block-of-mat-dims* [*simp*]: *dim-row* (*block-of-mat n A a b*) = *n dim-col* (*block-of-mat n A a b*) = *n*
⟨*proof*⟩

lemma *block-of-mat-idx*:
assumes *r* < *n* **and** *c* < *n*
shows *block-of-mat n A a b* $$ (*r*,*c*) = *A* $$ (*a*∗*n* + *r*, *b*∗*n* + *c*)
⟨*proof*⟩

lemma *operator-systemD*:
assumes *operator-system n OS*
shows *OS* ⊆ *carrier-mat n n* 1_m *n* ∈ *OS* ⋀*A*. *A* ∈ *OS* ⟹ $A^\dagger$ ∈ *OS*
⋀*A B*. *A* ∈ *OS* ⟹ *B* ∈ *OS* ⟹ *A* + *B* ∈ *OS* ⋀*A c*. *A* ∈ *OS* ⟹ *c* ·$_m$ *A* ∈ *OS*
⟨*proof*⟩

lemma *operator-system-zero*:

assumes *OS*: *operator-system n OS*
shows $0_m\ n\ n \in OS$
⟨*proof*⟩

lemma *operator-system-diff*:
assumes *OS*: *operator-system n OS* **and** *A*: $A \in OS$ **and** *B*: $B \in OS$
shows $A - B \in OS$
⟨*proof*⟩

lemma *operator-system-full*: *operator-system n* (*carrier-mat n n*)
⟨*proof*⟩

lemma *Mp-full*: *Mp p n* (*carrier-mat n n*) = *carrier-mat* $(p * n)$ $(p * n)$
⟨*proof*⟩

lemma *cp-on-full*: *cp-on n d* (*carrier-mat n n*) $\Phi \longleftrightarrow$ *completely-positive n d* Φ
⟨*proof*⟩

lemma *cp-on-mono*:
assumes *cp-on n d OS* Φ **and** $OS' \subseteq OS$
shows *cp-on n d OS'* Φ
⟨*proof*⟩

45.32 $M_p(OS)$ is an operator system

lemma *block-of-mat-add*:
assumes $A \in$ *carrier-mat* $(p * n)$ $(p * n)$ $B \in$ *carrier-mat* $(p * n)$ $(p * n)$ $a < p$ $b < p$
shows *block-of-mat n* $(A + B)$ *a b* = *block-of-mat n A a b* + *block-of-mat n B a b*
⟨*proof*⟩

lemma *block-of-mat-smult*:
assumes $A \in$ *carrier-mat* $(p * n)$ $(p * n)$ $a < p$ $b < p$
shows *block-of-mat n* $(k \cdot_m A)$ *a b* = $k \cdot_m$ *block-of-mat n A a b*
⟨*proof*⟩

lemma *block-of-mat-dagger*:
assumes $A \in$ *carrier-mat* $(p * n)$ $(p * n)$ $a < p$ $b < p$
shows *block-of-mat n* $(A^\dagger)$ *a b* = $($*block-of-mat n A b a*$)^\dagger$
⟨*proof*⟩

lemma *block-of-mat-one*:
assumes $a < p$ $b < p$
shows *block-of-mat n* $(1_m\ (p * n))$ *a b* = (*if a* = *b then* 1_m *n else* 0_m *n n*)
⟨*proof*⟩

The note of Day 24: $M_p(S)$ is again an operator system.

lemma *Mp-operator-system*:
assumes *OS*: *operator-system n OS*
shows *operator-system* $(p * n)$ (*Mp p n OS*)
⟨*proof*⟩

45.33 Every element is a combination of four positives

lemma *herm-bound*:
assumes *Hmc*: *Hm* ∈ *carrier-mat n n* **and** *Hmh*: *hermitian Hm*
shows ∃ *c*::*real*. $0 \le c \wedge$ *psd* (*complex-of-real c* $\cdot_m$ 1_m *n* − *Hm*) ∧ *psd* (*complex-of-real c* $\cdot_m$ 1_m *n* + *Hm*)
⟨*proof*⟩

Proposition (Day 24): $X = (P_1 - P_2) + i(P_3 - P_4)$ with $P_k \in S^+$.

proposition *operator-system-four-positives*:
assumes *OS*: *operator-system n OS* **and** *A*: *A* ∈ *OS*
shows ∃ *P1 P2 P3 P4*. *P1* ∈ *OS* ∧ *P2* ∈ *OS* ∧ *P3* ∈ *OS* ∧ *P4* ∈ *OS* ∧ *psd P1* ∧ *psd P2* ∧ *psd P3* ∧ *psd P4*
∧ *A* = (*P1* − *P2*) + i ·$_m$ (*P3* − *P4*)
⟨*proof*⟩

end
theory *Day25-Arveson-Correspondence*
imports *Day24-Operator-Systems*
begin

Days 24–25: Arveson's correspondence between maps $\Phi : S \to M_d$ and functionals $S_\Phi : M_d(S) \to \mathbb{C}$, $S_\Phi((A_{ij})) = \frac{1}{d}\sum_{i,j} \Phi(A_{ij})_{ij}$, with inverse $f \mapsto \Phi_f$, $\Phi_f(M) = d\,(f(E_{ij} \otimes M))_{ij}$.

45.34 Finite sums of matrices

definition *msum* :: *nat* ⇒ *nat* ⇒ (′*a* ⇒ *complex Matrix.mat*) ⇒ ′*a list* ⇒ *complex Matrix.mat* **where**
msum r c g xs = *foldr* (λ*x acc*. *g x* + *acc*) *xs* (*0*$_m$ *r c*)

lemma *msum-Nil* [*simp*]: *msum r c g* [] = *0*$_m$ *r c*
⟨*proof*⟩

lemma *msum-Cons* [*simp*]: *msum r c g* (*x* # *xs*) = *g x* + *msum r c g xs*
⟨*proof*⟩

lemma *msum-carrier*: ∀ *x* ∈ *set xs*. *g x* ∈ *carrier-mat r c* ⟹ *msum r c g xs* ∈ *carrier-mat r c*
⟨*proof*⟩

lemma *msum-index*:
assumes ∀ *x* ∈ *set xs*. *g x* ∈ *carrier-mat r c* **and** *i* < *r* **and** *j* < *c*
shows *msum r c g xs* $$ (*i*,*j*) = (∑ *x*←*xs*. *g x* $$ (*i*,*j*))
⟨*proof*⟩

lemma *msum-in-os*:
assumes *OS*: *operator-system n OS* **and** *g*: ∀ *x* ∈ *set xs*. *g x* ∈ *OS*
shows *msum n n g xs* ∈ *OS*
⟨*proof*⟩

lemma *lin-on-zero*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ
shows Φ (*0*$_m$ *n n*) = *0*$_m$ *d d*
⟨*proof*⟩

lemma *lin-on-msum*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ **and** *g*: ∀ *x* ∈ *set xs*. *g x* ∈ *OS*
shows Φ (*msum n n g xs*) = *msum d d* (λ*x*. Φ (*g x*)) *xs*
⟨*proof*⟩

lemma *sum-list-product-upt*:
(∑ *x*←*List.product* [*0*..<*a*] [*0*..<*b*]. *h x*) = (∑ *i*<*a*. ∑ *j*<*b*. *h* (*i*,*j*))
⟨*proof*⟩

lemma *sum-delta2*:
fixes *a d b e* :: *nat*
assumes *a*: *a* < *d* **and** *b*: *b* < *e*
shows (∑ *i*<*d*. ∑ *j*<*e*. *if i* = *a* ∧ *j* = *b then v else 0*) = *v*

⟨*proof*⟩

45.35 $E_{ij} \otimes M$ and the block decomposition

definition *unit-block* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
unit-block d n i j M = *Matrix.mat* (*d* ∗ *n*) (*d* ∗ *n*)
(λ(*r*,*c*). *if r div n* = *i* ∧ *c div n* = *j then M* $$ (*r mod n*, *c mod n*) *else 0*)

lemma *unit-block-carrier* [*simp*]: *unit-block d n i j M* ∈ *carrier-mat* (*d* ∗ *n*) (*d* ∗ *n*)
⟨*proof*⟩

lemma *unit-block-dims* [*simp*]: *dim-row* (*unit-block d n i j M*) = *d* ∗ *n dim-col* (*unit-block d n i j M*) = *d* ∗ *n*
⟨*proof*⟩

unit-block d n i j M is $E_{ij} \otimes M$.

lemma *unit-block-tensor*:
assumes *M*: *M* ∈ *carrier-mat n n* **and** *i*: *i* < *d* **and** *j*: *j* < *d*
shows *unit-block d n i j M* = *Tensor.tensor-mat* (*Matrix.mat d d* (λ(*a*,*b*). *if a* = *i* ∧ *b* = *j then 1 else 0*)) *M*
⟨*proof*⟩

lemma *block-unit-block*:
assumes *M*: *M* ∈ *carrier-mat n n* **and** *a*: *a* < *d* **and** *b*: *b* < *d*
shows *block-of-mat n* (*unit-block d n i j M*) *a b* = (*if a* = *i* ∧ *b* = *j then M else* 0_m *n n*)
⟨*proof*⟩

lemma *unit-block-Mp*:
assumes *OS*: *operator-system n OS* **and** *M*: *M* ∈ *OS* **and** *i*: *i* < *d* **and** *j*: *j* < *d*
shows *unit-block d n i j M* ∈ *Mp d n OS*
⟨*proof*⟩

lemma *block-decomposition*:
assumes *A*: *A* ∈ *carrier-mat* (*d* ∗ *n*) (*d* ∗ *n*)
shows *A* = *msum* (*d* ∗ *n*) (*d* ∗ *n*) (λ(*i*,*j*). *unit-block d n i j* (*block-of-mat n A i j*)) (*List.product* [*0*..<*d*] [*0*..<*d*])
⟨*proof*⟩

45.36 The correspondence

definition *arveson-functional* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* ⇒ *complex* **where**
arveson-functional n d Φ *A* = (∑ *i*<*d*. ∑ *j*<*d*. Φ (*block-of-mat n A i j*) $$ (*i*,*j*)) / *of-nat d*

definition *arveson-map* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex*) ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
arveson-map n d f M = *Matrix.mat d d* (λ(*i*,*j*). *of-nat d* ∗ *f* (*unit-block d n i j M*))

definition *lin-fun-on* :: *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex*) ⇒ *bool* **where**
lin-fun-on TS f ⟷ (∀ *A* ∈ *TS*. ∀ *B* ∈ *TS*. *f* (*A* + *B*) = *f A* + *f B*) ∧ (∀ *A* ∈ *TS*. ∀ *c*. *f* (*c* ·$_m$ *A*) = *c* ∗ *f A*)

definition *positive-fun-on* :: *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex*) ⇒ *bool* **where**
positive-fun-on TS f ⟷ (∀ *A* ∈ *TS*. *psd A* ⟶ 0 ≤ *f A*)

lemma *amplification-carrier'*: *amplification n d p* Φ *A* ∈ *carrier-mat* (*p* ∗ *d*) (*p* ∗ *d*)
⟨*proof*⟩

lemma *amplification-idx*:
assumes *r*: $r < q$ **and** *s*: $s < q$ **and** *i*: $i < d$ **and** *j*: $j < d$
shows *amplification n d q Φ A $$ (r * d + i, s * d + j) = Φ (block-of-mat n A r s) $$ (i,j)*
⟨*proof*⟩

lemma *arveson-map-index*: $i < d \Longrightarrow j < d \Longrightarrow$ *arveson-map n d f M $$ (i,j) = of-nat d * f (unit-block d n i j M)*
⟨*proof*⟩

lemma *arveson-map-carrier* [*simp*]: *arveson-map n d f M ∈ carrier-mat d d*
⟨*proof*⟩

lemma *arveson-map-dims* [*simp*]: *dim-row (arveson-map n d f M) = d dim-col (arveson-map n d f M) = d*
⟨*proof*⟩

lemma *lin-fun-zero*:
assumes *TS*: *operator-system N TS* **and** *f*: *lin-fun-on TS f*
shows *f* $(0_m\ N\ N) = 0$
⟨*proof*⟩

lemma *lin-fun-msum*:
assumes *TS*: *operator-system N TS* **and** *f*: *lin-fun-on TS f* **and** *g*: $\forall x \in$ *set xs. g x ∈ TS*
shows *f (msum N N g xs) =* $(\sum x \leftarrow xs.\ f\ (g\ x))$
⟨*proof*⟩

$S_{\Phi_f} = f$ on $M_d(S)$.

theorem *arveson-functional-of-map*:
assumes *OS*: *operator-system n OS* **and** *f*: *lin-fun-on (Mp d n OS) f* **and** *A*: *A ∈ Mp d n OS* **and** *d*: $0 < d$
shows *arveson-functional n d (arveson-map n d f) A = f A*
⟨*proof*⟩

$\Phi_{S_\Phi} = \Phi$ on S.

theorem *arveson-map-of-functional*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS Φ* **and** *M*: *M ∈ OS* **and** *d*: $0 < d$
shows *arveson-map n d (arveson-functional n d Φ) M = Φ M*
⟨*proof*⟩

lemma *unit-block-add*:
assumes *M*: *M ∈ carrier-mat n n* **and** *M′*: *M′ ∈ carrier-mat n n*
shows *unit-block d n i j (M + M′) = unit-block d n i j M + unit-block d n i j M′*
⟨*proof*⟩

lemma *unit-block-smult*:
assumes *M*: *M ∈ carrier-mat n n*
shows *unit-block d n i j (c ·m M) = c ·m unit-block d n i j M*
⟨*proof*⟩

lemma *arveson-map-lin*:
assumes *OS*: *operator-system n OS* **and** *f*: *lin-fun-on (Mp d n OS) f*
shows *lin-on n d OS (arveson-map n d f)*
⟨*proof*⟩

lemma *arveson-functional-lin*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS Φ*
shows *lin-fun-on (Mp d n OS) (arveson-functional n d Φ)*
⟨*proof*⟩

lemma *quad-split*:
fixes M :: *complex Matrix.mat* **and** w :: *complex Matrix.vec*
assumes M: $M \in$ *carrier-mat* $(q * d)$ $(q * d)$ **and** w: *dim-vec* $w = q * d$
shows $\langle w \mid M *_v w\rangle = (\sum r{<}q.\ \sum i{<}d.\ \sum s{<}q.\ \sum j{<}d.$
$cnj\ (w\ \$\ (r * d + i)) * M\ \$\$\ (r * d + i,\ s * d + j) * w\ \$\ (s * d + j))$
⟨*proof*⟩

45.37 Theorem (Arveson): completely positive iff d-positive iff S_Φ positive

lemma *if-zero-mult*: $(if\ P\ then\ x\ else\ 0) * (y{::}complex) = (if\ P\ then\ x * y\ else\ 0)$
⟨*proof*⟩

lemma *mult-if-zero*: $(y{::}complex) * (if\ P\ then\ x\ else\ 0) = (if\ P\ then\ y * x\ else\ 0)$
⟨*proof*⟩

lemma *cnj-if-zero*: $cnj\ (if\ P\ then\ x\ else\ 0) = (if\ P\ then\ cnj\ x\ else\ 0)$
⟨*proof*⟩

lemma *nonneg-of-real-quad*:
fixes z :: *complex*
assumes $z \in \mathbb{R}$ **and** $0 \le Re\ z$ **and** d: $0 < d$
shows $0 \le z\ /\ of\text{-}nat\ d$
⟨*proof*⟩

lemma *diag-quad-amplification*:
fixes $d\ n$ :: *nat* **and** Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat* **and** A :: *complex Matrix.mat*
defines $v \equiv$ *Matrix.vec* $(d * d)$ $(\lambda x.\ if\ x\ div\ d = x\ mod\ d\ then\ 1\ else\ (0{::}complex))$
shows $\langle v \mid amplification\ n\ d\ d\ \Phi\ A *_v v\rangle = (\sum i{<}d.\ \sum j{<}d.\ \Phi\ (block\text{-}of\text{-}mat\ n\ A\ i\ j)\ \$\$\ (i,j))$
⟨*proof*⟩

lemma *dpos-imp-functional-pos*:
assumes *OS*: *operator-system n OS* **and** d: $0 < d$ **and** *P*: *p-positive-on n d d OS* Φ
shows *positive-fun-on* $(Mp\ d\ n\ OS)$ $(arveson\text{-}functional\ n\ d\ \Phi)$
⟨*proof*⟩

lemma *conj-tensor-index*:
fixes w :: *complex Matrix.vec*
assumes s: $s < q$ **and** k: $k < d$ **and** b': $b' < n$ **and** b: $b < n$
defines $V \equiv$ *Tensor.tensor-mat* $(Matrix.mat\ q\ d\ (\lambda(r,t).\ w\ \$\ (r * d + t)))\ (1_m\ n)$
shows $V\ \$\$\ (s * n + b',\ k * n + b) = (if\ b' = b\ then\ w\ \$\ (s * d + k)\ else\ 0)$
⟨*proof*⟩

lemma *conj-block*:
fixes A :: *complex Matrix.mat* **and** w :: *complex Matrix.vec*
assumes A: $A \in$ *carrier-mat* $(q * n)$ $(q * n)$ **and** i: $i < d$ **and** j: $j < d$ **and** a: $a < n$ **and** b: $b < n$
defines $V \equiv$ *Tensor.tensor-mat* $(Matrix.mat\ q\ d\ (\lambda(r,t).\ w\ \$\ (r * d + t)))\ (1_m\ n)$
shows $((V^\dagger) * A * V)\ \$\$\ (i * n + a,\ j * n + b)$
$= (\sum r{<}q.\ \sum s{<}q.\ cnj\ (w\ \$\ (r * d + i)) * w\ \$\ (s * d + j) * A\ \$\$\ (r * n + a,\ s * n + b))$
⟨*proof*⟩

lemma *functional-pos-imp-cp*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ **and** d: $0 < d$
and *Pos*: *positive-fun-on* $(Mp\ d\ n\ OS)$ $(arveson\text{-}functional\ n\ d\ \Phi)$
shows *cp-on n d OS* Φ
⟨*proof*⟩

Theorem (Arveson, Day 24): for linear $\Phi : S \to M_d$, completely positive $\iff$ d-positive $\iff$

S_Φ positive.

theorem *arveson-cp-iff*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ **and** *d*: *0 < d*
shows *cp-on n d OS* Φ ⟷ *p-positive-on n d d OS* Φ
and *p-positive-on n d d OS* Φ ⟷ *positive-fun-on* (*Mp d n OS*) (*arveson-functional n d* Φ)
⟨*proof*⟩

Theorem (Arveson correspondence, Day 25): $\Phi \mapsto S_\Phi$ maps $CP(S, M_d)$ onto the positive functionals on $M_d(S)$, with inverse $f \mapsto \Phi_f$; it is affine and sends unital maps to states.

theorem *arveson-correspondence*:
assumes *OS*: *operator-system n OS* **and** *d*: *0 < d*
shows *lin-on n d OS* Φ ⟹ *cp-on n d OS* Φ ⟷ *positive-fun-on* (*Mp d n OS*) (*arveson-functional n d* Φ)
and *lin-fun-on* (*Mp d n OS*) *f* ⟹ *cp-on n d OS* (*arveson-map n d f*) ⟷ *positive-fun-on* (*Mp d n OS*) *f*
and *lin-on n d OS* Φ ⟹ *M* ∈ *OS* ⟹ *arveson-map n d* (*arveson-functional n d* Φ) *M* = Φ *M*
and *lin-fun-on* (*Mp d n OS*) *f* ⟹ *A* ∈ *Mp d n OS* ⟹ *arveson-functional n d* (*arveson-map n d f*) *A* = *f A*
⟨*proof*⟩

lemma *arveson-functional-affine*:
assumes *OS*: *operator-system n OS* **and** *L1*: *lin-on n d OS* Φ **and** *L2*: *lin-on n d OS* Ψ **and** *A*: *A* ∈ *Mp d n OS*
shows *arveson-functional n d* (λ*M*. *t* ·$_m$ Φ *M* + (*1* − *t*) ·$_m$ Ψ *M*) *A*
= *t* ∗ *arveson-functional n d* Φ *A* + (*1* − *t*) ∗ *arveson-functional n d* Ψ *A*
⟨*proof*⟩

lemma *arveson-functional-unital*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ **and** *u*: Φ (*1*$_m$ *n*) = *1*$_m$ *d* **and** *d*: *0 < d*
shows *arveson-functional n d* Φ (*1*$_m$ (*d* ∗ *n*)) = *1*
⟨*proof*⟩

end
theory *Day25-Unital-Functional-Norm*
imports *Day25-Arveson-Correspondence*
HOL−Analysis.L2-Norm
begin

Day 25 proposition (D25-4): for an operator system $S \subseteq M_n$ and a linear $f : S \to \mathbb{C}$ with $f(I) = 1$, f is positive iff $\|f\| = 1$. The operator norm is $\|A\| = \sup\{\|Av\| : \|v\| \le 1\}$ and, as in the notes, $\|f\|$ is defined for bounded f. The proof differs from the notes in one step: for $P \ge 0$ the bound $\|P - sI + itI\|^2 \le s^2 + t^2$ ($s = \|P\|/2$) comes from $P^2 \le \|P\|\,P$, which uses the square root of P, instead of from the spectrum of P. Cauchy–Schwarz is *cauchy-schwarz-ineq* of Isabelle_Marries_Dirac.

definition *op-norm* :: *nat* ⇒ *complex Matrix.mat* ⇒ *real* **where**
op-norm n A = *Sup* {‖*A* ∗$_v$ *v*‖ | *v*. *dim-vec v* = *n* ∧ ‖*v*‖ ≤ *1*}

definition *fun-norm* :: *nat* ⇒ *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex*) ⇒ *real* **where**
fun-norm n TS f = *Sup* {*cmod* (*f A*) | *A*. *A* ∈ *TS* ∧ *op-norm n A* ≤ *1*}

45.38 Vectors

lemma *vnorm-nonneg*: ‖*v*‖ ≥ *0*
⟨*proof*⟩

lemma *inner-self-norm*: ⟨*v*|*v*⟩ = *complex-of-real* (‖*v*‖^2)

⟨*proof*⟩

lemma *cauchy-schwarz-norm*:
assumes *dim-vec* v = *dim-vec* w
shows *cmod* $\langle v|w\rangle \leq \|v\| * \|w\|$
⟨*proof*⟩

lemma *inner-add-smult-expand*:
assumes *dim-vec* u = *dim-vec* x
shows $\langle u + z \cdot_v x \mid u + z \cdot_v x\rangle = \langle u|u\rangle + z * \langle u|x\rangle + cnj\ z * \langle x|u\rangle + cnj\ z * z * \langle x|x\rangle$
⟨*proof*⟩

lemma *smult-mat-vec*: $A \in$ *carrier-mat* $n\ m \Longrightarrow$ *dim-vec* $v = m \Longrightarrow (c \cdot_m A) *_v v = c \cdot_v (A *_v v)$
⟨*proof*⟩

lemma *entry-le-vnorm*: $i <$ *dim-vec* $v \Longrightarrow$ *cmod* $(v\ \$\ i) \leq \|v\|$
⟨*proof*⟩

lemma *vnorm-le-sum*: $\|w\| \leq (\sum i{<}$*dim-vec* w. *cmod* $(w\ \$\ i))$
⟨*proof*⟩

lemma *vnorm-zero-iff*: *dim-vec* $v = n \Longrightarrow \|v\| = 0 \longleftrightarrow v = 0_v\ n$
⟨*proof*⟩

45.39 The operator norm

lemma *mult-vec-norm-bound*:
assumes A: $A \in$ *carrier-mat* $n\ n$ **and** v: *dim-vec* $v = n$
shows $\|A *_v v\| \leq (\sum i{<}n.\ \sum k{<}n.$ *cmod* $(A\ \$\$\ (i,k))) * \|v\|$
⟨*proof*⟩

lemma *op-norm-bdd*:
assumes A: $A \in$ *carrier-mat* $n\ n$
shows *bdd-above* $\{\|A *_v v\| \mid v.$ *dim-vec* $v = n \wedge \|v\| \leq 1\}$
⟨*proof*⟩

lemma *mult-zero-vec'*: $A \in$ *carrier-mat* $n\ n \Longrightarrow A *_v 0_v\ n = 0_v\ n$
⟨*proof*⟩

lemma *op-norm-ge*:
assumes A: $A \in$ *carrier-mat* $n\ n$ **and** v: *dim-vec* $v = n$ $\|v\| \leq 1$
shows $\|A *_v v\| \leq$ *op-norm* $n\ A$
⟨*proof*⟩

lemma *op-norm-nonneg*:
assumes A: $A \in$ *carrier-mat* $n\ n$
shows $0 \leq$ *op-norm* $n\ A$
⟨*proof*⟩

lemma *op-norm-mult*:
assumes A: $A \in$ *carrier-mat* $n\ n$ **and** v: *dim-vec* $v = n$
shows $\|A *_v v\| \leq$ *op-norm* $n\ A * \|v\|$
⟨*proof*⟩

lemma *op-norm-le*:
assumes A: $A \in$ *carrier-mat* $n\ n$ **and** c: $0 \leq c$ **and** b: $\bigwedge v.$ *dim-vec* $v = n \Longrightarrow \|A *_v v\| \leq c * \|v\|$
shows *op-norm* $n\ A \leq c$
⟨*proof*⟩

lemma *op-norm-one*:
assumes *n*: $0 < n$
shows *op-norm n* $(1_m\ n) = 1$
⟨*proof*⟩

lemma *op-norm-smult-le*:
assumes *A*: $A \in$ *carrier-mat n n*
shows *op-norm n* $(c \cdot_m A) \leq$ *cmod c* $*$ *op-norm n A*
⟨*proof*⟩

lemma *op-norm-zero-imp*:
assumes *A*: $A \in$ *carrier-mat n n* **and** *z*: *op-norm n A* $= 0$
shows $A = 0_m\ n\ n$
⟨*proof*⟩

45.40 Unital functionals

context
fixes *n* :: *nat* **and** *TS* :: *complex Matrix.mat set* **and** *f* :: *complex Matrix.mat* $\Rightarrow$ *complex*
assumes *OS*: *operator-system n TS* **and** *L*: *lin-fun-on TS f* **and** *f1*: *f* $(1_m\ n) = 1$
begin

lemma *TSc*: $A \in TS \Longrightarrow A \in$ *carrier-mat n n* ⟨*proof*⟩

lemma *lin2*: $A \in TS \Longrightarrow B \in TS \Longrightarrow f\ (a \cdot_m A + b \cdot_m B) = a * f\ A + b * f\ B$
⟨*proof*⟩

lemma *comb-in*: $A \in TS \Longrightarrow B \in TS \Longrightarrow a \cdot_m A + b \cdot_m B \in TS$
⟨*proof*⟩

lemma *f-zero*: *f* $(0_m\ n\ n) = 0$
⟨*proof*⟩

A Hermitian $A \in S$ with $|\Re\langle x|Ax\rangle| \leq c\|x\|^2$ satisfies $cI \pm A \geq 0$, so a positive f has $f(A)$ real with $|f(A)| \leq c$.

lemma *herm-bound*:
assumes *pos*: *positive-fun-on TS f* **and** *A*: $A \in$ *TS hermitian A* **and** *c*: $0 \leq c$
and *b*: $\bigwedge x$. *dim-vec x* $= n \Longrightarrow |Re\ \langle x|A *_v x\rangle| \leq c * \|x\|^2$
shows *Im* $(f\ A) = 0$ **and** $|Re\ (f\ A)| \leq c$
⟨*proof*⟩

lemma *adjoint-inner*:
assumes *A*: $A \in$ *carrier-mat n n* **and** *x*: *dim-vec x* $= n$
shows $\langle x|A^\dagger *_v x\rangle =$ *cnj* $\langle x|A *_v x\rangle$
⟨*proof*⟩

($\Rightarrow$): a positive unital f has $|f(A)| \leq \|A\|$.

lemma *positive-imp-bound*:
assumes *pos*: *positive-fun-on TS f* **and** *A*: $A \in TS$
shows *cmod* $(f\ A) \leq$ *op-norm n A*
⟨*proof*⟩

($\Leftarrow$): if $|f(A)| \leq \|A\|$ on S then f is positive.

lemma *sq-le-norm-quad*:
assumes *P*: $P \in$ *carrier-mat n n psd P* **and** *x*: *dim-vec x* $= n$
shows $\|P *_v x\|^2 \leq$ *op-norm n P* $*$ *Re* $\langle x|P *_v x\rangle$

⟨*proof*⟩

lemma *bound-imp-positive*:
assumes *n*: *0* < *n* **and** *b*: ∀ *A* ∈ *TS*. *cmod* (*f A*) ≤ *op-norm n A*
shows *positive-fun-on TS f*
⟨*proof*⟩

lemma *fun-norm-one-iff*:
assumes *n*: *0* < *n* **and** *bdd*: ∃ *C*. ∀ *A* ∈ *TS*. *cmod* (*f A*) ≤ *C* ∗ *op-norm n A*
shows *fun-norm n TS f* = *1* ⟷ (∀ *A* ∈ *TS*. *cmod* (*f A*) ≤ *op-norm n A*)
⟨*proof*⟩

D25-4.

theorem *unital-positive-iff-norm-one*:
assumes *n*: *0* < *n* **and** *bdd*: ∃ *C*. ∀ *A* ∈ *TS*. *cmod* (*f A*) ≤ *C* ∗ *op-norm n A*
shows *positive-fun-on TS f* ⟷ *fun-norm n TS f* = *1*
⟨*proof*⟩

end

end
theory *Day26-Arveson-Extension*
imports *Day25-Arveson-Correspondence Vern-Paulsen-QC-3-10-Lmat.Separable-Cone*
begin

Day 26: Arveson's extension theorem. The notes extend the state S_Φ by Hahn–Banach (norm one) and use that a unital functional of norm one is positive. Here the positive functional S_Φ on the operator system $M_d(S)$ is extended to a positive functional on $M_d(M_n)$ directly, by separating the positive definite cone from the kernel of S_Φ (*separating-hyperplane-set-0* in the Euclidean space of typed matrices); the extension is $X \mapsto \mathrm{Tr}(DX)$ with $D \geq 0$. Dimensions are cardinalities of finite types, as in *Separable-Cone*.

45.41 Auxiliary facts

lemma *operator-system-herm-parts*:
assumes *OS*: *operator-system n OS* **and** *A*: *A* ∈ *OS*
shows ∃ *Hp Kp*. *Hp* ∈ *OS* ∧ *Kp* ∈ *OS* ∧ *hermitian Hp* ∧ *hermitian Kp* ∧ *A* = *Hp* + i ·$_m$ *Kp*
⟨*proof*⟩

lemma *Tr-dagger*:
assumes *A*: *A* ∈ *carrier-mat n n*
shows *Tr* (*A*†) = *cnj* (*Tr A*)
⟨*proof*⟩

lemma *Tr-herm-mult-real*:
assumes *Wc*: *W* ∈ *carrier-mat n n* **and** *Mc*: *M* ∈ *carrier-mat n n* **and** *Wh*: *hermitian W* **and** *Mh*: *hermitian M*
shows *Tr* (*W* ∗ *M*) ∈ ℝ
⟨*proof*⟩

lemma *Tr-mult-outer*:
fixes *A* :: *complex Matrix.mat* **and** *v* :: *complex Matrix.vec*
assumes *A*: *A* ∈ *carrier-mat n n* **and** *v*: *dim-vec v* = *n*
shows *Tr* (*A* ∗ (*v* ⊘ *v*)) = ⟨*v* | *A* ∗$_v$ *v*⟩
⟨*proof*⟩

lemma *Tr-psd-mult-nonneg*:

assumes *Dc*: $D \in$ *carrier-mat n n* **and** *Pc*: $P \in$ *carrier-mat n n* **and** *Dp*: *psd D* **and** *Pp*: *psd P*
shows $0 \leq Tr\ (D * P)$
⟨*proof*⟩

lemma *herm-entry*:
assumes $M \in$ *carrier-mat n n hermitian M* $i < n$ $j < n$
shows $M\ \$\$\ (i,j) = cnj\ (M\ \$\$\ (j,i))$
⟨*proof*⟩

lemma *herm-real-comb*:
assumes *Ac*: $A \in$ *carrier-mat n n* **and** *Bc*: $B \in$ *carrier-mat n n* **and** *Ah*: *hermitian A* **and** *Bh*: *hermitian B*
shows *hermitian* (*complex-of-real u* $\cdot_m$ *A* + *complex-of-real v* $\cdot_m$ *B*)
⟨*proof*⟩

lemma *herm-real-smult*:
assumes *Ac*: $A \in$ *carrier-mat n n* **and** *Ah*: *hermitian A*
shows *hermitian* (*complex-of-real t* $\cdot_m$ *A*)
⟨*proof*⟩

lemma *hermitian-one*: *hermitian* (1_m *n*)
⟨*proof*⟩

lemma *lin-t-zero*:
fixes *b c* :: *real*
assumes $\forall t.\ 0 \leq c + t * b$
shows $b = 0$
⟨*proof*⟩

lemma *eps-nonneg*:
fixes *x y* :: *real*
assumes $\forall \varepsilon > 0.\ 0 \leq x + \varepsilon * y$
shows $0 \leq x$
⟨*proof*⟩

lemma *psd-one-mat*: *psd* (1_m *n*)
⟨*proof*⟩

45.42 Matrices as points of the Euclidean space *lmat*

lemma *unwrap-wrap-lm*:
fixes *M* :: *complex Matrix.mat*
assumes *M* ∈ *carrier-mat CARD('k::finite) CARD('k)*
shows *unwrap-lm* (*wrap-lm M* :: (*complex*, *'k*, *'k*) *lmat*) = *M*
⟨*proof*⟩

lemma *wrap-lm-add*:
fixes *M M'* :: *complex Matrix.mat*
assumes *M*: *M* ∈ *carrier-mat CARD('k::finite) CARD('k)* **and** *M'*: *M'* ∈ *carrier-mat CARD('k) CARD('k)*
shows (*wrap-lm* (*M* + *M'*) :: (*complex*, *'k*, *'k*) *lmat*) = *wrap-lm M* + *wrap-lm M'*
⟨*proof*⟩

lemma *wrap-lm-scale*:
fixes *M* :: *complex Matrix.mat*
assumes *M*: *M* ∈ *carrier-mat CARD('k::finite) CARD('k)*
shows (*wrap-lm* (*complex-of-real r* $\cdot_m$ *M*) :: (*complex*, *'k*, *'k*) *lmat*) = *r* $*_R$ *wrap-lm M*
⟨*proof*⟩

lemma *wrap-lm-zero*: (*wrap-lm* (0_m *CARD*('k::*finite*) *CARD*('k)) :: (*complex*, 'k, 'k) *lmat*) = *0*
⟨*proof*⟩

45.43 Extension of positive functionals

A positive functional on an operator system $T \subseteq M_N$ is $X \mapsto \mathrm{Tr}(DX)$ on T for some $D \geq 0$; hence it extends to a positive functional on M_N.

theorem *positive-functional-extension*:
fixes *TS* :: *complex Matrix.mat set* **and** *f* :: *complex Matrix.mat* ⇒ *complex*
assumes *TS*: *operator-system CARD*('k::*finite*) *TS* **and** *f*: *lin-fun-on TS f* **and** *fpos*: *positive-fun-on TS f*
shows ∃ *D*. *D* ∈ *carrier-mat CARD*('k) *CARD*('k) ∧ *psd D* ∧ (∀ *A* ∈ *TS*. *f A* = *Tr* (*D* ∗ *A*))
⟨*proof*⟩

45.44 Arveson's extension theorem

theorem *arveson-extension*:
fixes *OS* :: *complex Matrix.mat set* **and** Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *OS*: *operator-system CARD*('n::*finite*) *OS* **and** *L*: *lin-on CARD*('n) *CARD*('d::*finite*) *OS* Φ
and *CP*: *cp-on CARD*('n) *CARD*('d) *OS* Φ
shows ∃ Ψ. *lin-on CARD*('n) *CARD*('d) (*carrier-mat CARD*('n) *CARD*('n)) Ψ
∧ *completely-positive CARD*('n) *CARD*('d) Ψ ∧ (∀ *M* ∈ *OS*. Ψ *M* = Φ *M*)
⟨*proof*⟩

Corollary: a completely positive map on an operator system has a Kraus form.

corollary *arveson-kraus*:
fixes *OS* :: *complex Matrix.mat set* **and** Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *OS*: *operator-system CARD*('n::*finite*) *OS* **and** *L*: *lin-on CARD*('n) *CARD*('d::*finite*) *OS* Φ
and *CP*: *cp-on CARD*('n) *CARD*('d) *OS* Φ
shows ∃ *Ws*. (∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat CARD*('d) *CARD*('n)) ∧ (∀ *M* ∈ *OS*. Φ *M* = *kraus-map-dim CARD*('d) *Ws M*)
⟨*proof*⟩

end

Part IV
Functional-analytic infrastructure (Lecture 25: Hahn–Banach)

theory *Matrix-Typed*
imports
HOL−Hahn-Banach.Hahn-Banach
Jordan-Normal-Form.Matrix
HOL−Library.Cardinality
Vern-Paulsen-QC-3-10.Basic-Math
begin

46 Typed-dimension matrix wrapper

The type ('a, 'm :: *finite*, 'n :: *finite*) *lmat* represents matrices of size $CARD('m) \times CARD('n)$ with entries in 'a, by typedef of the JNF matrix type at the appropriate dimension.

typedef (**overloaded**) ('a, 'm :: *finite*, 'n :: *finite*) *lmat* =
  $\{M :: 'a$ *Matrix.mat*. *dim-row* $M = CARD('m) \wedge$ *dim-col* $M = CARD('n)\}$
  morphisms *unwrap-lm wrap-lm*
⟨*proof*⟩

setup-lifting *type-definition-lmat*

lemma *unwrap-lm-dim-row* [*simp*]:
  dim-row (*unwrap-lm* (M :: ('a, 'm :: *finite*, 'n :: *finite*) *lmat*)) = $CARD('m)$
  ⟨*proof*⟩

lemma *unwrap-lm-dim-col* [*simp*]:
  dim-col (*unwrap-lm* (M :: ('a, 'm :: *finite*, 'n :: *finite*) *lmat*)) = $CARD('n)$
  ⟨*proof*⟩

lemma *unwrap-lm-carrier* [*simp*]:
  unwrap-lm (M :: ('a, 'm :: *finite*, 'n :: *finite*) *lmat*)
    ∈ *carrier-mat* ($CARD('m)$) ($CARD('n)$)
  ⟨*proof*⟩

47 Typeclass instances

The unique zero matrix of the right shape.

instantiation *lmat* :: (*zero*, *finite*, *finite*) *zero*
begin
lift-definition *zero-lmat* :: ('a, 'm :: *finite*, 'n :: *finite*) *lmat*
  is 0_m ($CARD('m)$) ($CARD('n)$) ⟨*proof*⟩
instance ⟨*proof*⟩
end

lemma *unwrap-lm-zero* [*simp*]:
  unwrap-lm (0 :: ('a :: *zero*, 'm :: *finite*, 'n :: *finite*) *lmat*)
    = 0_m ($CARD('m)$) ($CARD('n)$)
  ⟨*proof*⟩

instantiation *lmat* :: (*plus*, *finite*, *finite*) *plus*
begin
lift-definition *plus-lmat* ::
  ('a, 'm :: *finite*, 'n :: *finite*) *lmat* ⇒ ('a, 'm :: *finite*, 'n :: *finite*) *lmat* ⇒ ('a, 'm :: *finite*, 'n :: *finite*) *lmat*
  is (+) ⟨*proof*⟩
instance ⟨*proof*⟩
end

lemma *unwrap-lm-plus* [*simp*]:
  unwrap-lm ((A :: ('a :: *plus*, 'm :: *finite*, 'n :: *finite*) *lmat*) + B)
    = *unwrap-lm* A + *unwrap-lm* B
  ⟨*proof*⟩

instantiation *lmat* :: (*uminus*, *finite*, *finite*) *uminus*
begin
lift-definition *uminus-lmat* ::

('a, 'm :: finite, 'n :: finite) lmat ⇒ ('a, 'm :: finite, 'n :: finite) lmat
is uminus ⟨proof⟩
instance ⟨proof⟩
end

lemma unwrap-lm-uminus [simp]:
unwrap-lm (− (A :: ('a :: uminus, 'm :: finite, 'n :: finite) lmat))
= − unwrap-lm A
⟨proof⟩

instantiation lmat :: (minus, finite, finite) minus
begin
lift-definition minus-lmat ::
('a, 'm :: finite, 'n :: finite) lmat ⇒ ('a, 'm :: finite, 'n :: finite) lmat ⇒ ('a, 'm :: finite, 'n :: finite) lmat
is (−) ⟨proof⟩
instance ⟨proof⟩
end

lemma unwrap-lm-minus [simp]:
unwrap-lm ((A :: ('a :: minus, 'm :: finite, 'n :: finite) lmat) − B)
= unwrap-lm A − unwrap-lm B
⟨proof⟩

Lift the additive structure laws via ⟦⋀i j. ⟦i < dim-row ?B; j < dim-col ?B⟧ ⟹ ?A $$ (i, j) = ?B $$ (i, j); dim-row ?A = dim-row ?B; dim-col ?A = dim-col ?B⟧ ⟹ ?A = ?B on the underlying JNF matrices.

instance lmat :: (semigroup-add, finite, finite) semigroup-add
⟨proof⟩

instance lmat :: (ab-semigroup-add, finite, finite) ab-semigroup-add
⟨proof⟩

instance lmat :: (monoid-add, finite, finite) monoid-add
⟨proof⟩

instance lmat :: (comm-monoid-add, finite, finite) comm-monoid-add
⟨proof⟩

instance lmat :: (group-add, finite, finite) group-add
⟨proof⟩

instance lmat :: (ab-group-add, finite, finite) ab-group-add
⟨proof⟩

48 Real scalar multiplication and Hahn-Banach overloading

HOL's (·) is the abstract real-scalar multiplication used by the Hahn-Banach locales. We overload it on the type (complex, 'm, 'n) lmat as multiplication by the embedded real, lifted from JNF's ($\cdot_m$).

overloading
prod-lmat ≡
Vector-Space.prod ::
real ⇒ (complex, 'm :: finite, 'n :: finite) lmat
⇒ (complex, 'm, 'n) lmat
begin

definition *prod-lmat* ::
  real ⇒ (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
        ⇒ (*complex*, *'m*, *'n*) *lmat*
where
  prod-lmat r M = *wrap-lm* (*complex-of-real r* ·$_m$ *unwrap-lm M*)
end

lemma *unwrap-lm-prod* [*simp*]:
  unwrap-lm (*Vector-Space.prod r* (*M* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*))
    = *complex-of-real r* ·$_m$ *unwrap-lm M*
  ⟨*proof*⟩

49 Real-vectorspace locale on the full type

Every (*complex*, *'m*, *'n*) *lmat* with the universe set is a real vector space in HOL/Hahn_Banach's sense.

lemma *vectorspace-UNIV-lmat*:
  Vector-Space.vectorspace (*UNIV* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*)
⟨*proof*⟩

50 The Hilbert-Schmidt norm on (*complex*, *'m*, *'n*) *lmat*

The HS / Frobenius norm: $\|M\|_H S$ = *sqrt* (*Re* ⟨⟨*M* | *M*⟩⟩). We define it on (*complex*, *'m*, *'n*) *lmat* using the underlying matrix's HS inner product from *Basic-Math*.

definition *hs-norm-lmat* ::
  (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat* ⇒ *real*
where
  hs-norm-lmat M = *sqrt* (*Re* (⟨⟨ *unwrap-lm M* | *unwrap-lm M* ⟩⟩))

lemma *hs-norm-lmat-nonneg*: *0* ≤ *hs-norm-lmat M*
  ⟨*proof*⟩

lemma *hs-norm-lmat-sq*:
  (*hs-norm-lmat M*)^2 = *Re* (⟨⟨ *unwrap-lm M* | *unwrap-lm M* ⟩⟩)
⟨*proof*⟩

lemma *hs-norm-lmat-smult*:
  hs-norm-lmat (*Vector-Space.prod a M*) = |*a*| ∗ *hs-norm-lmat M*
⟨*proof*⟩

51 Triangle inequality for the HS norm

Cauchy-Schwarz on the HS inner product (real form), used in the triangle inequality proof.

lemma *hs-inner-prod-real-self*:
  ⟨⟨ *A* | *A* ⟩⟩ = *complex-of-real* (*Re* (⟨⟨ *A* | *A* ⟩⟩))
  ⟨*proof*⟩

lemma *hs-inner-prod-left-add*:
  assumes *dim-row A1* = *dim-row B* **and** *dim-col A1* = *dim-col B*
    and *dim-row A2* = *dim-row B* **and** *dim-col A2* = *dim-col B*
  shows ⟨⟨ *A1* + *A2* | *B* ⟩⟩ = ⟨⟨ *A1* | *B* ⟩⟩ + ⟨⟨ *A2* | *B* ⟩⟩
⟨*proof*⟩

lemma *hs-cauchy-schwarz-re*:

fixes A B :: *complex Matrix.mat*
assumes *dr*: *dim-row* A = *dim-row* B **and** *dc*: *dim-col* A = *dim-col* B
shows $(Re\ (\langle\langle\ A\ |\ B\ \rangle\rangle))^2 \leq Re\ (\langle\langle\ A\ |\ A\ \rangle\rangle) * Re\ (\langle\langle\ B\ |\ B\ \rangle\rangle)$
⟨*proof*⟩

lemma *hs-norm-lmat-triangle*:
fixes M N :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
shows *hs-norm-lmat* $(M + N) \leq$ *hs-norm-lmat* M + *hs-norm-lmat* N
⟨*proof*⟩

52 Subspace, seminorm, normed-vectorspace instances

HS norm is a seminorm on the whole real vector space of (*complex*, *'m*, *'n*) *lmat*.

lemma *seminorm-UNIV-hs-norm-lmat*:
seminorm (*UNIV* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*) *hs-norm-lmat*
⟨*proof*⟩

HS norm is a (real) norm: it vanishes only on the zero matrix.

lemma *hs-norm-lmat-zero-iff*:
fixes M :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
shows (*hs-norm-lmat* $M = 0$) $\longleftrightarrow$ $(M = 0)$
⟨*proof*⟩

lemma *norm-UNIV-hs-norm-lmat*:
Normed-Space.norm
(*UNIV* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*) *hs-norm-lmat*
⟨*proof*⟩

lemma *normed-vectorspace-UNIV-lmat*:
normed-vectorspace
(*UNIV* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*) *hs-norm-lmat*
⟨*proof*⟩

A complex-linear subspace V of (*complex*, *'m*, *'n*) *lmat* (i.e. nonempty, closed under addition and complex-scalar multiplication) is in particular a real-linear subspace, hence a *subspace* of *UNIV* in HOL/Hahn_Banach's sense.

definition *cmat-subspace-lmat* ::
(*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set* ⇒ *bool*
where
cmat-subspace-lmat $V \equiv$
$V \neq \{\} \wedge$
$(\forall\, x \in V.\ \forall\, y \in V.\ x + y \in V) \wedge$
$(\forall\, x \in V.\ \forall\, c$ :: *complex*.
wrap-lm $(c \cdot_m$ *unwrap-lm* $x) \in V)$

lemma *subspace-of-cmat-subspace-lmat*:
fixes V :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*
assumes *cmat-subspace-lmat* V
shows *Subspace.subspace* V (*UNIV* :: (*complex*, *'m*, *'n*) *lmat set*)
⟨*proof*⟩

53 Complex scalar multiplication on lmat

Complex scalar multiplication on (*complex*, *'m*, *'n*) *lmat*, extending the *Vector-Space.prod* overload (which is only by reals).

definition *cmult-lmat* ::
  complex ⇒ (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
    ⇒ (*complex*, *'m*, *'n*) *lmat*
where
  cmult-lmat c M = *wrap-lm* (*c* ·$_m$ *unwrap-lm M*)

lemma *unwrap-lm-cmult* [*simp*]:
  unwrap-lm (*cmult-lmat c* (*M* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*))
    = *c* ·$_m$ *unwrap-lm M*
  ⟨*proof*⟩

lemma *cmult-lmat-real*:
  cmult-lmat (*complex-of-real a*) *M* = *Vector-Space.prod a M*
  ⟨*proof*⟩

lemma *cmult-lmat-add*:
  fixes *x y* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
  shows *cmult-lmat c* (*x* + *y*) = *cmult-lmat c x* + *cmult-lmat c y*
⟨*proof*⟩

lemma *cmult-lmat-scalar-add*:
  fixes *x* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
  shows *cmult-lmat* (*c* + *d*) *x* = *cmult-lmat c x* + *cmult-lmat d x*
⟨*proof*⟩

lemma *cmult-lmat-scalar-mult*:
  fixes *x* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
  shows *cmult-lmat* (*c* ∗ *d*) *x* = *cmult-lmat c* (*cmult-lmat d x*)
⟨*proof*⟩

lemma *cmult-lmat-one*:
  fixes *x* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
  shows *cmult-lmat 1 x* = *x*
⟨*proof*⟩

lemma *cmult-lmat-uminus-one*:
  fixes *x* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
  shows *cmult-lmat* (− *1*) *x* = − *x*
⟨*proof*⟩

lemma *cmult-lmat-i-sq*:
  fixes *x* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
  shows *cmult-lmat* i (*cmult-lmat* i *x*) = − *x*
  ⟨*proof*⟩

lemma *hs-norm-lmat-cmult*:
  fixes *M* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
  shows *hs-norm-lmat* (*cmult-lmat c M*) = *cmod c* ∗ *hs-norm-lmat M*
⟨*proof*⟩

lemma *hs-norm-lmat-cmult-i* [*simp*]:
  fixes *M* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
  shows *hs-norm-lmat* (*cmult-lmat* i *M*) = *hs-norm-lmat M*
  ⟨*proof*⟩

54 Day 25 (26 October 2011): the Hahn–Banach theorem

Definition (notes). Let W be a normed space. A linear functional $f : W \to \mathbb{C}$ is *bounded* if $|f(w)| \le C\|w\|$ for some constant C; the least such C is the norm $\|f\| = \sup\{|f(w)| : \|w\| \le 1\}$.

Theorem (notes) (Hahn–Banach)**.** Let W be a normed space, $V \subseteq W$ a subspace and $g : V \to \mathbb{C}$ a bounded linear functional. Then there is a bounded linear functional $f : W \to \mathbb{C}$ with $\|f\| = \|g\|$ and $f(v) = g(v)$ for all $v \in V$.

In the notes the theorem is used (Day 26) to extend the state S_Φ from $M_d(S)$ to $M_d(M_n)$ in the proof of Arveson's extension theorem, which is not formalised. Here it is proved for $W = (\mathbb{C}$-valued $m \times n$ matrices) with the Hilbert–Schmidt norm: HOL's Hahn–Banach gives a real-linear norm-preserving extension of $\Re g$, and the Bohnenblust–Sobczyk construction $f(x) = F(x) - iF(ix)$ recovers a complex-linear extension with the same bound.

55 Bounded complex-linear functionals on lmat

definition *clinear-functional-on* ::
  (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*
  ⇒ ((*complex*, *'m*, *'n*) *lmat* ⇒ *complex*) ⇒ *bool*
where
  clinear-functional-on V g ≡
    (∀ *x* ∈ *V*. ∀ *y* ∈ *V*. *g* (*x* + *y*) = *g x* + *g y*) ∧
    (∀ *x* ∈ *V*. ∀ *c*. *g* (*cmult-lmat c x*) = *c* ∗ *g x*)

definition *bounded-clinear-functional-on* ::
  (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*
  ⇒ ((*complex*, *'m*, *'n*) *lmat* ⇒ *complex*) ⇒ *bool*
where
  bounded-clinear-functional-on V g ≡
    clinear-functional-on V g ∧
    (∃ *C* ≥ *0*. ∀ *x* ∈ *V*. *cmod* (*g x*) ≤ *C* ∗ *hs-norm-lmat x*)

56 Real-part functional and linearform/continuous instances

definition *re-part-lmat* ::
  ((*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat* ⇒ *complex*)
  ⇒ ((*complex*, *'m*, *'n*) *lmat* ⇒ *real*)
where
  re-part-lmat g x = *Re* (*g x*)

lemma *linearform-re-part-lmat*:
  fixes *V* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*
    and *g* :: (*complex*, *'m*, *'n*) *lmat* ⇒ *complex*
  assumes *V-sub*: *Subspace.subspace V UNIV*
    and *g-clin*: *clinear-functional-on V g*
  shows *linearform V* (*re-part-lmat g*)
⟨*proof*⟩

lemma *continuous-re-part-lmat*:
  fixes *V* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*
    and *g* :: (*complex*, *'m*, *'n*) *lmat* ⇒ *complex*
  assumes *V-sub*: *Subspace.subspace V UNIV*
    and *g-bnd*: *bounded-clinear-functional-on V g*
  shows *continuous V* (*re-part-lmat g*) *hs-norm-lmat*
⟨*proof*⟩

57 Application of HOL/Hahn_Banach (norm-preserving extension)

Apply HOL/Hahn_Banach to extend the real-linearform *re-part-lmat g* to all of UNIV preserving the function norm.

theorem *hahn-banach-re-part-lmat*:
  fixes V :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat set*
    and g :: (*complex*, $'m$, $'n$) *lmat* $\Rightarrow$ *complex*
  assumes *V-sub*: *Subspace.subspace V* (*UNIV* :: (*complex*, $'m$, $'n$) *lmat set*)
    and *g-bnd*: *bounded-clinear-functional-on V g*
  shows
    $\exists F$ :: (*complex*, $'m$, $'n$) *lmat* $\Rightarrow$ *real*.
      linearform UNIV F
    $\wedge$ *continuous UNIV F hs-norm-lmat*
    $\wedge$ ($\forall x \in V$. *F x* = *re-part-lmat g x*)
⟨*proof*⟩

58 Bohnenblust-Sobczyk: complex extension from real extension

Given a real-linear extension F of *re-part-lmat g* on all of UNIV, define the complex-linear extension $f\,x = F\,x - i \cdot F\,(i \cdot x)$.

definition *bs-complex-extension* ::
  ((*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat* $\Rightarrow$ *real*)
  $\Rightarrow$ ((*complex*, $'m$, $'n$) *lmat* $\Rightarrow$ *complex*)
where
  bs-complex-extension F x =
    complex-of-real (*F x*) − i ∗ *complex-of-real* (*F* (*cmult-lmat* i *x*))

lemma *bs-complex-extension-add*:
  fixes F :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat* $\Rightarrow$ *real*
  assumes *F-lin*: *linearform UNIV F*
  shows *bs-complex-extension F* (*x* + *y*)
      = *bs-complex-extension F x* + *bs-complex-extension F y*
⟨*proof*⟩

lemma *bs-complex-extension-real-smult*:
  fixes F :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat* $\Rightarrow$ *real*
  assumes *F-lin*: *linearform UNIV F*
  shows *bs-complex-extension F* (*Vector-Space.prod a x*)
      = *complex-of-real a* ∗ *bs-complex-extension F x*
⟨*proof*⟩

lemma *bs-complex-extension-i-smult*:
  fixes F :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat* $\Rightarrow$ *real*
  assumes *F-lin*: *linearform UNIV F*
  shows *bs-complex-extension F* (*cmult-lmat* i *x*)
      = i ∗ *bs-complex-extension F x*
⟨*proof*⟩

lemma *bs-complex-extension-csmult*:
  fixes F :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat* $\Rightarrow$ *real*
  assumes *F-lin*: *linearform UNIV F*
  shows *bs-complex-extension F* (*cmult-lmat c x*)
      = *c* ∗ *bs-complex-extension F x*

⟨*proof*⟩

lemma *bs-complex-extension-clinear*:
fixes F :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat* ⇒ *real*
assumes *F-lin*: *linearform UNIV F*
shows *clinear-functional-on UNIV* (*bs-complex-extension F*)
⟨*proof*⟩

The exact Bohnenblust-Sobczyk bound: any C bound on F lifts verbatim to a bound on *bs-complex-extension F*. This is the standard $e^{\wedge}\{-i\vartheta\}$-rotation argument used to prove the complex Hahn-Banach theorem from the real one.

lemma *bs-complex-extension-tight-bound*:
fixes F :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat* ⇒ *real*
assumes *F-lin*: *linearform UNIV F*
and *F-C*: $\bigwedge x.\ |F\ x| \leq C * \textit{hs-norm-lmat}\ x$
shows *cmod* (*bs-complex-extension F x*) ≤ C * *hs-norm-lmat x*
⟨*proof*⟩

lemma *bs-complex-extension-bounded*:
fixes F :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat* ⇒ *real*
assumes *F-lin*: *linearform UNIV F*
and *F-cont*: *continuous UNIV F hs-norm-lmat*
shows $\exists\, C \geq 0.\ \forall x.$ *cmod* (*bs-complex-extension F x*) ≤ C * *hs-norm-lmat x*
⟨*proof*⟩

lemma *bs-complex-extension-extends*:
fixes V :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat set*
and g :: (*complex*, $'m$, $'n$) *lmat* ⇒ *complex*
and F :: (*complex*, $'m$, $'n$) *lmat* ⇒ *real*
assumes *g-clin*: *clinear-functional-on V g*
and *V-clos-i*: $\forall\, x \in V.$ *cmult-lmat* i $x \in V$
and *F-ext*: $\forall\, x \in V.\ F\ x =$ *re-part-lmat g x*
shows $\forall\, x \in V.$ *bs-complex-extension F x* = *g x*
⟨*proof*⟩

59 The matrix-space Hahn-Banach theorem

Combine all the steps: a bounded complex-linear functional on a complex-linear subspace V of (*complex*, $'m$, $'n$) *lmat* extends to a bounded complex-linear functional on all of (*complex*, $'m$, $'n$) *lmat* (without claiming norm preservation; for the application to Arveson Extension this is enough together with the bound chase).

theorem *matrix-hahn-banach-lmat*:
fixes V :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat set*
and g :: (*complex*, $'m$, $'n$) *lmat* ⇒ *complex*
assumes *V-csub*: *cmat-subspace-lmat V*
and *g-bnd*: *bounded-clinear-functional-on V g*
shows $\exists f$ :: (*complex*, $'m$, $'n$) *lmat* ⇒ *complex*.
bounded-clinear-functional-on UNIV f
$\wedge\ (\forall\, x \in V.\ f\ x = g\ x)$
⟨*proof*⟩

The explicit-bound form of the matrix Hahn-Banach theorem: any C-bound on $|Re \circ g|$ over the subspace V lifts verbatim to a C-bound on *cmod* ∘ f over the whole space.

The proof unfolds HOL/Hahn_Banach's locale-local *fn-norm* to its *Sup*-based definition, bounds the supremum on the subspace side by C via *g-re-C*, transports the bound to the extension side

via *norm-Hahn-Banach*'s norm-preservation conclusion, then uses *bs-complex-extension-tight-bound* to turn the bound on the real extension into a bound on the complex extension.

theorem *matrix-hahn-banach-lmat-with-bound*:
  fixes V :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat set*
    and g :: (*complex*, $'m$, $'n$) *lmat* $\Rightarrow$ *complex*
    and C :: *real*
  assumes *V-csub*: *cmat-subspace-lmat V*
    and *g-clin*: *clinear-functional-on V g*
    and *g-re-C*: $\forall\, x \in V.\ |Re\ (g\ x)| \leq C * hs\text{-}norm\text{-}lmat\ x$
    and *C-nn*: $0 \leq C$
  shows $\exists f$ :: (*complex*, $'m$, $'n$) *lmat* $\Rightarrow$ *complex*.
      clinear-functional-on UNIV f
    $\wedge$ $(\forall\, x \in V.\ f\ x = g\ x)$
    $\wedge$ $(\forall\, x.\ cmod\ (f\ x) \leq C * hs\text{-}norm\text{-}lmat\ x)$
⟨*proof*⟩

end

theory *Essential-Functional-Analysis*
  imports
    Matrix-Typed
    Quantum-Computing-3-10
    Jordan-Normal-Form.Schur-Decomposition
    Jordan-Normal-Form.DL-Rank
begin

60 Overview

The notes take the analysis of finite-dimensional spaces for granted ("it is easy to see that it is closed", Day 22; "C is a closed cone", Day 28). This theory supplies those facts for the matrix space M_n with the Hilbert–Schmidt norm $\|A\| = \sqrt{\mathrm{Tr}(A^* A)}$: the norm axioms, the parallelogram law, entrywise bounds, completeness, closedness of the sets of Hermitian and of positive semidefinite matrices, and Carathéodory's theorem in the form needed on Day 28: a non-negative combination of product matrices $\sigma \otimes \tau$ in M_{mp} can be rewritten using at most $2(mp)^2 + 1$ of them.

61 HS inner-product algebra

lemma *hs-inner-prod-self-eq-of-real*:
  $\langle\langle\ A\ |\ A\ \rangle\rangle$ = *complex-of-real* (*Re* ($\langle\langle\ A\ |\ A\ \rangle\rangle$))
  ⟨*proof*⟩

lemma *hs-inner-prod-self-zero-iff*:
  fixes A :: *complex Matrix.mat*
  shows $\langle\langle\ A\ |\ A\ \rangle\rangle = 0 \longleftrightarrow A = 0_m\ (dim\text{-}row\ A)\ (dim\text{-}col\ A)$
⟨*proof*⟩

lemma *Re-hs-inner-prod-right-smult-real*:
  fixes $A\ B$ :: *complex Matrix.mat* **and** t :: *real*
  assumes *dim-row A* = *dim-row B* **and** *dim-col A* = *dim-col B*
  shows *Re* ($\langle\langle\ A\ |\ complex\text{-}of\text{-}real\ t \cdot_m B\ \rangle\rangle$) = t * *Re* ($\langle\langle\ A\ |\ B\ \rangle\rangle$)
⟨*proof*⟩

lemma *Re-hs-inner-prod-swap*:

fixes *A B* :: *complex Matrix.mat*
assumes *dim-row A* = *dim-row B* **and** *dim-col A* = *dim-col B*
shows *Re* (⟨⟨ *A* | *B* ⟩⟩) = *Re* (⟨⟨ *B* | *A* ⟩⟩)
⟨*proof*⟩

62 Trace lemmas

lemma *Tr-dagger*:
fixes *M* :: *complex Matrix.mat*
assumes *M-c*: *M* ∈ *carrier-mat n n*
shows *Tr* (*M*†) = *cnj* (*Tr M*)
⟨*proof*⟩

lemma *Tr-smult-mat*:
fixes *M* :: *complex Matrix.mat* **and** *c* :: *complex*
assumes *M-c*: *M* ∈ *carrier-mat n n*
shows *Tr* (*c* ·$_m$ *M*) = *c* ∗ *Tr M*
⟨*proof*⟩

lemma *Re-hs-inner-eq-Re-Tr-dagger*:
fixes *A B* :: *complex Matrix.mat*
assumes *A-c*: *A* ∈ *carrier-mat n n*
and *B-c*: *B* ∈ *carrier-mat n n*
shows *Re* (⟨⟨ *A* | *B* ⟩⟩) = *Re* (*Tr* ((*A*†) ∗ *B*))
⟨*proof*⟩

63 HS norm definition and basic facts

HS / Frobenius norm on raw complex matrices. Definition duplicates the one in *Matrix-Typed* for the typed wrapper, but here for the raw matrices.

definition *hs-norm* :: *complex Matrix.mat* ⇒ *real* **where**
hs-norm A ≡ *sqrt* (*Re* (⟨⟨ *A* | *A* ⟩⟩))

lemma *hs-norm-nonneg*: *hs-norm A* ≥ *0*
⟨*proof*⟩

lemma *hs-norm-sq*:
(*hs-norm A*)^2 = *Re* (⟨⟨ *A* | *A* ⟩⟩)
⟨*proof*⟩

lemma *hs-norm-zero*: *hs-norm* (*0*$_m$ *m n*) = *0*
⟨*proof*⟩

lemma *hs-norm-smult*:
hs-norm (*c* ·$_m$ *A*) = *cmod c* ∗ *hs-norm A*
⟨*proof*⟩

Triangle inequality for the HS norm via Cauchy-Schwarz.

lemma *hs-norm-triangle*:
fixes *A B* :: *complex Matrix.mat*
assumes *A-c*: *A* ∈ *carrier-mat n n*
and *B-c*: *B* ∈ *carrier-mat n n*
shows *hs-norm* (*A* + *B*) ≤ *hs-norm A* + *hs-norm B*
⟨*proof*⟩

HS-norm-squared of $U + (-t) \cdot V$, expanded along the bilinear structure. We extract this into

a stand-alone lemma so the parameter names do not clash with let-bound names in callers.

lemma *hs-norm-sq-diff-expand*:
  fixes *U V* :: *complex Matrix.mat* **and** *t* :: *real*
  assumes *U-c*: *U* ∈ *carrier-mat n n*
    and *V-c*: *V* ∈ *carrier-mat n n*
  shows (*hs-norm* (*U* + *complex-of-real* (−*t*) ·$_m$ *V*))^2
      = *Re* (⟨⟨ *U* | *U* ⟩⟩) − *2* ∗ *t* ∗ *Re* (⟨⟨ *U* | *V* ⟩⟩)
        + *t*^{2} ∗ *Re* (⟨⟨ *V* | *V* ⟩⟩)
⟨*proof*⟩

HS norm satisfies the parallelogram identity.

lemma *hs-norm-parallelogram*:
  fixes *U V* :: *complex Matrix.mat*
  assumes *U-c*: *U* ∈ *carrier-mat n n*
    and *V-c*: *V* ∈ *carrier-mat n n*
  shows (*hs-norm* (*U* + *V*))^2 + (*hs-norm* (*U* − *V*))^2
      = *2* ∗ (*hs-norm U*)^2 + *2* ∗ (*hs-norm V*)^2
⟨*proof*⟩

lemma *hs-norm-zero-iff*:
  fixes *A* :: *complex Matrix.mat*
  shows *hs-norm A* = *0* ⟷ *A* = *0*$_m$ (*dim-row A*) (*dim-col A*)
⟨*proof*⟩

Every entry of a matrix is bounded in modulus-squared by the HS-norm squared. Useful for transferring HS-norm convergence to entry-wise convergence.

lemma *cmod-entry-sq-le-hs-norm-sq*:
  fixes *A* :: *complex Matrix.mat* **and** *i j* :: *nat*
  assumes *A-c*: *A* ∈ *carrier-mat n n*
    and *i-lt*: *i* < *n* **and** *j-lt*: *j* < *n*
  shows (*cmod* (*A* $$ (*i*,*j*)))^2 ≤ (*hs-norm A*)^2
⟨*proof*⟩

Modulus form of the entry bound: $|A_{i,j}| \le \|A\|_H S$.

lemma *cmod-entry-le-hs-norm*:
  fixes *A* :: *complex Matrix.mat* **and** *i j* :: *nat*
  assumes *A-c*: *A* ∈ *carrier-mat n n*
    and *i-lt*: *i* < *n* **and** *j-lt*: *j* < *n*
  shows *cmod* (*A* $$ (*i*,*j*)) ≤ *hs-norm A*
⟨*proof*⟩

64 HS-norm convergence vs entry-wise convergence

HS-norm convergence implies entry-wise convergence: if (M_n) converges to L in HS-norm then for every (i, j) the entry sequence M_n \$\$ (i, j) tends to L \$\$ (i, j).

lemma *HS-lim-entry-lim*:
  fixes *M-seq* :: *nat* ⇒ *complex Matrix.mat* **and** *r* :: *nat*
    and *i j* :: *nat* **and** *L* :: *complex Matrix.mat*
  assumes *carrier*: ∀ *n*. *M-seq n* ∈ *carrier-mat r r*
    and *L-c*: *L* ∈ *carrier-mat r r*
    and *lim*: (λ*n*. *hs-norm* (*M-seq n* − *L*)) ⟶ *0*
    and *i-lt*: *i* < *r* **and** *j-lt*: *j* < *r*
  shows (λ*n*. *M-seq n* $$ (*i*,*j*)) ⟶ *L* $$ (*i*,*j*)
⟨*proof*⟩

HS-Cauchy implies entry-wise Cauchy: the dual of *HS-lim-entry-lim*. Combined with the completeness of $\mathbb{C}$ it yields a candidate limit for every entry.

lemma *HS-Cauchy-entry-Cauchy*:
  fixes *M-seq* :: *nat* ⇒ *complex Matrix.mat* **and** *r* :: *nat*
    and *i j* :: *nat*
  assumes *carrier*: ∀ *n*. *M-seq n* ∈ *carrier-mat r r*
    and *hs-cauchy*: ∀ *ε*>*0*. ∃ *N*. ∀ *x* ≥ *N*. ∀ *y* ≥ *N*.
                    hs-norm (*M-seq x* − *M-seq y*) < *ε*
    and *i-lt*: *i* < *r* **and** *j-lt*: *j* < *r*
  shows *Cauchy* (λ*n*. *M-seq n* $$ (*i*,*j*))
⟨*proof*⟩

HS-norm-squared as a real double sum of entry-modulus squares. This is the entry-wise expansion of $\|A\|^2 = Re\ \langle\langle\ A\ |\ A\ \rangle\rangle$.

lemma *hs-norm-sq-as-entry-sum*:
  fixes *A* :: *complex Matrix.mat*
  assumes *A-c*: *A* ∈ *carrier-mat r r*
  shows (*hs-norm A*)²
        = (∑ *i*∈{*0*..<*r*}. ∑ *j*∈{*0*..<*r*}. (*cmod* (*A* $$ (*i*,*j*)))²)
⟨*proof*⟩

Pointwise entry bounds give an HS-norm-squared bound.

lemma *hs-norm-sq-le-from-entry-bounds*:
  fixes *A* :: *complex Matrix.mat*
  assumes *A-c*: *A* ∈ *carrier-mat r r*
    and *entries*: ⋀*i j*. *i* < *r* ⟹ *j* < *r* ⟹ (*cmod* (*A* $$ (*i*,*j*)))² ≤ *M*
  shows (*hs-norm A*)² ≤ *real r* ∗ *real r* ∗ *M*
⟨*proof*⟩

Entry-cmod bound gives an HS-norm bound. Combines the *hs-norm-sq-le-from-entry-bounds* bound with *sqrt*.

lemma *hs-norm-le-from-entry-bounds*:
  fixes *A* :: *complex Matrix.mat* **and** *δ* :: *real*
  assumes *A-c*: *A* ∈ *carrier-mat r r*
    and *δ-nn*: *0* ≤ *δ*
    and *entries*: ⋀*i j*. *i* < *r* ⟹ *j* < *r* ⟹ *cmod* (*A* $$ (*i*,*j*)) ≤ *δ*
  shows *hs-norm A* ≤ *real r* ∗ *δ*
⟨*proof*⟩

Entry-wise convergence implies HS-norm convergence. This is the slow half of HS-completeness; we use the entry-bound helper to keep it tractable.

lemma *entry-lim-imp-HS-lim*:
  fixes *M-seq* :: *nat* ⇒ *complex Matrix.mat* **and** *M-lim* :: *complex Matrix.mat*
    and *r* :: *nat*
  assumes *carrier*: ∀ *n*. *M-seq n* ∈ *carrier-mat r r*
    and *M-lim-c*: *M-lim* ∈ *carrier-mat r r*
    and *entry-lim*: ⋀*i j*. *i* < *r* ⟹ *j* < *r*
                ⟹ (λ*n*. *M-seq n* $$ (*i*,*j*)) ⟶ *M-lim* $$ (*i*,*j*)
  shows (λ*n*. *hs-norm* (*M-seq n* − *M-lim*)) ⟶ *0*
⟨*proof*⟩

HS-completeness of the matrix space: every HS-Cauchy sequence in *carrier-mat r r* converges in HS norm to a matrix in *carrier-mat r r*. Combines *HS-Cauchy-entry-Cauchy* + completeness of $\mathbb{C}$ + *entry-lim-imp-HS-lim*.

lemma *HS-Cauchy-imp-convergent*:
  fixes *M-seq* :: *nat* ⇒ *complex Matrix.mat* **and** *r* :: *nat*

assumes *carrier*: $\forall\, n.$ *M-seq* $n \in$ *carrier-mat* $r\ r$
and *hs-cauchy*: $\forall\, \varepsilon{>}0.\ \exists\, N.\ \forall\, x \geq N.\ \forall\, y \geq N.$
hs-norm (*M-seq* x − *M-seq* y) $< \varepsilon$
shows $\exists$ *M-lim*. *M-lim* $\in$ *carrier-mat* $r\ r$
$\wedge\ (\forall\, \varepsilon{>}0.\ \exists\, N.\ \forall\, n \geq N.$ *hs-norm* (*M-seq* n − *M-lim*) $< \varepsilon$)
⟨*proof*⟩

Symmetry of HS-distance: $\|A - B\| = \|B - A\|$.

lemma *hs-norm-sub-swap*:
fixes $A\ B ::$ *complex Matrix.mat*
assumes *A-c*: $A \in$ *carrier-mat* $n\ n$
and *B-c*: $B \in$ *carrier-mat* $n\ n$
shows *hs-norm* $(A - B) =$ *hs-norm* $(B - A)$
⟨*proof*⟩

HS-norm closedness of the Hermitian set.

lemma *hermitian-HS-closed*:
fixes *M-seq* :: *nat* ⇒ *complex Matrix.mat* **and** $L ::$ *complex Matrix.mat*
and $r ::$ *nat*
assumes *carrier*: $\forall\, n.$ *M-seq* $n \in$ *carrier-mat* $r\ r$
and *L-c*: $L \in$ *carrier-mat* $r\ r$
and *herm-n*: $\forall\, n.$ *hermitian* (*M-seq* n)
and *lim*: $(\lambda n.$ *hs-norm* (*M-seq* $n - L$)) ⟶ 0
shows *hermitian* L
⟨*proof*⟩

HS-norm closedness of the PSD set. The proof combines hermitian_HS_closed for the Hermitian part with continuity of the quadratic form $\sigma \mapsto \langle x \mid \sigma *_v x \rangle$ (linear in the entries of σ, hence HS-continuous via entry convergence).

lemma *psd-HS-closed*:
fixes *σ-n* :: *nat* ⇒ *complex Matrix.mat* **and** $\sigma ::$ *complex Matrix.mat*
and $r ::$ *nat*
assumes *carrier*: $\forall\, n.$ *σ-n* $n \in$ *carrier-mat* $r\ r$
and *σ-c*: $\sigma \in$ *carrier-mat* $r\ r$
and *psd-n*: $\forall\, n.$ *psd* (*σ-n* n)
and *lim*: $(\lambda n.$ *hs-norm* (*σ-n* $n - \sigma$)) ⟶ 0
shows *psd* σ
⟨*proof*⟩

Reverse triangle inequality: $|\|A\| - \|B\|| \leq \|A - B\|$.

lemma *hs-norm-sub-triangle-ineq*:
fixes $A\ B ::$ *complex Matrix.mat*
assumes *A-c*: $A \in$ *carrier-mat* $n\ n$
and *B-c*: $B \in$ *carrier-mat* $n\ n$
shows |*hs-norm* A − *hs-norm* B| ≤ *hs-norm* $(A - B)$
⟨*proof*⟩

65 Carathéodory's theorem for conic combinations

Carathéodory's theorem for cones: in a real vector space V of dimension d, every element of the conic hull of a set S (a finite non-negative combination of elements of S) is already a non-negative combination of at most d elements of S. We need it for V the real space of Hermitian $(mp)\times(mp)$ matrices, of real dimension $(mp)^2$, and S the product positive matrices $\sigma \otimes \tau$: it bounds the length of the decompositions of elements of the separable cone uniformly, which is what makes the cone closed (Separable_Cone). HOL-Analysis has Carathéodory for *euclidean-space*; using

it directly would require a Euclidean-space instance on the Hermitian matrices, so the theorem is proved here directly for lists: any $d+1$ vectors are linearly dependent, and renormalising the weights along a dependence relation removes one term while keeping all weights non-negative.

Below is a fold-based weighted sum of the cone-style list, suitable for stating Carathéodory at the matrix level.

fun *cone-list-sum* ::
  nat ⇒ *nat* ⇒
   (*real* × *complex Matrix.mat* × *complex Matrix.mat*) *list* ⇒
   complex Matrix.mat
where
  cone-list-sum m p [] = 0_m (*m*∗*p*) (*m*∗*p*)
| *cone-list-sum m p* ((*w*, σ, τ) # *ts*) =
     complex-of-real w ·$_m$ (σ ⨂ τ) + *cone-list-sum m p ts*

lemma *cone-list-sum-carrier*:
  cone-list-sum m p ts ∈ *carrier-mat* (*m*∗*p*) (*m*∗*p*)
⟨*proof*⟩

cone-list-sum is additive over list-append.

lemma *cone-list-sum-append*:
  cone-list-sum m p (*xs* @ *ys*)
    = *cone-list-sum m p xs* + *cone-list-sum m p ys*
⟨*proof*⟩

A zero-weight singleton over a properly-sized matrix pair contributes the zero matrix.

lemma *cone-list-sum-zero-singleton*:
  assumes σ-*c*: σ ∈ *carrier-mat m m* **and** τ-*c*: τ ∈ *carrier-mat p p*
  shows *cone-list-sum m p* [(*0*, σ, τ)] = 0_m (*m*∗*p*) (*m*∗*p*)
⟨*proof*⟩

Dropping a zero-weight entry from a cone list (with properly typed matrix part) preserves the cone sum.

lemma *cone-list-sum-drop-zero*:
  assumes *i-lt*: *i* < *length xs*
     and *zero-w*: *fst* (*xs* ! *i*) = *0*
     and *typing*: ∀(*w*, σ, τ) ∈ *set xs*.
                  σ ∈ *carrier-mat m m* ∧ τ ∈ *carrier-mat p p*
  shows *cone-list-sum m p* (*take i xs* @ *drop* (*Suc i*) *xs*)
         = *cone-list-sum m p xs*
⟨*proof*⟩

Linearity of *cone-list-sum* in the weight. Two lists with matching matrix parts but weights related by $w_k = {w_k}^1 + \mu * r_k$ satisfy the corresponding sum-equation.

lemma *cone-list-sum-weight-split*:
  fixes *f g h* :: *nat* ⇒ *real*
    and σ*f* τ*f* :: *nat* ⇒ *complex Matrix.mat*
    and μ :: *real* **and** *L* :: *nat*
  assumes *typing*: ⋀*k*. *k* < *L* ⟹
                  σ*f k* ∈ *carrier-mat m m* ∧ τ*f k* ∈ *carrier-mat p p*
     and *split*: ⋀*k*. *k* < *L* ⟹ *f k* = *g k* + μ ∗ *h k*
  shows *cone-list-sum m p* (*map* (λ*k*. (*f k*, σ*f k*, τ*f k*)) [*0*..<*L*])
     = *cone-list-sum m p* (*map* (λ*k*. (*g k*, σ*f k*, τ*f k*)) [*0*..<*L*])
       + *complex-of-real* μ ·$_m$
          cone-list-sum m p (*map* (λ*k*. (*h k*, σ*f k*, τ*f k*)) [*0*..<*L*])
  ⟨*proof*⟩

Entry-wise expression for *cone-list-sum*: each entry is a HOL sum over the list indices of weight times tensor entry.

lemma *cone-list-sum-entry*:
fixes $m\ p :: nat$
and $ts :: (real \times complex\ Matrix.mat \times complex\ Matrix.mat)\ list$
assumes *typing*: $\forall (w, \sigma, \tau) \in set\ ts.$
$\sigma \in carrier\text{-}mat\ m\ m \wedge \tau \in carrier\text{-}mat\ p\ p$
and *i-lt*: $i < m * p$ **and** *j-lt*: $j < m * p$
shows *cone-list-sum m p ts* \$\$ (i, j)
$= (\sum k{<}length\ ts.$
$complex\text{-}of\text{-}real\ (fst\ (ts!k))$
$* (fst\ (snd\ (ts!k)) \bigotimes snd\ (snd\ (ts!k)))$ \$\$ $(i, j))$
⟨*proof*⟩

An entry-wise zero criterion: if every entry is 0, the matrix is 0_m.

lemma *matrix-eq-zero-via-entries*:
fixes $M :: complex\ Matrix.mat$
assumes *M-c*: $M \in carrier\text{-}mat\ (m{*}p)\ (m{*}p)$
and *zero*: $\bigwedge i\ j.\ i < m{*}p \Longrightarrow j < m{*}p \Longrightarrow M\ \$\$\ (i, j) = 0$
shows $M = 0_m\ (m{*}p)\ (m{*}p)$
⟨*proof*⟩

Top-level linear-algebra core. A list of more than $2\ (m\ p)^2$ complex $(m\ p) \times (m\ p)$ matrix pairs admits a non-trivial real linear combination of their tensor products that vanishes.

The proof is the standard JNF *vec-space TYPE*(*real*) $(2\ (m\ p)^2)$ *li-le-dim* contrapositive applied to a real-flatten of the tensor products. Admitted here as a single named lemma so it can be discharged independently of the cone Carathéodory reduction machinery (which is fully proved below modulo this one lemma).

lemma *cone-list-real-lin-dep*:
fixes $m\ p :: nat$
and $pairs :: (complex\ Matrix.mat \times complex\ Matrix.mat)\ list$
assumes *m-pos*: $0 < m$ **and** *p-pos*: $0 < p$
and *typing*: $\forall (\sigma, \tau) \in set\ pairs.$
$\sigma \in carrier\text{-}mat\ m\ m \wedge \tau \in carrier\text{-}mat\ p\ p$
and *long*: $length\ pairs > 2 * (m{*}p)^2$
shows $\exists r :: nat \Rightarrow real.$
$(\exists k < length\ pairs.\ r\ k \neq 0) \wedge$
$(\forall k \geq length\ pairs.\ r\ k = 0) \wedge$
$cone\text{-}list\text{-}sum\ m\ p$
$(map\ (\lambda k.\ (r\ k, fst\ (pairs\ !\ k), snd\ (pairs\ !\ k)))$
$[0..{<}length\ pairs])$
$= 0_m\ (m{*}p)\ (m{*}p)$
⟨*proof*⟩

One reduction step: a length-L cone list with $L > (m\ p)^2 + 1$ admits a length-$(L{-}1)$ list with the same cone sum, weights non-negative, and matrix pairs inherited from the original list.

NOTE on the conclusion: the standard Carathéodory reduction *reweights* rather than *selects*, so we require only that the *matrix pair* (σ, τ) of each new entry occurs in the original list the weight may differ. This is what the downstream closedness argument actually needs (typing + non-negativity inherited from the original cone).

lemma *caratheodory-matrix-cone-step*:
fixes $m\ p :: nat$
and $ts :: (real \times complex\ Matrix.mat \times complex\ Matrix.mat)\ list$
assumes *m-pos*: $0 < m$ **and** *p-pos*: $0 < p$
and *ts-typing*: $\forall (w, \sigma, \tau) \in set\ ts.$

$\sigma \in$ *carrier-mat m m* $\wedge$ $\tau \in$ *carrier-mat p p*
and *ts-nn*: $\forall (w, -, -) \in$ *set ts*. $0 \leq w$
and *ts-long*: *length ts* $> 2 * (m*p)^2 + 1$
shows $\exists$ *ts'*. *length ts'* $=$ *length ts* $- 1$
$\wedge$ $(\forall (w, \sigma, \tau) \in$ *set ts'*.
$\sigma \in$ *carrier-mat m m* $\wedge$ $\tau \in$ *carrier-mat p p* $\wedge$ $0 \leq w$
$\wedge$ $(\exists w_0.\ (w_0, \sigma, \tau) \in$ *set ts*$))$
$\wedge$ *cone-list-sum m p ts* $=$ *cone-list-sum m p ts'*
⟨*proof*⟩

The Carathéodory bound follows from the reduction step by strong induction on *length ts*. This part is now closed unconditionally; only the two focused sub-sorries inside *caratheodory-matrix-cone-step* remain.

lemma *caratheodory-matrix-cone*:
fixes *m p* :: *nat*
and *ts* :: (*real* $\times$ *complex Matrix.mat* $\times$ *complex Matrix.mat*) *list*
assumes *m-pos*: $0 < m$ **and** *p-pos*: $0 < p$
and *ts-typing*: $\forall (w, \sigma, \tau) \in$ *set ts*.
$\sigma \in$ *carrier-mat m m* $\wedge$ $\tau \in$ *carrier-mat p p*
and *ts-nn*: $\forall (w, -, -) \in$ *set ts*. $0 \leq w$
shows $\exists$ *ts'*. *length ts'* $\leq 2 * (m*p)^2 + 1$
$\wedge$ $(\forall (w, \sigma, \tau) \in$ *set ts'*.
$\sigma \in$ *carrier-mat m m* $\wedge$ $\tau \in$ *carrier-mat p p* $\wedge$ $0 \leq w$
$\wedge$ $(\exists w_0.\ (w_0, \sigma, \tau) \in$ *set ts*$))$
$\wedge$ *cone-list-sum m p ts* $=$ *cone-list-sum m p ts'*
⟨*proof*⟩

How the bound is used (Separable_Cone): a sequence in the separable cone has decompositions of length at most $K = 2(mp)^2+1$; padding with zero-weight entries and normalising traces makes the weights bounded, so coordinatewise Bolzano–Weierstrass extracts a convergent subsequence of weights and factors; tensoring and addition are continuous, so the limit lies in the cone. The compactness of the unit-trace positive matrices is obtained from the Euclidean-space instance of Lmat_Euclidean (real dimension $2(mp)^2$) rather than from the sequential-compactness lemma *HS-Cauchy-imp-convergent* proved above.

end

theory *Lmat-Euclidean*
imports
Vern-Paulsen-QC-3-10.Matrix-Typed
HOL−Analysis.Euclidean-Space
HOL−Analysis.L2-Norm
HOL−Analysis.Elementary-Metric-Spaces
HOL−Analysis.Topology-Euclidean-Space
begin

The HOL/Hahn_Banach development imports a locale also called *norm* under the qualifier *Normed-Space*. Hide its short names so that, inside our instantiation blocks, *norm* resolves to the class method from *HOL.Real-Vector-Spaces*.

hide-const (**open**) *Normed-Space.norm*
hide-const (**open**) *Normed-Space.seminorm*

66 Overview

Nothing in this theory is stated in the notes; it is the bridge that lets the HOL-Analysis theorems on convex sets (separating hyperplanes, compactness) be applied to matrices. ($'a$, $'m$,

$'n$) *lmat* is given the typeclass tower *real-vector*, *metric-space*, *real-normed-vector*, *real-inner*, *euclidean-space*, *heine-borel*; for $'a = complex$ the norm is the Hilbert–Schmidt norm, the inner product its real part, and the dimension is $2\,\mathrm{CARD}('m)\,\mathrm{CARD}('n)$.

67 Real scalar multiplication and *real-vector* instance

For $'a :: real\text{-}vector$, lift element-wise scalar multiplication via *map-mat* (*scaleR a*).

instantiation *lmat* :: (*real-vector*, *finite*, *finite*) *scaleR*
begin

lift-definition *scaleR-lmat* ::
real ⇒ ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat* ⇒ ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
is λ*r M*. *map-mat* (λ*x*. *r* $*_R$ *x*) *M* ⟨*proof*⟩

instance ⟨*proof*⟩

end

lemma *unwrap-lm-scaleR* [*simp*]:
unwrap-lm ((*r* :: *real*) $*_R$ (*M* :: ('*a* :: *real-vector*, '*m* :: *finite*, '*n* :: *finite*) *lmat*))
= *map-mat* (λ*x*. *r* $*_R$ *x*) (*unwrap-lm M*)
⟨*proof*⟩

lemma *unwrap-lm-scaleR-index* [*simp*]:
fixes *M* :: ('*a* :: *real-vector*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
assumes *i* < *CARD*('*m*) *j* < *CARD*('*n*)
shows *unwrap-lm* (*r* $*_R$ *M*) $$ (*i*, *j*) = *r* $*_R$ (*unwrap-lm M* $$ (*i*, *j*))
⟨*proof*⟩

instance *lmat* :: (*real-vector*, *finite*, *finite*) *real-vector*
⟨*proof*⟩

68 Auxiliary: pointwise unwrapping (used by later instances)

lemma *unwrap-lm-plus-index* [*simp*]:
fixes *M N* :: ('*a* :: *plus*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
assumes *i* < *CARD*('*m*) *j* < *CARD*('*n*)
shows *unwrap-lm* (*M* + *N*) $$ (*i*, *j*) = *unwrap-lm M* $$ (*i*, *j*) + *unwrap-lm N* $$ (*i*, *j*)
⟨*proof*⟩

lemma *unwrap-lm-minus-index* [*simp*]:
fixes *M N* :: ('*a* :: *ab-group-add*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
assumes *i* < *CARD*('*m*) *j* < *CARD*('*n*)
shows *unwrap-lm* (*M* − *N*) $$ (*i*, *j*) = *unwrap-lm M* $$ (*i*, *j*) − *unwrap-lm N* $$ (*i*, *j*)
⟨*proof*⟩

lemma *unwrap-lm-zero-index* [*simp*]:
fixes *i j* :: *nat*
assumes *i* < *CARD*('*m* :: *finite*) *j* < *CARD*('*n* :: *finite*)
shows *unwrap-lm* (*0* :: ('*a* :: *zero*, '*m*, '*n*) *lmat*) $$ (*i*, *j*) = *0*
⟨*proof*⟩

lemma *lmat-eq-iff-entries*:
fixes *M N* :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
shows *M* = *N* ⟷

$(\forall\, i\ j.\ i < CARD('m) \longrightarrow j < CARD('n)$
$\longrightarrow$ *unwrap-lm* M \$\$ $(i, j) =$ *unwrap-lm* N \$\$ $(i, j))$
⟨*proof*⟩

69 Distance, uniformity, open: *metric-space* instance

Distance is the L2 sum of entry-wise distances, indexed by $\{0..<CARD('m)\} \times \{0..<CARD('n)\}$. Uniformity and *open* are defined via the standard metric-induced filter so that the *open-uniformity* axiom is discharged by reflexivity.

instantiation *lmat* :: (*metric-space*, *finite*, *finite*) *dist*
begin

definition *dist-lmat* :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
$\Rightarrow$ ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat* $\Rightarrow$ *real* **where**
dist-lmat M N = *L2-set* $(\lambda(i, j).$ *dist* (*unwrap-lm M* \$\$ $(i, j))$ (*unwrap-lm N* \$\$ $(i, j)))$
$(\{0..<CARD('m)\} \times \{0..<CARD('n)\})$

instance ⟨*proof*⟩

end

instantiation *lmat* :: (*metric-space*, *finite*, *finite*) *uniformity-dist*
begin

definition *uniformity-lmat* ::
(('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat* $\times$ ('*a*, '*m*, '*n*) *lmat*) *filter* **where**
(*uniformity-lmat* :: (('*a*, '*m*, '*n*) *lmat* $\times$ ('*a*, '*m*, '*n*) *lmat*) *filter*)
= (*INF* $e \in \{0<..\}.$ *principal* $\{(x, y).$ *dist* $x\ y < e\}$)

instance ⟨*proof*⟩

end

instantiation *lmat* :: (*metric-space*, *finite*, *finite*) *metric-space*
begin

definition *open-lmat* :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat set* $\Rightarrow$ *bool* **where**
open-lmat U = $(\forall\, x \in U.\ \forall_F\ (x', y)$ *in uniformity*. $x' = x \longrightarrow y \in U)$

instance ⟨*proof*⟩

end

70 Norm and *real-normed-vector* instance

instantiation *lmat* :: (*real-normed-vector*, *finite*, *finite*) *real-normed-vector*
begin

definition *norm-lmat* :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat* $\Rightarrow$ *real* **where**
norm-lmat M = *L2-set* $(\lambda(i, j).$ *norm* (*unwrap-lm M* \$\$ $(i, j)))$
$(\{0..<CARD('m)\} \times \{0..<CARD('n)\})$

definition *sgn-lmat* ::
('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat* $\Rightarrow$ ('*a*, '*m*, '*n*) *lmat* **where**
sgn-lmat (*M* :: ('*a*, '*m*, '*n*) *lmat*) = *scaleR* (*inverse* (*norm M*)) *M*

instance ⟨*proof*⟩

end

71 Real inner product

For $'a :: \textit{real-inner}$, the lmat inner product is the entry-wise sum of inner products. When $'a = \textit{complex}$ with the standard real-inner instance $\textit{inner}\ z\ w = \textit{Re}\ (\textit{cnj}\ z * w)$, this is the real part of the Hilbert-Schmidt inner product.

instantiation *lmat* :: (*real-inner*, *finite*, *finite*) *real-inner*
begin

definition *inner-lmat* :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
⇒ ('*a*, '*m*, '*n*) *lmat* ⇒ *real* **where**
inner-lmat M N = (∑ (*i*, *j*) ∈ {*0*..<*CARD*('*m*)} × {*0*..<*CARD*('*n*)}.
inner (*unwrap-lm M* $$ (*i*, *j*)) (*unwrap-lm N* $$ (*i*, *j*)))

instance ⟨*proof*⟩

end

72 Basis: matrix units carrying a basis vector

definition *lmat-axis* :: *nat* ⇒ *nat* ⇒ '*a* :: *zero*
⇒ ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat* **where**
lmat-axis i j u = *wrap-lm* (*Matrix.mat* (*CARD*('*m*)) (*CARD*('*n*))
(λ(*r*, *s*). *if r* = *i* ∧ *s* = *j then u else 0*))

lemma *unwrap-lmat-axis* [*simp*]:
unwrap-lm (*lmat-axis i j u* :: ('*a* :: *zero*, '*m* :: *finite*, '*n* :: *finite*) *lmat*)
= *Matrix.mat* (*CARD*('*m*)) (*CARD*('*n*))
(λ(*r*, *s*). *if r* = *i* ∧ *s* = *j then u else 0*)
⟨*proof*⟩

lemma *lmat-axis-index* [*simp*]:
fixes *u* :: '*a* :: *zero*
assumes *r* < *CARD*('*m* :: *finite*) *s* < *CARD*('*n* :: *finite*)
shows *unwrap-lm* (*lmat-axis i j u* :: ('*a*, '*m*, '*n*) *lmat*) $$ (*r*, *s*)
= (*if r* = *i* ∧ *s* = *j then u else 0*)
⟨*proof*⟩

Inner product against a single matrix-unit basis vector picks out one matrix entry.

lemma *inner-lmat-axis*:
fixes *u* :: '*a* :: *real-inner*
and *M* :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
assumes *ij*: *i* < *CARD*('*m*) *j* < *CARD*('*n*)
shows *inner M* (*lmat-axis i j u* :: ('*a*, '*m*, '*n*) *lmat*)
= *inner* (*unwrap-lm M* $$ (*i*, *j*)) *u*
⟨*proof*⟩

Two matrix-unit basis vectors are equal iff their entries and positions match (or both vectors are zero).

lemma *lmat-axis-eq-iff*:
fixes *u v* :: '*a* :: *zero*
assumes *i* < *CARD*('*m* :: *finite*) *j* < *CARD*('*n* :: *finite*)

and $i' < CARD('m)$ $j' < CARD('n)$
shows $(lmat\text{-}axis\ i\ j\ u :: ('a, 'm, 'n)\ lmat) = lmat\text{-}axis\ i'\ j'\ v$
$\longleftrightarrow (u = v \wedge i = i' \wedge j = j') \vee (u = 0 \wedge v = 0)$
⟨*proof*⟩

Inner product between two matrix-unit basis vectors: 1 if same position and basis component, 0 otherwise.

lemma *inner-lmat-axis-axis*:
fixes $u\ v :: 'a :: real\text{-}inner$
assumes *ij*: $i < CARD('m :: finite)$ $j < CARD('n :: finite)$
and *ij'*: $i' < CARD('m)$ $j' < CARD('n)$
shows $inner\ (lmat\text{-}axis\ i\ j\ u :: ('a, 'm, 'n)\ lmat)\ (lmat\text{-}axis\ i'\ j'\ v)$
$= (if\ i = i' \wedge j = j'\ then\ inner\ u\ v\ else\ 0)$
⟨*proof*⟩

73 Basis set

definition *Basis-lmat-set* :: $('a :: euclidean\text{-}space, 'm :: finite, 'n :: finite)\ lmat\ set$
where $Basis\text{-}lmat\text{-}set = (\bigcup i \in \{0..<CARD('m)\}.$
$\bigcup j \in \{0..<CARD('n)\}.$
$\bigcup u \in Basis.\ \{lmat\text{-}axis\ i\ j\ u :: ('a, 'm, 'n)\ lmat\})$

lemma *Basis-lmat-set-finite*:
$finite\ (Basis\text{-}lmat\text{-}set :: ('a :: euclidean\text{-}space, 'm :: finite, 'n :: finite)\ lmat\ set)$
⟨*proof*⟩

lemma *Basis-lmat-set-nonempty*:
$Basis\text{-}lmat\text{-}set \neq (\{\} :: ('a :: euclidean\text{-}space, 'm :: finite, 'n :: finite)\ lmat\ set)$
⟨*proof*⟩

lemma *Basis-lmat-orthonormal*:
fixes $u\ v :: ('a :: euclidean\text{-}space, 'm :: finite, 'n :: finite)\ lmat$
assumes *uB*: $u \in Basis\text{-}lmat\text{-}set$ **and** *vB*: $v \in Basis\text{-}lmat\text{-}set$
shows $inner\ u\ v = (if\ u = v\ then\ 1\ else\ 0)$
⟨*proof*⟩

lemma *axis-mem-Basis-lmat-set*:
fixes $u :: 'a :: euclidean\text{-}space$
assumes *i-lt*: $i < CARD('m :: finite)$
and *j-lt*: $j < CARD('n :: finite)$
and *uB*: $u \in Basis$
shows $(lmat\text{-}axis\ i\ j\ u :: ('a, 'm, 'n)\ lmat) \in Basis\text{-}lmat\text{-}set$
⟨*proof*⟩

lemma *Basis-lmat-perp-imp-zero*:
fixes $x :: ('a :: euclidean\text{-}space, 'm :: finite, 'n :: finite)\ lmat$
assumes $*$: $\forall u \in Basis\text{-}lmat\text{-}set.\ inner\ x\ u = 0$
shows $x = 0$
⟨*proof*⟩

lemma *zero-inner-Basis-lmat*:
fixes $u :: ('a :: euclidean\text{-}space, 'm :: finite, 'n :: finite)\ lmat$
shows $inner\ (0 :: ('a, 'm, 'n)\ lmat)\ u = 0$
⟨*proof*⟩

lemma *Basis-lmat-perp-zero*:
fixes $x :: ('a :: euclidean\text{-}space, 'm :: finite, 'n :: finite)\ lmat$

shows $(\forall\, u \in \textit{Basis-lmat-set}.\ \textit{inner}\ x\ u = 0) \longleftrightarrow x = 0$
⟨*proof*⟩

74 Euclidean space instance

instantiation *lmat* :: (*euclidean-space*, *finite*, *finite*) *euclidean-space*
begin

definition *Basis-lmat* :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat set* **where**
Basis-lmat = *Basis-lmat-set*

instance ⟨*proof*⟩

end

75 Heine-Borel

instance *lmat* :: (*euclidean-space*, *finite*, *finite*) *heine-borel* ⟨*proof*⟩

76 Dimension

Two distinct $(i,\ j,\ u)$ with $u \in \textit{Basis-}'a$ give distinct matrix-axis vectors.

lemma *lmat-axis-inj-on-Basis-indices*:
fixes *u v* :: '*a* :: *euclidean-space*
assumes *ij*: $i < \textit{CARD}('m :: \textit{finite})\ j < \textit{CARD}('n :: \textit{finite})$
and *ij'*: $i' < \textit{CARD}('m)\ j' < \textit{CARD}('n)$
and *uB*: $u \in \textit{Basis}$ **and** *vB*: $v \in \textit{Basis}$
shows $(\textit{lmat-axis}\ i\ j\ u :: ('a, 'm, 'n)\ \textit{lmat}) = \textit{lmat-axis}\ i'\ j'\ v$
$\longleftrightarrow i = i' \wedge j = j' \wedge u = v$
⟨*proof*⟩

lemma *DIM-lmat*:
$\textit{DIM}(('a :: \textit{euclidean-space}, 'm :: \textit{finite}, 'n :: \textit{finite})\ \textit{lmat})$
$= \textit{CARD}('m) * \textit{CARD}('n) * \textit{DIM}('a)$
⟨*proof*⟩

77 Component-wise bounds via the L2 norm

lemma *norm-lmat-index-le*:
fixes $M :: ('a :: \textit{real-normed-vector}, 'm :: \textit{finite}, 'n :: \textit{finite})\ \textit{lmat}$
assumes $i < \textit{CARD}('m)\ j < \textit{CARD}('n)$
shows $\textit{norm}\ (\textit{unwrap-lm}\ M\ \$\$\ (i, j)) \leq \textit{norm}\ M$
⟨*proof*⟩

lemma *dist-lmat-index-le*:
fixes $M\ N :: ('a :: \textit{real-normed-vector}, 'm :: \textit{finite}, 'n :: \textit{finite})\ \textit{lmat}$
assumes $i < \textit{CARD}('m)\ j < \textit{CARD}('n)$
shows $\textit{dist}\ (\textit{unwrap-lm}\ M\ \$\$\ (i, j))\ (\textit{unwrap-lm}\ N\ \$\$\ (i, j)) \leq \textit{dist}\ M\ N$
⟨*proof*⟩

78 Convergence: lmat sequence convergence implies entry convergence

lemma *tendsto-unwrap-lm-index*:

fixes $f :: 'b \Rightarrow ('a :: \textit{real-normed-vector}, 'm :: \textit{finite}, 'n :: \textit{finite})\ \textit{lmat}$
and $L :: ('a, 'm, 'n)\ \textit{lmat}$
assumes $\textit{lim}: (f \longrightarrow L)\ F$
and $\textit{ij}: i < \textit{CARD}('m)\ j < \textit{CARD}('n)$
shows $((\lambda x.\ \textit{unwrap-lm}\ (f\ x)\ \$\$\ (i, j)) \longrightarrow \textit{unwrap-lm}\ L\ \$\$\ (i, j))\ F$
⟨*proof*⟩

lemma *LIMSEQ-unwrap-lm-index*:
fixes $f :: \textit{nat} \Rightarrow ('a :: \textit{real-normed-vector}, 'm :: \textit{finite}, 'n :: \textit{finite})\ \textit{lmat}$
and $L :: ('a, 'm, 'n)\ \textit{lmat}$
assumes $f \longrightarrow L\ i < \textit{CARD}('m)\ j < \textit{CARD}('n)$
shows $(\lambda n.\ \textit{unwrap-lm}\ (f\ n)\ \$\$\ (i, j)) \longrightarrow \textit{unwrap-lm}\ L\ \$\$\ (i, j)$
⟨*proof*⟩

end

Part V
Lectures 26–28: separability and entanglement witnesses

theory *Separable-Cone*
imports
Lmat-Euclidean
Vern-Paulsen-QC-3-10.Quantum-Computing-3-10
HOL−Analysis.Convex
HOL−Analysis.Convex-Euclidean-Space
HOL−Analysis.Polytope
HOL−Analysis.Starlike
begin

HOL-Hahn_Banach defines a locale also called *continuous*. Hide its short name so that, in the proofs below, *continuous* resolves to the topological-space predicate from *HOL.Topological-Spaces*.

hide-const (**open**) *Function-Norm.continuous*

79 Overview (Days 27–28 of the notes)

This theory proves the results restated lecture by lecture in theory Day24_Separability. The notes argue in the real vector space of Hermitian matrices with a closed-cone separation theorem; formally we work in the typed matrix space $(\textit{complex}, 'm \times 'p, 'm \times 'p)\ \textit{lmat}$, a real inner-product space (Lmat_Euclidean), and use HOL-Analysis' *separating-hyperplane-closed-point*. Steps: the positive matrices form a closed set; the unit-trace positive matrices form a compact set; the tensor product is continuous, so the product density matrices *Sprime* form a compact set not containing 0; by Carathéodory (Essential_Functional_Analysis) every element of the separable cone is a bounded combination of at most $2(mp)^2 + 1$ generators, and the cone is closed. Separation then yields a real functional, which is converted into a Hermitian witness W with $\Re\,\mathrm{Tr}(W\rho) < 0 \le \Re\,\mathrm{Tr}(W\sigma)$ on the cone.

80 Typed PSD predicate

definition *psd-lmat* :: (*complex*, *'k* :: *finite*, *'k*) *lmat* ⇒ *bool* **where**
psd-lmat M ⟷ *psd* (*unwrap-lm M*)

lemma *psd-lmat-unwrap* [*iff*]:
psd-lmat (*M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*) ⟷ *psd* (*unwrap-lm M*)
⟨*proof*⟩

81 Typed tensor product

Two square matrices with type-level dimensions *'m*, *'p* tensor to a square matrix with type-level dimension *'m* × *'p*.

lemma *tensor-mat-carrier-lmat*:
fixes *A B* :: *complex Matrix.mat*
assumes *A*: *A* ∈ *carrier-mat* (*CARD*(*'m* :: *finite*)) (*CARD*(*'m*))
and *B*: *B* ∈ *carrier-mat* (*CARD*(*'p* :: *finite*)) (*CARD*(*'p*))
shows *A* ⨂ *B* ∈ *carrier-mat* (*CARD*(*'m* × *'p*)) (*CARD*(*'m* × *'p*))
⟨*proof*⟩

definition *tensor-lmat* ::
(*complex*, *'m* :: *finite*, *'m*) *lmat* ⇒ (*complex*, *'p* :: *finite*, *'p*) *lmat*
⇒ (*complex*, *'m* × *'p*, *'m* × *'p*) *lmat* **where**
tensor-lmat σ τ = *wrap-lm* (*unwrap-lm σ* ⨂ *unwrap-lm τ*)

lemma *unwrap-lm-tensor-lmat* [*simp*]:
fixes *σ* :: (*complex*, *'m* :: *finite*, *'m*) *lmat*
and *τ* :: (*complex*, *'p* :: *finite*, *'p*) *lmat*
shows *unwrap-lm* (*tensor-lmat σ τ* :: (*complex*, *'m* × *'p*, *'m* × *'p*) *lmat*)
= *unwrap-lm σ* ⨂ *unwrap-lm τ*
⟨*proof*⟩

82 The separable cone

The set of *separable matrices*: all nonneg combinations of tensor products of typed PSD matrices. Equivalently, the convex conic hull of the product PSD set.

definition *separable-lmat-gen* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*,
'm × *'p*) *lmat set* **where**
separable-lmat-gen =
{*tensor-lmat σ τ* | *σ τ*.
(*psd-lmat σ* :: *bool*) ∧ (*psd-lmat τ* :: *bool*)}

definition *separable-cone-lmat* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*,
'm × *'p*) *lmat set* **where**
separable-cone-lmat = *convex-cone hull separable-lmat-gen*

Sanity: the cone contains 0, is convex, and is conic.

lemma *separable-cone-lmat-convex-cone*:
convex-cone (*separable-cone-lmat*
:: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat set*)
⟨*proof*⟩

lemma *zero-in-separable-cone-lmat*:
(*0* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat*)
∈ *separable-cone-lmat*
⟨*proof*⟩

83 Step 1: PSD set is closed

Sub-lemmas for the closedness of PSD: the Hermitian half and the nonneg-quadratic-form half.

The entry-projection $M \mapsto$ *unwrap-lm M $$ (i, j)* is continuous on lmat for valid index (i, j).

lemma *isCont-unwrap-lm-index*:
fixes *x* :: (*'a* :: *real-normed-vector*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
assumes *ij*: *i* < *CARD*('*m*) *j* < *CARD*('*n*)
shows *isCont* (λ*M*. *unwrap-lm M $$ (i, j)*) *x*
⟨*proof*⟩

lemma *continuous-on-unwrap-lm-index*:
fixes *S* :: (*'a* :: *real-normed-vector*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*
assumes *ij*: *i* < *CARD*('*m*) *j* < *CARD*('*n*)
shows *continuous-on S* (λ*M*. *unwrap-lm M $$ (i, j)*)
⟨*proof*⟩

lemma *closed-entry-hermitian-eq*:
fixes *i j* :: *nat*
assumes *ij*: *i* < *CARD*('*k* :: *finite*) *j* < *CARD*('*k*)
shows *closed* {*M* :: (*complex*, *'k*, *'k*) *lmat*.
unwrap-lm M $$ (i, j) = *cnj* (*unwrap-lm M $$ (j, i)*)}
⟨*proof*⟩

lemma *closed-hermitian-unwrap-lmat*:
closed {*M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*. *hermitian* (*unwrap-lm M*)}
⟨*proof*⟩

lemma *closed-nonneg-quad-form-lmat*:
fixes *x* :: *complex Matrix.vec*
assumes *xdim*: *dim-vec x* = *CARD*('*k* :: *finite*)
shows *closed* {*M* :: (*complex*, *'k*, *'k*) *lmat*.
Re (⟨*x* | *unwrap-lm M* ∗$_v$ *x*⟩) ≥ *0*}
⟨*proof*⟩

lemma *closed-psd-lmat*:
closed {*M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*. *psd-lmat M*}
⟨*proof*⟩

84 Trace on lmat

definition *Tr-lmat* :: (*complex*, *'k* :: *finite*, *'k*) *lmat* ⇒ *complex* **where**
Tr-lmat M = (∑ *i* = *0*..<*CARD*('*k*). *unwrap-lm M $$ (i, i)*)

lemma *continuous-on-Tr-lmat*:
fixes *S* :: (*complex*, *'k* :: *finite*, *'k*) *lmat set*
shows *continuous-on S Tr-lmat*
⟨*proof*⟩

lemma *Tr-lmat-add*:
fixes *M N* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*
shows *Tr-lmat* (*M* + *N*) = *Tr-lmat M* + *Tr-lmat N*
⟨*proof*⟩

lemma *Tr-lmat-scaleR*:
fixes *M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat* **and** *a* :: *real*
shows *Tr-lmat* (*a* ∗$_R$ *M*) = *a* ∗$_R$ *Tr-lmat M*

⟨*proof*⟩

lemma *Tr-lmat-zero* [*simp*]:
Tr-lmat (*0* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*) = *0*
⟨*proof*⟩

lemma *Tr-lmat-sum-scaleR*:
fixes *T* :: *'a set* **and** *u* :: *'a* ⇒ *real*
and *f* :: *'a* ⇒ (*complex*, *'k* :: *finite*, *'k*) *lmat*
assumes *finite T*
shows *Tr-lmat* (∑ *t* ∈ *T*. *u t* ∗*R f t*) = (∑ *t* ∈ *T*. *u t* ∗*R Tr-lmat* (*f t*))
⟨*proof*⟩

lemma *closed-unit-trace-lmat*:
closed {*M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*. *Tr-lmat M* = *1*}
⟨*proof*⟩

85 Compactness of the unit-trace PSD slice

For PSD σ with *Tr* σ = *1*: ${\sigma^2}_{HS} = \sum {\lambda_i}^2 \leq (\sum \lambda_i)^2 = 1$ where λ_i are the (nonneg) eigenvalues.

lemma *psd-entry-bound-by-diag*:
fixes *M* :: *complex Matrix.mat*
assumes *Mcar*: *M* ∈ *carrier-mat n n*
and *Mpsd*: *psd M*
and *i*: *i* < *n* **and** *j*: *j* < *n*
shows (*norm* (*M* $$ (*i*, *j*)))^2 ≤ *Re* (*M* $$ (*i*, *i*)) ∗ *Re* (*M* $$ (*j*, *j*))
⟨*proof*⟩

lemma *norm-lmat-le-of-psd-unit-trace*:
fixes *M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*
assumes *psd*: *psd-lmat M* **and** *tr*: *Tr-lmat M* = *1*
shows *norm M* ≤ *1*
⟨*proof*⟩

lemma *compact-unit-trace-psd-lmat*:
compact {*M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*. *psd-lmat M* ∧ *Tr-lmat M* = *1*}
⟨*proof*⟩

86 Tensor continuity

The typed tensor product is jointly continuous in both factors.

lemma *continuous-on-tensor-lmat*:
fixes *T* :: ((*complex*, *'m* :: *finite*, *'m*) *lmat*
× (*complex*, *'p* :: *finite*, *'p*) *lmat*) *set*
shows *continuous-on T* (λ(*σ*, *τ*). *tensor-lmat σ τ*
:: (*complex*, *'m* × *'p*, *'m* × *'p*) *lmat*)
⟨*proof*⟩

87 The unit-trace product slice

definition *Sprime* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat set*
where
Sprime =
{*ϱ*. ∃(*σ* :: (*complex*, *'m*, *'m*) *lmat*) (*τ* :: (*complex*, *'p*, *'p*) *lmat*).
ϱ = *tensor-lmat σ τ*

$\wedge$ *psd-lmat* $\sigma \wedge$ *Tr-lmat* $\sigma = 1$
$\wedge$ *psd-lmat* $\tau \wedge$ *Tr-lmat* $\tau = 1\}$

lemma *Sprime-image*:
(*Sprime* :: (*complex*, $'m$ :: *finite* $\times$ $'p$ :: *finite*, $'m \times 'p$) *lmat set*)
= $(\lambda(\sigma, \tau).$ *tensor-lmat* σ $\tau)$ ‘
$(\{\sigma$ :: (*complex*, $'m$, $'m$) *lmat*. *psd-lmat* $\sigma \wedge$ *Tr-lmat* $\sigma = 1\}$
$\times \{\tau$ :: (*complex*, $'p$, $'p$) *lmat*. *psd-lmat* $\tau \wedge$ *Tr-lmat* $\tau = 1\})$
⟨*proof*⟩

lemma *compact-Sprime*:
compact (*Sprime* :: (*complex*, $'m$ :: *finite* $\times$ $'p$ :: *finite*, $'m \times 'p$) *lmat set*)
⟨*proof*⟩

Reindex $\{0..<n*q\}$ via (div, mod) into $\{0..<n\} \times \{0..<q\}$.

lemma *sum-div-mod-split*:
fixes f :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ $'a$ :: *comm-monoid-add*
assumes *q-pos*: $0 < q$
shows $(\sum a = 0..<n.\ \sum b = 0..<q.\ f\ a\ b)$
$= (\sum i = 0..<n * q.\ f\ (i\ div\ q)\ (i\ mod\ q))$
⟨*proof*⟩

lemma *Tr-lmat-tensor*:
fixes σ :: (*complex*, $'m$ :: *finite*, $'m$) *lmat*
and τ :: (*complex*, $'p$ :: *finite*, $'p$) *lmat*
shows *Tr-lmat* (*tensor-lmat* σ τ :: (*complex*, $'m \times 'p$, $'m \times 'p$) *lmat*)
= *Tr-lmat* σ $*$ *Tr-lmat* τ
⟨*proof*⟩

lemma *zero-not-in-Sprime*:
(0 :: (*complex*, $'m$ :: *finite* $\times$ $'p$ :: *finite*, $'m \times 'p$) *lmat*) $\notin$ *Sprime*
⟨*proof*⟩

88 Closedness of the separable cone

Proof strategy (uses the typeclass tower from *Lmat-Euclidean*):
Let $S' = \{$*tensor-lmat* σ τ | σ τ. *psd-lmat* $\sigma \wedge$ *psd-lmat* $\tau \wedge$ *Tr* (*unwrap-lm* σ) $= 1 \wedge$ *Tr* (*unwrap-lm* τ) $= 1\}$ be the unit-trace product slice.

1. **S’ is compact.**
   - $\{\sigma$ :: *lmat*. *psd-lmat* $\sigma\}$ is closed (entry-wise lift of the Hermitian condition + entry-wise lift of the quadratic-form nonnegativity, both via *tendsto-unwrap-lm-index*).
   - $\{\sigma$ :: *lmat*. *Re* (*Tr* (*unwrap-lm* σ)) $= 1\}$ is closed (trace is continuous in entries, via *tendsto-unwrap-lm-index*).
   - PSD + unit-trace is bounded: *norm* $\sigma^2 =$ *Re Tr* $(\sigma^\dagger\ \sigma) \leq$ *Tr* $\sigma \cdot \lambda_{max} \leq 1$.
   - Closed + bounded in finite-dimensional *real-inner* = compact (*bounded-closed-imp-seq-compact* + heine_borel).
   - *tensor-lmat* is continuous (entries are bilinear in the entries of the factors, lifts via the typeclass).
   - S' is the image of (compact) $\times$ (compact) under *tensor-lmat*, hence compact.
2. **convex hull S’ is compact** via *compact-convex-hull* (applies because every euclidean_space is a real_inner finite-dimensional space).

3. **0 ∉ convex hull S'.** Every element of *convex hull S'* has trace 1 (linear combinations of trace-1 matrices with nonneg coefficients summing to 1), while 0 has trace 0.

4. **separable_lmat_gen = conic hull S'.** Each *tensor-lmat σ τ* with *σ*, *τ* PSD equals (*Tr σ · Tr τ*) · *tensor-lmat* (*σ*/*Tr σ*) (*τ*/*Tr τ*) when traces are positive, and equals *0* when one trace is 0 (in which case the corresponding factor is itself 0 because PSD + Tr = 0 ⇒ 0).

5. **Putting it together.** *separable-cone-lmat* = *convex-cone hull separable-lmat-gen* = *convex-cone hull* (*conic hull S'*) = *convex-cone hull S'* = *conic hull* (*convex hull S'*) (the last by *convex-cone-hull-separate-nonempty*). Apply *closed-conic-hull* with the (compact, *0* ∉ ...) clause.

The infrastructure for all of these steps is now available (*norm*, *inner*, *dist*, *compact*, *convex*, *convex-cone*, etc. all defined for typed lmat via the euclidean_space + heine_borel instances).

The convex cone hulls of *separable-lmat-gen* and *Sprime* coincide: every *tensor-lmat σ τ* with *σ*, *τ* PSD can be rescaled into *Sprime* by dividing by traces (when nonzero) or replaced by 0 when a trace is zero.

The hard direction depends on three facts that each lift a spectral / bilinearity property:
psd_lmat_scaleR PSD preserved under nonneg real scaling psd_lmat_zero_trace_zero PSD with Tr = 0 is the zero matrix tensor_lmat_scaleR_left,right tensor is bilinear over real scaling psd_lmat_Tr_real_nonneg Tr of PSD is a real nonneg complex.
These reduce *tensor σ τ* with *σ*, *τ* PSD to (*Tr σ* ∗ *Tr τ*) ∗$_R$ *tensor* (*σ*/*Tr σ*) (*τ*/*Tr τ*) when traces are positive, or to 0 when one is zero both forms in *convex-cone hull Sprime*.

lemma *Sprime-subset-separable-lmat-gen*:
(*Sprime* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat set*)
⊆ *separable-lmat-gen*
⟨*proof*⟩

The four spectral / bilinearity helpers needed for the hard direction. Each is a one-line lift of a JNF fact, but the JNF facts (spectral closure and tensor scaleR) live outside the parent heap so they are stated as targeted sorries here for the moment.

lemma *psd-lmat-Tr-real-nonneg*:
fixes *M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*
assumes *M-psd*: *psd-lmat M*
shows *Im* (*Tr-lmat M*) = *0* ∧ *0* ≤ *Re* (*Tr-lmat M*)
⟨*proof*⟩

lemma *psd-lmat-zero-trace-zero*:
fixes *M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*
assumes *M-psd*: *psd-lmat M* **and** *tr-zero*: *Tr-lmat M* = *0*
shows *M* = *0*
⟨*proof*⟩

lemma *psd-smult-nonneg-local*:
fixes *A* :: *complex Matrix.mat* **and** *c* :: *real*
assumes *c*: *0* ≤ *c*
and *A*: *psd A* **and** *Ac*: *A* ∈ *carrier-mat n n*
shows *psd* (*complex-of-real c* ·$_m$ *A*)
⟨*proof*⟩

lemma *unwrap-lm-scaleR-eq*:
fixes *M* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat* **and** *a* :: *real*

shows *unwrap-lm* $(a *_R M) =$ *complex-of-real* $a \cdot_m$ *unwrap-lm* M
⟨*proof*⟩

lemma *psd-lmat-scaleR-nonneg*:
fixes $M ::$ (*complex*, $'k ::$ *finite*, $'k$) *lmat* **and** $a ::$ *real*
assumes *M-psd*: *psd-lmat* M **and** *a-nn*: $0 \leq a$
shows *psd-lmat* $(a *_R M)$
⟨*proof*⟩

lemma *tensor-lmat-scaleR-left*:
fixes $a ::$ *real*
and $\sigma ::$ (*complex*, $'m ::$ *finite*, $'m$) *lmat*
and $\tau ::$ (*complex*, $'p ::$ *finite*, $'p$) *lmat*
shows (*tensor-lmat* $(a *_R \sigma)$ τ
:: (*complex*, $'m \times 'p$, $'m \times 'p$) *lmat*)
$= a *_R$ *tensor-lmat* σ τ
⟨*proof*⟩

lemma *tensor-lmat-scaleR-right*:
fixes $a ::$ *real*
and $\sigma ::$ (*complex*, $'m ::$ *finite*, $'m$) *lmat*
and $\tau ::$ (*complex*, $'p ::$ *finite*, $'p$) *lmat*
shows (*tensor-lmat* σ $(a *_R \tau)$
:: (*complex*, $'m \times 'p$, $'m \times 'p$) *lmat*)
$= a *_R$ *tensor-lmat* σ τ
⟨*proof*⟩

lemma *tensor-lmat-zero-left*:
fixes $\tau ::$ (*complex*, $'p ::$ *finite*, $'p$) *lmat*
shows (*tensor-lmat* $(0 ::$ (*complex*, $'m ::$ *finite*, $'m$) *lmat*) τ
:: (*complex*, $'m \times 'p$, $'m \times 'p$) *lmat*) $= 0$
⟨*proof*⟩

lemma *tensor-lmat-zero-right*:
fixes $\sigma ::$ (*complex*, $'m ::$ *finite*, $'m$) *lmat*
shows (*tensor-lmat* σ $(0 ::$ (*complex*, $'p ::$ *finite*, $'p$) *lmat*)
:: (*complex*, $'m \times 'p$, $'m \times 'p$) *lmat*) $= 0$
⟨*proof*⟩

lemma *convex-cone-hull-separable-eq-Sprime*:
(*convex-cone hull*
(*separable-lmat-gen*
:: (*complex*, $'m ::$ *finite* $\times$ $'p ::$ *finite*, $'m \times 'p$) *lmat set*))
$=$ *convex-cone hull Sprime*
⟨*proof*⟩

theorem *separable-cone-lmat-closed*:
closed (*separable-cone-lmat*
:: (*complex*, $'m ::$ *finite* $\times$ $'p ::$ *finite*, $'m \times 'p$) *lmat set*)
⟨*proof*⟩

89 The lmat inner product as a real-trace inner product

The real inner product on (*complex*, $'m$, $'n$) *lmat* is the real part of the JNF Hilbert-Schmidt inner product on the underlying matrices. For square matrices it is also $Re\ (Tr\ (M^\dagger * N))$.

lemma *inner-lmat-eq-re-hs-inner-prod*:
fixes $M\ N ::$ (*complex*, $'m ::$ *finite*, $'n ::$ *finite*) *lmat*

shows *inner M N = Re ⟨⟨ unwrap-lm M | unwrap-lm N ⟩⟩*
⟨*proof*⟩

lemma *inner-lmat-eq-re-tr-dagger*:
fixes *M N* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*
shows *inner M N = Re (Tr ((unwrap-lm M)† ∗ unwrap-lm N))*
⟨*proof*⟩

90 Separating hyperplane for non-separable states

The closedness of the separable cone, combined with HOL-Analysis *separating-hyperplane-closed-point*, yields the geometric Hahn-Banach witness: for every ϱ outside the separable cone there is a linear functional (an HS inner product against some matrix a) that is strictly negative on ϱ and nonneg on every separable element. Because the separable set is a CONE, the strict inequality *inner a x > b* on the cone forces $b < 0$ and *inner a x* ≥ 0 on the cone.

theorem *separable-cone-lmat-separating-hyperplane*:
fixes *ϱ* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat*
assumes *notin*: *ϱ* ∉ *separable-cone-lmat*
shows ∃ *a* :: (*complex*, *'m* × *'p*, *'m* × *'p*) *lmat*.
inner a ϱ < 0
∧ (∀ *σ* ∈ *separable-cone-lmat*. *0 ≤ inner a σ*)
⟨*proof*⟩

91 Entanglement witness in real-trace form

Reformulating the separating hyperplane via the *inner = Re Tr* identity gives a JNF matrix W whose adjoint times ϱ has strictly negative real trace, while $W^\dagger * \sigma$ has nonneg real trace for every σ in the separable cone.

theorem *entanglement-witness-re-tr*:
fixes *ϱ* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat*
assumes *notin*: *ϱ* ∉ *separable-cone-lmat*
shows ∃ *W* :: *complex Matrix.mat*.
W ∈ *carrier-mat* (*CARD*(*'m* × *'p*)) (*CARD*(*'m* × *'p*))
∧ *Re* (*Tr* (*W* ∗ *unwrap-lm ϱ*)) < *0*
∧ (∀ *σ* :: (*complex*, *'m* × *'p*, *'m* × *'p*) *lmat*.
σ ∈ *separable-cone-lmat*
⟶ *0* ≤ *Re* (*Tr* (*W* ∗ *unwrap-lm σ*)))
⟨*proof*⟩

92 Hermitian entanglement witness

For ϱ outside the separable cone with hermitian unwrap, the witness can be chosen hermitian. Direct entrywise construction of $W = (A + A^\dagger)/2$ using *Matrix.mat*; we avoid lemmas about *dagger* distributing over sums or scalar multiplication.

theorem *entanglement-witness-hermitian*:
fixes *ϱ* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat*
assumes *notin*: *ϱ* ∉ *separable-cone-lmat*
and *ϱ-herm*: *hermitian* (*unwrap-lm ϱ*)
shows ∃ *W* :: *complex Matrix.mat*.
W ∈ *carrier-mat* (*CARD*(*'m* × *'p*)) (*CARD*(*'m* × *'p*))
∧ *hermitian W*
∧ *Re* (*Tr* (*W* ∗ *unwrap-lm ϱ*)) < *0*
∧ (∀ *σ* :: (*complex*, *'m* × *'p*, *'m* × *'p*) *lmat*.

$\sigma \in$ *separable-cone-lmat*
$\wedge$ *hermitian* (*unwrap-lm* σ)
$\longrightarrow$ $0 \leq$ *Re* (*Tr* (*W* $*$ *unwrap-lm* σ)))
⟨*proof*⟩

93 Hermitian closure of the separable cone

Every element of *separable-cone-lmat* has hermitian unwrap. Generators *tensor-lmat* σ τ with σ, τ PSD are hermitian, and the hermitian set is closed under + and nonneg real scaling.

lemma *hermitian-zero-mat*:
hermitian (0_m *n n* :: *complex Matrix.mat*)
⟨*proof*⟩

lemma *hermitian-add-mat*:
fixes *P Q* :: *complex Matrix.mat*
assumes *Pcar*: *P* $\in$ *carrier-mat n n* **and** *Qcar*: *Q* $\in$ *carrier-mat n n*
and *Pherm*: *hermitian P* **and** *Qherm*: *hermitian Q*
shows *hermitian* (*P* + *Q*)
⟨*proof*⟩

lemma *hermitian-smult-real-mat*:
fixes *M* :: *complex Matrix.mat* **and** *c* :: *real*
assumes *Mcar*: *M* $\in$ *carrier-mat n n* **and** *Mherm*: *hermitian M*
shows *hermitian* (*complex-of-real c* $\cdot_m$ *M*)
⟨*proof*⟩

Tensor product of hermitians is hermitian (entrywise via *index-tensor-mat*).

lemma *hermitian-tensor-mat*:
fixes *P Q* :: *complex Matrix.mat*
assumes *Pcar*: *P* $\in$ *carrier-mat m m* **and** *Qcar*: *Q* $\in$ *carrier-mat p p*
and *m-pos*: $0 <$ *m* **and** *p-pos*: $0 <$ *p*
and *Pherm*: *hermitian P* **and** *Qherm*: *hermitian Q*
shows *hermitian* (*P* $\bigotimes$ *Q*)
⟨*proof*⟩

Generators of *separable-cone-lmat* have hermitian unwrap.

lemma *separable-lmat-gen-hermitian*:
fixes σ :: (*complex*, $'m$ :: *finite* $\times$ $'p$ :: *finite*, $'m \times 'p$) *lmat*
assumes $\sigma \in$ *separable-lmat-gen*
shows *hermitian* (*unwrap-lm* σ)
⟨*proof*⟩

The hermitian set in lmat is a convex cone, hence contains the hull of the generators.

lemma *hermitian-lmat-set-convex-cone*:
convex-cone {*M* :: (*complex*, $'k$ :: *finite*, $'k$) *lmat*.
hermitian (*unwrap-lm M*)}
⟨*proof*⟩

theorem *separable-cone-lmat-hermitian*:
fixes σ :: (*complex*, $'m$ :: *finite* $\times$ $'p$ :: *finite*, $'m \times 'p$) *lmat*
assumes $\sigma \in$ *separable-cone-lmat*
shows *hermitian* (*unwrap-lm* σ)
⟨*proof*⟩

Final form of the entanglement witness theorem: the side condition "σ has hermitian unwrap" is automatic for elements of the separable cone.

theorem *entanglement-witness-hermitian'*:
fixes ϱ :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat*
assumes *notin*: $\varrho \notin$ *separable-cone-lmat*
and ϱ-*herm*: *hermitian* (*unwrap-lm* ϱ)
shows $\exists W$ :: *complex Matrix.mat*.
W ∈ *carrier-mat* (*CARD*($'m$ × $'p$)) (*CARD*($'m$ × $'p$))
∧ *hermitian W*
∧ *Re* (*Tr* (*W* ∗ *unwrap-lm* ϱ)) < *0*
∧ (∀ σ :: (*complex*, $'m$ × $'p$, $'m$ × $'p$) *lmat*.
σ ∈ *separable-cone-lmat*
⟶ *0* ≤ *Re* (*Tr* (*W* ∗ *unwrap-lm* σ)))
⟨*proof*⟩

94 Main separability characterisation

The Horodecki-style witness characterisation: for hermitian ϱ, membership in the separable cone is equivalent to the nonneg-real-trace property holding against every hermitian witness that is itself nonneg-real-trace on the cone.

theorem *separable-iff-witness*:
fixes ϱ :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat*
assumes ϱ-*herm*: *hermitian* (*unwrap-lm* ϱ)
shows ϱ ∈ *separable-cone-lmat*
⟷
(∀ *W* :: *complex Matrix.mat*.
W ∈ *carrier-mat* (*CARD*($'m$ × $'p$)) (*CARD*($'m$ × $'p$))
∧ *hermitian W*
∧ (∀ σ :: (*complex*, $'m$ × $'p$, $'m$ × $'p$) *lmat*.
σ ∈ *separable-cone-lmat*
⟶ *0* ≤ *Re* (*Tr* (*W* ∗ *unwrap-lm* σ)))
⟶ *0* ≤ *Re* (*Tr* (*W* ∗ *unwrap-lm* ϱ)))
⟨*proof*⟩

95 Density-matrix specialisation

For an actual quantum density matrix ϱ (PSD with unit trace), the conditions *psd-lmat* ϱ and *Tr-lmat* ϱ = *1* imply *hermitian* (*unwrap-lm* ϱ), so the witness theorem and the characterisation specialise cleanly.

theorem *entanglement-witness-for-density*:
fixes ϱ :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat*
assumes ϱ-*psd*: *psd-lmat* ϱ
and ϱ-*tr*: *Tr-lmat* ϱ = *1*
and ϱ-*not-sep*: $\varrho \notin$ *separable-cone-lmat*
shows $\exists W$ :: *complex Matrix.mat*.
W ∈ *carrier-mat* (*CARD*($'m$ × $'p$)) (*CARD*($'m$ × $'p$))
∧ *hermitian W*
∧ *Re* (*Tr* (*W* ∗ *unwrap-lm* ϱ)) < *0*
∧ (∀ σ :: (*complex*, $'m$ × $'p$, $'m$ × $'p$) *lmat*.
σ ∈ *separable-cone-lmat*
⟶ *0* ≤ *Re* (*Tr* (*W* ∗ *unwrap-lm* σ)))
⟨*proof*⟩

The witness is also nonneg on every separable density matrix (*Sprime*) since *Sprime* ⊆ *separable-cone-lmat*.

theorem *entanglement-witness-for-density-on-Sprime*:

fixes ϱ :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat*
assumes *ϱ-psd*: *psd-lmat* ϱ
and *ϱ-tr*: *Tr-lmat* $\varrho = 1$
and *ϱ-not-sep*: $\varrho \notin$ *separable-cone-lmat*
shows $\exists W$ :: *complex Matrix.mat*.
$W \in$ *carrier-mat* (*CARD*($'m \times 'p$)) (*CARD*($'m \times 'p$))
$\wedge$ *hermitian W*
$\wedge$ *Re* (*Tr* (*W* $*$ *unwrap-lm* ϱ)) < 0
$\wedge$ ($\forall \sigma$ :: (*complex*, $'m \times 'p$, $'m \times 'p$) *lmat*.
$\sigma \in$ *Sprime* $\longrightarrow 0 \leq$ *Re* (*Tr* (*W* $*$ *unwrap-lm* σ)))
⟨*proof*⟩

The full Horodecki characterisation, specialised to density matrices.

theorem *separable-density-iff-witness*:
fixes ϱ :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat*
assumes *ϱ-psd*: *psd-lmat* ϱ
and *ϱ-tr*: *Tr-lmat* $\varrho = 1$
shows $\varrho \in$ *separable-cone-lmat*
$\longleftrightarrow$
($\forall W$ :: *complex Matrix.mat*.
$W \in$ *carrier-mat* (*CARD*($'m \times 'p$)) (*CARD*($'m \times 'p$))
$\wedge$ *hermitian W*
$\wedge$ ($\forall \sigma$ :: (*complex*, $'m \times 'p$, $'m \times 'p$) *lmat*.
$\sigma \in$ *separable-cone-lmat*
$\longrightarrow 0 \leq$ *Re* (*Tr* (*W* $*$ *unwrap-lm* σ)))
$\longrightarrow 0 \leq$ *Re* (*Tr* (*W* $*$ *unwrap-lm* ϱ)))
⟨*proof*⟩

The trace is identically *1* on the convex hull of *Sprime*.

lemma *Tr-lmat-convex-hull-Sprime-eq-one*:
fixes ϱ :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat*
assumes $\varrho \in$ *convex hull Sprime*
shows *Tr-lmat* $\varrho = 1$
⟨*proof*⟩

A density matrix is separable iff it is a convex combination of separable pure-product states. This is the standard textbook definition of separability for mixed states.

theorem *separable-density-iff-in-convex-hull-Sprime*:
fixes ϱ :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat*
assumes *ϱ-psd*: *psd-lmat* ϱ
and *ϱ-tr*: *Tr-lmat* $\varrho = 1$
shows $\varrho \in$ *separable-cone-lmat* $\longleftrightarrow \varrho \in$ *convex hull Sprime*
⟨*proof*⟩

Topology of the separable density-matrix set.

lemma *compact-convex-hull-Sprime*:
compact (*convex hull*
(*Sprime* :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat set*))
⟨*proof*⟩

lemma *closed-convex-hull-Sprime*:
closed (*convex hull*
(*Sprime* :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat set*))
⟨*proof*⟩

lemma *bounded-convex-hull-Sprime*:
bounded (*convex hull*

```
      (Sprime :: (complex, 'm :: finite × 'p :: finite, 'm × 'p) lmat set))
⟨proof⟩
```

Strict separation in the inner-product form: by closedness of *convex hull Sprime* we get a strictly negative-inner-product hyperplane. The Re-trace strict-form is then an immediate corollary once the hermitian witness construction is replayed.

```
lemma separable-density-separating-hyperplane:
  fixes ϱ :: (complex, 'm :: finite × 'p :: finite, 'm × 'p) lmat
  assumes ϱ-psd: psd-lmat ϱ
    and ϱ-tr: Tr-lmat ϱ = 1
    and ϱ-not-sep: ϱ ∉ separable-cone-lmat
  shows ∃ a b.
        inner a ϱ < b
      ∧ (∀ σ :: (complex, 'm × 'p, 'm × 'p) lmat.
          σ ∈ convex hull Sprime ⟶ b < inner a σ)
⟨proof⟩
```

The inner-product-to-Re-trace bridge for hermitian arguments, extracted as a separate lemma so we can reuse it for the strict separation theorem. This is the same hermitisation construction that drives *entanglement-witness-hermitian*.

```
lemma hermitian-witness-from-lmat:
  fixes a :: (complex, 'k :: finite, 'k) lmat
  shows ∃ W :: complex Matrix.mat.
        W ∈ carrier-mat (CARD('k)) (CARD('k))
      ∧ hermitian W
      ∧ (∀ M :: (complex, 'k, 'k) lmat.
          hermitian (unwrap-lm M)
            ⟶ Re (Tr (W ∗ unwrap-lm M)) = inner a M)
⟨proof⟩
```

The strict Re-trace form of the entanglement witness.

```
theorem entanglement-witness-strict:
  fixes ϱ :: (complex, 'm :: finite × 'p :: finite, 'm × 'p) lmat
  assumes ϱ-psd: psd-lmat ϱ
    and ϱ-tr: Tr-lmat ϱ = 1
    and ϱ-not-sep: ϱ ∉ separable-cone-lmat
  shows ∃ (W :: complex Matrix.mat) c.
        W ∈ carrier-mat (CARD('m × 'p)) (CARD('m × 'p))
      ∧ hermitian W
      ∧ Re (Tr (W ∗ unwrap-lm ϱ)) < c
      ∧ (∀ σ :: (complex, 'm × 'p, 'm × 'p) lmat.
          σ ∈ convex hull Sprime
            ⟶ c < Re (Tr (W ∗ unwrap-lm σ)))
⟨proof⟩


end


theory Day24-Separability
  imports
    Quantum-Computing11
    Vern-Paulsen-QC-3-10-Lmat.Separable-Cone
begin
```

96 Days 26–28 (28 October – 2 November 2011): separability and entanglement witnesses

Entanglement revisited. Ensembles $\{\psi_i, p_i\}$ on $\mathcal{H}_A$ and $\{\phi_j, q_j\}$ on $\mathcal{H}_B$ compose to the density matrix $\sum_{i,j} p_i q_j |\psi_i\rangle\langle\psi_i| \otimes |\phi_j\rangle\langle\phi_j|$.

Definition (notes) (Day 26)**.** A density matrix $P \in L(\mathcal{H}_A \otimes \mathcal{H}_B)$ is *separable* if $P = \sum_l p_l |\psi_l\rangle\langle\psi_l| \otimes |\phi_l\rangle\langle\phi_l|$ with unit vectors $\psi_l \in \mathcal{H}_A$, $\phi_l \in \mathcal{H}_B$, $p_l \geq 0$, $\sum p_l = 1$; otherwise P is *entangled*. More generally (Day 27) $Sep \subseteq M_r(M_n)$ is the set of matrices $\sum_l P_l \otimes Q_l$ with $P_l \geq 0$, $Q_l \geq 0$.

Are there entangled density matrices, and how can one tell? The notes answer with the theory of convex cones:

Theorem (notes) (Days 27–28)**.** Let V be a finite-dimensional real vector space, $K \subseteq V$ closed and convex, $y \notin K$. Then there is a linear $f : V \to \mathbb{R}$ and $\alpha \in \mathbb{R}$ with $f(y) < \alpha \leq f(K)$; if C is a closed cone and $w \notin C$ there is f with $f(w) < 0 \leq f(C)$.

The separable matrices form a closed cone C in the real vector space of Hermitian matrices; an entangled R is separated from C by a real functional, which complexifies to $\tilde{f}(H + iK) = f(H) + if(K)$, and $\tilde{f}$ corresponds to a Hermitian matrix W with $\mathrm{Tr}(WR) < 0 \leq \mathrm{Tr}(W\sigma)$ for all separable σ: an *entanglement witness*. The notes phrase the final criterion through positive maps Φ_f obtained from the Arveson correspondence; that form is *separable-iff-positive-maps* in *Day27-Positive-Map-Criterion*, derived from the witness form proved here.

The formal development uses the typed matrices *lmat* so that HOL-Analysis' separation theorem applies: *separable-cone-lmat* is the convex cone generated by tensor products of positive matrices and *Sprime* is the set of product density matrices.

96.1 The separable cone (Day 27)

The cone is convex, conic, contains 0, and consists of Hermitian matrices.

theorem *D27-separable-cone-convex-cone*:
  convex-cone (*separable-cone-lmat*
    :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat set*)
  ⟨*proof*⟩

theorem *D27-zero-separable*:
  (*0* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat*)
    ∈ *separable-cone-lmat*
  ⟨*proof*⟩

theorem *D27-separable-state-hermitian*:
  fixes *ϱ* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat*
  assumes *ϱ* ∈ *separable-cone-lmat*
  shows *hermitian* (*unwrap-lm ϱ*)
  ⟨*proof*⟩

96.2 Separable density matrices (Day 26)

For a density matrix ρ (positive with $\mathrm{Tr}\,\rho = 1$), membership in the cone is the notes' definition: ρ is a convex combination of product density matrices $\sigma_i \otimes \tau_i$.

theorem *D26-separable-density-iff-convex-combination*:
  fixes *ϱ* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat*
  assumes *ϱ-psd*: *psd-lmat ϱ*

and *ϱ-tr*: *Tr-lmat ϱ = 1*
shows *ϱ ∈ separable-cone-lmat ⟷ ϱ ∈ convex hull Sprime*
⟨*proof*⟩

96.3 The separable cone is closed (Day 28)

The notes record that C is a closed cone (needed for the separation theorem); here the cone is closed in the Hilbert–Schmidt topology and the separable density matrices form a compact convex set.

theorem *D28-separable-cone-closed*:
closed (*separable-cone-lmat*
:: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat set*)
⟨*proof*⟩

theorem *D28-separable-density-compact*:
compact (*convex hull*
(*Sprime* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat set*))
⟨*proof*⟩

theorem *D28-separable-density-closed*:
closed (*convex hull*
(*Sprime* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat set*))
⟨*proof*⟩

theorem *D28-separable-density-bounded*:
bounded (*convex hull*
(*Sprime* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat set*))
⟨*proof*⟩

96.4 Separation (Days 27–28)

The separation theorem applied to the cone: a non-separable ρ is separated from C by a real-linear functional (an inner product against some a), strictly for density matrices because the separable density matrices are compact.

theorem *D27-separating-hyperplane-cone*:
fixes *ϱ* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat*
assumes *ϱ ∉ separable-cone-lmat*
shows ∃ *a* :: (*complex*, *'m* × *'p*, *'m* × *'p*) *lmat*.
inner a ϱ < 0
∧ (∀ *σ* ∈ *separable-cone-lmat*. *0 ≤ inner a σ*)
⟨*proof*⟩

theorem *D27-strict-hyperplane-density*:
fixes *ϱ* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat*
assumes *ϱ-psd*: *psd-lmat ϱ*
and *ϱ-tr*: *Tr-lmat ϱ = 1*
and *ϱ-not-sep*: *ϱ ∉ separable-cone-lmat*
shows ∃ *a b*.
inner a ϱ < b
∧ (∀ *σ* :: (*complex*, *'m* × *'p*, *'m* × *'p*) *lmat*.
σ ∈ convex hull Sprime ⟶ b < inner a σ)
⟨*proof*⟩

96.5 Entanglement witnesses (Day 28)

An *entanglement witness* for ρ is a Hermitian W with $\mathrm{Tr}(W\sigma) \geq 0$ on every separable σ and $\mathrm{Tr}(W\rho) < 0$. (i) Every non-separable Hermitian ρ has a witness; (ii) density matrices have a witness with a strict margin; (iii) (Horodecki) ρ is separable iff every Hermitian W that is non-negative on the cone is non-negative on ρ.

theorem *D28-witness-exists-hermitian*:
fixes ϱ :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat*
assumes *notin*: $\varrho \notin$ *separable-cone-lmat*
and *ϱ-herm*: *hermitian* (*unwrap-lm* ϱ)
shows ∃ W :: *complex Matrix.mat*.
$W \in$ *carrier-mat* (*CARD*($'m$ × $'p$)) (*CARD*($'m$ × $'p$))
∧ *hermitian W*
∧ *Re* (*Tr* (*W* ∗ *unwrap-lm* ϱ)) < *0*
∧ (∀ σ :: (*complex*, $'m$ × $'p$, $'m$ × $'p$) *lmat*.
$\sigma \in$ *separable-cone-lmat*
⟶ *0* ≤ *Re* (*Tr* (*W* ∗ *unwrap-lm* σ)))
⟨*proof*⟩

theorem *D28-witness-for-density*:
fixes ϱ :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat*
assumes *ϱ-psd*: *psd-lmat* ϱ
and *ϱ-tr*: *Tr-lmat* ϱ = *1*
and *ϱ-not-sep*: $\varrho \notin$ *separable-cone-lmat*
shows ∃ W :: *complex Matrix.mat*.
$W \in$ *carrier-mat* (*CARD*($'m$ × $'p$)) (*CARD*($'m$ × $'p$))
∧ *hermitian W*
∧ *Re* (*Tr* (*W* ∗ *unwrap-lm* ϱ)) < *0*
∧ (∀ σ :: (*complex*, $'m$ × $'p$, $'m$ × $'p$) *lmat*.
$\sigma \in$ *separable-cone-lmat*
⟶ *0* ≤ *Re* (*Tr* (*W* ∗ *unwrap-lm* σ)))
⟨*proof*⟩

theorem *D28-witness-strict-density*:
fixes ϱ :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat*
assumes *ϱ-psd*: *psd-lmat* ϱ
and *ϱ-tr*: *Tr-lmat* ϱ = *1*
and *ϱ-not-sep*: $\varrho \notin$ *separable-cone-lmat*
shows ∃ (*W* :: *complex Matrix.mat*) *c*.
$W \in$ *carrier-mat* (*CARD*($'m$ × $'p$)) (*CARD*($'m$ × $'p$))
∧ *hermitian W*
∧ *Re* (*Tr* (*W* ∗ *unwrap-lm* ϱ)) < *c*
∧ (∀ σ :: (*complex*, $'m$ × $'p$, $'m$ × $'p$) *lmat*.
$\sigma \in$ *convex hull Sprime*
⟶ *c* < *Re* (*Tr* (*W* ∗ *unwrap-lm* σ)))
⟨*proof*⟩

theorem *D28-separable-iff-witness*:
fixes ϱ :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat*
assumes *ϱ-herm*: *hermitian* (*unwrap-lm* ϱ)
shows $\varrho \in$ *separable-cone-lmat*
⟷
(∀ W :: *complex Matrix.mat*.
$W \in$ *carrier-mat* (*CARD*($'m$ × $'p$)) (*CARD*($'m$ × $'p$))
∧ *hermitian W*
∧ (∀ σ :: (*complex*, $'m$ × $'p$, $'m$ × $'p$) *lmat*.
$\sigma \in$ *separable-cone-lmat*

$\longrightarrow$ *0* $\leq$ *Re* (*Tr* (*W* $*$ *unwrap-lm* σ)))
$\longrightarrow$ *0* $\leq$ *Re* (*Tr* (*W* $*$ *unwrap-lm* ϱ)))
⟨*proof*⟩

theorem *D28-separable-density-iff-witness*:
fixes ϱ :: (*complex*, $'m$ :: *finite* $\times$ $'p$ :: *finite*, $'m \times 'p$) *lmat*
assumes ϱ*-psd*: *psd-lmat* ϱ
and ϱ*-tr*: *Tr-lmat* ϱ = *1*
shows $\varrho \in$ *separable-cone-lmat*
$\longleftrightarrow$
(∀ *W* :: *complex Matrix.mat*.
W $\in$ *carrier-mat* (*CARD*($'m \times 'p$)) (*CARD*($'m \times 'p$))
$\wedge$ *hermitian W*
$\wedge$ (∀ σ :: (*complex*, $'m \times 'p$, $'m \times 'p$) *lmat*.
$\sigma \in$ *separable-cone-lmat*
$\longrightarrow$ *0* $\leq$ *Re* (*Tr* (*W* $*$ *unwrap-lm* σ)))
$\longrightarrow$ *0* $\leq$ *Re* (*Tr* (*W* $*$ *unwrap-lm* ϱ)))
⟨*proof*⟩

This closes the entanglement chapter of the notes. The Horodecki–Peres criterion for two qubits is *Day28-Horodecki-Peres*, the positive-map formulation *Day27-Positive-Map-Criterion*; the universal-witness question (Day 28) is not formalised.

end
theory *Day27-Positive-Map-Criterion*
imports *Day26-Arveson-Extension Day34-Knill-Laflamme*
begin

Days 27–28: positive maps detect entanglement. For $R \in M_r(M_n)$ (block size n), $\Phi^{(r)}(R) \geq 0$ for every positive Φ when R is separable (D27-1); a functional f on $M_r(M_n)$ is nonnegative on the separable matrices iff Φ_f is positive (D27-3); and with the entanglement witnesses of *Separable-Cone* this gives the positive-map criterion (D28-3).

lemma *block-of-mat-carrier'* [*simp*]: *block-of-mat n A a b* $\in$ *carrier-mat n n*
⟨*proof*⟩

96.6 Tensor products with a matrix in the first factor

lemma *tensor-add-first*:
fixes *A B P* :: *complex Matrix.mat*
assumes *A*: *A* $\in$ *carrier-mat r r* **and** *B*: *B* $\in$ *carrier-mat r r* **and** *P*: *P* $\in$ *carrier-mat n n* **and** *n*: *0* $< n$
shows *Tensor.tensor-mat* (*A* + *B*) *P* = *Tensor.tensor-mat A P* + *Tensor.tensor-mat B P*
⟨*proof*⟩

lemma *tensor-zero-first*:
fixes *P* :: *complex Matrix.mat*
assumes *P*: *P* $\in$ *carrier-mat n n* **and** *n*: *0* $< n$
shows *Tensor.tensor-mat* (0_m *r r*) *P* = 0_m (*r* $*$ *n*) (*r* $*$ *n*)
⟨*proof*⟩

lemma *tensor-msum-first*:
fixes *P* :: *complex Matrix.mat*
assumes *g*: ∀ *x* $\in$ *set xs*. *g x* $\in$ *carrier-mat r r* **and** *P*: *P* $\in$ *carrier-mat n n* **and** *n*: *0* $< n$
shows *Tensor.tensor-mat* (*msum r r g xs*) *P* = *msum* (*r* $*$ *n*) (*r* $*$ *n*) (λx. *Tensor.tensor-mat* (*g x*) *P*) *xs*
⟨*proof*⟩

96.7 The quadratic form of Φ_f

definition *rank1* :: *complex Matrix.vec* ⇒ *complex Matrix.mat* **where**
rank1 w = *Matrix.mat* (*dim-vec w*) (*dim-vec w*) (λ(*i*,*j*). *cnj* (*w* $ *i*) ∗ *w* $ *j*)

lemma *rank1-carrier* [*simp*]: *rank1 w* ∈ *carrier-mat* (*dim-vec w*) (*dim-vec w*)
⟨*proof*⟩

lemma *rank1-dims* [*simp*]: *dim-row* (*rank1 w*) = *dim-vec w dim-col* (*rank1 w*) = *dim-vec w*
⟨*proof*⟩

lemma *rank1-outer*: *rank1* (*conjugate u*) = *u* ⊘ *u*
⟨*proof*⟩

lemma *rank1-psd*: *psd* (*rank1 w*)
⟨*proof*⟩

lemma *quad-arveson-map*:
fixes *f* :: *complex Matrix.mat* ⇒ *complex* **and** *w* :: *complex Matrix.vec*
assumes *f*: *lin-fun-on* (*carrier-mat* (*r* ∗ *n*) (*r* ∗ *n*)) *f* **and** *w*: *dim-vec w* = *r* **and** *P*: *P* ∈ *carrier-mat n n* **and** *n*: *0* < *n*
shows ⟨*w* | *arveson-map n r f P* ∗*v w*⟩ = *of-nat r* ∗ *f* (*Tensor.tensor-mat* (*rank1 w*) *P*)
⟨*proof*⟩

96.8 Proposition (Day 27): $f(\mathrm{Sep}) \geq 0$ iff Φ_f is positive

theorem *positive-map-iff-sep-nonneg*:
fixes *f* :: *complex Matrix.mat* ⇒ *complex*
assumes *f*: *lin-fun-on* (*carrier-mat* (*r* ∗ *n*) (*r* ∗ *n*)) *f* **and** *r*: *0* < *r* **and** *n*: *0* < *n*
shows *positive-map n r* (*arveson-map n r f*)
⟷ (∀ *Q P*. *Q* ∈ *carrier-mat r r* ⟶ *psd Q* ⟶ *P* ∈ *carrier-mat n n* ⟶ *psd P*
⟶ *0* ≤ *f* (*Tensor.tensor-mat Q P*))
⟨*proof*⟩

96.9 Proposition (Day 27): positive maps keep separable matrices positive

lemma *block-tensor*:
fixes *Q P* :: *complex Matrix.mat*
assumes *Q*: *Q* ∈ *carrier-mat r r* **and** *P*: *P* ∈ *carrier-mat n n* **and** *a*: *a* < *r* **and** *b*: *b* < *r*
shows *block-of-mat n* (*Tensor.tensor-mat Q P*) *a b* = *Q* $$ (*a*,*b*) ·*m P*
⟨*proof*⟩

lemma *amplification-tensor*:
fixes *Q P* :: *complex Matrix.mat* **and** Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *Q*: *Q* ∈ *carrier-mat r r* **and** *P*: *P* ∈ *carrier-mat n n*
and *d*: *0* < *d* **and** *n*: *0* < *n*
shows *amplification n d r* Φ (*Tensor.tensor-mat Q P*) = *Tensor.tensor-mat Q* (Φ *P*)
⟨*proof*⟩

lemma *amplification-lin*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *A*: *A* ∈ *carrier-mat* (*p* ∗ *n*) (*p* ∗ *n*) **and** *B*: *B* ∈ *carrier-mat* (*p* ∗ *n*) (*p* ∗ *n*)
shows *amplification n d p* Φ (*A* + *B*) = *amplification n d p* Φ *A* + *amplification n d p* Φ *B*
and *amplification n d p* Φ (*c* ·*m A*) = *c* ·*m amplification n d p* Φ *A*
⟨*proof*⟩

lemma *unwrap-lm-scaleR-mat*:
fixes *R* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*

shows *unwrap-lm* $(u *_R R) = $ *complex-of-real* $u \cdot_m$ *unwrap-lm* R
⟨*proof*⟩

theorem *positive-map-separable*:
fixes R :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat* **and** Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *L*: *lin-on CARD*($'p$) *d* (*carrier-mat CARD*($'p$) *CARD*($'p$)) Φ **and** *pos*: *positive-map CARD*($'p$) *d* Φ
and *d*: $0 < d$ **and** *R*: $R \in$ *separable-cone-lmat*
shows *psd* (*amplification CARD*($'p$) *d CARD*($'m$) Φ (*unwrap-lm R*))
⟨*proof*⟩

96.10 Theorem (Day 28): the positive-map criterion

A Hermitian $R \in M_r(M_n)$ is separable iff $\Phi^{(r)}(R) \geq 0$ for every positive $\Phi : M_n \to M_r$.

theorem *separable-iff-positive-maps*:
fixes R :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat*
assumes *Rh*: *hermitian* (*unwrap-lm R*)
shows $R \in$ *separable-cone-lmat* ⟷
($\forall \Phi$. *lin-on CARD*($'p$) *CARD*($'m$) (*carrier-mat CARD*($'p$) *CARD*($'p$)) Φ ⟶ *positive-map CARD*($'p$) *CARD*($'m$) Φ
⟶ *psd* (*amplification CARD*($'p$) *CARD*($'m$) *CARD*($'m$) Φ (*unwrap-lm R*)))
⟨*proof*⟩

end
theory *Day28-Horodecki-Peres*
imports *QC-Prelims*
begin

97 Day 28: the Horodecki–Peres criterion for two qubits (D28-5)

Entries of the partial transpose $\Phi^{(2)}(X)$ of a 4×4 matrix.

lemma *partial-transpose-2-index*:
fixes M :: *complex Matrix.mat*
assumes *i*: $i < 4$ **and** *j*: $j < 4$
shows *amplification 2 2 2* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*) M $$ (*i*,*j*)
= M $$ ((*i div 2*) * *2* + *j mod 2*, (*j div 2*) * *2* + *i mod 2*)
⟨*proof*⟩

lemma *product-vec-iff-det-zero*:
fixes v :: *complex Matrix.vec*
assumes *vd*: *dim-vec* $v = 4$
shows ($\exists \alpha\ \beta$. *dim-vec* $\alpha = 2 \wedge$ *dim-vec* $\beta = 2 \wedge v =$ *tensor-vec* $\alpha\ \beta$)
⟷ $v \$ 0 * v \$ 3 - v \$ 1 * v \$ 2 = 0$
⟨*proof*⟩

The notes' witness $w = (-\overline{v_2}, \overline{v_1})$ gives $\langle w|\Phi^{(2)}(|v\rangle\langle v|)w\rangle = -2|v_0 v_3 - v_1 v_2|^2$ $(= 2|\langle v_2|v_1\rangle|^2 - 2\|v_1\|^2\|v_2\|^2$ by Lagrange's identity).

lemma *partial-transpose-witness-value*:
fixes v :: *complex Matrix.vec*
assumes *vd*: *dim-vec* $v = 4$
defines $w \equiv$ *Matrix.vec 4* (λk. *if* $k = 0$ *then* − *cnj* ($v \$ 2$) *else if* $k = 1$ *then* − *cnj* ($v \$ 3$)
else if $k = 2$ *then cnj* ($v \$ 0$) *else cnj* ($v \$ 1$))
shows $\langle w \mid$ *amplification 2 2 2* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*) ($v \oslash v$) $*_v$ $w\rangle$

$= -\ 2 * cnj\ (v\ \$\ 0 * v\ \$\ 3 - v\ \$\ 1 * v\ \$\ 2) * (v\ \$\ 0 * v\ \$\ 3 - v\ \$\ 1 * v\ \$\ 2)$
⟨*proof*⟩

theorem *horodecki-peres-two-qubits*:
fixes v :: *complex Matrix.vec*
assumes *vd*: *dim-vec* $v = 4$
shows *psd* (*amplification 2 2 2* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*) ($v \oslash v$))
$\longleftrightarrow$ ($\exists \alpha\ \beta$. *dim-vec* $\alpha = 2 \wedge$ *dim-vec* $\beta = 2 \wedge v =$ *tensor-vec* $\alpha\ \beta$)
⟨*proof*⟩

corollary *horodecki-peres-entangled*:
fixes v :: *complex Matrix.vec*
assumes *vd*: *dim-vec* $v = 4$
shows *entangled 2 2 v* $\longleftrightarrow$
¬ *psd* (*amplification 2 2 2* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*) ($v \oslash v$))
⟨*proof*⟩

end

Part VI
Quantum Algorithms (beyond the notes)

theory *Hermitian-Functional-Calculus*
imports *Vern-Paulsen-QC-3-10.Quantum-Computing-3-10*
begin

98 Functional calculus for Hermitian matrices

For Hermitian A and $g : \mathbb{R} \to \mathbb{C}$, $g(A)$ is the matrix that acts on every eigenvector v of A with eigenvalue λ as multiplication by $g(\lambda)$. The eigenvectors span $\mathbb{C}^n$, so this determines $g(A)$; for a spectral decomposition $A = U\,\mathrm{diag}(D)\,U^*$ it is $U\,\mathrm{diag}(g \circ D)\,U^*$.

definition *diag-cmat* :: *complex list* ⇒ *complex Matrix.mat* **where**
diag-cmat xs = *Matrix.mat* (*length xs*) (*length xs*) (λ(*i*,*j*). *if i* = *j then xs* ! *i else 0*)

lemma *diag-cmat-carrier* [*simp*]: *diag-cmat xs* ∈ *carrier-mat* (*length xs*) (*length xs*)
⟨*proof*⟩

lemma *dim-diag-cmat* [*simp*]: *dim-row* (*diag-cmat xs*) = *length xs dim-col* (*diag-cmat xs*) = *length xs*
⟨*proof*⟩

lemma *diag-mat-of-real-cmat*: *diag-mat-of-real D* = *diag-cmat* (*map complex-of-real D*)
⟨*proof*⟩

lemma *diag-cmat-mult-vec*:
assumes *dim-vec w* = *length xs*
shows *diag-cmat xs* $*_v$ *w* = *Matrix.vec* (*length xs*) (λ*i*. *xs* ! *i* ∗ *w* \$ *i*)
⟨*proof*⟩

lemma *col-mult-diag*:
assumes *M*: *M* ∈ *carrier-mat n* (*length xs*) **and** *j*: *j* < *length xs*
shows *Matrix.col* (*M* ∗ *diag-cmat xs*) *j* = *xs* ! *j* $\cdot_v$ *Matrix.col M j*
⟨*proof*⟩

lemma *diag-cmat-dagger*: $(\mathit{diag\text{-}cmat}\ \mathit{xs})^\dagger = \mathit{diag\text{-}cmat}\ (\mathit{map}\ \mathit{cnj}\ \mathit{xs})$
⟨*proof*⟩

lemma *smult-mat-mult-vec*:
$A \in \mathit{carrier\text{-}mat}\ n\ m \Longrightarrow \mathit{dim\text{-}vec}\ v = m \Longrightarrow (c \cdot_m A) *_v v = c \cdot_v (A *_v v)$
⟨*proof*⟩

lemma *diag-conj-carrier*:
assumes *U*: $U \in \mathit{carrier\text{-}mat}\ n\ n$ **and** *xs*: $\mathit{length}\ \mathit{xs} = n$
shows $U * \mathit{diag\text{-}cmat}\ \mathit{xs} * (U^\dagger) \in \mathit{carrier\text{-}mat}\ n\ n$
⟨*proof*⟩

98.1 Spectral decompositions

spectral-decomp A U D: U unitary and $U^*AU = \mathrm{diag}(D)$.

definition *spectral-decomp* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* ⇒ *real list* ⇒ *bool* **where**
$\mathit{spectral\text{-}decomp}\ A\ U\ D \longleftrightarrow U \in \mathit{carrier\text{-}mat}\ (\mathit{dim\text{-}row}\ A)\ (\mathit{dim\text{-}row}\ A) \wedge \mathit{unitary}\ U \wedge$
$\mathit{length}\ D = \mathit{dim\text{-}row}\ A \wedge U^\dagger * A * U = \mathit{diag\text{-}mat\text{-}of\text{-}real}\ D$

lemma *spectral-decomp-exists*:
assumes $A \in \mathit{carrier\text{-}mat}\ n\ n\ \mathit{hermitian}\ A$
shows $\exists\, U\ D.\ \mathit{spectral\text{-}decomp}\ A\ U\ D$
⟨*proof*⟩

lemma *spectral-decompD*:
assumes *A*: $A \in \mathit{carrier\text{-}mat}\ n\ n$ **and** *sd*: $\mathit{spectral\text{-}decomp}\ A\ U\ D$
shows $U \in \mathit{carrier\text{-}mat}\ n\ n\ \mathit{unitary}\ U\ \mathit{length}\ D = n\ U^\dagger * U = 1_m\ n\ U * (U^\dagger) = 1_m\ n$
$A = U * \mathit{diag\text{-}mat\text{-}of\text{-}real}\ D * (U^\dagger)\ A * U = U * \mathit{diag\text{-}mat\text{-}of\text{-}real}\ D$
⟨*proof*⟩

The columns of U are eigenvectors: $Au_j = D_j u_j$.

lemma *spectral-decomp-col*:
assumes *A*: $A \in \mathit{carrier\text{-}mat}\ n\ n$ **and** *sd*: $\mathit{spectral\text{-}decomp}\ A\ U\ D$ **and** *j*: $j < n$
shows $A *_v \mathit{Matrix.col}\ U\ j = \mathit{complex\text{-}of\text{-}real}\ (D\ !\ j) \cdot_v \mathit{Matrix.col}\ U\ j$
⟨*proof*⟩

98.2 The definition

definition *fun-on-eigen* :: *complex Matrix.mat* ⇒ (*real* ⇒ *complex*) ⇒ *complex Matrix.mat* ⇒ *bool* **where**
$\mathit{fun\text{-}on\text{-}eigen}\ A\ g\ B \longleftrightarrow (\forall\, v\ l.\ \mathit{dim\text{-}vec}\ v = \mathit{dim\text{-}row}\ A \longrightarrow$
$A *_v v = \mathit{complex\text{-}of\text{-}real}\ l \cdot_v v \longrightarrow B *_v v = g\ l \cdot_v v)$

definition *mat-fun* :: (*real* ⇒ *complex*) ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
$\mathit{mat\text{-}fun}\ g\ A = (\mathit{SOME}\ B.\ B \in \mathit{carrier\text{-}mat}\ (\mathit{dim\text{-}row}\ A)\ (\mathit{dim\text{-}row}\ A) \wedge \mathit{fun\text{-}on\text{-}eigen}\ A\ g\ B)$

definition *herm-spectrum* :: *complex Matrix.mat* ⇒ *real set* **where**
$\mathit{herm\text{-}spectrum}\ A = \{l.\ \exists\, v.\ \mathit{dim\text{-}vec}\ v = \mathit{dim\text{-}row}\ A \wedge v \neq 0_v\ (\mathit{dim\text{-}row}\ A) \wedge$
$A *_v v = \mathit{complex\text{-}of\text{-}real}\ l \cdot_v v\}$

lemma *fun-on-eigenD*:
$\mathit{fun\text{-}on\text{-}eigen}\ A\ g\ B \Longrightarrow \mathit{dim\text{-}vec}\ v = \mathit{dim\text{-}row}\ A \Longrightarrow A *_v v = \mathit{complex\text{-}of\text{-}real}\ l \cdot_v v \Longrightarrow$
$B *_v v = g\ l \cdot_v v$
⟨*proof*⟩

$U\,\mathrm{diag}(g \circ D)\,U^*$ acts correctly on eigenvectors: if $Av = \lambda v$ then $w = U^*v$ has $D_i w_i = \lambda w_i$, so $g(D_i)w_i = g(\lambda)w_i$.

lemma *spectral-decomp-fun-on-eigen*:
assumes *A*: $A \in$ *carrier-mat n n* **and** *sd*: *spectral-decomp A U D*
shows *fun-on-eigen A g* $(U *$ *diag-cmat* $(map\ g\ D) * (U^\dagger))$
⟨*proof*⟩

Uniqueness: two matrices that agree on the eigenvectors u_j agree on U, hence are equal.

lemma *fun-on-eigen-unique*:
assumes *A*: $A \in$ *carrier-mat n n hermitian A*
and *B1*: *B1* $\in$ *carrier-mat n n fun-on-eigen A g B1*
and *B2*: *B2* $\in$ *carrier-mat n n fun-on-eigen A g B2*
shows *B1* = *B2*
⟨*proof*⟩

lemma *mat-fun-spec*:
assumes *A*: $A \in$ *carrier-mat n n hermitian A*
shows *mat-fun g A* $\in$ *carrier-mat n n fun-on-eigen A g* (*mat-fun g A*)
⟨*proof*⟩

lemma *mat-fun-carrier* [*simp*]: $A \in$ *carrier-mat n n* $\Longrightarrow$ *hermitian A* $\Longrightarrow$ *mat-fun g A* $\in$ *carrier-mat n n*
⟨*proof*⟩

theorem *mat-fun-eigen*:
assumes *A*: $A \in$ *carrier-mat n n hermitian A*
and *v*: *dim-vec v* = *n* **and** *ev*: $A *_v$ *v* = *complex-of-real l* $\cdot_v$ *v*
shows *mat-fun g A* $*_v$ *v* = *g l* $\cdot_v$ *v*
⟨*proof*⟩

lemma *mat-fun-eqI*:
assumes *A*: $A \in$ *carrier-mat n n hermitian A*
and *B*: *B* $\in$ *carrier-mat n n fun-on-eigen A g B*
shows *mat-fun g A* = *B*
⟨*proof*⟩

theorem *mat-fun-decomp*:
assumes *A*: $A \in$ *carrier-mat n n hermitian A* **and** *sd*: *spectral-decomp A U D*
shows *mat-fun g A* = *U* $*$ *diag-cmat* $(map\ g\ D) * (U^\dagger)$
⟨*proof*⟩

Only the values of g on the spectrum matter.

lemma *mat-fun-cong*:
assumes *A*: $A \in$ *carrier-mat n n hermitian A*
and *gh*: $\bigwedge l.\ l \in$ *herm-spectrum A* $\Longrightarrow$ *g l* = *h l*
shows *mat-fun g A* = *mat-fun h A*
⟨*proof*⟩

98.3 Algebra

lemma *mat-fun-id*: $A \in$ *carrier-mat n n* $\Longrightarrow$ *hermitian A* $\Longrightarrow$ *mat-fun complex-of-real A* = *A*
⟨*proof*⟩

lemma *mat-fun-const*:
assumes *A*: $A \in$ *carrier-mat n n hermitian A*
shows *mat-fun* $(\lambda$-. *c*$)$ *A* = *c* $\cdot_m$ 1_m *n*
⟨*proof*⟩

lemma *mat-fun-one*: $A \in$ *carrier-mat n n* $\Longrightarrow$ *hermitian A* $\Longrightarrow$ *mat-fun* $(\lambda$-. *1*$)$ *A* = 1_m *n*
⟨*proof*⟩

theorem *mat-fun-mult*:
assumes *A*: $A \in$ *carrier-mat n n hermitian A*
shows *mat-fun g A* $*$ *mat-fun h A* $=$ *mat-fun* $(\lambda x.\ g\ x * h\ x)$ *A*
⟨*proof*⟩

theorem *mat-fun-dagger*:
assumes *A*: $A \in$ *carrier-mat n n hermitian A*
shows $(\textit{mat-fun}\ g\ A)^\dagger = \textit{mat-fun}\ (\lambda x.\ \textit{cnj}\ (g\ x))\ A$
⟨*proof*⟩

lemma *mat-fun-hermitian*:
assumes *A*: $A \in$ *carrier-mat n n hermitian A*
shows *hermitian* (*mat-fun* $(\lambda x.$ *complex-of-real* $(f\ x))$ *A*)
⟨*proof*⟩

theorem *mat-fun-unitary*:
assumes *A*: $A \in$ *carrier-mat n n hermitian A*
and *g*: $\bigwedge l.\ l \in \textit{herm-spectrum}\ A \Longrightarrow \textit{cmod}\ (g\ l) = 1$
shows *unitary* (*mat-fun g A*)
⟨*proof*⟩

98.4 Time evolution

e^{itA}, the evolution under the Hamiltonian A (phase estimation uses $U = e^{itA}$).

definition *evolution* :: *complex Matrix.mat* ⇒ *real* ⇒ *complex Matrix.mat* **where**
evolution A t $=$ *mat-fun* $(\lambda x.\ \textit{cis}\ (t * x))$ *A*

context
fixes *A* :: *complex Matrix.mat* **and** *n* :: *nat*
assumes *A*: $A \in$ *carrier-mat n n hermitian A*
begin

lemma *evolution-carrier* [*simp*]: *evolution A t* $\in$ *carrier-mat n n*
⟨*proof*⟩

lemma *evolution-eigen*:
dim-vec v $= n \Longrightarrow A *_v\ v =$ *complex-of-real l* $\cdot_v\ v \Longrightarrow$ *evolution A t* $*_v\ v =$ *cis* $(t * l) \cdot_v\ v$
⟨*proof*⟩

lemma *evolution-unitary*: *unitary* (*evolution A t*)
⟨*proof*⟩

lemma *evolution-zero*: *evolution A 0* $= 1_m\ n$
⟨*proof*⟩

lemma *evolution-add*: *evolution A* $(s + t) =$ *evolution A s* $*$ *evolution A t*
⟨*proof*⟩

lemma *evolution-pow*: *evolution A t* $\hat{}_m\ k =$ *evolution A* $(\textit{real}\ k * t)$
⟨*proof*⟩

end

98.5 The inverse

$A^{-1} = g(A)$ for $g(x) = 1/x$; since $1/0 = 0$ in HOL this is the pseudo-inverse in general.

definition *herm-inv* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
herm-inv A = *mat-fun* (λ*x*. *complex-of-real* (*inverse x*)) *A*

theorem *herm-inv*:
assumes *A*: *A* ∈ *carrier-mat n n hermitian A* **and** *z*: *0* ∉ *herm-spectrum A*
shows *herm-inv A* ∗ *A* = 1_m *n A* ∗ *herm-inv A* = 1_m *n*
⟨*proof*⟩

lemma *herm-inv-eigen*:
assumes *A*: *A* ∈ *carrier-mat n n hermitian A*
and *v*: *dim-vec v* = *n* **and** *ev*: *A* ∗$_v$ *v* = *complex-of-real l* ·$_v$ *v*
shows *herm-inv A* ∗$_v$ *v* = *complex-of-real* (*inverse l*) ·$_v$ *v*
⟨*proof*⟩

lemma *herm-inv-hermitian*: *A* ∈ *carrier-mat n n* ⟹ *hermitian A* ⟹ *hermitian* (*herm-inv A*)
⟨*proof*⟩

end
theory *Phase-Estimation*
imports *Hermitian-Functional-Calculus HOL−Analysis.Convex*
begin

99 Phase estimation

The Fourier matrix F_N, $(F_N)_{jk} = \omega^{jk}/\sqrt{N}$ with $\omega = e^{2\pi i/N}$, and phase estimation: for $Uu = e^{2\pi i\varphi}u$, the circuit $(F_N^* \otimes I)\,CU\,(\frac{1}{\sqrt{N}}\sum_k |k\rangle \otimes u)$ leaves the register in $\sum_j \alpha_j|j\rangle$ with $\alpha_j = \frac{1}{N}\sum_k e^{2\pi ik(\varphi - j/N)}$.

99.1 Sums of roots of unity

lemma *root-of-unity-sum*:
assumes *N*: *0* < *N* **and** *j*: *j* < *N* **and** *l*: *l* < *N*
shows (∑ *k*<*N*. *cis* (*2* ∗ *pi* ∗ *real k* ∗ (*real l* − *real j*) / *real N*)) = (*if l* = *j then of-nat N else 0*)
⟨*proof*⟩

99.2 The Fourier matrix

definition *fourier* :: *nat* ⇒ *complex Matrix.mat* **where**
fourier N = *Matrix.mat N N* (λ(*j*,*k*). *cis* (*2* ∗ *pi* ∗ *real j* ∗ *real k* / *real N*) / *complex-of-real* (*sqrt* (*real N*)))

lemma *fourier-carrier* [*simp*]: *fourier N* ∈ *carrier-mat N N*
⟨*proof*⟩

lemma *dim-fourier* [*simp*]: *dim-row* (*fourier N*) = *N dim-col* (*fourier N*) = *N*
⟨*proof*⟩

lemma *fourier-index*:
j < *N* ⟹ *k* < *N* ⟹ *fourier N* $$ (*j*, *k*) = *cis* (*2* ∗ *pi* ∗ *real j* ∗ *real k* / *real N*) / *complex-of-real* (*sqrt* (*real N*))
⟨*proof*⟩

lemma *sqrt-N-sq*: *complex-of-real* (*sqrt* (*real N*)) ∗ *complex-of-real* (*sqrt* (*real N*)) = *of-nat N*
⟨*proof*⟩

The entries pair to a root of unity: $\overline{F_{kj}}F_{kl} = \omega^{k(l-j)}/N$.

lemma *fourier-entry-pair*:
assumes $j < N$ $k < N$ $l < N$
shows *cnj* (*fourier N* $$ (*k*, *j*)) ∗ *fourier N* $$ (*k*, *l*) = *cis* (2 ∗ *pi* ∗ *real k* ∗ (*real l* − *real j*) / *real N*) / *of-nat N*
⟨*proof*⟩

theorem *fourier-unitary*:
assumes *N*: 0 < *N*
shows *unitary* (*fourier N*)
⟨*proof*⟩

99.3 Controlled powers and phase kickback

$\mathrm{C}U = \sum_k |k\rangle\langle k| \otimes U^k$ on $\mathbb{C}^N \otimes \mathbb{C}^m$ (index $km + i$).

definition *ctrl-pow* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
ctrl-pow N m U = *Matrix.mat* (*N* ∗ *m*) (*N* ∗ *m*)
(λ(*r*, *c*). *if r div m* = *c div m then* (*U* ^*m* (*r div m*)) $$ (*r mod m*, *c mod m*) *else* 0)

lemma *ctrl-pow-carrier* [*simp*]: *ctrl-pow N m U* ∈ *carrier-mat* (*N* ∗ *m*) (*N* ∗ *m*)
⟨*proof*⟩

lemma *dim-ctrl-pow* [*simp*]: *dim-row* (*ctrl-pow N m U*) = *N* ∗ *m dim-col* (*ctrl-pow N m U*) = *N* ∗ *m*
⟨*proof*⟩

lemma *ctrl-pow-index*:
r < *N* ∗ *m* ⟹ *c* < *N* ∗ *m* ⟹
ctrl-pow N m U $$ (*r*, *c*) = (*if r div m* = *c div m then* (*U* ^*m* (*r div m*)) $$ (*r mod m*, *c mod m*) *else* 0)
⟨*proof*⟩

lemma *sum-blocks*:
fixes *N m* :: *nat*
shows (∑ *c*<*N* ∗ *m*. *f c*) = (∑ *k*<*N*. ∑ *i*<*m*. *f* (*k* ∗ *m* + *i*))
⟨*proof*⟩

lemma *block-index*:
fixes *k i N m* :: *nat*
assumes *k* < *N i* < *m*
shows *k* ∗ *m* + *i* < *N* ∗ *m* (*k* ∗ *m* + *i*) *div m* = *k* (*k* ∗ *m* + *i*) *mod m* = *i*
⟨*proof*⟩

lemma *pow-eigen*:
fixes *U* :: *complex Matrix.mat*
assumes *U*: *U* ∈ *carrier-mat m m* **and** *u*: *dim-vec u* = *m* **and** *ev*: *U* ∗*v u* = *z* ·*v u*
shows (*U* ^*m k*) ∗*v u* = *z* ^ *k* ·*v u*
⟨*proof*⟩

Phase kickback: if $Uu = zu$ then $\mathrm{C}U(a \otimes u) = (\sum_k z^k a_k |k\rangle) \otimes u$.

theorem *ctrl-pow-kickback*:
fixes *U* :: *complex Matrix.mat*
assumes *U*: *U* ∈ *carrier-mat m m* **and** *u*: *dim-vec u* = *m* **and** *ev*: *U* ∗*v u* = *z* ·*v u*
and *a*: *dim-vec a* = *N*
shows *ctrl-pow N m U* ∗*v tensor-vec a u* = *tensor-vec* (*Matrix.vec N* (λ*k*. *z* ^ *k* ∗ *a* $ *k*)) *u*
⟨*proof*⟩

99.4 The phase estimation circuit

The register starts in the uniform state $\frac{1}{\sqrt{N}} \sum_k |k\rangle$ ($= H^{\otimes t}|0\rangle$ for $N = 2^t$).

definition *uniform-state* :: *nat* ⇒ *complex Matrix.vec* **where**
uniform-state N = *Matrix.vec N* (λ-. *1* / *complex-of-real* (*sqrt* (*real N*)))

definition *pe-amp* :: *nat* ⇒ *real* ⇒ *nat* ⇒ *complex* **where**
pe-amp N φ j = (∑ *k*<*N*. *cis* (*2* ∗ *pi* ∗ *real k* ∗ (*φ* − *real j* / *real N*))) / *of-nat N*

lemma *fourier-dagger-kick*:
assumes *j*: *j* < *N*
shows ((*fourier N*)† ∗$_v$ *Matrix.vec N* (λ*k*. *cis* (*2* ∗ *pi* ∗ *φ*) ^ *k* ∗ *uniform-state N* $ *k*)) $ *j* = *pe-amp N φ j*
⟨*proof*⟩

The output of phase estimation for an eigenvector u, $Uu = e^{2\pi i\varphi}u$.

theorem *phase-estimation*:
assumes *U*: *U* ∈ *carrier-mat m m* **and** *u*: *dim-vec u* = *m*
and *ev*: *U* ∗$_v$ *u* = *cis* (*2* ∗ *pi* ∗ *φ*) ·$_v$ *u*
shows (((*fourier N*)†) ⨂ *1*$_m$ *m*) ∗$_v$ (*ctrl-pow N m U* ∗$_v$ *tensor-vec* (*uniform-state N*) *u*) =
tensor-vec (*Matrix.vec N* (*pe-amp N φ*)) *u*
⟨*proof*⟩

If $\varphi = j_0/N$ exactly, the register ends in $|j_0\rangle$.

theorem *phase-estimation-exact*:
assumes *N*: *0* < *N* **and** *j0*: *j0* < *N*
shows *Matrix.vec N* (*pe-amp N* (*real j0* / *real N*)) = *unit-vec N j0*
⟨*proof*⟩

99.5 Measuring the register

Probability of outcome j when the register of $\psi \in \mathbb{C}^N \otimes \mathbb{C}^m$ is measured.

definition *reg-prob* :: *nat* ⇒ *complex Matrix.vec* ⇒ *nat* ⇒ *real* **where**
reg-prob m ψ j = (∑ *i*<*m*. (*cmod* (*ψ* $ (*j* ∗ *m* + *i*)))²)

lemma *reg-prob-tensor*:
assumes *b*: *dim-vec b* = *N* **and** *u*: *dim-vec u* = *m* **and** *j*: *j* < *N*
and *unit*: (∑ *i*<*m*. (*cmod* (*u* $ *i*))²) = *1*
shows *reg-prob m* (*tensor-vec b u*) *j* = (*cmod* (*b* $ *j*))²
⟨*proof*⟩

99.6 The success probability

Jordan's inequality $\sin x \geq 2x/\pi$ on $[0, \pi/2]$, by concavity (as in *Gauss-Sums.Polya-Vinogradov*).

lemma *jordan-inequality*:
assumes *0* ≤ *x x* ≤ *pi* / *2*
shows *2* / *pi* ∗ *x* ≤ *sin x*
⟨*proof*⟩

lemma *jordan-inequality-abs*:
assumes |*x*| ≤ *pi* / *2*
shows *2* / *pi* ∗ |*x*| ≤ |*sin x*|
⟨*proof*⟩

lemma *cmod-cis-minus-one*: *cmod* (*cis x* − *1*) = *2* ∗ |*sin* (*x* / *2*)|
⟨*proof*⟩

The key estimate: $\left|\frac{1}{N}\sum_k e^{2\pi i k\delta}\right| \geq 2/\pi$ for $|\delta| \leq \frac{1}{2N}$.

theorem *geometric-amp-bound*:
assumes *N*: *0 < N* **and** *d*: $|\delta| \leq 1 / (2 * real\ N)$
shows *2 / pi* $\leq$ *cmod* $((\sum k{<}N.\ cis\ (2 * pi * real\ k * \delta)) / of\text{-}nat\ N)$
⟨*proof*⟩

With φ within $\frac{1}{2N}$ of j/N, outcome j has probability at least $4/\pi^2$.

theorem *phase-estimation-bound*:
assumes *N*: *0 < N* **and** *close*: $|\varphi - real\ j / real\ N| \leq 1 / (2 * real\ N)$
shows *4 / pi*^{2} $\leq$ $(cmod\ (pe\text{-}amp\ N\ \varphi\ j))^2$
⟨*proof*⟩

The phase is periodic: α_0 is unchanged by $\varphi \mapsto \varphi - 1$.

lemma *pe-amp-shift*: *pe-amp N* φ *0 = pe-amp N* $(\varphi - 1)$ *0*
⟨*proof*⟩

lemma *scaled-close*:
assumes *N*: *0 < N* **and** *x*: $|real\ N * x| \leq 1 / 2$
shows $|x| \leq 1 / (2 * real\ N)$
⟨*proof*⟩

Phase estimation succeeds: for $0 \leq \varphi < 1$ and a unit eigenvector, some outcome $j < N$ (the nearest to $N\varphi$, read mod N) is observed with probability at least $4/\pi^2$.

theorem *phase-estimation-success*:
fixes *U* :: *complex Matrix.mat*
assumes *N*: *0 < N* **and** *U*: *U* $\in$ *carrier-mat m m* **and** *u*: *dim-vec u = m*
and *unit*: $(\sum i{<}m.\ (cmod\ (u\ \$\ i))^2) = 1$
and *ev*: *U* $*_v$ *u = cis* $(2 * pi * \varphi)$ $\cdot_v$ *u* **and** φ: $0 \leq \varphi\ \varphi < 1$
shows $\exists j{<}N.\ 4 / pi^2 \leq$
reg-prob m ((((*fourier N*)†) $\bigotimes$ 1_m *m*) $*_v$ (*ctrl-pow N m U* $*_v$ *tensor-vec* (*uniform-state N*) *u*)) *j*
⟨*proof*⟩

end
theory *QFT-Fourier*
imports *Phase-Estimation*
begin

100 The AFP Fourier transform circuit is the Fourier matrix

The circuit *ordered-QFT* of the AFP entry *Quantum-Fourier-Transform* is shown there to map $|j\rangle$ to the product state $2^{-n/2}\bigotimes_{l=1}^{n}(|0\rangle + e^{2\pi i j/2^l}|1\rangle)$. Its k-th entry is $2^{-n/2}e^{2\pi i j k/2^n}$ (the binary digits of k select the factors), so the circuit is F_{2^n} and phase estimation runs on it.

lemma *kron-phase-index*:
fixes *f* :: *nat* $\Rightarrow$ *complex Matrix.mat* **and** *j* :: *nat*
assumes *f*: $\bigwedge l.\ f\ l \in carrier\text{-}mat\ 2\ 1$
and *fi*: $\bigwedge l\ c.\ c < 2 \Longrightarrow f\ l\ \$\$\ (c,\ 0) = exp\ (2 * \mathrm{i} * pi * of\text{-}nat\ j * of\text{-}nat\ c\ /\ 2\ \hat{}\ l)$
shows *1* $\leq$ *a* $\Longrightarrow$ *nat* $(b - a + 1) = m \Longrightarrow k < 2\ \hat{}\ m \Longrightarrow$
kron f (*map nat* [*a*..*b*]) $\$\$\ (k,\ 0) = exp\ (2 * \mathrm{i} * pi * of\text{-}nat\ j * of\text{-}nat\ k\ /\ 2\ \hat{}\ nat\ b)$
⟨*proof*⟩

lemma *QFT-product-representation-index*:
assumes *k*: *k* $< 2\ \hat{}\ n$
shows *QFT-product-representation j n* $\$\$\ (k,\ 0) =$
cis $(2 * pi * real\ j * real\ k\ /\ 2\ \hat{}\ n)$ / *complex-of-real* $(sqrt\ (2\ \hat{}\ n))$

⟨*proof*⟩

theorem *ordered-QFT-fourier*: *ordered-QFT n* = *fourier* (2 ^ *n*)
⟨*proof*⟩

Phase estimation with the AFP circuit: the theorems of *Phase-Estimation* hold with F_{2^t} replaced by *ordered-QFT t*.

corollary *phase-estimation-circuit*:
fixes *U* :: *complex Matrix.mat*
assumes *U* ∈ *carrier-mat m m dim-vec u* = *m U* ∗$_v$ *u* = *cis* (2 ∗ *pi* ∗ *φ*) ·$_v$ *u*
shows (((*ordered-QFT t*)†) ⨂ 1$_m$ *m*) ∗$_v$ (*ctrl-pow* (2 ^ *t*) *m U* ∗$_v$ *tensor-vec* (*uniform-state* (2 ^ *t*)) *u*) =
tensor-vec (*Matrix.vec* (2 ^ *t*) (*pe-amp* (2 ^ *t*) *φ*)) *u*
⟨*proof*⟩

end
theory *HHL*
imports *Phase-Estimation*
begin

101 The HHL algorithm

Harrow, Hassidim and Lloyd's algorithm for $Ax = b$, A Hermitian: phase estimation of $e^{it_0 A}$ writes the eigenvalue λ of each eigencomponent into a register, a rotation controlled by the register puts C/λ into the amplitude of an ancilla $|1\rangle$, and phase estimation is undone. Measuring the ancilla in $|1\rangle$ leaves the system in $A^{-1}b/\|A^{-1}b\|$. The space is register ⊗ (system ⊗ ancilla).

101.1 Block-diagonal controlled operations

$\sum_k |k\rangle\langle k| \otimes B_k$ on $\mathbb{C}^N \otimes \mathbb{C}^m$.

definition *ctrl-blocks* :: *nat* ⇒ *nat* ⇒ (*nat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* **where**
ctrl-blocks N m B = *Matrix.mat* (*N* ∗ *m*) (*N* ∗ *m*)
(λ(*r*, *c*). *if r div m* = *c div m then B* (*r div m*) $$ (*r mod m*, *c mod m*) *else 0*)

lemma *ctrl-blocks-carrier* [*simp*]: *ctrl-blocks N m B* ∈ *carrier-mat* (*N* ∗ *m*) (*N* ∗ *m*)
⟨*proof*⟩

lemma *dim-ctrl-blocks* [*simp*]: *dim-row* (*ctrl-blocks N m B*) = *N* ∗ *m dim-col* (*ctrl-blocks N m B*) = *N* ∗ *m*
⟨*proof*⟩

lemma *ctrl-blocks-index*:
r < *N* ∗ *m* ⟹ *c* < *N* ∗ *m* ⟹
ctrl-blocks N m B $$ (*r*, *c*) = (*if r div m* = *c div m then B* (*r div m*) $$ (*r mod m*, *c mod m*) *else 0*)
⟨*proof*⟩

lemma *ctrl-pow-blocks*: *ctrl-pow N m U* = *ctrl-blocks N m* (λ*k*. *U* ^$_m$ *k*)
⟨*proof*⟩

lemma *div-mod-eq-iff*:
fixes *r c m* :: *nat*
shows *r* = *c* ⟷ *r div m* = *c div m* ∧ *r mod m* = *c mod m*
⟨*proof*⟩

lemma *ctrl-blocks-cong*:

assumes $\bigwedge k.\ k < N \Longrightarrow B\ k = B'\ k$
shows *ctrl-blocks N m B = ctrl-blocks N m B'*
⟨*proof*⟩

lemma *ctrl-blocks-one*: *ctrl-blocks N m* (λ-. 1_m *m*) = 1_m (*N* ∗ *m*)
⟨*proof*⟩

lemma *sum-if-const*: (∑ *i*∈*A*. *if P then f i else 0*) = (*if P then* (∑ *i*∈*A*. *f i*) *else 0*)
⟨*proof*⟩

lemma *tensor-unit-index*:
assumes *k' < N i' < m dim-vec w = m*
shows *tensor-vec* (*unit-vec N k*) *w* $ (*k'* ∗ *m* + *i'*) = (*if k' = k then w* $ *i' else 0*)
⟨*proof*⟩

lemma *ctrl-blocks-col-index*:
assumes *r < N* ∗ *m k' < N i' < m*
shows *ctrl-blocks N m B* $$ (*r*, *k'* ∗ *m* + *i'*) = (*if r div m = k' then B k'* $$ (*r mod m*, *i'*) *else 0*)
⟨*proof*⟩

lemma *ctrl-blocks-row-index*:
assumes *c < N* ∗ *m k' < N i' < m*
shows *ctrl-blocks N m B* $$ (*k'* ∗ *m* + *i'*, *c*) = (*if k' = c div m then B k'* $$ (*i'*, *c mod m*) *else 0*)
⟨*proof*⟩

On a basis state of the register, $\sum_k |k\rangle\langle k| \otimes B_k$ applies B_k.

lemma *ctrl-blocks-basis*:
assumes *k*: *k < N* **and** *w*: *dim-vec w = m* **and** *B*: *B k* ∈ *carrier-mat m m*
shows *ctrl-blocks N m B* ∗$_v$ *tensor-vec* (*unit-vec N k*) *w* = *tensor-vec* (*unit-vec N k*) (*B k* ∗$_v$ *w*)
⟨*proof*⟩

lemma *ctrl-blocks-dagger*:
assumes *B*: $\bigwedge k.\ k < N \Longrightarrow B\ k \in$ *carrier-mat m m*
shows (*ctrl-blocks N m B*)† = *ctrl-blocks N m* (λ*k*. (*B k*)†)
⟨*proof*⟩

lemma *ctrl-blocks-mult*:
assumes *B1*: $\bigwedge k.\ k < N \Longrightarrow B1\ k \in$ *carrier-mat m m* **and** *B2*: $\bigwedge k.\ k < N \Longrightarrow B2\ k \in$ *carrier-mat m m*
shows *ctrl-blocks N m B1* ∗ *ctrl-blocks N m B2 = ctrl-blocks N m* (λ*k*. *B1 k* ∗ *B2 k*)
⟨*proof*⟩

lemma *ctrl-blocks-unitary*:
assumes *B*: $\bigwedge k.\ k < N \Longrightarrow B\ k \in$ *carrier-mat m m* **and** *u*: $\bigwedge k.\ k < N \Longrightarrow$ *unitary* (*B k*)
shows *unitary* (*ctrl-blocks N m B*)
⟨*proof*⟩

lemma *unitary-pow*:
assumes *U*: *U* ∈ *carrier-mat m m* **and** *u*: *unitary U*
shows *unitary* (*U* ^$_m$ *k*)
⟨*proof*⟩

101.2 The circuit

The ancilla rotation $R(c)|0\rangle = \sqrt{1-c^2}|0\rangle + c|1\rangle$.

definition *rot* :: *real* ⇒ *complex Matrix.mat* **where**
rot c = Matrix.mat 2 2 (λ(*i*, *j*). *if i = j then complex-of-real* (*sqrt* (1 − c^2)))

else if i = 1 then complex-of-real c else − complex-of-real c)

lemma *rot-carrier* [*simp*]: *rot c ∈ carrier-mat 2 2*
⟨*proof*⟩

lemma *dim-rot* [*simp*]: *dim-row (rot c) = 2 dim-col (rot c) = 2*
⟨*proof*⟩

lemma *rot-e0*: *rot c *$_v$ *unit-vec 2 0 =*
complex-of-real (sqrt (1 − c²)) ·$_v$ unit-vec 2 0 + complex-of-real c ·$_v$ unit-vec 2 1
⟨*proof*⟩

lemma *rot-unitary*:
assumes *c*: *|c| ≤ 1*
shows *unitary (rot c)*
⟨*proof*⟩

Register value k encodes the eigenvalue $\lambda_k = 2\pi k/(t_0 N)$ for $k < N/2$ and $2\pi(k-N)/(t_0 N)$ otherwise (negative eigenvalues), since $e^{it_0\lambda_k} = e^{2\pi ik/N}$.

definition *reg-eig* :: *real ⇒ nat ⇒ nat ⇒ real* **where**
*reg-eig t0 N k = 2 * pi * (if 2 * k < N then real k else real k − real N) / (t0 * real N)*

lemma *reg-eig-phase*:
assumes *t0*: *0 < t0* **and** *N*: *0 < N*
shows *cis (t0 * reg-eig t0 N k) = cis (2 * pi * (real k / real N))*
⟨*proof*⟩

lemma *reg-eig-nonzero*: *0 < t0 ⟹ 0 < k ⟹ k < N ⟹ reg-eig t0 N k ≠ 0*
⟨*proof*⟩

definition *hhl-rot* :: *real ⇒ real ⇒ nat ⇒ nat ⇒ nat ⇒ complex Matrix.mat* **where**
hhl-rot C t0 N n k = 1$_m$ n ⊗ rot (C / reg-eig t0 N k)

definition *pe-unitary* :: *nat ⇒ nat ⇒ complex Matrix.mat ⇒ complex Matrix.mat* **where**
*pe-unitary N m V = (((fourier N)†) ⊗ 1$_m$ m) * ctrl-pow N m V*

definition *hhl-W* :: *complex Matrix.mat ⇒ real ⇒ nat ⇒ complex Matrix.mat* **where**
*hhl-W A t0 N = pe-unitary N (dim-row A * 2) (evolution A t0 ⊗ 1$_m$ 2)*

definition *hhl-circuit* :: *complex Matrix.mat ⇒ real ⇒ nat ⇒ real ⇒ complex Matrix.mat* **where**
*hhl-circuit A t0 N C = (hhl-W A t0 N)† * (ctrl-blocks N (dim-row A * 2) (hhl-rot C t0 N (dim-row A)) * hhl-W A t0 N)*

lemma *hhl-rot-carrier* [*simp*]: *hhl-rot C t0 N n k ∈ carrier-mat (n * 2) (n * 2)*
⟨*proof*⟩

lemma *pe-unitary-carrier* [*simp*]: *pe-unitary N m V ∈ carrier-mat (N * m) (N * m)*
⟨*proof*⟩

lemma *pe-unitary-unitary*:
assumes *N*: *0 < N* **and** *V*: *V ∈ carrier-mat m m unitary V*
shows *unitary (pe-unitary N m V)*
⟨*proof*⟩

On $\frac{1}{\sqrt{N}}\sum_k |k\rangle \otimes w$ with $Vw = e^{2\pi ik/N}w$, phase estimation gives $|k\rangle \otimes w$.

lemma *pe-unitary-exact*:
assumes *N*: *0 < N* **and** *k*: *k < N* **and** *V*: *V ∈ carrier-mat m m* **and** *w*: *dim-vec w = m*

and *ev*: $V *_v w = cis\ (2 * pi * (real\ k\ /\ real\ N)) \cdot_v w$
shows *pe-unitary N m V* $*_v$ *tensor-vec* (*uniform-state N*) *w* = *tensor-vec* (*unit-vec N k*) *w*
⟨*proof*⟩

lemma *unitary-undo*:
assumes *W*: *W* ∈ *carrier-mat K K unitary W* **and** *x*: *dim-vec x* = *K* **and** *y*: *W* $*_v$ *x* = *y*
shows $W^\dagger *_v y = x$
⟨*proof*⟩

101.3 The circuit on an eigenvector

theorem *hhl-eigen*:
assumes *A*: *A* ∈ *carrier-mat n n hermitian A* **and** *t0*: *0* < *t0* **and** *N*: *0* < *N*
and *k*: *k* < *N* **and** *u*: *dim-vec u* = *n* **and** *ev*: *A* $*_v$ *u* = *complex-of-real* (*reg-eig t0 N k*) $\cdot_v$ *u*
shows *hhl-circuit A t0 N C* $*_v$ *tensor-vec* (*uniform-state N*) (*tensor-vec u* (*unit-vec 2 0*)) =
tensor-vec (*uniform-state N*) (*tensor-vec u* (*rot* (*C* / *reg-eig t0 N k*) $*_v$ *unit-vec 2 0*))
⟨*proof*⟩

101.4 All right-hand sides

lemma *mult-zero-vec-right*: *B* ∈ *carrier-mat K n* ⟹ *B* $*_v$ 0_v *n* = 0_v *K*
⟨*proof*⟩

Two matrices that agree on every eigenvector of a Hermitian A are equal.

lemma *eigvec-mat-eqI*:
assumes *A*: *A* ∈ *carrier-mat n n hermitian A*
and *B1*: *B1* ∈ *carrier-mat K n* **and** *B2*: *B2* ∈ *carrier-mat K n*
and *eq*: ⋀*v l*. *dim-vec v* = *n* ⟹ *A* $*_v$ *v* = *complex-of-real l* $\cdot_v$ *v* ⟹ *B1* $*_v$ *v* = *B2* $*_v$ *v*
shows *B1* = *B2*
⟨*proof*⟩

The matrix of $b \mapsto a \otimes (M_0 b \otimes |0\rangle + M_1 b \otimes |1\rangle)$.

definition *lift-mat* :: *nat* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
lift-mat n a M0 M1 = *Matrix.mat* (*dim-vec a* * (*n* * *2*)) *n*
(λ(*r*, *s*). *a* $ (*r div* (*n* * *2*)) *
(*if r mod 2* = *0 then M0* $$ (*r mod* (*n* * *2*) *div 2*, *s*) *else M1* $$ (*r mod* (*n* * *2*) *div 2*, *s*)))

lemma *lift-mat-carrier* [*simp*]: *lift-mat n a M0 M1* ∈ *carrier-mat* (*dim-vec a* * (*n* * *2*)) *n*
⟨*proof*⟩

lemma *lift-mat-mult-vec*:
assumes *M0*: *M0* ∈ *carrier-mat n n* **and** *M1*: *M1* ∈ *carrier-mat n n* **and** *b*: *dim-vec b* = *n*
shows *lift-mat n a M0 M1* $*_v$ *b* =
tensor-vec a (*tensor-vec* (*M0* $*_v$ *b*) (*unit-vec 2 0*) + *tensor-vec* (*M1* $*_v$ *b*) (*unit-vec 2 1*))
⟨*proof*⟩

theorem *hhl*:
assumes *A*: *A* ∈ *carrier-mat n n hermitian A* **and** *t0*: *0* < *t0* **and** *N*: *0* < *N*
and *spec*: ⋀*l*. *l* ∈ *herm-spectrum A* ⟹ ∃*k*. *0* < *k* ∧ *k* < *N* ∧ *l* = *reg-eig t0 N k*
and *b*: *dim-vec b* = *n*
shows *hhl-circuit A t0 N C* $*_v$ *tensor-vec* (*uniform-state N*) (*tensor-vec b* (*unit-vec 2 0*)) =
tensor-vec (*uniform-state N*)
(*tensor-vec* (*mat-fun* (λ*x*. *complex-of-real* (*sqrt* (*1* − (*C* / *x*)^2^))) *A* $*_v$ *b*) (*unit-vec 2 0*) +
tensor-vec ((*complex-of-real C* $\cdot_m$ *herm-inv A*) $*_v$ *b*) (*unit-vec 2 1*))
⟨*proof*⟩

Reading off the solution: $x = A^{-1}b$ solves $Ax = b$, the ancilla-$|1\rangle$ part of the output is $C\,(u \otimes x \otimes |1\rangle)$, and its squared norm (the probability of measuring the ancilla in $|1\rangle$) is $C^2\|x\|^2$.

lemma *uniform-state-norm*: $0 < N \Longrightarrow \langle \textit{uniform-state } N | \textit{uniform-state } N \rangle = 1$
⟨*proof*⟩

corollary *hhl-solution*:
assumes *A*: $A \in \textit{carrier-mat } n\ n$ *hermitian A* **and** *t0*: $0 < t0$ **and** *N*: $0 < N$
and *spec*: $\bigwedge l.\ l \in \textit{herm-spectrum } A \Longrightarrow \exists k.\ 0 < k \wedge k < N \wedge l = \textit{reg-eig } t0\ N\ k$
and *b*: $\textit{dim-vec } b = n$
defines $x \equiv \textit{herm-inv } A *_v b$
shows $A *_v x = b$
and $\textit{hhl-circuit } A\ t0\ N\ C *_v \textit{tensor-vec } (\textit{uniform-state } N)\ (\textit{tensor-vec } b\ (\textit{unit-vec } 2\ 0)) =$
$\textit{tensor-vec } (\textit{uniform-state } N)\ (\textit{tensor-vec } (\textit{mat-fun } (\lambda x.\ \textit{complex-of-real } (\textit{sqrt } (1 - (C\ /\ x)^2)))\ A *_v b)\ (\textit{unit-vec } 2\ 0)) +$
$\textit{complex-of-real } C \cdot_v \textit{tensor-vec } (\textit{uniform-state } N)\ (\textit{tensor-vec } x\ (\textit{unit-vec } 2\ 1))$
and $\langle \textit{tensor-vec } (\textit{uniform-state } N)\ (\textit{tensor-vec } (\textit{complex-of-real } C \cdot_v x)\ (\textit{unit-vec } 2\ 1)) \mid$
$\textit{tensor-vec } (\textit{uniform-state } N)\ (\textit{tensor-vec } (\textit{complex-of-real } C \cdot_v x)\ (\textit{unit-vec } 2\ 1)) \rangle =$
$\textit{complex-of-real } (C^2) * \langle x | x \rangle$
⟨*proof*⟩

101.5 The success probability

The diagonal of a spectral decomposition consists of eigenvalues.

lemma *spectral-decomp-eig-in-spectrum*:
assumes *A*: $A \in \textit{carrier-mat } n\ n$ **and** *sd*: *spectral-decomp A U D* **and** *j*: $j < n$
shows $D\ !\ j \in \textit{herm-spectrum } A$
⟨*proof*⟩

A real function non-negative on the spectrum gives a positive matrix.

lemma *mat-fun-psd*:
assumes *A*: $A \in \textit{carrier-mat } n\ n$ *hermitian A* **and** *f*: $\bigwedge l.\ l \in \textit{herm-spectrum } A \Longrightarrow 0 \leq f\ l$
shows $\textit{psd } (\textit{mat-fun } (\lambda x.\ \textit{complex-of-real } (f\ x))\ A)$
⟨*proof*⟩

lemma *mat-fun-add*:
assumes *A*: $A \in \textit{carrier-mat } n\ n$ *hermitian A*
shows $\textit{mat-fun } (\lambda x.\ g\ x + h\ x)\ A = \textit{mat-fun } g\ A + \textit{mat-fun } h\ A$
⟨*proof*⟩

lemma *inner-prod-add-right*:
assumes $\textit{dim-vec } a = n\ \textit{dim-vec } b = n\ \textit{dim-vec } c = n$
shows $\langle a | b + c \rangle = \langle a | b \rangle + \langle a | c \rangle$
⟨*proof*⟩

If $C \leq |\lambda| \leq \Lambda$ on the spectrum, $C > 0$, the probability $C^2 \|A^{-1}b\|^2$ of the ancilla outcome $|1\rangle$ is at least $(C/\Lambda)^2 \|b\|^2$: for unit b and $C = \lambda_{\min}$ this is $1/\kappa^2$.

theorem *hhl-success-bound*:
assumes *A*: $A \in \textit{carrier-mat } n\ n$ *hermitian A* **and** *C*: $0 < C$
and *spec*: $\bigwedge l.\ l \in \textit{herm-spectrum } A \Longrightarrow C \leq |l| \wedge |l| \leq \Lambda$
and *b*: $\textit{dim-vec } b = n$
shows $(C\ /\ \Lambda)^2 * \textit{Re } \langle b | b \rangle \leq C^2 * \textit{Re } \langle \textit{herm-inv } A *_v b | \textit{herm-inv } A *_v b \rangle$
⟨*proof*⟩

101.6 Non-Hermitian systems

For square A, the Hermitian $\tilde{A} = \begin{pmatrix} 0 & A \\ A^* & 0 \end{pmatrix}$ has $\tilde{A}(0 \oplus x) = b \oplus 0$ when $Ax = b$, so HHL applied to $\tilde{A}$ and $b \oplus 0$ returns $0 \oplus x$.

definition *herm-embed* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
herm-embed A = *four-block-mat* (*0*$_m$ (*dim-row A*) (*dim-row A*)) *A* (*A*†) (*0*$_m$ (*dim-col A*) (*dim-col A*))

lemma *herm-embed-carrier*: *A* ∈ *carrier-mat n n* ⟹ *herm-embed A* ∈ *carrier-mat* (*n* + *n*) (*n* + *n*)
⟨*proof*⟩

lemma *zero-dagger*: (*0*$_m$ *n m* :: *complex Matrix.mat*)† = *0*$_m$ *m n*
⟨*proof*⟩

lemma *herm-embed-hermitian*:
assumes *A*: *A* ∈ *carrier-mat n n*
shows *hermitian* (*herm-embed A*)
⟨*proof*⟩

lemma *zero-mat-mult-vec*: *dim-vec v* = *n* ⟹ *0*$_m$ *K n* ∗$_v$ *v* = *0*$_v$ *K*
⟨*proof*⟩

lemma *herm-embed-solve*:
assumes *A*: *A* ∈ *carrier-mat n n* **and** *x*: *dim-vec x* = *n* **and** *ax*: *A* ∗$_v$ *x* = *b*
shows *herm-embed A* ∗$_v$ (*0*$_v$ *n* @$_v$ *x*) = *b* @$_v$ *0*$_v$ *n*
⟨*proof*⟩

lemma *herm-embed-inverse*:
assumes *A*: *A* ∈ *carrier-mat n n* **and** *x*: *dim-vec x* = *n* **and** *ax*: *A* ∗$_v$ *x* = *b*
and *z*: *0* ∉ *herm-spectrum* (*herm-embed A*)
shows *herm-inv* (*herm-embed A*) ∗$_v$ (*b* @$_v$ *0*$_v$ *n*) = *0*$_v$ *n* @$_v$ *x*
⟨*proof*⟩

end
theory *HHL-Approximate*
imports *HHL*
begin

102 HHL with inexact phase estimation

When the eigenvalue λ of u is not one of the register values λ_k, phase estimation leaves the register in $\sum_j \alpha_j|j\rangle$ with α_j =*pe-amp*, the rotation acts with C/λ_j on the j-th branch, and uncomputation no longer returns the register to the uniform state. The output is compared with the ideal output $u\otimes R(C/\lambda)|0\rangle$ of the exact case; the squared distance is $\sum_j |\alpha_j|^2 \, \|R(C/\lambda_j)|0\rangle - R(C/\lambda)|0\rangle\|^2$.

102.1 Register sums

$\sum_{j<N} |j\rangle \otimes f_j$ on $\mathbb{C}^N \otimes \mathbb{C}^m$.

definition *reg-sum* :: *nat* ⇒ *nat* ⇒ (*nat* ⇒ *complex Matrix.vec*) ⇒ *complex Matrix.vec* **where**
reg-sum N m f = *Matrix.vec* (*N* ∗ *m*) (λ*r*. *f* (*r div m*) $ (*r mod m*))

lemma *dim-reg-sum* [*simp*]: *dim-vec* (*reg-sum N m f*) = *N* ∗ *m*
⟨*proof*⟩

lemma *reg-sum-index*:
k < *N* ⟹ *i* < *m* ⟹ *reg-sum N m f* $ (*k* ∗ *m* + *i*) = *f k* $ *i*
⟨*proof*⟩

lemma *reg-sum-cong*: (⋀*k*. *k* < *N* ⟹ *f k* = *g k*) ⟹ *reg-sum N m f* = *reg-sum N m g*
⟨*proof*⟩

lemma *tensor-reg-sum*:
assumes *a*: *dim-vec a* = *N* **and** *w*: *dim-vec w* = *m*
shows *tensor-vec a w* = *reg-sum N m* (λ*k*. *a* $ *k* ·$_v$ *w*)
⟨*proof*⟩

lemma *reg-sum-index-div*:
fixes *r N m* :: *nat*
assumes *r* < *N* ∗ *m*
shows *r div m* < *N r mod m* < *m*
⟨*proof*⟩

lemma *reg-sum-add*:
assumes *f*: ⋀*k*. *k* < *N* ⟹ *dim-vec* (*f k*) = *m* **and** *g*: ⋀*k*. *k* < *N* ⟹ *dim-vec* (*g k*) = *m*
shows *reg-sum N m f* + *reg-sum N m g* = *reg-sum N m* (λ*k*. *f k* + *g k*)
⟨*proof*⟩

lemma *reg-sum-diff*:
assumes *g*: ⋀*k*. *k* < *N* ⟹ *dim-vec* (*g k*) = *m*
shows *reg-sum N m f* − *reg-sum N m g* = *reg-sum N m* (λ*k*. *f k* − *g k*)
⟨*proof*⟩

lemma *reg-sum-smult*:
assumes *f*: ⋀*k*. *k* < *N* ⟹ *dim-vec* (*f k*) = *m*
shows *c* ·$_v$ *reg-sum N m f* = *reg-sum N m* (λ*k*. *c* ·$_v$ *f k*)
⟨*proof*⟩

Block-diagonal operators act branch by branch.

lemma *ctrl-blocks-reg-sum*:
assumes *B*: ⋀*k*. *k* < *N* ⟹ *B k* ∈ *carrier-mat m m* **and** *f*: ⋀*k*. *k* < *N* ⟹ *dim-vec* (*f k*) = *m*
shows *ctrl-blocks N m B* ∗$_v$ *reg-sum N m f* = *reg-sum N m* (λ*k*. *B k* ∗$_v$ *f k*)
⟨*proof*⟩

Branches are orthogonal: $\langle \sum_j |j\rangle \otimes f_j, \sum_j |j\rangle \otimes g_j\rangle = \sum_j \langle f_j, g_j\rangle$.

lemma *reg-sum-inner*:
assumes *f*: ⋀*k*. *k* < *N* ⟹ *dim-vec* (*f k*) = *m* **and** *g*: ⋀*k*. *k* < *N* ⟹ *dim-vec* (*g k*) = *m*
shows ⟨*reg-sum N m f*|*reg-sum N m g*⟩ = (∑ *k*<*N*. ⟨*f k*|*g k*⟩)
⟨*proof*⟩

102.2 Phase estimation on a branch

$W(u_N \otimes u \otimes w) = \alpha \otimes u \otimes w$ for every ancilla vector w, where $\alpha_j = \alpha_j(\varphi)$, $e^{it_0\lambda} = e^{2\pi i\varphi}$.

lemma *unitary-inner-self*:
assumes *W*: *W* ∈ *carrier-mat K K unitary W* **and** *y*: *dim-vec y* = *K*
shows ⟨*W*† ∗$_v$ *y*|*W*† ∗$_v$ *y*⟩ = ⟨*y*|*y*⟩
⟨*proof*⟩

lemma *rot-e0-vec*:
rot c ∗$_v$ *unit-vec 2 0* = *Matrix.vec 2* (λ*i*. *if i* = *0 then complex-of-real* (*sqrt* (*1* − *c*^{2})) *else complex-of-real c*)
⟨*proof*⟩

lemma *rot-diff-inner*:
⟨*rot c1* ∗$_v$ *unit-vec 2 0* − *rot c2* ∗$_v$ *unit-vec 2 0*|*rot c1* ∗$_v$ *unit-vec 2 0* − *rot c2* ∗$_v$ *unit-vec 2 0*⟩ =
complex-of-real ((*sqrt* (*1* − *c1*^{2}) − *sqrt* (*1* − *c2*^{2}))^2 + (*c1* − *c2*)^2)
⟨*proof*⟩

lemma *cnj-mult-self*: *cnj z * z = complex-of-real ((cmod z)^2)*
⟨*proof*⟩

lemma *inner-self-sum*: *dim-vec v = N ⟹ ⟨v|v⟩ = (∑ i<N. complex-of-real ((cmod (v $ i))^2))*
⟨*proof*⟩

The register amplitudes have total weight one: $\alpha = F_N^* b$ with b a unit vector.

lemma *pe-amp-sum-sq*:
assumes *N*: *0 < N*
shows *(∑ j<N. (cmod (pe-amp N φ j))^2) = 1*
⟨*proof*⟩

Tail bound: $|\alpha_j| \leq 1/(2N|\delta|)$ for $\delta = \varphi - j/N$, $0 < |\delta| \leq \frac{1}{2}$.

lemma *geometric-amp-tail*:
assumes *N*: *0 < N* **and** *d*: *|δ| ≤ 1 / 2* **and** *d0*: *δ ≠ 0*
shows *cmod ((∑ k<N. cis (2 * pi * real k * δ)) / of-nat N) ≤ 1 / (2 * real N * |δ|)*
⟨*proof*⟩

lemma *pe-amp-tail*:
assumes *N*: *0 < N* **and** *d*: *|φ − real j / real N| ≤ 1 / 2* **and** *d0*: *φ ≠ real j / real N*
shows *cmod (pe-amp N φ j) ≤ 1 / (2 * real N * |φ − real j / real N|)*
⟨*proof*⟩

The circuit with an arbitrary angle $f(k)$ on register value k; *hhl-circuit* is $f(k) = C/\lambda_k$, the physical circuit clamps the angle to $[-1, 1]$.

definition *hhl-gen* :: *complex Matrix.mat ⇒ real ⇒ nat ⇒ (nat ⇒ real) ⇒ complex Matrix.mat* **where**
*hhl-gen A t0 N f = (hhl-W A t0 N)† * (ctrl-blocks N (dim-row A * 2) (λk. 1_m (dim-row A) ⨂ rot (f k)) * hhl-W A t0 N)*

definition *clamp* :: *real ⇒ real* **where**
clamp x = max (−1) (min 1 x)

definition *hhl-circuit-clamped* :: *complex Matrix.mat ⇒ real ⇒ nat ⇒ real ⇒ complex Matrix.mat* **where**
hhl-circuit-clamped A t0 N C = hhl-gen A t0 N (λk. clamp (C / reg-eig t0 N k))

lemma *hhl-circuit-gen*: *hhl-circuit A t0 N C = hhl-gen A t0 N (λk. C / reg-eig t0 N k)*
⟨*proof*⟩

lemma *rot-blk-carrier* [*simp*]: *1_m n ⨂ rot x ∈ carrier-mat (n * 2) (n * 2)*
⟨*proof*⟩

lemma *clamp-abs*: *|clamp x| ≤ 1*
⟨*proof*⟩

lemma *clamp-id*: *|x| ≤ 1 ⟹ clamp x = x*
⟨*proof*⟩

definition *hhl-phi* :: *real ⇒ real ⇒ real* **where**
*hhl-phi t0 l = t0 * l / (2 * pi)*

context
fixes *A* :: *complex Matrix.mat* **and** *n N* :: *nat* **and** *t0 l* :: *real* **and** *u* :: *complex Matrix.vec*
and *f* :: *nat ⇒ real* **and** *c* :: *real*
assumes *A*: *A ∈ carrier-mat n n hermitian A* **and** *t0*: *0 < t0* **and** *N*: *0 < N*
and *u*: *dim-vec u = n* **and** *ev*: *A *_v u = complex-of-real l ·_v u*
begin

The phase of u, the register amplitudes, the ideal and the branch rotations.

abbreviation *phi* ≡ *hhl-phi t0 l*
definition *alpha* = *Matrix.vec N* (*pe-amp N phi*)
abbreviation *r-ideal* ≡ *rot c* $*_v$ *unit-vec 2 0*
definition *r-br k* = *rot* (*f k*) $*_v$ *unit-vec 2 0*

lemma *phi-phase*: *cis* (*t0* ∗ *l*) = *cis* (*2* ∗ *pi* ∗ *phi*)
⟨*proof*⟩

lemma *dim-alpha* [*simp*]: *dim-vec alpha* = *N* ⟨*proof*⟩
lemma *dim-r-ideal*: *dim-vec r-ideal* = *2* ⟨*proof*⟩
lemma *dim-r-br* [*simp*]: *dim-vec* (*r-br k*) = *2* ⟨*proof*⟩

lemma *W-branch*:
assumes *w*: *dim-vec w* = *2*
shows *hhl-W A t0 N* $*_v$ *tensor-vec* (*uniform-state N*) (*tensor-vec u w*) = *tensor-vec alpha* (*tensor-vec u w*)
⟨*proof*⟩

The output of the circuit and the ideal output as register sums.

lemma *hhl-output-branches*:
hhl-gen A t0 N f $*_v$ *tensor-vec* (*uniform-state N*) (*tensor-vec u* (*unit-vec 2 0*)) =
(*hhl-W A t0 N*)† $*_v$ *reg-sum N* (*n* ∗ *2*) (λ*k*. *alpha* $ *k* $\cdot_v$ *tensor-vec u* (*r-br k*))
⟨*proof*⟩

lemma *hhl-ideal-branches*:
tensor-vec (*uniform-state N*) (*tensor-vec u r-ideal*) =
(*hhl-W A t0 N*)† $*_v$ *reg-sum N* (*n* ∗ *2*) (λ*k*. *alpha* $ *k* $\cdot_v$ *tensor-vec u r-ideal*)
⟨*proof*⟩

The squared distance between output and ideal output.

theorem *hhl-approx-error*:
⟨*hhl-gen A t0 N f* $*_v$ *tensor-vec* (*uniform-state N*) (*tensor-vec u* (*unit-vec 2 0*)) − *tensor-vec* (*uniform-state N*) (*tensor-vec u r-ideal*) |
hhl-gen A t0 N f $*_v$ *tensor-vec* (*uniform-state N*) (*tensor-vec u* (*unit-vec 2 0*)) − *tensor-vec* (*uniform-state N*) (*tensor-vec u r-ideal*)⟩ =
⟨*u*|*u*⟩ ∗ (∑ *k*<*N*. *complex-of-real* ((*cmod* (*pe-amp N phi k*))^2) ∗ ⟨*r-br k* − *r-ideal*|*r-br k* − *r-ideal*⟩)
⟨*proof*⟩

The branch distance in real terms.

corollary *hhl-approx-error-real*:
⟨*hhl-gen A t0 N f* $*_v$ *tensor-vec* (*uniform-state N*) (*tensor-vec u* (*unit-vec 2 0*)) − *tensor-vec* (*uniform-state N*) (*tensor-vec u r-ideal*) |
hhl-gen A t0 N f $*_v$ *tensor-vec* (*uniform-state N*) (*tensor-vec u* (*unit-vec 2 0*)) − *tensor-vec* (*uniform-state N*) (*tensor-vec u r-ideal*)⟩ =
⟨*u*|*u*⟩ ∗ *complex-of-real* (∑ *k*<*N*. (*cmod* (*pe-amp N phi k*))^2 ∗
((*sqrt* (*1* − (*f k*)^2) − *sqrt* (*1* − *c*^{2}))^2 + (*f k* − *c*)^2))
⟨*proof*⟩

end

102.3 Bounds on the branch distances and amplitudes

$\||R(a)|0\rangle - R(b)|0\rangle\|^2 \leq 4$ for $|a|, |b| \leq 1$.

lemma *rot-dist-le-four*:
assumes |*a*| ≤ *1* |*b*| ≤ *1*

shows $(sqrt\ (1 - a^2) - sqrt\ (1 - b^2))^2 + (a - b)^2 \leq 4$
⟨*proof*⟩

Lipschitz estimate for $0 \leq a, b \leq \theta < 1$: $\|R(a)|0\rangle - R(b)|0\rangle\|^2 \leq (a-b)^2/(1-\theta^2)$.

lemma *rot-dist-lipschitz*:
assumes *th*: $0 < \vartheta\ \vartheta < 1$ **and** *a*: $0 \leq a\ a \leq \vartheta$ **and** *b*: $0 \leq b\ b \leq \vartheta$
shows $(sqrt\ (1 - a^2) - sqrt\ (1 - b^2))^2 + (a - b)^2 \leq (a - b)^2\ /\ (1 - \vartheta^2)$
⟨*proof*⟩

The amplitudes are 1-periodic in φ.

lemma *pe-amp-shift-up*: $pe\text{-}amp\ N\ \varphi\ j = pe\text{-}amp\ N\ (\varphi + 1)\ j$
⟨*proof*⟩

102.4 The error bound

lemma *pe-amp-tail-sq*:
assumes *N*: $0 < N$ **and** *d*: $|\varphi - real\ j\ /\ real\ N| \leq 1\ /\ 2$ **and** *d0*: $\varphi \neq real\ j\ /\ real\ N$
shows $(cmod\ (pe\text{-}amp\ N\ \varphi\ j))^2 \leq 1\ /\ (4 * (real\ N)^2 * (\varphi - real\ j\ /\ real\ N)^2)$
⟨*proof*⟩

Standing hypotheses of the bound: $0 < C$, $0 < \theta < 1$, $\lambda \geq 2C/\theta$ (the eigenvalue is at least twice the threshold $\lambda_{\mathrm{lo}} = C/\theta$ below which the angle is clamped) and $t_0\lambda \leq \pi/2$ (the phase $\varphi = t_0\lambda/2\pi$ is at most $\frac{1}{4}$).

lemma *hhl-param-facts*:
assumes *t0*: $0 < t0$ **and** *C*: $0 < C$ **and** *th*: $0 < \vartheta\ \vartheta < 1$
and *l-lo*: $2 * C\ /\ \vartheta \leq l$ **and** *l-hi*: $t0 * l \leq pi\ /\ 2$
shows $0 < l\ 0 < hhl\text{-}phi\ t0\ l\ hhl\text{-}phi\ t0\ l \leq 1\ /\ 4\ C\ /\ l \leq \vartheta\ /\ 2\ 0 \leq C\ /\ l\ |C\ /\ l| \leq 1$
$2 * C \leq \vartheta * l$
⟨*proof*⟩

Good branches: register value $\lambda_k \geq C/\theta$ in the positive half.

lemma *hhl-branch-good*:
assumes *N*: $0 < N$ **and** *t0*: $0 < t0$ **and** *C*: $0 < C$ **and** *th*: $0 < \vartheta\ \vartheta < 1$
and *l-lo*: $2 * C\ /\ \vartheta \leq l$ **and** *l-hi*: $t0 * l \leq pi\ /\ 2$
and *k*: $2 * k < N$ **and** *lam*: $C\ /\ \vartheta \leq reg\text{-}eig\ t0\ N\ k$
shows $(cmod\ (pe\text{-}amp\ N\ (hhl\text{-}phi\ t0\ l)\ k))^2 *$
$((sqrt\ (1 - (clamp\ (C\ /\ reg\text{-}eig\ t0\ N\ k))^2) - sqrt\ (1 - (C\ /\ l)^2))^2 + (clamp\ (C\ /\ reg\text{-}eig\ t0\ N\ k) - C\ /\ l)^2)$
$\leq pi^2 * \vartheta\ \hat{}\ 4\ /\ ((real\ N)^2 * t0^2 * C^2 * (1 - \vartheta^2))$
⟨*proof*⟩

Branches below the threshold in the positive half: weight at most $1/(N\varphi)^2$.

lemma *hhl-branch-low*:
assumes *N*: $0 < N$ **and** *t0*: $0 < t0$ **and** *C*: $0 < C$ **and** *th*: $0 < \vartheta\ \vartheta < 1$
and *l-lo*: $2 * C\ /\ \vartheta \leq l$ **and** *l-hi*: $t0 * l \leq pi\ /\ 2$
and *k*: $2 * k < N$ **and** *lam*: $reg\text{-}eig\ t0\ N\ k < C\ /\ \vartheta$
shows $(cmod\ (pe\text{-}amp\ N\ (hhl\text{-}phi\ t0\ l)\ k))^2 \leq 1\ /\ ((real\ N)^2 * (hhl\text{-}phi\ t0\ l)^2)$
⟨*proof*⟩

Branches in the negative half: weight at most $1/(N\varphi)^2$.

lemma *hhl-branch-neg*:
assumes *N*: $0 < N$ **and** *t0*: $0 < t0$ **and** *C*: $0 < C$ **and** *th*: $0 < \vartheta\ \vartheta < 1$
and *l-lo*: $2 * C\ /\ \vartheta \leq l$ **and** *l-hi*: $t0 * l \leq pi\ /\ 2$
and *k*: $k < N$ **and** *k2*: $\neg\ 2 * k < N$
shows $(cmod\ (pe\text{-}amp\ N\ (hhl\text{-}phi\ t0\ l)\ k))^2 \leq 1\ /\ ((real\ N)^2 * (hhl\text{-}phi\ t0\ l)^2)$
⟨*proof*⟩

Every branch: $|\alpha_k|^2 d_k^2 \le b_1 + b_2$ with $b_1 = \pi^2\theta^4/(N^2 t_0^2 C^2(1-\theta^2))$, $b_2 = 4/(N\varphi)^2$.

lemma *hhl-branch-bound*:
assumes *N*: *0 < N* **and** *t0*: *0 < t0* **and** *C*: *0 < C* **and** *th*: *0 < ϑ ϑ < 1*
and *l-lo*: *2 * C / ϑ ≤ l* **and** *l-hi*: *t0 * l ≤ pi / 2* **and** *k*: *k < N*
shows (*cmod* (*pe-amp N* (*hhl-phi t0 l*) *k*))^2 *
((*sqrt* (*1 − (clamp* (*C / reg-eig t0 N k*))^2) − *sqrt* (*1 − (C / l*)^2))^2 + (*clamp* (*C / reg-eig t0 N k*) − *C / l*)^2)
≤ *pi*^{2} * *ϑ ^ 4 / ((real N*)^2 * *t0*^{2} * *C*^{2} * (*1 − ϑ*^{2})) + *4 / ((real N*)^2 * (*hhl-phi t0 l*)^2)
⟨*proof*⟩

The error bound for the clamped circuit on an eigenvector: with $\varphi = t_0\lambda/2\pi$,

$$\|\text{output} - u_N \otimes u \otimes R(C/\lambda)|0\rangle\|^2 \le \|u\|^2 \,\frac{\pi^2}{N t_0^2}\Big(\frac{\theta^4}{C^2(1-\theta^2)} + \frac{16}{\lambda^2}\Big).$$

lemma *hhl-bound-algebra*:
assumes *N*: *0 < N* **and** *t0*: *0 < t0* **and** *C*: *0 < C* **and** *th*: *0 < ϑ ϑ < 1* **and** *l0*: *0 < l*
shows *real N * (pi*^{2} * *ϑ ^ 4 / ((real N*)^2 * *t0*^{2} * *C*^{2} * (*1 − ϑ*^{2})) + *4 / ((real N*)^2 * (*hhl-phi t0 l*)^2))
=
pi^2 */ (real N * t0*^{2}*) * (ϑ ^ 4 / (C*^{2} * (*1 − ϑ*^{2})) + *16 / l*^{2})
⟨*proof*⟩

theorem *hhl-clamped-error-bound*:
fixes *A* :: *complex Matrix.mat* **and** *u* :: *complex Matrix.vec*
assumes *A*: *A ∈ carrier-mat n n hermitian A* **and** *t0*: *0 < t0* **and** *N*: *0 < N*
and *u*: *dim-vec u = n* **and** *ev*: *A *$_v$ u = complex-of-real l ·$_v$ u*
and *C*: *0 < C* **and** *th*: *0 < ϑ ϑ < 1*
and *l-lo*: *2 * C / ϑ ≤ l* **and** *l-hi*: *t0 * l ≤ pi / 2*
shows *Re* ⟨*hhl-circuit-clamped A t0 N C *$_v$ tensor-vec* (*uniform-state N*) (*tensor-vec u* (*unit-vec 2 0*))
−
tensor-vec (*uniform-state N*) (*tensor-vec u* (*rot* (*C / l*) *$_v$ *unit-vec 2 0*)) |
*hhl-circuit-clamped A t0 N C *$_v$ tensor-vec* (*uniform-state N*) (*tensor-vec u* (*unit-vec 2 0*)) −
tensor-vec (*uniform-state N*) (*tensor-vec u* (*rot* (*C / l*) *$_v$ *unit-vec 2 0*))⟩
≤ *Re* ⟨*u*|*u*⟩ * (*pi*^{2} */ (real N * t0*^{2}*) * (ϑ ^ 4 / (C*^{2} * (*1 − ϑ*^{2})) + *16 / l*^{2}))
⟨*proof*⟩

end